

Integ_specificMessage Builder

Axway Integration Broker

Version 2.1.1



Copyright © Axway Software, 2014
All rights reserved.

This documentation describes the following Axway software:
Axway Integration Broker.

In this document, the name of the Axway Integration Broker software product often appears as "XIB". This usage is not to be construed as an attempt by Axway Software to establish legal title over the mark "XIB", and is only for purposes of clarity, convenience, and brevity in the documentation that describes the Axway Integration Broker product.

No part of this publication may be reproduced, transmitted, stored in a retrieval system, or translated into any human or computer language, in any form or by any means, electronic, mechanical, magnetic, optical, chemical, manual, or otherwise, without the prior written permission of the copyright owner, Axway Software.

This document, provided for informational purposes only, may be subject to significant modification. The descriptions and information in this document may not necessarily accurately represent or reflect the current or planned functionalities of this product. Axway Software may change this publication, the product described herein, or both. These changes will be incorporated in new versions of this document. Axway Software does not warrant that this document is error free.

Axway Software recognizes the rights of the holders of all trademarks used in its publications.

Preface

PURPOSE OF THIS MANUAL

This document describes:

Chapters	Contains
Introduction	provides a general overview of Message Builder.
Introductory Message Builder examples	contains a number of examples introducing the Message Builder language.
Constants and variables	describes how constants and variables are used in Message Builder.
Expressions	explains how expressions are used.
Built-in statements	describes some of the most important built-in statements.
Control flow	describes statements that are used to control the flow of execution.
Database access	describes the SQL database interface supported by Message Builder.
EDI parsing	describes statements used to parse EDI messages.
User functions and statements	describes user-written functions and statements.
Error handling and logging	describes error handling and logging.
Compiling and executing Message Builder programs	describes the start syntax of the Message Builder compiler and interpreter.
Message Builder language reference	contains the Message Builder Language Reference that describes all the features of the language in detail.
EDIFACT examples	contains examples that read and write EDIFACT messages.
An X12 example	contains an example of how to parse an X12 message.
Obsolete functions and statements	describes obsolete Message Builder functions and statements.
The Message Builder grammar	contains the YACC grammar for the Message Builder compiler.

WHO SHOULD READ THIS MANUAL

This manual is intended for Axway Integration Broker (hereafter referred to as "Integration Broker") users responsible for designing Message Builder programs.

The **Message Builder language** is a general-purpose programming language. The major part of the Integration Broker is written in the Message Builder language. This manual consists of an informal description of the language, beginning in chapter 1, and a reference section in chapter 12.

ASSUMED KNOWLEDGE

In this guide, we assume that you have a good understanding of the previous versions of XIB.

RELATED DOCUMENTATION

XIB is accompanied by a complete set of documentation, covering all aspects of using the component.

All XIB documentation is available from the Axway Support website: www.support.axway.com.

Table of contents

1	Introduction	12
1.1	Presentation	12
2	Introductory Message Builder examples	13
2.1	Presentation	13
2.2	A simple Message Builder program	13
2.3	File access	15
2.4	Database access	15
2.5	EDI parsing	16
3	Constants and variables	18
3.1	Presentation	18
3.2	Constants	18
3.3	Built-in variable types	19
3.3.1	The INTEGER variable type	19
3.3.2	The FLOAT variable type	19
3.3.3	The DATE variable type	19
3.3.4	The STRING variable type	20
3.3.5	The CHAR variable type	21
3.4	Data records	21
3.5	Arrays	22
3.6	Reserved variables	23
4	Expressions	24
4.1	Presentation	24
4.2	Operators and precedence	24
4.2.1	Arithmetic operators	25
4.2.2	Comparison operators	25
4.2.3	Logical operators	25
4.2.4	Bit manipulation operators	25
4.2.5	String operators	26
4.3	Types and conversion	27
5	Built-in statements	28

5.1	Presentation	28
5.2	The assignment statement	28
5.3	The PRINT statement	29
5.4	The READ statement	29
5.5	The OPEN statement	29
5.6	The CLOSE statement	30
6	Control flow	31
6.1	Presentation	31
6.2	The IF statement	31
6.3	The CASE statement	32
6.4	Looping statements	33
	6.4.1 The WHILE statement	34
	6.4.2 The REPEAT statement	34
	6.4.3 The FOR statement	34
	6.4.4 The BREAK and CONTINUE statements	35
7	Database Access	37
7.1	Presentation	37
7.2	Open and close the database	38
	7.2.1 Informix	38
	7.2.2 DB2 and ODBC	38
	7.2.3 Oracle	39
	7.2.4 Sybase	39
	7.2.5 AS/400 native	39
	7.2.6 MySQL	39
7.3	SQL statements	40
	7.3.1 Writing data to the database	40
	7.3.2 Reading data from the database	41
	7.3.3 Database and table location	42
7.4	Large string support	43
	7.4.1 Oracle	43
	7.4.2 DB2 (on UNIX or Windows, not OS/400) using ODBC or CLI	43
	7.4.3 IBM iSeries Access (IBM Universal Database on OS/400 accessed through ODBC on Windows)	44
	7.4.4 MySQL through ODBC	45
	7.4.5 Microsoft SQL Server	45
7.5	Unicode support for SQL data	45
	7.5.1 Oracle	45
	7.5.2 DB2 (on UNIX or Windows, not OS/400) using ODBC or CLI	46
	7.5.3 IBM iSeries Access (IBM Universal Database on OS/400 accessed through ODBC on Windows)	46
	7.5.4 MySQL through ODBC	47
	7.5.5 Microsoft SQL Server	47

8	EDI parsing	48
8.1	Presentation	48
8.2	The SEGMENT statement	48
8.3	The DATA ELEMENT statements	49
8.4	The COMPOSITE statement	50
8.5	EDI separator characters	50
8.6	An EDIFACT example	51
8.7	The SEGMENT GROUP statement	51
9	User functions and statements	53
9.1	Presentation	53
9.2	User functions	53
9.2.1	Default argument values	54
9.3	User statements	55
9.3.1	Default parameter values	56
10	Error handling and logging	57
10.1	Presentation	57
10.2	The <i>\$Error</i> variable	57
10.3	The <i>\$LogString</i> variable	57
10.4	Log messages	58
10.4.1	Errors	58
10.4.2	Fatal errors	59
10.4.3	Warnings	59
10.4.4	Informational "errors"	59
10.5	The LOG statement	59
10.6	The NOLOG statement	60
10.7	Error handling	60
10.8	Back trace information	61
11	Compiling and executing Message Builder programs	63
11.1	Presentation	63
11.2	Start syntax for the Message Builder compiler	63
11.2.1	Command Arguments	64
11.2.2	Environment variables	71
11.2.3	Library files	71
11.3	Start syntax for the Message Builder interpreter	72

11.3.1	Command arguments	72
11.3.2	Environment variables	78
11.3.3	Server mode	80
11.4	Examples	81
12	Message Builder language reference	82
12.1	Presentation	82
12.2	Language overview	83
12.2.1	General statements	83
12.2.2	Declaration statements	83
12.2.3	SQL statements	84
12.2.4	EDI statements	85
12.2.5	Interface statements	85
12.2.6	File management statements	85
12.2.7	String-handling functions and statements	86
12.2.8	Coroutine functions and statements	86
12.2.9	Debug statements	86
12.2.10	Miscellaneous functions	87
12.2.11	Miscellaneous statements	87
12.2.12	Other topics	88
12.3	Evaluation order	88
12.4	Reference pages	88
12.4.1	ANTINOLOG	89
12.4.2	ARGUMENT	90
12.4.3	ARGUMENTCOUNT	91
12.4.4	ARRAY DELETE	92
12.4.5	ARRAY INSERT	93
12.4.6	ARRAYDIM	94
12.4.7	ARRAYSIZE	95
12.4.8	ASSIGNMENT	96
12.4.9	BLOCK	97
12.4.10	BOUNDED LOOP	98
12.4.11	BREAK	101
12.4.12	CALL	101
12.4.13	CASE	103
12.4.14	CHARACTER	104
12.4.15	CLOSE	104
12.4.16	CODE	105
12.4.17	COMMENTS	106
12.4.18	COMPOSITE	106
12.4.19	CONSTANTS	108
12.4.20	CONTINUE	109
12.4.21	COROUTINE	110
12.4.22	COROUTINE KILL	111
12.4.23	COROUTINE LOCK	112
12.4.24	COROUTINE NEW	113
12.4.25	COROUTINE RESUME	115
12.4.26	COROUTINE TERMINATE	115
12.4.27	CURRENTDATE	117
12.4.28	DATA ELEMENT (EDIFACT)	117
12.4.29	DATA ELEMENT (X12)	120
12.4.30	DATABASE CLOSE	124
12.4.31	DATABASE EXECUTE	125
12.4.32	DATABASE OPEN	126
12.4.33	DATE	132

12.4.34	DEBUG	132
12.4.35	DECLARE EXTERNAL FUNCTION	135
12.4.36	DECLARE EXTERNAL STATEMENT	138
12.4.37	DECLARE FUNCTION	142
12.4.38	DECLARE MODULE	146
12.4.39	DECLARE RECORD	149
12.4.40	DECLARE STATEMENT	153
12.4.41	DECLARE VARIABLE	159
12.4.42	DUMP BACKTRACE	162
12.4.43	DUMP COROUTINES	164
12.4.44	DUMP PROFILE	165
12.4.45	DUMP VARIABLES	167
12.4.46	ESCAPE	169
12.4.47	EXEC SQL ALTER TABLE	169
12.4.48	EXEC SQL BEGIN WORK	170
12.4.49	EXEC SQL COMMIT WORK	171
12.4.50	EXEC SQL CREATE INDEX	172
12.4.51	EXEC SQL CREATE TABLE	173
12.4.52	EXEC SQL CREATE VIEW	175
12.4.53	EXEC SQL DELETE	175
12.4.54	EXEC SQL DROP INDEX	176
12.4.55	EXEC SQL DROP TABLE	177
12.4.56	EXEC SQL DROP VIEW	178
12.4.57	EXEC SQL GRANT	179
12.4.58	EXEC SQL INSERT	180
12.4.59	EXEC SQL REVOKE	181
12.4.60	EXEC SQL ROLLBACK WORK	182
12.4.61	EXEC SQL SELECT	182
12.4.62	EXEC SQL UPDATE	185
12.4.63	EXECUTE	186
12.4.64	EXIT	187
12.4.65	EXPORT	188
12.4.66	EXPRESSIONS	190
12.4.67	FIDENT	193
12.4.68	FILE APPEND	193
12.4.69	FILE COPY	195
12.4.70	FILE DELETE	196
12.4.71	FILE MOVE	197
12.4.72	FOR	198
12.4.73	FORMAT	199
12.4.74	FROM_ISO8859	202
12.4.75	GETDATE	202
12.4.76	IF	204
12.4.77	IMPORT	205
12.4.78	INCLUDE	206
12.4.79	INTERP_VERSION	207
12.4.80	IOSET	208
12.4.81	IOSET CREATE	208
12.4.82	IOSET DELETE	209
12.4.83	IOSET SWITCH	210
12.4.84	LOADPGM	211
12.4.85	LOG	213
12.4.86	NOLOG	214
12.4.87	OPEN	215
12.4.88	PIPE CLOSE	217
12.4.89	PIPE OPEN	218
12.4.90	PIPE READ	219
12.4.91	PIPE WRITE	220
12.4.92	PRAGMA	221
12.4.93	PRINT	223

12.4.94	PRINTERR	224
12.4.95	PROGRAMINFO	225
12.4.96	READ	227
12.4.97	REGEXP	228
12.4.98	RELATION	229
12.4.99	RELEASE	231
12.4.100	REPEAT	232
12.4.101	RESERVED VARIABLES	233
12.4.102	RETURN	237
12.4.103	SCAN	237
12.4.104	SEGMENT	239
12.4.105	SEGMENT GROUP	242
12.4.106	SERIALIZE	244
12.4.107	SETDATE	245
12.4.108	SIDENT	247
12.4.109	SLEEP	248
12.4.110	SOURCEFILE	248
12.4.111	SOURCELINE	249
12.4.112	SOURCEMODULE	249
12.4.113	SOURCEPROCEDURE	250
12.4.114	SPLIT CLOSE	250
12.4.115	SPLIT INTO	251
12.4.116	SPLIT OPEN	253
12.4.117	STRBACKWARD	254
12.4.118	STRFORWARD	255
12.4.119	STRLEN	257
12.4.120	STRREPEAT	257
12.4.121	SUBSTRING	258
12.4.122	SUSPEND	259
12.4.123	SWITCH	260
12.4.124	SYSTEM	261
12.4.125	THROW	262
12.4.126	TO_ISO8859	263
12.4.127	TRIMLEFT	263
12.4.128	TRIMRIGHT	264
12.4.129	TRUNCATE (function)	264
12.4.130	TRUNCATE (statement)	266
12.4.131	TRY	267
12.4.132	UNBOUNDED LOOP	269
12.4.133	UNLOADPGM	272
12.4.134	UNSERIALIZE	273
12.4.135	USE	274
12.4.136	VARIABLE TYPES	275
12.4.137	WHILE	277
12.4.138	WITH	278
13	Appendix A: EDIFACT examples	280
13.1	Presentation	280
13.2	Syntax check	280
13.2.1	Segment specification	280
13.3	Data extraction	283
13.3.1	Storing extracted data in an SQL database	284
13.3.2	Storing extracted data in a conventional text file	284
13.4	Creating an EDI message	284
13.4.1	Using data from an SQL database	284
13.4.2	Using data from a conventional text file	285

14	Appendix B: An X12 example	287
	14.1 Presentation	287
15	Appendix C: Obsolete functions and statements	291
	15.1 Presentation	291
	15.2 Reference Pages	291
	15.2.1 BINARY	291
	15.2.2 CENTER	292
	15.2.3 CONVERT	293
	15.2.4 EDI	294
	15.2.5 LEFT	295
	15.2.6 NDEC	296
	15.2.7 ON EDI ERROR	297
	15.2.8 ON EXECUTE ERROR	298
	15.2.9 READTAG	299
	15.2.10 RIGHT	300
	15.2.11 STRCNV	301
	15.2.12 STRMID	302
	15.2.13 TRUNC	303
	15.2.14 UNIQUE_ID	304
	15.2.15 UNIQUE_NAME	305
	15.2.16 WRITETAG	306
16	The Message Builder grammar	308
	16.1 Presentation	308

1 Introduction

1.1 Presentation

Message Builder is a general purpose programming language. Among other things, it contains special constructs to access SQL databases and to handle EDI messages. Large parts of the Integration Broker are written using Message Builder.

Add on packages add extra capabilities to the Message Builder language. Examples are graphical user interface programming and socket communication.

Message Builder runs on different versions of UNIX, on Windows, and AS/400. With one exception (the AS/400), compiled Message Builder programs can be moved between different computers and executed without recompilation.

The language can be extended with Message Builder written functions and statements that can be used in the same way as built-in functions and statements. It is also possible to extend the language with C-written functions and statements (refer to the *Extender manual*).

A Message Builder program consists of one or more text files containing the program text. These files are compiled using a compiler that produces an executable Message Builder program. This executable program consists of instructions for an imaginary machine. An interpreter implements this imaginary machine and is used to execute the program.

The following manuals are of interest to a Message Builder programmer:

- Message Builder Library
- Extender

The *Message Builder Library* manual contains documentation of Message Builder functions and statements that do not belong to the core language. It contains two parts:

- **Part 1:** *Extensions Libraries* describes C-written extensions to the core language (see the *Extender* manual for a description of how to create such extensions).
- **Part 2:** *Auxiliary Functions and Statements* describes Message-Builder-written functions and statements.

2 Introductory Message Builder examples

2.1 Presentation

This chapter contains a number of examples that introduce the Message Builder language. They are presented in order to give an idea of what a Message Builder program is, and to give examples of the main features of the Message Builder language.

2.2 A simple Message Builder program

The first example is a Message Builder program that contains some simple calculations. The results of these calculations are output to the screen. The task is to print a conversion table from Celsius to Fahrenheit. The table ranges from -5 to +5 degrees Celsius. First, the program and its results are presented. Then the program is discussed.

The program looks like this:

```
/*
 * A simple Message Builder program.
 */
DECLARE VARIABLE $celsius INTEGER;
DECLARE VARIABLE $fahrenheit FLOAT;

PRINT "Celsius    Fahrenheit\n";

$celsius = -5;
WHILE $celsius <= 5 {
    $fahrenheit = 1.8 * $celsius + 32;
    PRINT STRREPEAT(" ", 7 - STRLEN($celsius)) &
        $celsius & " = ";
    PRINT FORMAT("%.1f", $fahrenheit) & "\n";
    IF $celsius = 0 {
        PRINT "At last we're melting the ice!\n";
    }
    $celsius = $celsius + 1;
}
# end of program
```

The result written to the screen is:

```
Celsius    Fahrenheit
-5 = 23.0
-4 = 24.8
```

```

-3 = 26.6
-2 = 28.4
-1 = 30.2
0 = 32.0
At last we're melting the ice!
1 = 33.8
2 = 35.6
3 = 37.4
4 = 39.2
5 = 41.0

```

Except for string constants, it does not matter if upper or lowercase letters are used. The first statement could be replaced by:

```
DeclAre varIABle $CelsiuS InTeGer;
```

The meaning of the program would be the same. **In this manual, uppercase letters are used for language keywords.** Lowercase or a mix of lower- and uppercase is used for variables. User functions and statements are usually written using uppercase letters only.

The two `DECLARE VARIABLE` statements reserve memory for two variables. The first `PRINT` statement prints the table header on the screen. Notice the `"\n"` sequence that outputs a new-line to the screen.

After assigning the start value for `$celsius`, the main `WHILE` loop is entered. This loop takes care of the calculation and the printout of each row of the table. There is also an `IF` statement that checks if the `$celsius` variable has become equal to zero degrees Celsius and, if so, a message is printed.

The `STRREPEAT` and `FORMAT` functions are used to format the variables when they are printed. The `$celsius` variable will be right justified in a field of 7 characters and `$fahrenheit` will be printed with one fractional digit. The `&` character is a string concatenation operator.

This program example also demonstrates the two types of comments that can be used in a Message Builder program. The first three lines are a C-style comment. This type of comment starts with the sequence `/*` and ends with `*/`. A C-style comment may contain new-lines. The last line demonstrates the other type. It starts with a `#` character and continues to the end of the line.

If the above program is put in the file `conv.s4`, the program can be compiled using the Message Builder compiler:

```
c4edi conv.s4
```

If no syntax errors are found in the file, a file named `conv.x4` is the result of the compilation. It is an executable Message Builder program that can be executed using the Message Builder interpreter:

```
r4edi conv.x4
```

For further details on using the Message Builder compiler and interpreter, see chapter [11 Compiling and executing Message Builder programs](#).

2.3 File access

The next example demonstrates how data can be written to a file and how to switch between a file and the screen. The program is:

```
DECLARE VARIABLE $textFile STRING;
$textFile = "test.txt";

PRINT "First message. Printed on the screen.\n";

OPEN FILE OUTPUT $textFile;
PRINT "Second message. Printed to the file " &
      $textFile & ".\n";
CLOSE OUTPUT;

PRINT "Third message. Printed on the screen.\n";

OPEN FILE APPEND $textFile;
PRINT "Fourth message. Printed to the file " &
      $textFile & ".\n";
CLOSE OUTPUT;
```

The PRINT statement outputs data to the currently selected file. When a Message Builder program is started, the output file is the standard output (normally the screen). After a CLOSE OUTPUT statement has been executed, the standard output becomes the output file again.

The APPEND instead of the OUTPUT keyword to the OPEN FILE statement can be used if data in an existing file should not be overwritten.

2.4 Database access

The Message Builder language includes a subset of the X/Open SQL syntax standard for data processing. The interpreter sends the SQL statements to the Relational Database Management System (RDBMS) for execution. In the following example an RDBMS from Informix has been used (the only statement that is database dependent in this example is DATABASE OPEN).

The following is a short Message Builder program that inserts some data into the RDBMS and then reads it back. Finally, the data is deleted:

```
DECLARE VARIABLE $name STRING;
DECLARE VARIABLE $country STRING;

DATABASE OPEN "db";

$country = "Germany";

/*
 * Insert cities into the database.
 */
EXEC SQL INSERT INTO cities
          VALUES ("Stockholm", "Sweden");
EXEC SQL INSERT INTO cities
```

```

VALUES ("Dresden", :$country);
EXEC SQL INSERT INTO cities
VALUES ("Hamburg", :$country);

/*
 * Read cities from the database.
 */
EXEC SQL SELECT city_name INTO :$name FROM cities
WHERE country_name = :$country {
PRINT $name & " is a city in " & $country & "\n";
}

/*
 * Delete all cities from the database.
 */
EXEC SQL DELETE FROM cities;

```

Running the above program causes the following to be written to the output file:

```

Dresden is a city in Germany
Hamburg is a city in Germany

```

The following observations can be made from the above example: All SQL statements use the EXEC SQL prefix, and Message Builder variables in SQL statements are prefixed by a colon (:) character. This follows the X/Open rules for how SQL statements are embedded in other languages.

2.5 EDI parsing

The following example opens the *edi.dat* file as input and reads an EDIFACT UNB segment from it:

```

DECLARE VARIABLE $SyntaxIdentifier STRING;
DECLARE VARIABLE $SyntaxVersion INTEGER;
DECLARE VARIABLE $SenderId STRING;
DECLARE VARIABLE $IdCodeQualifier1 STRING;
DECLARE VARIABLE $ReverseRoutingAddress STRING;
DECLARE VARIABLE $RecipientId STRING;
DECLARE VARIABLE $IdCodeQualifier2 STRING;
DECLARE VARIABLE $RoutingAddress STRING;
DECLARE VARIABLE $Date STRING;
DECLARE VARIABLE $Time STRING;
DECLARE VARIABLE $IntChngRef STRING;
DECLARE VARIABLE $ReceipientRef STRING;
DECLARE VARIABLE $RecipientRefQual STRING;
DECLARE VARIABLE $ApplicationRef STRING;
DECLARE VARIABLE $ProcessingPrioCode STRING;
DECLARE VARIABLE $AckRequest INTEGER;
DECLARE VARIABLE $CommAgreementId STRING;
DECLARE VARIABLE $TestIndicator INTEGER;

OPEN FILE INPUT "edi.dat";

MANDATORY SEGMENT "UNB" {
MANDATORY COMPOSITE {
MANDATORY FIXED STRING 4 SET $SyntaxIdentifier;
MANDATORY FIXED NUMBER 1 SET $SyntaxVersion;
}
MANDATORY COMPOSITE {

```



```

        MANDATORY STRING 35 SET $SenderId;
        CONDITIONAL STRING 4 SET $IdCodeQualifier1;
        CONDITIONAL STRING 14 SET $ReverseRoutingAddress;
    }
    MANDATORY COMPOSITE {
        MANDATORY STRING 35 SET $RecipientId;
        CONDITIONAL STRING 4 SET $IdCodeQualifier2;
        CONDITIONAL STRING 14 SET $RoutingAddress;
    }
    MANDATORY COMPOSITE {
        MANDATORY FIXED STRING 6 SET $Date;
        MANDATORY FIXED STRING 4 SET $Time;
    }
    MANDATORY STRING 14 SET $IntChngRef;
    CONDITIONAL COMPOSITE {
        MANDATORY STRING 14 SET $ReceipientRef;
        CONDITIONAL FIXED STRING 2 SET $RecipientRefQual;
    }
    CONDITIONAL STRING 14 SET $ApplicationRef;
    CONDITIONAL FIXED STRING 1 SET $ProcessingPrioCode;
    CONDITIONAL FIXED NUMBER 1 SET $AckRequest;
    CONDITIONAL STRING 35 SET $CommAgreementId;
    CONDITIONAL FIXED NUMBER 1 SET $TestIndicator;
}

```

The MANDATORY SEGMENT "UNB" statement starts a mandatory "UNB" segment. It also reads a segment tag from the input file and checks that it is "UNB". The MANDATORY COMPOSITE and CONDITIONAL COMPOSITE statements start mandatory and conditional composite elements, respectively. The MANDATORY STRING, CONDITIONAL STRING, MANDATORY NUMBER, and CONDITIONAL NUMBER statements read data elements from the input file. The FIXED modifier indicates that the data element must be exactly as long as the specified length. Otherwise, the specified length is a maximum length. The SET modifier is used to put the contents of a data element in a variable.

3 Constants and variables

3.1 Presentation

A constant in the Message Builder language is an integer, floating-point, or string constant.

A variable in the Message Builder language is used to store a value. Five different types of built-in variables are supported. The difference between them is which type of value they can hold.

User declared data records are also supported. A data record is an aggregate variable type that consists of one or more fields. The type of each field is a built-in type, another record data type, or an array (see below).

Arrays of values of the same type can also be formed. Arrays can have any number of dimensions.

The term **scalar variable** is used to denote **a variable that is not a data record and not an array**. The term **non-array variable** is used to denote **a variable that is not an array**.

A variable name must start with a dollar sign (\$) followed by one or more alphanumeric and underscore characters. Variable names are not case sensitive, that is, there is no difference between using an uppercase and lowercase letter.

All variables must be declared before they are used.

3.2 Constants

A Message Builder program can contain integer, floating-point and string constants.

Examples of integer constants are:

23 -57

A numeric constant is a floating-point constant when it contains a decimal point and/or an exponent. Examples of floating-point constants are:

2.53 -9.75 2E5 2E-3

The last two constants contain exponents. They are equivalent to 200000 and 0.002, respectively.

Strings are enclosed inside double quotes ("). Examples of strings are:

```
"Integration Broker Message Builder" "A string with a newline\n"
```

The "\n" sequence in the last string is a special character sequence that denotes the new-line character.

3.3 Built-in variable types

The different built-in variable types supported by the Message Builder language are integer, floating-point, and date variables and two different types of string variables.

3.3.1 The INTEGER variable type

The INTEGER variable type is used to store a signed integer. A declaration of an integer variable may look like:

```
DECLARE VARIABLE $i INTEGER;
```

This statement declares an integer variable named \$i. The following short program writes 56 to the output file (which is the screen if nothing else has been specified):

```
DECLARE VARIABLE $a INTEGER;  
$a = 56;  
PRINT $a & "\n";
```

The second row assigns the value 56 to the \$a variable and the third row writes the value of \$a to the output file. The "\n" sequence is a string containing a new-line character and the & character concatenates the value of the \$a variable and the "\n" string. The PRINT statement is described on page 29.

3.3.2 The FLOAT variable type

The FLOAT variable type is used to store a floating-point number. A declaration of a floating-point variable may look like:

```
DECLARE VARIABLE $f FLOAT;
```

This statement declares a floating-point variable named \$f. The following short program outputs the value of the \$x variable:

```
DECLARE VARIABLE $x FLOAT;  
$x = 1.5;  
PRINT $x & "\n";
```

Note: 1.500000 is written to the output file. The reason is that the floating-point value is converted to a string before it is written to the output file and that six fractional digits are always used when a floating-point value is converted to a string.

3.3.3 The DATE variable type

The DATE variable type is used to store a date and time value. A declaration of a date variable may look like:

```
DECLARE VARIABLE $d DATE;
```

This statement declares a date variable named `$d`. The following short program outputs the current date:

```
DECLARE VARIABLE $date DATE;
$date = CURRENTDATE();
PRINT $date & "\n";
```

CURRENTDATE is a built-in function that returns the current date and time.

3.3.3.1 AS/400 DATE data type compared to Message Builder DATE variables

A DB2/400 DATE data type contains only a date: year to day, 6-10 characters long depending on the format used. A DB2/400 TIME data type contains hours - seconds and is 8 characters long. DB2/400 DATE and TIME fields must be handled through Message Builder CHAR or STRING variables.

3.3.3.2 AS/400 timestamp

An AS/400 database TIMESTAMP data type has the format:

```
"%Y-%m-%d-%H.%M.%S"
```

for example:

```
"1995-01-05.08.00.00".
```

This is equivalent to the IBM specification:

```
"yyyy-mm-dd-hh.mm.ss(.nnnnn)"
```

The microseconds part `nnnnn` cannot be used by Message Builder. The format above must be strictly followed when you insert a value into an AS/400 timestamp field as an explicit string.

You can insert a Message Builder DATE variable into an AS/400 timestamp database field. The Message Builder will convert the date to the timestamp format for you. You should set the reserved variable `$DateFormat` to the format you are using in your program before you convert a string to a DATE type.

Here is an example of Message Builder code to insert a value into a timestamp database field:

Suppose the table *table* has a field *birthday* of type `TIMESTAMP`.

```
# Declare a Message Builder variable
$Birthday DATE;
# Set your format (assuming you use colons in your program)
$DateFormat="%Y-%m-%d %H:%M:%S"
# Assign a value for your format
$Birthday = "1995-01-05 08:00:00"
# Insert the value in the database
EXEC SQL INSERT INTO= table (birthday) VALUES (: $Birthday);
```

The value in `$Birthday` is converted to timestamp format (1995-01-05.08.00.00) by Message Builder before it is inserted into the database.

3.3.4 The STRING variable type

The STRING variable type is used to store a string. A declaration of a string variable may look like:

```
DECLARE VARIABLE $s STRING;
```

This statement declares a string variable named `$s`. The following short program writes `Integration Broker` to the output file:

```
DECLARE VARIABLE $s STRING;
$s = "Integration Broker";
PRINT $s & "\n";
```

There is no upper limit for the length of the value of a `STRING` variable.

3.3.5 The `CHAR` variable type

The `CHAR` variable type is a variant of the `STRING` type. It is also used to store a string value. The difference is that the value is automatically padded with trailing spaces to a specified length. A declaration of a `CHAR` variable may look like:

```
DECLARE VARIABLE $c CHAR(10);
```

This statement declares a `CHAR` variable of length 10. The following short program:

```
DECLARE VARIABLE $c CHAR(10);
$c = "Integration Broker";
PRINT $c & "System\n";
```

outputs the string:

```
Integration Broker System
```

Note: The space between "Integration Broker" and "System" created by the automatic padding of the `$c` variable.

If a string that is longer than the declared length is assigned to a `CHAR` variable, the string is truncated.

3.4 Data records

Data records can be used to group multiple values of different types into one variable. A data record consisting of one integer field and one string field may be declared as follows:

```
DECLARE RECORD RRR {
    DECLARE FIELD $integer INTEGER;
    DECLARE FIELD $string STRING;
}
```

The name of the record is `RRR`. The name of the integer field is `$integer` and the name of the string field is `$string`.

A variable of type RECORD RRR may then be declared as follows:

```
DECLARE VARIABLE $r RECORD RRR;
```

Specifying the record variable name and the field separated by a dot accesses the individual fields of a record variable. The following example assigns the value 5 to the integer field and the "demo" string to the string field:

```
$r.$integer = 5;
$r.$string = "demo";
```

Record variables may be assigned as long as the left and right hand sides in the assignment are of the same type. The following example declares two record variables of type RRR assigns values to the fields of the first record variable, assigns the first record variable to the second and, finally, outputs the contents of the second record variable:

```
DECLARE VARIABLE $r1 RECORD RRR;
DECLARE VARIABLE $r2 RECORD RRR;
$r1.$integer = 123;
$r1.$string = "MB";
$r2 = $r1;
PRINT $r2.$integer & " " & $r2.$string & "\n";
```

The following is written to the output file:

```
123 MB
```

3.5 Arrays

Arrays can be used to group multiple values of the same type into one variable. A one-dimensional array of integers may be declared as:

```
DECLARE VARIABLE $a1[] INTEGER;
```

The square bracket pair indicates that it is an array. Note that no size is given. Arrays grow and shrink as needed. The number of bracket pairs specifies the number of dimensions for the array. A two dimensional array may thus be declared as:

```
DECLARE VARIABLE $a2[][] INTEGER;
```

The first element of an array has index 1. Assigning a value to the first element of the \$a1 array and then printing it may look like:

```
$a1[1] = 4;
PRINT $a1[1] & "\n";
```

When a value is assigned to a non-existing element, the array is automatically expanded. However, it is considered an error to reference a non-existing array element. For example, in the following program an error will occur at the last line because the fourth element does not exist:

```
DECLARE VARIABLE $x[] STRING;
$x[1] = "Integration Broker ";
$x[2] = "EDI ";
$x[3] = "System\n";
PRINT $x[1];
PRINT $x[2];
PRINT $x[3];
PRINT $x[4];
```

The `ARRAYSIZE` built-in function (see paragraph [12.4.7](#)) can be used to determine the size of an array and the `ARRAYDIM` built-in function (see paragraph [12.4.6](#)) can be used to obtain the number of dimensions.

The array index does not have to be a constant. The following program initializes the first 10 elements of an array to 1.0:

```
DECLARE VARIABLE $f[] FLOAT;  
FOR $i = 1 TO 10 {  
    $f[$i] = 1.0;  
}
```

The above program causes the array assignment to be executed 10 times while the value of `$i` change from 1 to 10. The `FOR` statement is described on paragraph [6.4.3](#).

3.6 Reserved variables

The Message Builder language contains a number of built-in variables called reserved variables. They are global variables that have special meaning in the language and they should not be declared using the `DECLARE VARIABLE` statement.

The most commonly used reserved variable is `$Error`. It contains the error code from the last statement or function that failed. Other commonly used reserved variables are the variables containing the EDI separator characters. For a complete list of all reserved variables, see chapter [12 Message Builder language reference](#).

4 Expressions

4.1 Presentation

Expressions combine values (or operands) with operators to produce new values. For example, the expression $4+2$ contains two operands, 4 and 2, and one operator, +; it evaluates to 6. **Message Builder statement parameters** and **function arguments** are expressions. For example, the PRINT statement evaluates its parameter and writes the result to the output file:

```
PRINT (3 + 2) * (2.1 - 0.1);
```

The above example writes 10.000000 to the output file. This chapter uses the PRINT statement for some of its examples.

4.2 Operators and precedence

The operators supported by the Message Builder language are:

```
-      (unary minus)
*      /      %
+      -
BNOT
BAND
BOR    BXOR
BSHIFT
&
=      <>      <      <=      >      >=
NOT
AND
OR     XOR
```

There is also a sub-string extraction operator and an array indexing operator.

Groups of operators with the same *precedence* are listed on the same line and operators with higher precedence appear above operators with lower precedence. For example, $4*2<7$ evaluates to 0 because the * operator has higher precedence than < and the multiplication is thus evaluated before the comparison. Parentheses can be used to change the precedence order. They can also be used to make the meaning of an expression clearer.

Operators with the same precedence group from left to right, for example, $10-4-3$ are the same as $(10-4)-3$; it evaluates to 3.

The evaluation order is undefined. This is important to remember when calling user functions with side effects. For a description of user functions, see chapter [9 User functions and statements](#).

Depending on the operator, the operands must be of a particular type. If not, they are automatically converted. For example:

```
PRINT "5.6" * 2 & "\n";
```

writes 11.200000 to the output file. Before the multiplication is performed, the "5.6" string is converted to the floating-point value 5.6 and the integer value 2 is converted to the floating-point value 2.0. Before the string concatenation, the result from the multiplication is converted to a string.

4.2.1 Arithmetic operators

The Message Builder language supports the arithmetic operators +, -, *, / and %. The - operator may be used either as a binary operator for subtraction, as in 4-2, or as a unary operator for negation, as in -(6*\$i). The operands are always converted to a floating-point value before the operation is performed and the result is a floating-point value.

4.2.2 Comparison operators

The operators < (less than), <= (less than or equal), > (greater than), >= (greater than or equal), = (equal), and <> (not equal) are used for comparing two values. Each operator produces an integer result of 1 (true) if its operands meet the condition, and 0 (false) if they do not.

If both operands are strings, the comparison is made lexically, for example, the comparisons:

```
"A" < "A"  
"A" < "B"  
"A" < "AA"  
"AA" < "A"
```

evaluates to 0, 1, 1, and 0, respectively.

If both operands are not strings, they are converted to floating-point values before the comparison.

4.2.3 Logical operators

The logical operators AND, OR and NOT are typically used for combining the results of comparison operators, as in the expression (note that the parentheses are not needed):

```
($x > 4) AND ($x < 10)
```

Each operator produces the integer value 0 or 1. AND produces a 1 result if both its operands are non-zero, OR produces a 1 result if either of its operands are non-zero, XOR produces a 1 result if exactly one of its operands is non-zero, and NOT produces a 1 result if its operand is 0.

In Message Builder, a 0 value is treated as false and anything other than 0 is treated as true. Whenever Message Builder generates a true/false value, it always uses 1 for true and 0 for false. The following example generates a true value because both of the operands are non-zero:

```
0.1 AND -0.98
```

4.2.4 Bit manipulation operators

The bit manipulation operators are BNOT, BAND, BOR, BXOR, and BSHIFT. All these operators convert their arguments to integer values and return an integer value.

The binary operator BAND performs a bitwise AND operation between its arguments, that is, the result bit is one if both bits are one, otherwise it is zero. The assignment:

```
$a = 3 BAND 7;
```

is equivalent to:

```
$a = 3;
```

The binary operator BOR performs a bitwise OR operation between its arguments, that is, the result bit is zero if both bits are zero and otherwise it is one. The assignment:

```
$a = 3 BOR 7;
```

is equivalent to:

```
$a = 7;
```

The binary operator BXOR performs a bitwise EXCLUSIVE-OR operation between its arguments, that is, the result bit is zero if both bits are equal and otherwise it is one. The assignment:

```
$a = 3 BXOR 6;
```

is equivalent to:

```
$a = 5;
```

The unary operator BNOT inverts all the bits in its argument. The assignment:

```
$a = (BNOT 3) BAND 15;
```

is equivalent to:

```
$a = 12;
```

The binary operator BSHIFT shifts the value of its left argument the number of positions specified by the absolute value of its right argument. If the right argument is positive, a left shift is performed and if it is negative, a right shift is performed. Zeros are always shifted in from the right or left respectively. The assignment:

```
$a = 3 BSHIFT 2;
```

is equivalent to:

```
$a = 12;
```

4.2.5 String operators

The string operators supported by Message Builder are string concatenation and sub-string extraction.

The string concatenation operator takes two arguments that are converted to strings and the result is a string. The following example writes `Integration Broker Message Builder` to the output file:

```
PRINT "Integration Broker " & "Message Builder";
```

The sub-string operator is used in the following way:

```
$string[$start, $end]
```

The string to extract a sub-string from is `$string` and the start and end positions for the sub-string is `$start` and `$end`, respectively.

The start and end positions may be arbitrary integer expressions. The sub-string operator may also be used as the left hand side in an assignment, in which case the sub-string is replaced by the right hand side of the assignment.

4.3 Types and conversion

Message Builder normally evaluates expressions numerically. The exceptions are the string concatenation operator and the comparison operators when both operands are strings.

If the operands for an operator have wrong types, they are automatically converted to the correct type. The result of an operation is always of the same type as the operands (after conversion) except for the comparison operators.

The following example demonstrates the automatic conversion between different types:

```
DECLARE VARIABLE $quantity STRING;
DECLARE VARIABLE $price STRING;
DECLARE VARIABLE $cost INTEGER;

/* assign strings from both an integer value and a string literal */
$quantity = 15;
$price = "245";

/* calculate the integer cost from the two strings quantity and price
*/
$cost = $quantity * $price;
PRINT "The cost is $" & $cost & "!\n";
```

5 Built-in statements

5.1 Presentation

A Message Builder program consists of a sequence of statements. A statement can be simple or compound. A compound statement contains other statements. This chapter describes some of the simple statements provided by the Message Builder language. For descriptions of the other statements, see chapter [12 Message Builder language reference](#).

5.2 The assignment statement

Assignment statements replace the current value of a variable with a new value specified by an expression. Examples of assignment statements are:

```
$x = $y + $z;  
$done = ($i >= 1) AND ($i < 100);  
$record.$field = $otherRecord.$otherField + 1;  
$a[$x+1] = $u[$y-1] * $v[$z+1];
```

An assignment statement consists of the variable to replace the value for, and an expression specifying the new value separated by an equal (=) character.

If the type of the value of the expression differs from the type of the variable, the expression value is, if possible, converted to the same type as the variable.

The variable to change the value for may be a scalar variable, a data record, or an array. If it is a data record, the expression must produce a value that is of the same type. For example a single reference to a data record variable:

```
DECLARE RECORD REC {  
    DECLARE FIELD $a STRING;  
    DECLARE FIELD $b FLOAT;  
}  
DECLARE VARIABLE $r1 RECORD REC;  
DECLARE VARIABLE $r2 RECORD REC;  
$r1.$a = "abc";  
$r1.$b = 5.77;  
$r2 = $r1;
```

If it is an array, the expression must produce a value that is an array with the same number of dimensions as the variable to change the value for. This often means that the expression is a single reference to an array as in the following example:

```
DECLARE VARIABLE $a[][] STRING;  
DECLARE VARIABLE $u[] STRING;  
DECLARE VARIABLE $v[] STRING;  
$u[1] = "u1";    /* set the 1st element of $u to "u1" */
```

```

$u[2] = "u2";      /* set the 2nd element of $u to "u2" */
$v[1] = "v1";      /* set the 1st element of $v to "v1" */
$v[2] = "v2";      /* set the 2nd element of $v to "v2" */
$a[1] = $u;        /* assign the $u array to element 1 */
$a[2] = $v;        /* assign the $v array to element 2 */

```

5.3 The PRINT statement

The PRINT statement writes data to the output file. Initially the output file is the standard output. Another output file can be chosen using the OPEN statement explained below.

The following example writes *Integration Broker Message Builder* to the output file:

```
PRINT "Integration Broker Message Builder\n";
```

The string concatenation operator is often used in PRINT statements. The following example outputs the value of the `$xyz` variable:

```
PRINT "$xyz = " & $xyz & "\n";
```

5.4 The READ statement

The READ statement reads data from the input file. Initially the input file is the standard input. **Another input file can be chosen using the OPEN statement.**

When reading data, one possibility is to specify the number of characters/bytes to read, for example,

```
READ $x COUNT 40;
```

This example **reads 40 characters from the input file** and puts them in the `$x` variable. Another possibility is **to read until a particular character is encountered**, for example,

```
READ $x UNTIL "\n";
```

This example reads characters from the input file until a new-line character is encountered. All characters read, except the new-line character, are put in the `$x` variable.

5.5 The OPEN statement

Initially, when a Message Builder program is started, there is one input and one output file open (not counting the error file). The input or the output file can be changed using the OPEN statement.

Opening the *input.dat* file as input may look like:

```
OPEN FILE INPUT "input.dat";
```

The old input file is automatically closed. Until another OPEN FILE INPUT statement has been executed, all READ statements will read data from the *input.dat* file.

Opening the *output.dat* file as output may look like:

```
OPEN FILE OUTPUT "output.dat";
```

The old output file is automatically closed. Until another OPEN FILE OUTPUT statement has been executed, all PRINT statements will write data to the *output.dat* file.

The following example reads one line of data from the file named *source.txt* and writes it to a file named *destination.txt*:

```
DECLARE VARIABLE $line STRING;  
OPEN FILE INPUT "source.txt";  
OPEN FILE OUTPUT "destination.txt";  
READ $line UNTIL "\n";  
PRINT $line & "\n";
```

5.6 The CLOSE statement

The CLOSE statement is used to close the input or output file. Closing the input and output files looks like:

```
CLOSE INPUT;  
CLOSE OUTPUT;
```

After the input file has been closed, the standard input becomes the input file, and after the output file has been closed, the standard output becomes the output file. If the input or output file is closed when the input or output file is already the standard input or output, a warning is generated.

6 Control flow

6.1 Presentation

This chapter describes the Message Builder statements for controlling the flow of execution in a program. The statements presented in this chapter are IF, CASE, WHILE, REPEAT, FOR, BREAK and CONTINUE.

6.2 The IF statement

In its simplest form, the IF statement evaluates an expression, tests its result, and conditionally executes a block of statements based on the result. For example, the following code fragment writes the string "positive" to the output file if the value of `$a` is positive:

```
IF $a > 0.0 {  
    PRINT "positive\n";  
}
```

The expression `$a>0.0` is first evaluated and if it is true (non-zero), that is, `$a` is greater than 0, the statements between the curly brackets (the `{` and `}` characters) are executed.

The IF statement also accepts an ELSE part. The following example writes "positive" if the test expression (`$a>0.0`) evaluates to true and "non-negative" otherwise:

```
IF $a > 0.0 {  
    PRINT "positive\n";  
}  
ELSE {  
    PRINT "non-negative\n";  
}
```

Finally, the ELSE IF part can be used to create a multi-choice statement. The following example prints one of the strings "greater than 9", "equal to 9", "equal to 8", or "less than 8" depending on the value of the integer variable `$i`:

```
IF $i > 9 {  
    PRINT "greater than 9\n";  
}  
ELSE IF $i = 9 {  
    PRINT "equal to 9\n";  
}  
ELSE IF $i = 8 {  
    PRINT "equal to 8\n";  
}  
ELSE {
```

```

        PRINT "less than 8\n";
    }

```

6.3 The CASE statement

The CASE statement compares the value of an expression against a number of constant values and executes zero, or one statement block, depending on which constant value matches. The following example converts an integer value in the range 0 to 9 to the words *zero*, *one*, *two*, *three*, etc.:

```

CASE $i
WHEN 0 {
    $s = "zero";
}
WHEN 1 {
    $s = "one";
}
WHEN 2 {
    $s = "two";
}
WHEN 3 {
    $s = "three";
}
WHEN 4 {
    $s = "four";
}
WHEN 5 {
    $s = "five";
}
WHEN 6 {
    $s = "six";
}
WHEN 7 {
    $s = "seven";
}
WHEN 8 {
    $s = "eight";
}
WHEN 9 {
    $s = "nine";
}
WHEN OTHERS {
    $s = "illegal integer value";
}

```

If `$i` is 5, `$s` is set to *five*. The WHEN OTHERS part is executed if no constant value equals `$i`.

The `$i` variable can be replaced by an expression and the constant values can be replaced by a comma-separated list of constants, as in the following example:

```
CASE 3 * $a - 1
WHEN 1, 2, 3, 4, 5 {
    $s = "in the range 1 - 5";
}
WHEN 6, 7, 8 {
    $s = "in the range 6 - 8";
}
```

This example does not have a WHEN OTHERS part. If the expression evaluates to a value less than 1 or greater than 8, no statement block is executed.

The constant values do not have to be integers. The inverse of the first example may look like:

```
CASE $s
WHEN "zero" {
    $i = 0;
}
WHEN "one" {
    $i = 1;
}
WHEN "two" {
    $i = 2;
}
WHEN "three" {
    $i = 3;
}
WHEN "four" {
    $i = 4;
}
WHEN "five" {
    $i = 5;
}
WHEN "six" {
    $i = 6;
}
WHEN "seven" {
    $i = 7;
}
WHEN "eight" {
    $i = 8;
}
WHEN "nine" {
    $i = 9;
}
```

6.4 Looping statements

Message Builder provides three statements for looping: WHILE, REPEAT and FOR. Each of these statements repeatedly executes a block of statements. They differ in **the types of setup that they perform before each iteration** and in the way they decide to terminate the loop.

6.4.1 The WHILE statement

The WHILE statement takes one test expression and one block of statements to be executed as long as the test expression evaluates to true (non-zero). The following program example writes 4, 6, 8, and 10 to the output file:

```
DECLARE VARIABLE $i INTEGER;
DECLARE VARIABLE $tmp INTEGER;
$i = 2;
WHILE $i <= 5 {
    $tmp = 2 * $i;
    PRINT $tmp & "\n";
    $i = $i + 1;
}
```

Before the statement block is executed, the test expression `$i<=5` is evaluated and, if it is true, the two assignments and the PRINT statement are executed. Then the test expression is evaluated again and if it is true the statement block is executed again, and so forth. If the test expression evaluates to false the first time, the statement block is never executed.

6.4.2 The REPEAT statement

The REPEAT statement is similar to the WHILE statement. The difference is that the test is made at the end of the loop. This means that the loop is always executed at least one time. Here is the WHILE statement example rewritten using the REPEAT statement:

```
DECLARE VARIABLE $i INTEGER;
DECLARE VARIABLE $tmp INTEGER;
$i = 2;
REPEAT {
    $tmp = 2 * $i;
    PRINT $tmp & "\n";
    $i = $i + 1;
} WHILE $i <= 5;
```

6.4.3 The FOR statement

The FOR statement is similar to WHILE except that it provides more explicit loop control. The WHILE statement example, which has been rewritten using the FOR statement, looks like:

```
DECLARE VARIABLE $tmp INTEGER;
FOR $i = 2 TO 5 {
    $tmp = 2 * $i;
    PRINT $tmp & "\n";
}
```

This example demonstrates two things: the loop variable does not have to be declared and it is automatically incremented.

The loop variable is `$i`. It is a scalar variable that is automatically declared as an integer in the FOR loop statement block. It shadows variables with the same name outside the statement block and it is read-only, that is, it cannot be modified.

A loop increment can also be specified using the STEP modifier. The above example may be simplified as follows:

```
FOR $i = 2 TO 10 STEP 2 {
    PRINT $i & "\n";
}
```

If the DOWNTO keyword is used instead of TO, the loop variable is decremented instead. The following example writes 10, 8, 6, 4, and 2 to the output file:

```
FOR $i = 10 DOWNTO 2 STEP 2 {  
    PRINT $i & "\n";  
}
```

The complete syntax for the FOR loop is:

```
FOR var = expression1 TO expression2 STEP expression3 {  
    ...  
}
```

Or

```
FOR var = expression1 DOWNTO expression2 STEP expression3 {  
    ...  
}
```

The STEP *expression3* part is optional (the default is 1). All the expressions (*expression1*, *expression2*, and *expression3*) are evaluated once when the loop is entered. This means that the end condition (*expression2*) or increment (*expression3*) cannot be changed from within the loop. The program example:

```
DECLARE VARIABLE $tmp INTEGER;  
$tmp = 4;  
FOR $i = 1 TO $tmp {  
    $tmp = 10;  
    PRINT $i & "\n";  
}  
PRINT $tmp;
```

writes the numbers 1, 2, 3, 4, and 10 to the output file.

6.4.4 The BREAK and CONTINUE statements

The BREAK and CONTINUE statements can be used to break out of a loop or to continue with the next iteration of a loop.

The BREAK statement causes the innermost enclosing loop to terminate immediately. The following example prints elements in an array as long as the elements are not empty:

```
DECLARE VARIABLE $s[] STRING;  
$s[1] = "one";  
$s[2] = "two";  
$s[3] = "three";  
$s[4] = "four";  
$s[5] = "five";  
$s[6] = "six";  
$s[7] = "";  
$s[8] = "eight";  
$s[9] = "nine";  
$s[10] = "ten";  
FOR $i = 1 TO 10 {  
    IF $s[$i] = "" {  
        BREAK;  
    }  
    PRINT $s[$i] & "\n";  
}
```

The output from the above example will be:

```
one  
two  
three  
four
```

```
five  
six
```

The CONTINUE statement causes only the current iteration of the innermost loop to be terminated; the loop continues with the next iteration. If the BREAK statement is replaced by CONTINUE in the above example, the output changes to:

```
one  
two  
three  
four  
five  
six  
eight  
nine  
ten
```

7 Database Access

7.1 Presentation

The Message Builder language includes statements to open and close a connection to a database and SQL statements to access the database.

The supported SQL statements are a subset of the X/Open SQL standard and are verified to comply with this syntax subset (to avoid discovering the syntax error at runtime). To use syntax outside of this subset, many syntax elements (such as column names, whole expressions, and so on) can be replaced with the custom blocks similar to the following:

```
@( " something=\"xx\" and "  
    & ' xyzzzy='  
    & :$MB_variable_to_become_SQL_host_variable  
    & " or x=y /* comment that will "  
    & "be sent to the database server */ "  
)
```

These custom blocks can contain anything, are not checked against the database system accepted syntax until at runtime, and do not pose an SQL injection risk since the strings are constants and the variables are converted to database-specific variable placeholders rather than being concatenated to the strings.

All SQL statements use the EXEC SQL prefix and all Message Builder variables used in SQL statements must be preceded by a colon (:) character. This follows the X/Open rules for how SQL statements shall be embedded in other languages.

If an error occurs in a database statement an exception is thrown, unless ON EXECUTE ERROR is used. The database connection is not closed and further SQL statements can still be sent through it (unless, of course, the error occurs in DATABASE OPEN).

The SQL statements supported by the Message Builder language are:

Statement	Description
EXEC SQL ALTER TABLE	Add new columns to a table.
EXEC SQL BEGIN WORK	Start a transaction.
EXEC SQL COMMIT WORK	End and store a transaction.
EXEC SQL CREATE INDEX	Create an index.
EXEC SQL CREATE TABLE	Create a table.
EXEC SQL CREATE VIEW	Create a viewed table.
EXEC SQL DELETE	Remove data from a table.
EXEC SQL DROP INDEX	Destroy an index.
EXEC SQL DROP TABLE	Destroy a table.
EXEC SQL DROP VIEW	Destroy a view.
EXEC SQL GRANT	Grant privileges on a table.

EXEC SQL INSERT	Insert data into a table.
EXEC SQL REVOKE	Revoke privileges on a table.
EXEC SQL ROLLBACK WORK	End and undo a transaction.
EXEC SQL SELECT	Read data from one or more tables.
EXEC SQL UPDATE	Update data in a table.

The SQL language is not described in this manual, only the interface between Message Builder and SQL. For more information about SQL and its syntax, see the RDBMS manuals and the *X/Open Portability Guide Issue 3 – Data Management*.

7.2 Open and close the database

Before the database can be accessed, a DATABASE OPEN statement must be executed. The purpose of this statement is to create a channel to the RDBMS and log in to it. The DATABASE OPEN statement takes one string parameter. The contents of this parameter are RDBMS dependent.

Opening a connection to an Informix database may look like:

```
DATABASE OPEN "dbname";
```

Closing the connection to the database does not differ between different databases. The DATABASE CLOSE statement is used to close a database connection:

```
DATABASE CLOSE;
```

Below, for each RDBMS type supported by the Message Builder language, the format of the DATABASE OPEN parameter string is listed.

7.2.1 Informix

When opening a connection to an Informix database, the format of the DATABASE OPEN statement is as follows:

```
DATABASE OPEN "database-name@server-name";
```

The *database-name* part is the name of the database and *server-name* is the name of the database server. The *@server-name* part is normally left out. It is only necessary if a connection to a database other than the current database server is desired.

When opening a connection to an Informix SE database, the database directory can also be specified:

```
DATABASE OPEN "pathname/database-name@server-name";
```

The *pathname* part is the path of the database directory up to the parent directory of the *.dbs* directory (the directory where Informix-SE database files reside).

7.2.2 DB2 and ODBC

When opening a connection to a DB2 or ODBC database, the format of the DATABASE OPEN statement is as follows:

```
DATABASE OPEN "data-source/user/password";
```

The *data-source* part specifies the data source name, the *user* part is the user identifier, and *password* is an authentication string (typically a password).

7.2.3 Oracle

When opening a connection to an Oracle database, the format of the DATABASE OPEN statement is as follows:

```
DATABASE OPEN "username/password@host-name";
```

The *username* and *password* parts are mandatory. The *host-name* part specifies the host that the RDBMS is running on. If the RDBMS is running on the local machine, the *@host-name* part may be left out.

7.2.4 Sybase

When opening a connection to a Sybase database, the format of the DATABASE OPEN statement is as follows:

```
DATABASE OPEN "user/password/database-name/server-name/char-set";
```

The *user* field is the user name and *char-set* is the name of the character set to use. The *database-name*, *server-name* and *char-set* fields may be left out and trailing slashes (/) can be removed.

7.2.5 AS/400 native

When opening a connection to an AS/400 Native database, the format of the DATABASE OPEN statement is as follows:

```
DATABASE OPEN "library-name/options"
```

The *library-name* field is mandatory. It is used to specify the location of an SQL package object, which is created by the AS/400 SQL interface. The *library-name* field for the SQL package is usually specified as QTEMP.

The database *options* field is optional. If it is omitted, it is taken from the DB_FORMATS environment variable (see paragraph [11.3.2](#)). See the DATABASE OPEN statement, page 126, for a description of the *options* field.

7.2.6 MySQL

If ODBC is available, it is recommended to use it and the MySQL ODBC connector instead of this somewhat limited native connector. In that case, when using the UnixODBC driver manager, it may be necessary to create a symbolic link to enable the MySQL driver to load:

```
ln -s lib/libodbcinst.so.2 lib/libodbcinst.so.1
```

When opening a connection using the MySQL native connector, the format of the DATABASE OPEN statement is as follows:

```
DATABASE OPEN "user/password/database/host/port/unix-socket";
```

All fields are optional; if they are empty or if the specification string has less than five slashes, they are assumed to be unused.

If the *user* field is unused, the current operating system user is used.

If the *password* field is unused, users with empty passwords in the MySQL user table will be searched for a match.

If the *database* field is unused the default database is selected.

If the *host* field is unused, or equal to "localhost", a connection to the local host is created using UNIX domain sockets rather than TCP/IP.

If the *host* is remote and the *port* field is used, it can specify a non-default port number where the remote MySQL server is listening. The *port* field must be a number and not a service name from /etc/services.

If the *host* field specifies the local host and the *unix-socket* field is used, it can specify the name of the UNIX domain socket where the local MySQL server is listening.

7.3 SQL statements

This section describes how data is communicated between the host language—Message Builder—and the embedded language—SQL. Familiarity with the SQL language is assumed.

7.3.1 Writing data to the database

The EXEC SQL INSERT statement is used to insert data into a table in the database:

```
DATABASE OPEN "db";
EXEC SQL INSERT INTO xyz VALUES ("data1", "data2");
DATABASE CLOSE;
```

The above example inserts one row of data into the xyz table. The inserted data consists of the fixed strings "data1" and "data2". It is often necessary to insert data that is obtained during run-time. This can be accomplished if we insert the values of variables:

```
DECLARE VARIABLE $s1 STRING;
DECLARE VARIABLE $s2 STRING;
DATABASE OPEN "db";
$s1 = "data1";
$s2 = "data2";
EXEC SQL INSERT INTO xyz VALUES (: $s1, : $s2);
DATABASE CLOSE;
```

Note: colon characters precede the Message Builder variables in the SQL statement.

Sometimes it is desired to insert null values into the database. To accomplish this for both the columns above, each Message Builder variable must be replaced by a variable pair. The second variable in such a pair is called an indicator variable. If it contains the integer value -1, a null value is inserted. The following example inserts null values instead of empty strings:

```
DECLARE VARIABLE $s1 STRING;
DECLARE VARIABLE $i1 INTEGER;
DECLARE VARIABLE $s2 STRING;
DECLARE VARIABLE $i2 INTEGER;
DATABASE OPEN "db";
.
.
.
IF $s1 = "" {
    $i1 = -1;
}
```



```

ELSE {
    $i1 = 0;
}
IF $s2 = "" {
    $i2 = -1;
}
ELSE {
    $i2 = 0;
}
EXEC SQL INSERT INTO xyz VALUES (:s1 :i1, :s2 :i2);
DATABASE CLOSE;

```

7.3.2 Reading data from the database

The EXEC SQL SELECT statement is used to read data from the database. In Message Builder the SELECT SQL statement **has been augmented by a statement block that is executed for each row of data returned from the database.**

Remember that many databases, such as recent Oracle or Microsoft SQL Server, do not preserve the order of inserted rows, even if no rows have ever been deleted from that database table.

The following example reads the *c1* and *c2* columns from the *xyz* table and write the read data to the output file:

```

DECLARE VARIABLE $c1 STRING;
DECLARE VARIABLE $c2 STRING;
DATABASE OPEN "db";
EXEC SQL SELECT c1, c2 INTO :$c1, :$c2 FROM xyz {
    PRINT "c1 = '" & $c1 & "'\n";
    PRINT "c2 = '" & $c2 & "'\n";
}
DATABASE CLOSE;

```

The INTO clause is used to specify a list of Message Builder destination variables. Note that colon characters precede the destination variables.

In the previous example, if the read data contains null values, the values of the destination variables will be empty strings. Indicator variables can be used to detect null values:

```

DECLARE VARIABLE $c1 STRING;
DECLARE VARIABLE $i1 INTEGER;
DECLARE VARIABLE $c2 STRING;
DECLARE VARIABLE $i2 INTEGER;
DATABASE OPEN "db";
EXEC SQL SELECT c1, c2 INTO :$c1 :$i1, :$c2 :$i2 FROM xyz {
    IF $i1 = -1 {
        PRINT "c1 = null\n";
    }
    ELSE {
        PRINT "c1 = '" & $c1 & "'\n";
    }
    IF $i2 = -1 {
        PRINT "c2 = null\n";
    }
    ELSE {
        PRINT "c2 = '" & $c2 & "'\n";
    }
}
DATABASE CLOSE;

```

The indicator variables \$i1 and \$i2 will receive the value -1 if the corresponding column contains a null value. The indicator variables are also used to indicate truncated data, in which case they receive the length of the non-truncated data (see the EXEC SQL SELECT statement reference, paragraph [12.4.61](#), for further information). If a column is not null and the data is not truncated, the indicator variable is set to 0.

7.3.2.1 Typecasting database variables on the AS/400

When using host variables, the SQL engine on AS/400 does not support typecasting. That is, the type of the host variable must match the data type of the database variable exactly.

Data type	Message Builder host variable type
CHAR(x)	CHAR(x)
INTEGER	INTEGER
FLOAT	FLOAT
TIMESTAMP	DATE or STRING(19)
DATE	STRING(6-10) or CHAR(6-10)
TIME	STRING(8) or CHAR(8)
DECIMAL	FLOAT

7.3.3 Database and table location

(For details, refer to: EXEC SQL ALTER TABLE and EXEC SQL UPDATE).

In the DB2/400 SQL environment, the native QSYS file system behaves as one large database. It is therefore possible to address any table (file in native AS/400 syntax) on an AS/400 if the correct SQL syntax is used. The syntax can be divided into two types, one for table creation and one for table access.

7.4 Large string support

If a variable is declared like this:

```
DECLARE VARIABLE $str STRING;
```

all SQL statements will use a maximum size of 255 bytes for this string. Therefore it is usually necessary to declare the maximum size explicitly when using strings in SQL.

Example:

```
DECLARE VARIABLE $str STRING(500000);  
DECLARE VARIABLE $chr CHAR(500000);
```

However, this will only eliminate the 255 bytes limit. The various database types have various limits on the size of a CHAR or VARCHAR column/data type. We support the larger, special data types used to circumvent these limitations only if the corresponding database type is listed in this chapter.

7.4.1 Oracle

We support the BLOB and CLOB data types. No special handling is necessary.

CLOBs cannot be compared (for example, they cannot be used in WHERE clauses) on this database system.

7.4.2 DB2 (on UNIX or Windows, not OS/400) using ODBC or CLI

We support the BLOB and CLOB data types. CLOBs cannot be compared (for example, they cannot be used in WHERE clauses) on this database system.

When used through ODBC you must

```
EXPORT "32672" AS "EDI_CLOB_THRESHOLD";
```

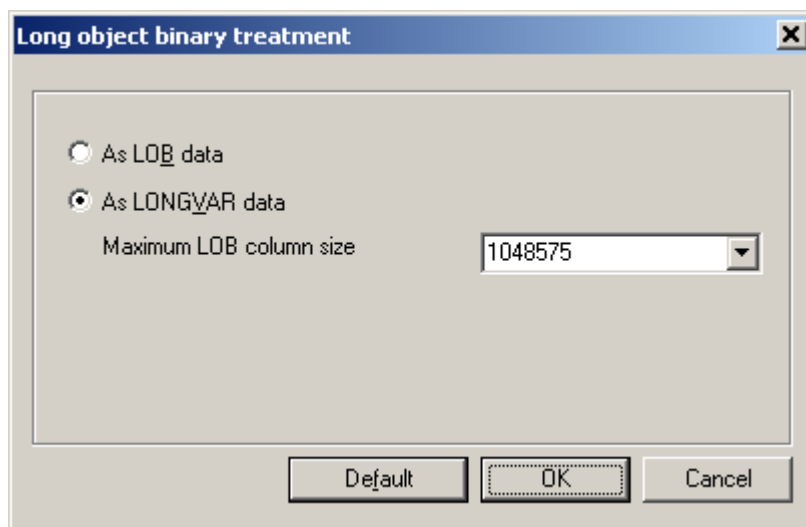
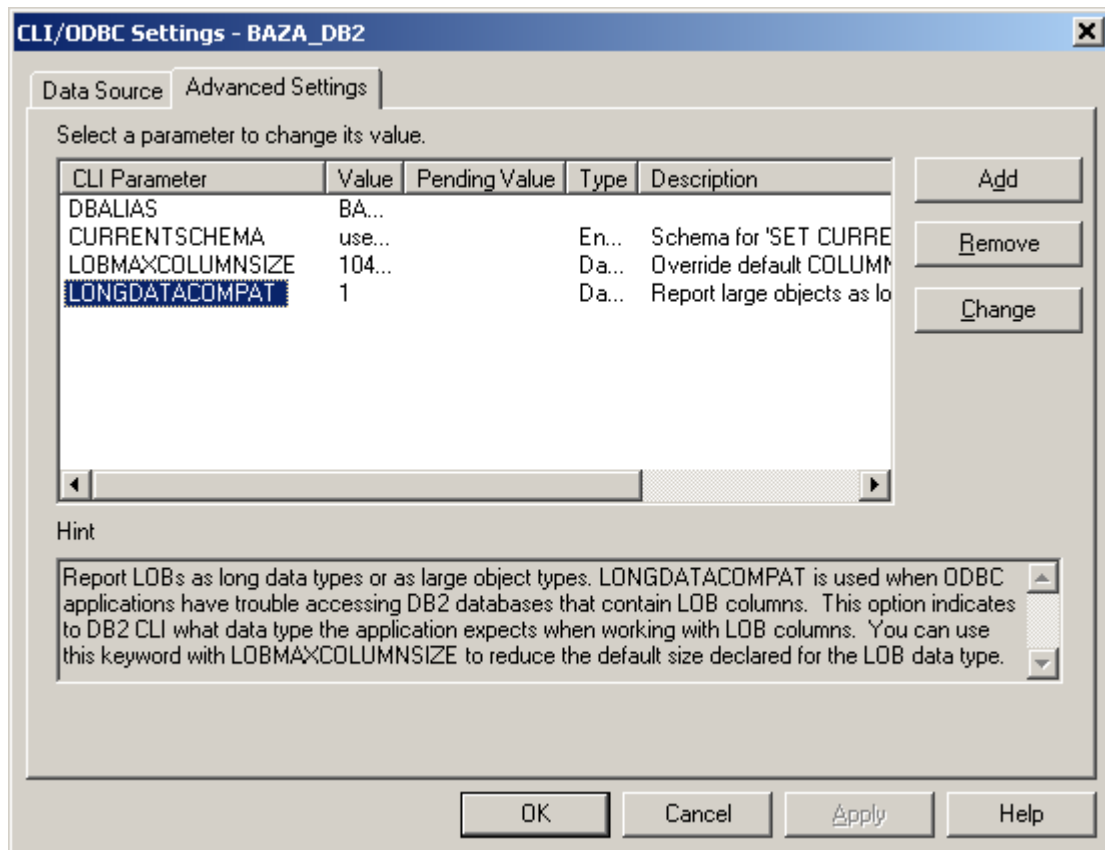
before calling DATABASE OPEN.

On UNIX,

```
longdatacompat=1
```

must be present in the db2cli.ini file.

On Windows, the data source needs to be configured like this:



7.4.3 IBM iSeries Access (IBM Universal Database on OS/400 accessed through ODBC on Windows)

We support the BLOB and CLOB data types. CLOBs cannot be compared (for example, used in WHERE clauses) on this database system.

You must

```
EXPORT "32672" AS "EDI_CLOB_THRESHOLD";
```

before calling DATABASE OPEN.

7.4.4 MySQL through ODBC

Note that we do not support large strings through the MySQL native database interface, only through ODBC. We support MySQL through ODBC on UNIX and Windows.

On MySQL, if the total size of the strings in a row exceeds 65535, some of the string columns need to be defined as MEDIUMTEXT (maximum size of 2 to the power of 24 bytes) or LARGETEXT (maximum size of 2 to the power of 32 bytes).

EDI_CLOB_THRESHOLD should be defined accordingly before calling DATABASE OPEN.

7.4.5 Microsoft SQL Server

We support VARBINARY(MAX) and VARCHAR(MAX) columns. In a table with a VARBINARY(MAX) column named looong, the correct insert command is this:

```
EXEC SQL INSERT INTO test_long(looong)
VALUES (CAST(CAST(:$big_string AS varchar(max)) AS
varbinary(max)));
```

The correct select command is this:

```
EXEC SQL SELECT CAST(looong AS varchar(max))
                                INTO :$received_big_string
                                FROM test_long
{
    ...
}
```

The correct WHERE clause is this:

```
EXEC SQL SELECT CAST(looong AS varchar(max))
                                INTO :$received_big_string
                                FROM test_long
                                WHERE looong=CAST(CAST(:$big_string AS
varchar(max)) AS varbinary(max))
{
    ...
}
```

You must

```
EXPORT "8000" AS "EDI_CLOB_THRESHOLD";
```

before calling DATABASE OPEN.

On UNIX, you need to set the maximum size of a VARBINARY string transmitted over wire, in bytes, in the [global] section of freetds.conf:

```
text size = 500000
```

7.5 Unicode support for SQL data

Unless listed below, Unicode strings are not supported on a database interface.

7.5.1 Oracle

We support the NVARCHAR2 data type.

```
DATABASE EXECUTE "SET 4EDI CODEPAGE=873";
```

will change the character set used in SQL string variables, received from or sent to the Oracle database, to UTF-8. The character set can be changed any number of times and is associated with the connection.

The 873 number was obtained through the following query (note that the dual table is a dummy table, it's just used to enable the select call and the nls_charset_id Oracle SQL function):

```
SQL> select nls_charset_id('AL32UTF8') from dual;

NLS_CHARSET_ID('AL32UTF8')
-----
                        873
```

The NLS codes, such as 'AL32UTF8' can be obtained from the Oracle documentation.

7.5.2 DB2 (on UNIX or Windows, not OS/400) using ODBC or CLI

We support the VARGRAPHIC data type and others.

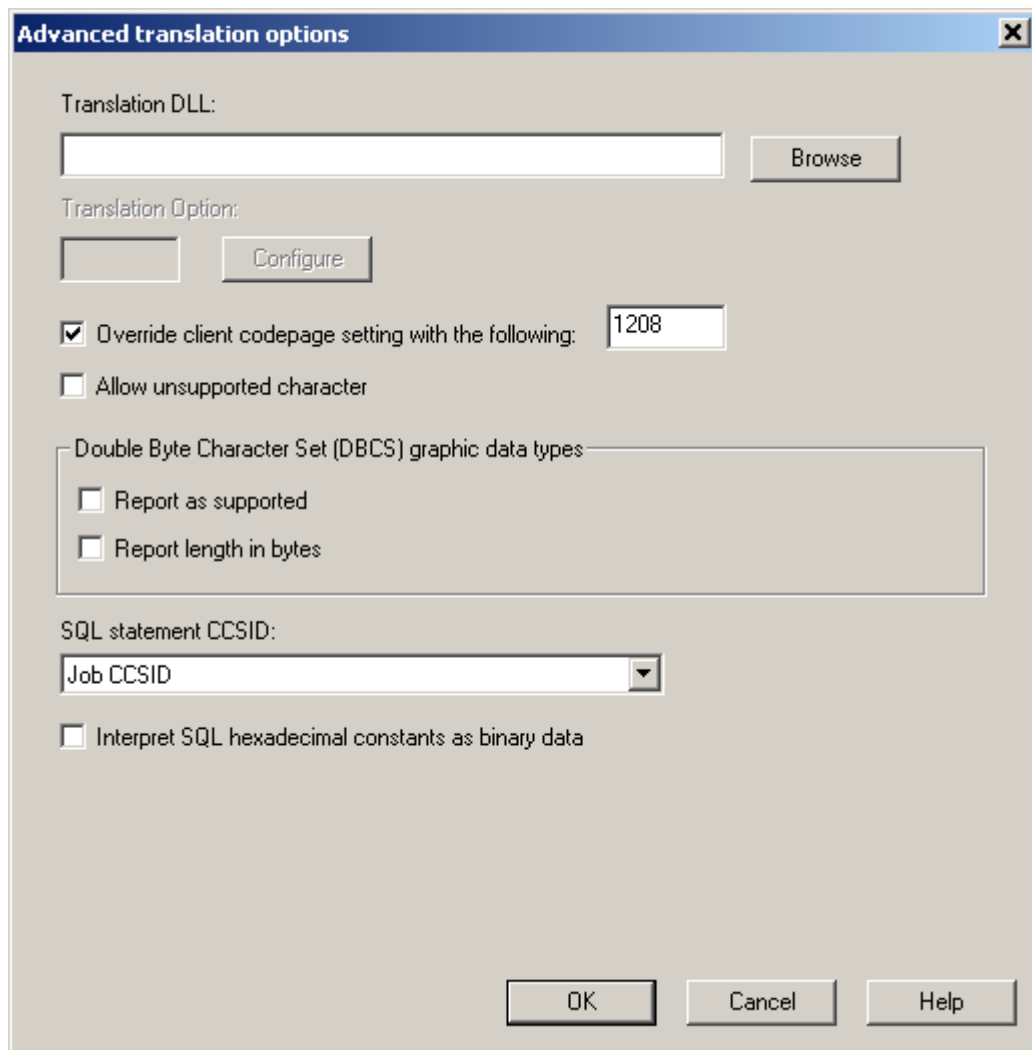
The environment variable DB2CODEPAGE must be configured to the value 1208 to enable all SQL strings to be treated as UTF-8.

7.5.3 IBM iSeries Access (IBM Universal Database on OS/400 accessed through ODBC on Windows)

We support the VARGRAPHIC data type. To create a UTF-16 column in a table on OS/400 use the following syntax:

```
CREATE TABLE test_edi_unicode ( unicode VARGRAPHIC(50) CCSID
1200 )
```

To treat all SQL strings as UTF-8 you need to set codepage 1208 for the ODBC data source:



7.5.4 MySQL through ODBC

Note that we do not support Unicode through the MySQL native database interface, only through ODBC.

We support data types such as VARCHAR CHARACTER SET ucs2.

You need to set the CHARSET = utf8 option in odbc.ini to enable all SQL strings to be treated as UTF-8. Testing was done with the libmyodbc5w.so driver.

7.5.5 Microsoft SQL Server

For this database server we only support UTF-16 binary data. This data can be inserted to and retrieved from a NVARCHAR column with a cast:

```
EXEC SQL INSERT INTO test_edi_unicode(unicode) VALUES(CAST(:$utf16 AS VARBINARY(2000)));
EXEC SQL SELECT CAST(CAST(unicode AS VARBINARY(2000)) AS VARCHAR(2000)) INTO :$result_utf16 FROM test_edi_unicode;
```

8 EDI parsing

8.1 Presentation

One purpose of the Message Builder language is to simplify the process of reading and parsing EDI messages. For this purpose, the language contains statements to read and verify the data elements that compose EDI messages. The data elements can be grouped into composite elements and composite and data elements can be grouped into segments. There are also statements that are used to group segments into segment groups (EDIFACT) and bounded and unbounded loops (X12).

EDI messages are always read from the input file. The OPEN statement is used to open the file containing the EDI message to read.

This chapter concentrates on parsing of EDIFACT messages. See the Message Builder Language Reference for information about parsing X12 and TRADACOMS messages.

To write EDI messages, use the PRINT statement. The ESCAPE built-in function and the TRUNCATE statement (see paragraph [12.4.130](#)) and function are also convenient for this purpose.

8.2 The SEGMENT statement

The SEGMENT statement starts a segment. It reads a segment tag from the input file and verifies that it is the expected tag. The segment can be MANDATORY (it must exist) or CONDITIONAL (it may exist). The SEGMENT statement also contains a statement block that is executed when the segment is found in the input file. The following example reads a segment tag named XYZ from the *edi.dat* file and writes a message to the output file indicating that the segment has been found:

```
OPEN FILE INPUT "edi.dat";
MANDATORY SEGMENT "XYZ" {
    PRINT "Segment XYZ found\n";
    .
    .
    .
}
```

The XYZ segment is mandatory, so the *edi.dat* file must start with an XYZ segment or an error is generated and the program terminates. To make the above example meaningful, the SEGMENT statement block also has to contain composite and/or data element statements (see below).

If the above example is changed to:

```
OPEN FILE INPUT "edi.dat";
CONDITIONAL SEGMENT "XYZ" {
    PRINT "Segment XYZ found\n";
    .
    .
    .
}
```

The message is output only if the *edi.dat* file starts with an XYZ segment. Otherwise the statements inside the statement block are not executed and no error is generated.

A repeat count may be given. It specifies how many times the segment may be repeated in the message. Specifying a repeat count of 999 for the XYZ segment looks like:

```
CONDITIONAL SEGMENT "XYZ" REPEAT 999 {
    PRINT "Segment XYZ found\n";
    .
    .
    .
}
```

If only the REPEAT keyword, without a count, is specified, the segment may be repeated an infinite number of times. If REPEAT is not specified, the segment may not be repeated.

8.3 The DATA ELEMENT statements

The DATA ELEMENT group of statements is used to read and verify data elements. This group contains statements to read BINARY, DATE, NUMBER, NUMERIC, STRING, and TIME data elements. This section will only cover the STRING version, see the DATA ELEMENT reference for information about the other types.

A data element statement must only be used inside a segment or composite statement block.

The following example reads a mandatory string data element with a length in the range 4 to 8, the string is put in the `$data` variable and a message is written to the output file:

```
MANDATORY VARIABLE STRING 4, 8 SET $data {
    PRINT "String data element found\n";
}
```

The number 4 specifies the minimum length of the data element. The minimum length may be left out if it is 1. If the contents of the data element is of no interest, the `SET $data` part may be left out. A semicolon may replace the statement block if no code is executed when the data element is found. The following statement reads a conditional string with a length in the range 1 to 10 (the contents of the element is discarded):

```
CONDITIONAL VARIABLE STRING 10;
```

A validation string may also be specified. The following statement tries to read a conditional string data element from the input file and if it exists checks that it equals EDI:

```
CONDITIONAL VARIABLE STRING 3 VALIDATE "EDI";
```

The VARIABLE keyword may be replaced by FIXED, if the length of the data element must be of a fixed length. The previous statement may be rewritten:

```
CONDITIONAL FIXED STRING 3 VALIDATE "EDI";
```

No minimum length can be specified when the FIXED keyword is used. The VARIABLE keyword is optional and may be left out if the data element is not of fixed length.

8.4 The COMPOSITE statement

The COMPOSITE statement is used to group data elements into composite data elements. COMPOSITE statements are only allowed inside SEGMENT statement blocks.

The following example reads and verifies a mandatory composite data element containing two string data elements:

```
MANDATORY COMPOSITE {  
    MANDATORY STRING 10 SET $s1;  
    MANDATORY STRING 10 SET $s2;  
}
```

The MANDATORY keyword can be replaced by CONDITIONAL if the composite data element is conditional.

8.5 EDI separator characters

The Message Builder interpreter uses reserved variables to determine the character that separates data elements in a segment, which character to use as segment separator, etc. The following reserved variables are used to control which separator characters to use when parsing EDIFACT messages:

- \$EdiSegSep
The segment separator, that is, the last character in a segment. The default value is `\x1c` (IS4).
- \$EdiElemSep
The data element separator, that is, it separates two data or composite elements in a segment. The default value is `\x1d` (IS3).
- \$EdiCompSep
The composite element separator, that is, it separates data elements inside a composite data element. The default value is `\x1f` (IS1).
- \$EdiRelChar
The release character, that is, it is used to indicate that the following character is not a separator character. By default, no release character is defined.

All these variables are of CHAR type with length 1. To change the separator characters, simply assign new values to these variables.

When writing EDI messages, these reserved variables should be used when separators are written. This makes it easier to switch to another separator set.

8.6 An EDIFACT example

The following example reads an EDIFACT 88.1 Alternative Currency Amount (ACA) segment:

```
DECLARE VARIABLE $currency STRING;
DECLARE VARIABLE $currencyQual STRING;
DECLARE VARIABLE $amount INTEGER;

$EdiSegSep = "'";
$EdiElemSep = "+";
$EdiCompSep = ":";
CONDITIONAL SEGMENT "ACA" REPEAT 5 {
    CONDITIONAL COMPOSITE {
        MANDATORY FIXED STRING 3 SET $currency;
        CONDITIONAL VARIABLE STRING 3 SET $currencyQual;
    }
    MANDATORY VARIABLE NUMBER 15 SET $amount;
    PRINT "Currency = " & $currency & "\n";
    PRINT "Currency qualifier = " & $currencyQual & "\n";
    PRINT "Amount = " & $amount & "\n";
}
```

If the contents of the input file are:

```
ACA+ZZZ:SKR+15790'ACA+ZZZ:ABC+7799'
```

the following is written to the output file:

```
Currency = ZZZ
Currency qualifier = SKR
Amount = 15790
Currency = ZZZ
Currency qualifier = ABC
Amount = 7799
```

8.7 The SEGMENT GROUP statement

The SEGMENT GROUP statement groups a number of segments into a segment group. Segment groups may be nested to any level (that is, segment groups inside other segment groups are allowed). The following example demonstrates the usage of segment groups:

```
CONDITIONAL SEGMENT GROUP "TDT" REPEAT 10 {
    MANDATORY SEGMENT "TDT" {
        .
        .
        .
    }
    CONDITIONAL SEGMENT GROUP "LOC" REPEAT 2 {
        MANDATORY SEGMENT "LOC" {
            .
            .
            .
        }
        CONDITIONAL SEGMENT "DTM" REPEAT 2 {
            .
            .
            .
        }
    }
}
```

```
}  
}
```

The syntax is similar to the SEGMENT statement. A MANDATORY or CONDITIONAL qualifier may be specified and a repeat count can be given. The first segment in a segment group should be mandatory and the segment group "tag" must equal the first segment tag.

9 User functions and statements

9.1 Presentation

Besides the built-in functions and statements, the user can also implement new functions and statements written using the Message Builder language. These are called user functions and statements. Calling a user function or statement is no different than calling a built-in one.

By combining user functions and statements with the library feature of the Message Builder compiler (see chapter [11 Compiling and executing Message Builder programs](#)), the Message Builder language is made truly extensible.

The difference between a user function and a user statement is that a function returns a value while a statement does not. Furthermore, the function arguments cannot be modified, while a statement may modify its parameters and in that way "return" one or more values.

9.2 User functions

The following code fragment declares a user function that removes the character specified by the second argument from the string specified by the first, and returns the result:

```
1  DECLARE FUNCTION REMOVE_CHAR($string, $remove)
2  {
3      DECLARE VARIABLE $s STRING;
4      DECLARE VARIABLE $i INTEGER;
5
6      $s = " " & $string & " ";
7      $i = 2;
8      WHILE $i < STRLEN($s) {
9          IF $s[$i, $i] = $remove {
10             $s = $s[1, $i - 1] & $s[$i + 1, STRLEN($s)];
11          }
12          ELSE {
13             $i = $i + 1;
14          }
15      }
16      RETURN $s[2, STRLEN($s) - 1];
17 }
```

Line 1 contains the function header. The name of the function is REMOVE_CHAR and it has two formal arguments: the first argument has been named `$string`, which is the string from which characters are to be removed; the second argument has been named

`$remove`, which specifies the characters to be removed. The `$string` and `$remove` arguments are automatically declared as constant local variables to the function.

The types of the formal arguments have not been specified. By default, the types are determined by the type of the expressions used as arguments when calling the function.

Lines 3 and 4 declare two variables local to the function. A local variable can only be used inside the function and it does not retain its value between invocations of it.

Line 6 initializes the local variable `$s` to the input string with one leading and one trailing space added. The reason that these spaces are added is to avoid having to check for special cases in the loop (that is, checking for the first or last character). Note that the `$string` argument cannot be used directly because it cannot be modified.

Lines 8 to 15 is the loop that removes all characters that equal the contents of `$remove`. `STRLEN` is a built-in function that return the length of a string and the `$s[$x, $y]` construct is an operator that extracts a sub-string from the `$s` string. See chapter [12 Message Builder language reference](#) for further information.

Line 16 contains a `RETURN` statement that returns the control to the caller. The value returned by the function is the expression specified as parameter to the `RETURN` statement. In this case, it is the resulting string with the first and last characters stripped off. The last statement in a user function must be a `RETURN`, `EXIT` or `THROW` statement. `RETURN` statements may be used in all places where other statements are allowed. The type of the `RETURN` parameter expression determines the type of the value returned by the function; in this case it is a `STRING`.

Calling the above function may look like:

```
PRINT REMOVE_CHAR("1.2.3.4.5", ".") & "\n";
```

The following will be written to the output file:

```
12345
```

User functions do not have to be declared before they are called. Hence, the above `PRINT` statement may appear either before or after the declaration of the `REMOVE_CHAR` function.

When compiling a Message Builder program containing calls to user functions that are not declared, the compiler can be instructed to automatically search for the missing functions in source code libraries. For a description of this feature, see chapter [11 Compiling and executing Message Builder programs](#).

9.2.1 Default argument values

Default values for the function arguments may be specified. Modifying the function header for the `REMOVE_CHAR` function so that the default value for the `$remove` argument is a space looks like:

```
DECLARE FUNCTION REMOVE_CHAR($string, $remove = " ")
```

Now, if the second argument is left out in a call to the function, the function will behave as if a space was specified as the second argument. The following two `PRINT` statements produce the same result:

```
PRINT REMOVE_CHAR("1 2 3 4 5") & "\n";
PRINT REMOVE_CHAR("1 2 3 4 5", " ") & "\n";
```

When calling a user function, an argument can only be left out if it has a default value. When declaring a function, if an argument has a default value, all the following arguments must also have default values.

9.3 User statements

The SPLIT_STRING user statement, as input, takes a string. It splits the string at the specified position and returns the resulting two strings in two output variables. The SPLIT_STRING user statement may be implemented as follows:

```
1  DECLARE STATEMENT SPLIT_STRING IN $string
2                                     POS IN $pos
3                                     FIRST OUT $first
4                                     LAST OUT $last
5  {
6      DECLARE VARIABLE $length INTEGER;
7
8      $length = STRLEN($string);
9      IF $pos = 1 {
10         $first = "";
11         $last = $string;
12     }
13     ELSE IF $pos = $length {
14         $first = $string;
15         $last = "";
16     }
17     ELSE {
18         $first = $string[1, $pos - 1];
19         $last = $string[$pos, $length];
20     }
21     RETURN;
22 }
```

Lines 1 to 4 contain the statement header. The name of the statement is SPLIT_STRING and it has four formal parameters: `$string`, which is the string to split; `$pos`, which is the position where the `$string` string is split; and `$first` and `$last`, which are the output variables where the result of the split is "returned". The IN and OUT keywords specify that the following parameter is an input or an output parameter, respectively. Input parameters are read-only in the user statement. Output parameters are used to "return" values from a user statement. The POS, FIRST and LAST words are noise words that are used to separate the parameters from each other. The same noise words must be used when calling the user statement.

The writer of the user statement is free to choose appropriate noise words. Strings containing alphanumeric and underscore characters or a single comma are valid noise words. It is not allowed to have a noise word before the first parameter.

Line 6 declares the local integer variable `$length`. Local variables can only be used inside the user statement where they are declared and they do not retain their values between invocations of it.

Lines 8 to 20 contain the Message Builder code that is executed when the user statement is called.

Line 21 contains a RETURN statement that returns the control to the caller. Note that the RETURN statement does not have any parameters since a user statement does not return anything. RETURN statements may be used in all places in the user statement where other statements are allowed.

Calling the SPLIT_STRING user statement may look like:

```
DECLARE VARIABLE $s1 STRING;
DECLARE VARIABLE $s2 STRING;
SPLIT_STRING "AxwayMessage Builder" POS 6 FIRST $s1 LAST $s2;
PRINT "" & $s1 & " " & $s2 & "\n";
```

The following will be written to the output file:

```
'Axway' 'Message Builder'
```

User statements do not have to be declared before they are called. Hence, the above call may appear either before or after the declaration of the SPLIT_STRING statement.

A user statement parameter may also be declared as an IN OUT parameter, specifying that it can be used both as an input and as an output parameter. See the Language Reference in chapter [12 Message Builder language reference](#) for further information.

When compiling a Message Builder program containing calls to user statements that are not declared, the compiler can be instructed to automatically search for the missing user statements in source code libraries. For a description of this feature, see chapter [11 Compiling and executing Message Builder programs](#).

9.3.1 Default parameter values

Default values for statement input parameters may be specified in the same way as for user function arguments. For output parameters it is possible to specify that the output value will be dropped, if the corresponding parameter is not specified when the statement is called. The DROP keyword is used for this purpose. Modifying the header for the SPLIT_STRING user statement so that both the output variables may be left out looks like:

```
DECLARE STATEMENT SPLIT_STRING IN $string
                                POS IN $pos
                                FIRST OUT $first DROP
                                LAST OUT $last DROP
```

Now, if the third and/or fourth parameters are left out in the call to the statement, the corresponding output values will be discarded. The following two calls are equivalent if the value of the first output parameter is not needed:

```
SPLIT_STRING "AxwayMB" POS 6 LAST $s2;
SPLIT_STRING "AxwayMB" POS 6 FIRST $s1 LAST $s2;
```

When declaring a user statement, any parameter may be given a default value (a dropped output value is considered to have a default value) as long as it is possible to uniquely determine which parameters are specified when the statement is called.

When calling a user statement, the first parameter, if any parameters are specified, must always be specified and it must be the first parameter. The remaining parameters may be given in any order as long as the Message Builder compiler can uniquely identify the parameters using the noise words.

10 Error handling and logging

10.1 Presentation

When an error occurs while a Message Builder program is executing and the error is not fatal, an error exception is generated. The default behavior when an error exception occurs is to assign an error code to the `$Error` reserved variable and an informational error string to the `$LogString` reserved variable, write an error message to a log file and then terminate. This behavior can be changed. This chapter describes what happens when errors occur and how to catch errors so that the execution can continue.

10.2 The `$Error` variable

When an error message of severity level `ERROR` or `WARNING` and, sometimes, when an `INFO` message is output, an error/warning code is assigned to the reserved integer variable named `$Error`. The contents of this variable can be used to detect a warning or to determine the reason for an error. Possible error codes for all built-in statements and functions are listed in the Message Builder Language Reference, together with an indication of the error severity levels.

The standard include file `error.s4h` contains declarations of integer variables containing all the different error codes. The integer variables in this file should be used instead of integer constants in order to make the program easier to read and to avoid future problems when moving Message Builder programs to newer versions of Integration Broker.

If the contents of the `$Error` variable are checked after the execution of a statement, it should be set to 0 before the statement is executed. The reason is that if the contents have not changed, no error has occurred. All error codes are positive integers; hence, a value of 0 can be used to indicate that no error has occurred. For an example, see the description of `WARNING` log messages below.

10.3 The `$LogString` variable

The `$LogString` reserved variable contains the last log message. Log messages suppressed by the `NOLOG` statement or the `I` interpreter option are also put in the `$LogString` variable.

10.4 Log messages

Log messages are divided into five types: INFO, DEBUG, WARNING, ERROR and FATAL. The INFO type is used to output informational messages and the DEBUG type is used to output debug information. The remaining three types indicate errors of various severity levels. The ERROR type is used when an error exception is generated. Log messages of all types are, by default, written to the standard error output. This behavior can be changed when the Message Builder interpreter is started. Integration Broker normally sends all log messages to the Trace Log. See the *System and Administration Manager* manual for further information.

The error written to the log when a division by zero error is encountered is used to describe the format of a log message:

```
ERROR :20000119:13.02.17:r4edi(demo):divide by zero
```

The log message is divided into a number of fields separated by colon characters. The first field is the log message type, the second is the date, and the third is the time of day that the log message was written. The fourth is the name of the Message Builder interpreter followed by the name of the Message Builder program inside parentheses (directory components and ".s4" or ".x4" extensions are removed). The fifth and last field is the message.

If the error occurs in a program loaded using the LOADPGM statement (see paragraph [12.4.84](#)), the name of the loaded program follows the Message Builder program inside parentheses. If, for example, the *demo.x4* program has loaded a program named *load.x4* and a divide by zero error occurs in *load.x4*, the following log message is output:

```
ERROR :20000119:13.02.17:r4edi(demo(load)):divide by zero
```

If the Message Builder program has been compiled with the trace option (**T**), two extra fields are inserted between the fourth and fifth fields. These extra fields are the name of the source code file and the source code line number corresponding to the error. If the source file name is *demo.s4* and the division that causes the error is on line 57 in that file, the log message will be:

```
ERROR :20000119:13.02.17:r4edi(demo):demo.s4:57:divide by zero
```

When the Message Builder interpreter is started, it is possible to specify different destinations for the different log message types. Individual log message types can also be suppressed using the **I** interpreter option.

The date field consists of 8 digits (4 for the year, 2 for the month, and 2 for the month day). For backward compatibility reasons, the **t** option to the interpreter can be used to obtain date fields that consist of 6 digits (2 digits for the year).

Log messages can also be suppressed using the NOLOG statement (see paragraph [10.6](#)).

10.4.1 Errors

Normally, when an error of severity level ERROR occurs, continued execution is not considered meaningful. After writing the appropriate error message to the log, the Message Builder interpreter will halt the execution.

ERRORs are usually generated by program errors that the Message Builder compiler cannot predict, such as failure to open a file or division by zero. An EDI message syntax error (the specification of an EDI message in the Message Builder program does not match the data in the input file) also generates an error of type ERROR. If it is not

acceptable that a run time error halts the execution of the program, the TRY statement (see paragraph 6.3) can be used to prevent this.

10.4.2 Fatal errors

When an error of severity level FATAL occurs, continued execution is not possible. After an error message has been written to the log, the Message Builder interpreter halts the execution.

Example of a FATAL error is when there is no more internal memory available.

10.4.3 Warnings

After an error of severity level WARNING, the Message Builder interpreter will update the `$Error` and `$LogString` reserved variables and write a warning message to the log. After that, the interpreter will continue to execute the program. By checking the `$Error` variable, it is possible to trap warnings that occur during execution. The following example demonstrates how a warning can be detected:

```
INCLUDE "error.s4h";
DECLARE VARIABLE $test STRING;
$Error = 0;
IMPORT "TEST_VALUE" INTO $test;
IF $Error = $Error_ImportKeyMissing {
    PRINT "TEST_VALUE does not exist\n";
}
```

Only the following built-in statements and operators generate warnings: CALL, CLOSE, IMPORT, PIPE CLOSE and the sub-string operator. The "standard" for built-in functions, statements and operators is to only generate errors of severity level ERROR or FATAL. The WARNINGS are kept for backward compatibility reasons.

10.4.4 Informational "errors"

After an "error" of severity level INFO, the Message Builder interpreter will update the `$Error` and `$LogString` reserved variables and write an informational message to the log. After that, the interpreter will continue to execute the program. In the same way as for warnings, by checking the `$Error` variable, it is possible to trap informational "errors" that occur during execution.

Only the PIPE READ, READ and SCAN statements generate informational "errors". The "standard" for built-in functions, statements and operators is to only generate errors of severity level ERROR or FATAL. The INFOs are kept for backward compatibility reasons.

10.5 The LOG statement

The LOG statement can be used to write messages to the log. The following example writes the warning message "file not found" to the log:

```
LOG "file not found" TYPE "WARNING";
```

The other fields of a log message (date, time, etc.) are automatically added. No new-line character should be present in the message string. The TYPE parameter specifies the type of the message. If the type is not specified, it defaults to ERROR.

This statement is typically used to write error messages to the log when trapping errors using the TRY statement (see below).

10.6 The NOLOG statement

The NOLOG statement can be used to suppress messages that would otherwise be written to the log. It specifies a block of statements where no logging will be done.

The following example reads lines from the file *in.txt* and writes them to the file *out.txt* while adding line numbers. The end-of-file message from the READ statement, which is normally obtained when end-of-file is encountered, is suppressed.

```
DECLARE VARIABLE $line STRING;
DECLARE VARIABLE $lineNumber INTEGER;

OPEN FILE INPUT "in.txt";
OPEN FILE OUTPUT "out.txt";
$error = 0;
$lineNumber = 0;
NOLOG {
    READ $line UNTIL "\n";
    WHILE $error <> 0 {
        PRINT $lineNumber & " " & $line & "\n";
        READ $line UNTIL "\n";
    }
}
```

NOLOG statements may be nested. Logging is enabled when the outermost NOLOG statement block is exited.

If user functions and statements are called from within a NOLOG statement block, logging is also disabled when executing the user functions and statements.

10.7 Error handling

The TRY statement can be used to take care of errors in a statement block. The following example tries to open a file. If the file does not exist or another error occurs, an error is written to the log (by the OPEN statement) and the execution continues:

```
TRY {
    OPEN FILE INPUT "file.txt";
}
CATCH $exception
WHEN "EXECUTE" {
    /* do nothing when the open fails */
}
```

If an error occurs while executing the statements in the TRY statement block (in this case the OPEN statement), the execution continues at the point specified by the CATCH keyword. A string indicating the type of error is put in the variable following the CATCH keyword and then the CATCH part behaves as the CASE statement (see paragraph 6.3). Note that the `$exception` variable is automatically declared as a string variable in all the WHEN statement blocks.

When an error occurs in a TRY statement block, the string put in the variable following the CATCH keyword is the error type. Message Builder can directly generate four different types of errors; EXECUTE, EDI, AGGREGATE and PGMLOGIC. Other error types can also be generated if the THROW statement (see paragraph [12.4.125](#)) is used. EXECUTE errors are the most common. EDI errors are generated when EDI messages being parsed do not conform to the format described by the Message Builder code. AGGREGATE errors are generated when arrays and data records are used in illegal ways. Some Message Builder statements must only or must not be executed at certain points, for example a SEGMENT statement must not be executed inside another SEGMENT statement. PGMLOGIC errors are generated in these cases. In the Message Builder Language Reference, all errors of type EDI, AGGREGATE or PGMLOGIC are indicated as such. If the error type is not mentioned, it is an EXECUTE error.

The *exception.s4h* header file contains declarations of constant variables representing the EXECUTE, EDI, AGGREGATE and PGMLOGIC error types. The names of these constant variables are `$Exception_Execute`, `$Exception_Edi`, `$Exception_Aggregate` and `$Exception_Pgmlogic`, respectively.

In the example above, if the error message generated by the OPEN statement should be suppressed and another message generated instead, the NOLOG and LOG statements can be used:

```
TRY {
    NOLOG {
        OPEN FILE INPUT "file.txt";
    }
}
CATCH $exception
WHEN "EXECUTE" {
    LOG "failed to open file.txt";
}
```

If user functions and statements are called from within a TRY statement block, errors are also caught when executing the user functions and statements.

10.8 Back trace information

When a program terminates abnormally, that is, when a fatal error occurs, an uncaught exception is generated, or the interpreter receives an interrupt signal, the interpreter outputs back trace information. The back trace information contains the position for where the error occurred and the positions for all the user function and statement calls used to come there. If trace information is available (the **T** option was specified when the program was compiled), all positions are given using source code line numbers and file names. Otherwise, if trace information is not available, pseudo code addresses and program names are used to represent the positions. If trace information is available, the back trace information may look something like:

```
INFO :20000102:12.13.55:r4edi(bt):terminating at line 5, file "bt.s4"
INFO :20000102:12.13.55:r4edi(bt):call from line 10, file "bt.s4"
```

If trace information is not available, the back trace information may instead look like:

```
INFO :20000102:12.16.58:r4edi(bt):terminating at address 12, program
"bt.x4"
INFO :20000102:12.16.58:r4edi(bt):call from address 16, program
"bt.x4"
```

By using the **A** compiler option it is possible to find out what source code line and file name a particular pseudo code address corresponds to.

11 Compiling and executing Message Builder programs

11.1 Presentation

Message Builder consists of a compiler and an interpreter. The input to the compiler is one or more ordinary text files containing a Message Builder program. All program files should have the suffix ".s4". Before the Message Builder interpreter can execute the program it must be compiled. By default, the output file from the compiler has the suffix ".x4".

Except for the AS/400, compiled Message Builder programs can be moved between different computers and executed without recompilation.

The AS/400 format is different from other platforms (UNIX and Windows), because the native character set of AS/400 is EBCDIC, not ASCII. An ".x4" file consists of a mixture of binary data and character strings, and as such is dependent on the native character set. As a result, Message Builder programs compiled on other platforms will not run on AS/400, and vice versa. All Message Builder programs to be run on AS/400 must be compiled on AS/400.

If the file to execute has the suffix ".s4", the interpreter replaces the ".s4" suffix with ".x4".

11.2 Start syntax for the Message Builder compiler

For UNIX and AS/400 systems, the formal syntax for the Message Builder compiler is:

```
c4edi [-?6NOTVZflnpsvz] [-O file] [-1 file] [-2 file] [-3 file]
      [-A addr] [-t option]
      [-D file] [-F func] [-I dir] [-K file] [-L dir] [-P pragma]
      [-S stmnt] [-a file] [-b file] [-c string] [-d when] [-o file]
      [-q old=new] [-r old=new] [-w spec] [-x ver] [-y file] [files]
```

For Windows systems, the formal syntax is (the UNIX syntax is also accepted by the Windows compiler):

```
c4edi [/?] [/N] [/O] [/T] [/V] [/f] [/l] [/n] [/p] [/s] [/v] [/Z] [/z]
      [/O file] [/l file] [/2 file] [/3 file] [/A addr] [/D file]
      [/F func] [/I dir] [/K file] [/L dir] [/P pragma] [/S stmnt]
      [/a file] [/b file] [/c string] [/d when] [/o file] [/q old=new]
      [/r old=new] [/w spec] [/x ver] [/y file] [/6] [/t option]
      [files]
```

11.2.1 Command Arguments

The following options can be specified:

?

Outputs a short command syntax description and then exits.

0

Valid but not used.

1

Valid but not used.

2

All error, warning and informational messages output by the compiler are written to *file*. If this option is not specified, these messages are written to the standard error output.

3

Valid but not used.

6

Valid only for 64-bit c4edi. Create an .x4 file with a modified format, which will not be loaded by 32-bit r4edi, but which will preserve 64-bit integer constants.

A

When a program terminates abnormally, that is, when a fatal error occurs or an uncaught exception is generated, the interpreter outputs back trace information. The back trace information contains the position for where the error occurred and the positions for all the user function and statement calls used to come there. If trace information is available (the **T** option was specified when the program was compiled), all positions are given using source code line numbers and file names. Otherwise, if trace information is not available, pseudo code addresses and program names are used to represent the positions. The **A** compiler option can be used to map a pseudo code address to a source code line and file name. The compile command line (except for the **A** option) and the source code files should be identical to the compile command line and the source code files used when the terminating program was created.

D

The output generated when the **d** option is specified is written to *file*. The default is to write the output to the standard error output.

F

The compiler will output information about the *func* function so it will be possible to call it from another program. See the LOADPGM (paragraph [12.4.84](#)) and USE (paragraph [12.4.35](#)) statements for more details. This option has the same effect as the PRAGMA EXPORT FUNCTION directive (paragraph [12.4.92](#)).

I

Directory to search for include files. The *dir* option argument is a directory name that is added to the beginning of the list of directories to search for include files. See the description of the EDI_INCLUDE environment variable (see paragraph [11.3.2](#)) for further details. Multiple **I** options may be specified.

K

Read options from *file*. All options that can be specified on the command line can also be specified in an options file. Each option is specified using one line in the file and each line consists of one or two fields – the option character and an optional option argument. Space and/or tab characters separate the fields. The option argument should only be present if the option takes an argument.

The single quote ('), double quote (") and backslash (\) characters have special meaning in the option argument. They are treated in the same way as in the UNIX Bourne shell.

Environment variable expansion is also performed. On UNIX and AS/400 machines the following constructs are recognized:

```
$variable
${variable}
${variable:-word}
${variable:+word}
```

On Windows machines, the `%variable%` construct is recognized.

A number sign (#), not inside quote characters and not escaped by a backslash, starts a comment. All characters to the end of the line are ignored.

A line can be continued on the next line by ending the line with a backslash character.

Multiple **K** options may be specified.

L

Directory to search for library files. The *dir* option argument is a directory name that is added to the beginning of the list of directories to search for undeclared modules, functions and statements. See the description of the EDI_LIB environment variable (page 78) for further details. Multiple **L** options may be specified.

N

Check option. Perform the compilation without producing any executable (".x4") file.

O

Do not optimize. By default, the compiler tries to replace constant expressions with constant values, remove non-reachable code, make some peephole optimizations, etc. This option disables these actions.

P

Ignore the specified PRAGMA directive. The option argument can be `file`, `line`, `trace` or `warnings`. For example, if `-Pfile` is specified, the PRAGMA FILE directive will be ignored. Multiple **P** options may be specified.

S

The compiler will output information about the *stmt* statement so it will be possible to call it from another program. See the LOADPGM (paragraph [12.4.84](#)) and USE (paragraph [12.4.135](#)) statements for more details. This option has the same effect as the PRAGMA EXPORT STATEMENT directive (paragraph [12.4.92](#)).

T

Generate code for source code trace. The compiler generates code that makes it possible for the Message Builder interpreter to keep track of the current source code file name and line number when executing the program. When a message is written to the log and the program has been compiled with this option, the file name and line number is output together with the rest of the message. For trace debug (see the DEBUG statement, paragraph [12.4.34](#)) to work when running the interpreter, this option must have been used when the program was compiled. Note that a program compiled with this option executes slower than if it was compiled without it.

V

Shows the version number and then exits.

Z

Output informational messages about some of the performed optimizations.

a

This option is used together with the **w** option to instruct the compiler which source code files to output warnings for. If no **a** options are specified, warnings are output for all source files (except those specified using the **b** option). The *file* option argument is the source code file. Multiple **a** options may be specified.

b

This option is used together with the **w** option to instruct the compiler which source code files to not output warnings for. The *file* option argument is the source code file. Multiple **b** options may be specified.

c

The comment section of the output file is set to *string*. All special character sequences supported in Message Builder string constants can also be used in *string*. This option is intended for future use.

d

Output debug information. This option causes the parse tree to be written in a readable format to the standard error output or the file specified by the **D** option. The *when* option argument is either *before* or *after*, specifying that the parse tree shall be output before or after the optimization, respectively. This option is intended for debug purposes and is of no interest to the regular user.

f

If this option is specified, for each include and library file (see the description of the EDI_LIB environment variable, paragraph [11.2.2](#)), an informational message is output. Include files that are ignored because of the ONCE modifier to the INCLUDE statement are also indicated to be so.

l

Output label names, that is, user function and statement names, to the output file. This option is intended for debug purposes and is of no interest to the regular user.

n

New lines are, by default, not allowed in Message Builder string constants. For backward compatibility reasons, this option can be used to allow them.

o

The name of the output file. The default name of the output file is the name of the first source code file with the ".s4" extension replaced with ".x4". If there is no ".s4" extension, the default name is formed by appending the ".x4" extension to the name. If the Message Builder program to compile is read from the standard input, the default name is *output.x4*.

p

While the input source code files are processed, the compiler outputs the current position in the input files. This option is intended for debugging purposes and is of no interest to the regular user.

q

This option is used to change names of variables. The word to the left of the equal sign is the variable name to be replaced and the word to the right is the new variable name. This option only has effect if the **x** option is specified. For example, in version 5.x.x of Message Builder, `$LogString` was a valid global variable name. In version 6.x.x, `$LogString` is a reserved variable and it is therefore not a valid global variable name. Specifying:

```
$LogString=$NewLogString
```

changes all occurrences of `$LogString` to `$NewLogString`.

r

This option is used to change names of identifiers. The word to the left of the equal sign is the identifier name to be replaced and the word to the right is the new name. This option only has effect if the **x** option is specified. It can be used to change the names of functions, statements, modules, and data records. For example, in version 4.x.x of Message Builder, the `USE` identifier was valid to use as a function, statement, and module name. In version 5.x.x, `USE` is a built-in statement and it is therefore not a valid function, statement, module, or data record name. Specifying:

```
USE=MYUSE
```

changes all occurrences of `USE` to `MYUSE`.

s

Output statistics. When the compilation has completed without errors, the number of read characters, lines and files are written to the standard error output.

t

Language options. Currently the only supported option is **convzerodate1970**, which changes how `DECLARE $d DATE; $d = 0;` is compiled when optimizations are enabled. The backwards compatible default is to initialize the date with January 1st 1900; when this option is enabled the date is initialized with January 1st 1970 (adjusted according to the time zone).

v

Outputs variable names to the output file. This option is intended for debug purposes and is of no interest to the regular user.

w

Output warnings about various things. The *spec* option argument specifies what to output warnings for:

Output warning	Meaning
unused-non-constant-globals	Report unused non-constant global variables.
unused-constant-globals	Report unused constant global variables.
unused-non-constant-locals	Report unused non-constant local variables.
unused-constant-locals	Report unused constant local variables.
unused-formals	Report unused formal arguments or parameters.
assign-only-globals	Report assigned only global variables.
assign-only-locals	Report assigned only local variables.
refer-only-globals	Report non-constant global variables that are never assigned.
refer-only-locals	Report non-constant local variables that are never assigned.
unused-user-functions	Report unused user declared functions.
unused-extension-functions	Report unused extension functions.

Output warning	Meaning
unused-external-functions	Report unused external declared functions.
unused-user-statements	Report unused user declared statements.
unused-extension-statements	Report unused extension statements.
unused-external-statements	Report unused external declared statements.
empty-statements	Report empty statements.
variables-different-case	Report when the same variable name is used with different case.
reserved-variables-different-case	Report when the case in reserved variables does not equal the "standard". The standard for the different reserved variables is: \$Error, \$LogString, \$DateFormat, \$SqlCode, \$EdiPos, \$EdiSeg, \$EdiSegCnt, \$EdiElemCnt, \$EdiCompCnt, \$EdiSegCodeSep, \$EdiRelChar, \$EdiPadChar, \$EdiFile, \$EdiSegSep, \$EdiElemSep, \$EdiCompSep, \$EdiCharSet, \$EdiInitMode, \$EdiResetFlag, \$EdiPreFlag, \$EdiByteFlag, \$EdiCharValid and \$Serial.
functions-different-case	Report when the same function name is used with different case.
statements-different-case	Report when the same statement name is used with different case.
modules-different-case	Report when the same module name is used with different case.
records-different-case	Report when the same record name is used with different case.
fields-different-case	Report when the same record field is used with different case.
pragma-warnings	Report when there is a different number of PRAGMA WARNINGS ON and PRAGMA WARNINGS OFF directives in a source code file.
pragma-trace	Report when there is a different number of PRAGMA TRACE ON and PRAGMA TRACE OFF directives in a source code file.

The following compound warning options arguments may also be used:

Compound warning	Meaning
unused-globals	unused-non-constant-globals and unused-constant-globals.
unused-locals	unused-non-constant-locals and unused-constant-locals.
unused-non-constant-variables	unused-non-constant-globals and unused-non-constant-locals.
unused-constant-variables	unused-constant-globals and unused-constant-locals.
unused-variables	unused-non-constant-globals, unused-non-constant-locals, unused-constant-globals, unused-constant-locals and unused-formals.
assign-only-variables	assign-only-globals and assign-only-locals.
refer-only-variables	refer-only-globals and refer-only-locals.
unused-functions	unused-user-functions, unused-extension-functions and unused-external-functions.
unused-statements	unused-user-statements, unused-extension-statements and unused-external-statements.
unused-procedures	unused-user-functions, unused-extension-functions, unused-external-functions, unused-user-statements, unused-extension-statements and unused-external-statements.
different-case	variables-different-case, reserved-variables-different-case,

	functions-different-case, statements-different-case, modules-different-case, and records-different-case.
all	All the warning options.

Multiple **w** options may be specified. All warning option arguments may be prefixed by **no-** in order to disable warnings of a particular type. For example to output all warnings except warnings for unused formal arguments and parameters, the following two options can be used:

```
-wall -wno-unused-formals
```

The **a** option can be used to specify which source code files to output warnings for and the **b** option to specify which source code files to not output warnings for. The PRAGMA WARNINGS directive (see paragraph [12.4.92](#)) can also be used to turn warnings on and off.

x

This option can be used to compile Message Builder programs that conform to an old syntax. The *ver* option argument specifies which version of Message Builder the code to be compiled conforms to. Valid values are 4, 5 and 6. Specifying 6 only changes the identifiers and variables specified using the **r** and **q** options, respectively.

y

This option is used together with the **x** option to convert Message Builder programs using an old syntax to the syntax used by the current version. The *file* option argument is the name of the file where the converted source code shall be written. The program is not compiled and INCLUDE directives are not processed if this option is specified. This option is ignored if the **x** option is not specified.

z

By default, the conversions performed when the **x** option is specified are done without user notification. If this option is specified, informational messages, describing all changes made, are written to the standard error output. This option is ignored if the **x** option is not specified.

The remaining arguments are the files to compile:

files Message Builder source code files that are read in sequence. If no files are specified, the source code to compile is read from the standard input. By specifying multiple source code files on the command line, the code from several files can be used when producing the executable file. This can be seen as a way to do "linking" on the source code level.

Instead of specifying a source code file, it is possible to enter a string surrounded by '{' and '}' characters. All text between these characters are assumed to be Message Builder code and compiled. The following example declares a constant variable, for example a configuration constant, before the *ex.s4* file is compiled:

```
c4edi -o ex.s4 '{ DECLARE $Conf = 1 CONSTANT INTEGER; }' ex.s4
```

Note the single quotes around the source code; they are necessary in order to prevent the command interpreter from interpreting the source code in any special way.

11.2.2 Environment variables

The following environment variables have special meaning to the Message Builder compiler:

C4EDI_OPTIONS

Before processing the command line options, the compiler processes the options specified by this environment variable. A Bourne Shell example:

```
C4EDI_OPTIONS="-T -x4"
```

The double quotes are needed by the Shell. Double and/or single quotes may be used to include space and/or tab characters in an option argument:

```
C4EDI_OPTIONS="-c'This is a comment'"
```

It is an error to specify a non-option argument in this environment variable.

EDI_INCLUDE

The environment variable EDI_INCLUDE is used by the compiler to find files to include. It specifies which directories to search for include files. The format of the value of EDI_INCLUDE is identical to the standard PATH variable. If the EDI_INCLUDE variable does not exist, the compiler searches for files to include in the current directory.

The **I** command option can also be used to add directories to the list of directories to be searched for include files. For UNIX and AS/400 systems, the Integration Broker normally sets this environment variable to:

```
.: $CORE_LOCAL/4edi/include:$CORE_ROOT/4edi/include
```

and for Windows systems it is set to:

```
.; %CORE_LOCAL%\4edi\include;%CORE_ROOT%\4edi\include
```

EDI_LIB

The environment variable EDI_LIB is used by the compiler to find the Message Builder code for user functions, user statements, and modules that are undeclared when the source code files have been processed. It specifies which directories to search for missing functions, statements, and/or modules. The format of the value of EDI_LIB is identical to the standard PATH variable. If the EDI_LIB variable does not exist, the compiler searches for missing functions, statements and modules in the current directory. The **L** command option can also be used to add directories to the library search path. For UNIX and AS/400 systems, the Integration Broker normally sets this environment variable to:

```
.: $CORE_LOCAL/4edi/lib:$CORE_ROOT/4edi/lib
```

and for Windows systems it is set to:

```
.; %CORE_LOCAL%\4edi\lib;%CORE_ROOT%\4edi\lib
```

11.2.3 Library files

When all the source code files specified on the command line have been processed, and there are functions, statements, and/or modules that are missing, the compiler tries to find them using table files. Table files are searched for in the directories specified by the library search path and have the extension ".t4". The compiler reads all files with this extension. It then uses the information in the table files to try to find the missing functions, statements, and modules. An alternative to using table files is to give the

library file the same name as the function, statement, or module (only lowercase letters should be used) that it contains and appending the extension ".s4f", ".s4s" or ".s4m", respectively.

Each line in a table file consists of three fields separated by space and/or tab characters. The first field should contain one of the keywords `function`, `statement`, or `module` (the case does not matter), the second field a function, statement, or module name (depending on if the first field was `function`, `statement`, or `module`, respectively) and the third field the name of the file containing the function, statement, or module (the file is assumed to be located in the same directory as the table file). The second field must be a valid function, statement, or module name and the third field must not contain, on UNIX or AS/400 systems, a slash (/) character, and on Windows systems it must not contain a backslash (\) character. Comments may be inserted in the table files. All characters from and including a # character to the end of the line are discarded.

Note: functions and statements inside a module do not have to be listed in the table file if the module itself is listed.

11.3 Start syntax for the Message Builder interpreter

For UNIX and AS/400 systems, the formal syntax for the Message Builder interpreter is:

```
r4edi [-?EFSTVfintv] [-O file] [-1 file] [-2 file] [-3 file] [-C size]
      [-D dir] [-I file] [-K file] [-L file] [-M file] [-N file]
      [-O dir] [-P dir] [-b spec] [-d file] [-e file] [-l type]
      [-p file] [-s spec] program [arguments]
```

For Windows systems, the formal syntax is (the UNIX syntax is also accepted by the Windows interpreter):

```
r4edi [/?] [/E] [/S] [/T] [/V] [/f] [/i] [/n] [/t] [/v] [/O file]
      [/1 file] [/2 file] [/3 file] [/C size] [/D dir] [/I file]
      [/K file] [/L file] [/M file] [/N file] [/O dir] [/P dir]
      [/b spec] [/d file] [/e file] [/l type] [/p file] [/s spec]
      program [arguments]
```

11.3.1 Command arguments

The following options can be specified:

?

Outputs a short command syntax description and then exits.

0

The name of the standard input file. On Windows, this option opens the file in shared mode. This makes it possible for several programs to have the file open. Special files, such as pipes and mail slots, are not supported.

1

The name of the standard output file. On Windows, this option opens the file in shared mode. This makes it possible for several programs to have the file open. Special files, such as pipes and mail slots, are not supported.

2

The name of the standard error output file. On Windows, this option opens the file in shared mode. This makes it possible for several programs to have the file open. Special files, such as pipes and mail slots, are not supported.

3

The name of the error log file. If this option is not specified, errors are written to the standard error output.

On Windows, this option opens the file in shared mode. This makes it possible for several programs to have the file open. Mail slots are supported.

On AS/400, if *file* starts with a percentage character (%), *file* is assumed to be the name of an AS/400 user queue.

C

The size of the SQL statement cache. In order to improve the performance when executing SQL statements, the interpreter keeps a cache of prepared SQL statements. The default size of the cache is one. For maximum performance, the cache size should be set greater than or equal to the number of SQL statements in the Message Builder program. If *size* is zero, an infinite cache will be used.

D

Directory to search for database interfaces. The *dir* option argument is a directory name that is added to the beginning of the list of directories that the interpreter shall search for database interfaces. See the description of the EDI_DBLOAD environment variable (paragraph [11.2.2](#)) for further details. Multiple **D** options may be specified.

E

Enable EDI debug logging. This is equivalent to have DEBUG EDI ON (see paragraph [12.4.34](#)) as the first statement in the program.

F

Increase the limit on number of open files on non-Windows platforms. The limit is raised to the following limit or the administrator-defined hard ulimit, whichever is smaller:

Operating system	Open files limit
HP-UX	60000
AIX	32767
Linux	100000

Solaris	100000
OS/400	50000

I

The name of the fatal error log file. If this option is not specified, fatal errors are written to the error log file.

K

The name of the warning log file. If this option is not specified, warnings are written to the error log file.

L

The name of the debug log file. If this option is not specified, debug messages are written to the error log file.

M

The name of the informational messages log file. If this option is not specified informational messages are written to the error log file.

N

The name of the trace log file. If this option is not specified, trace output is written to the standard error output.

O

Directory to search for object extension module files (see the *Extender* manual). The *dir* option argument is a directory name that is added to the beginning of the list of directories that the interpreter shall search for object extension module files. See the description of the EDI_LOAD environment variable (paragraph [11.2.2](#)) for further details. Multiple **O** options may be specified.

P

Directory to search for executable Message Builder programs. The *dir* option argument is a directory name that is added to the beginning of the list of directories that the interpreter shall search for executable program files. See the description of the EDI_PATH environment variable (paragraph [11.2.2](#)) for further details. Multiple **P** options may be specified.

S

Enable SQL debug logging. This is equivalent to having a DEBUG SQL ON (see paragraph [12.4.34](#)) as the first statement in the program.

T

Enable trace debug logging. This is equivalent to having a `DEBUG TRACE ON` (see paragraph [12.4.34](#)) as the first statement in the program. Note that the program must have been compiled with the **T** option for trace debug to work.

V

If no Message Builder program is specified on the command line, show the version number of the interpreter and then exits. If a Message Builder program is specified, output version information specified by `PRAGMA VERSION` directives and then exits.

b

Output buffering method, that is, the buffering used when writing to the output file. The *spec* option argument can be one of `full`, `line` or `none`, specifying full buffering, line buffering or no buffering, respectively. The default is full buffering. Note that this option only affects the behavior of the `PRINT` statement (see paragraph [12.4.93](#)).

d

Generate profiling information, that is, information about how much time are spent executing different parts of the program. A program that shall be profiled must have been compiled using the **T** option. The *file* option argument is a file where the profiling information is written. A hyphen (-) instead of a file name can be used to specify that the information shall be written to the standard output file. Note that profiling information is always appended to *file*.

The profiling information for one program starts with the line:

```
PROGRAM name
```

Where *name* is the name of the program. Each program consists of one or more source code files. The profiling information for each source code file starts with the line:

```
FILE name
```

Where *name* is the name of the source code file.

For each line in the source code that has been executed the following line is output:

```
line-no execution-count execution-time[ acc-execution-time]
```

Where *line-no* is the line number, *execution-count* is the number of times the line has been executed, *execution-time* is the time spent executing the line (in seconds), and, if the line is a user function or statement call, *acc-execution-time* is the accumulated execution time (in seconds) for the function or statement.

Note: The execution times are very approximate. For the accumulated execution times for functions and statements to be reliable, the **-P_{trace}** option should be used when the program is compiled.

If, for example, the following program (with name *prog.s4*) is profiled:

```
FOR $i = 1 TO 10000 {  
    PRINT_INTEGER $i;  
}  
  
DECLARE STATEMENT  
PRINT_INTEGER IN $integer PRIMITIVE  
{
```

```

    PRINT $integer;
    PRINT "\n";
    RETURN;
}

```

The information appended to the profiling file will look something like:

```

PROGRAM prog.s4
FILE prog.s4
1 10001 0.738990
2 10000 0.263004 0.726286
8 10000 0.240108
9 10000 0.255566
10 10000 0.250075

```

e

Set the initial input file to *file*. This is equivalent to having an OPEN FILE INPUT (see paragraph [12.4.87](#)) as the first statement in the program.

f

If this option is specified, fractions of a second (using two digits) are output in the time stamps of log messages.

i

Inhibits all informational messages. This option is obsolete, use the **I** option instead.

I

Suppress all log messages of the type specified by *type*. For example:

```
-lDEBUG
```

suppresses all debug log messages.

n

The interpreter normally optimizes the pseudo code. This option prevents this optimization. The only reason to use this option is when it is suspected that the optimization doesn't working correctly.

p

The process ID for the Message Builder interpreter is written to *file*.

s

Starts the interpreter in server mode (see paragraph [11.3.3](#)). The *spec* argument is a comma-separated list of up to five items.

The first item is the TCP service name (for UNIX systems, a name in the */etc/services* file, for Windows systems a name in the *%SystemRoot%\system32\drivers\etc\services* file, and for AS/400 systems a Service Table Entry accessible through the AS/400 command WRKSRVTBLE) or the TCP port number (a decimal number). If a port number

of 0 is specified, the default port number 30510 is used. The clients must use this service name or port number when connecting to the server.

The second item is a configuration file containing known Message Builder servers. The configuration file consists of lines of three fields separated by white space. The first field is the name of a Message Builder server, the second field is the host name or an IP address using dot notation and the third field is the service name or port number for the server. Comments may be inserted using the # character. The # character and all the remaining characters on the line are ignored. If this item is empty, no configuration file is read.

The third item is the server's own name. If this field is specified, the server will broadcast information about itself in order to make it possible for other servers on the same LAN (Local Area Network) to find it. The time between each broadcast will be 1 minute. It will also listen for broadcasts from other servers in order to find them. If it has not received a broadcast from a particular server for the last 3 minutes, the server will be forgotten. The UDP protocol is used for this purpose. The UDP port number used is the same as the TCP port number. If this item is specified, the fourth item must also be specified.

The fourth item is a host name or an IP address using dot notation. It specifies the broadcast address to use. This item must and must only be specified if the third item is present.

The fifth item is the name of a file where communication debug information will be written. If this item is not specified, no debug information will be produced.

As an example, the following argument to the **s** option:

```
s4ediport,conf,myname,1.2.3.255
```

specifies that the TCP (and UDP) port to use is given by the *s4ediport* service name, the *conf* file contains known Message Builder servers, broadcasts containing the server name *myname* should be transmitted and the broadcast address to use is 1.2.3.255.

An example of a known Message Builder server configuration file may be:

```
# Other known Message Builder servers.
server1  host1      port1
server2  host2      port2
# example of dot notation
server3  45.46.34.18 port3
# example of port number
server4  host4      33333
```

t

The date-field width in log messages is normally 8 digits (4 for the year, 2 for the month, and 2 for the month day). This option tells the interpreter to use 6 digits for the date field (2 digits for the year). A date-field width of 6 digits was used by older versions of Message Builder.

v

Outputs the version number of the compiler that compiled the program specified by *program* and then exists.

The remaining command arguments are:

- *program*
The file name of the Message Builder executable program file to execute.

The EDI_PATH environment variable (see paragraph [11.2.2](#)) is used to search for the program file.

- *arguments*
Arguments entered on the command line are accessible from the Message Builder program with the ARGUMENT function (see paragraph [12.4.2](#)). The number of entered arguments can be obtained using the ARGUMENTCOUNT function (see paragraph [12.4.3](#)).

11.3.2 Environment variables

The following environment variables have special meaning to the Message Builder interpreter:

CORE_PROCEngine_DUMP_PATH

When the 4edi interpreter inside a procengine or r4edi executable terminates with a fatal error, a dump of all Message Builder variables (with truncated strings) is created in the \$CORE_ROOT directory. This variable can be used to override that path, since these dumps can be huge and some installations do not have enough space in \$CORE_ROOT.

DB_FORMATS

The DB_FORMATS environment variable is only used on the AS/400. It is used when the *options* field of the DATABASE OPEN statement (see paragraph [12.4.32](#)) is not specified.

EDI_DB

If the TYPE parameter to the DATABASE OPEN statement is not specified, the interpreter uses the EDI_DB environment variable to decide which database interface to use. If EDI_DB is not set, the interpreter uses the built-in database interface, if any. Otherwise, it uses the database interface whose file name is the first when all database interface file names (found using the first item in the EDI_DBLOAD variable) are sorted in lexicographical order.

The database type can be one *Db2_81* (DB2 8.x and later), *Informix_73* (Informix 7.3.x and later), *Informix_92* (Informix 9.2 and later, on certain platforms), *MySQL_41* (MySQL 4.1 native driver, on certain platforms), *MySQL_50* (MySQL 5.0 and later native driver, on certain platforms), *Odbc_2* (ODBC 2.x), *Odbc_3* (ODBC 3.x), *Odbc3_UnixODBC_post_2_2_14* (on 64-bit Unix platforms the UnixODBC ABI has changed in version 2.2.14 and later – the DB2 and MySQL drivers require the new ABI), *Oracle_81* (on Windows only, name kept to avoid breaking existing applications; it is actually an Oracle 9.2 or later interface), *Oracle_92* (Oracle 9.2.x), *Oracle_101* (Oracle 10.1.x and later, including Oracle 11 on platforms without a \$CORE_ROOT/4edi/dbload/oracle111.db; tested up to Oracle 12.1), *Oracle_111* (Oracle 11.1.x and later), *OS400_SQL* (AS/400 Native), *Sybase_111* (Sybase 11.1) and *Sybase_1251* (Sybase 12.5.1 and later).

The case does not matter.

EDI_DBLOAD

The interpreter can dynamically load database interfaces. The interpreter uses the environment variable EDI_DBLOAD to find dynamically loadable database interfaces. It specifies database interfaces and directories to search for database interfaces.

The format of the value of EDI_DBLOAD is identical to the standard PATH variable. If the EDI_DBLOAD variable does not exist, the interpreter searches for database interfaces in the current directory. All files with the *.db* extension are assumed to be database interfaces. The **D** command option can also be used to add database interfaces and directories to the list to be searched for database interfaces.

For UNIX systems with an Informix 7.1.x database, the installation program normally sets this environment variable to:

```
$CORE_ROOT/4edi/dbload/informix71.db:$CORE_ROOT/4edi/dbload
```

For Windows systems with an Informix 7.2.x database, the environment variable is normally set to:

```
%CORE_ROOT%\4edi\dbload\informix72.db:%CORE_ROOT%\4edi\dbload
```

For AS/400 systems using the native database, the environment variable is normally set to:

```
$CORE_ROOT/4edi/dbload/os400.db:$CORE_ROOT/4edi/dbload
```

EDI_DBLOAD_DISABLED

Opposite of EDI_DBLOAD. The specified database interfaces will not be loaded.

EDI_DEBUG_OPTIONS

The environment variable EDI_DEBUG_OPTIONS can be used to set options that cause the interpreter to output debug information in certain circumstances. The value of the EDI_DEBUG_OPTIONS variable should contain the options separated by white space. The following options are currently supported (case is not significant):

- **LOAD** : Output information about all loaded extensions.
- **LOADPGM** : Output information about all loaded Message Builder programs.
- **UNLOADPGM** : Output information about all unloaded Message Builder programs.

EDI_ORACLE_xx, EDI_CLOB_THRESHOLD, EDI_SYBASExx_CS_IFILE

See 12.4.32 DATABASE OPEN.

EDI_OVERRIDE_SQL_CACHE_SIZE

Overrides the -C r4edi option, see page 73

EDI_LOAD

The environment variable EDI_LOAD is used by the interpreter to find object extension module files when the LOAD statement is executed (see the *Extender* manual). It specifies directories to search for the file to load. The format of the value of EDI_LOAD is identical to the standard PATH variable. If the EDI_LOAD variable does not exist, the interpreter searches for files to load in the current directory. The **O** command option can also be used to add directories to the list of directories to be searched for files to load. For UNIX and AS/400 systems, the Integration Broker normally sets this environment variable to:

```
.: $CORE_LOCAL/4edi/load:$CORE_ROOT/4edi/load
```

and for Windows systems it is set to:

```
.;%CORE_LOCAL%\4edi\load;%CORE_ROOT%\4edi\load
```

EDI_PATH

The interpreter uses the environment variable EDI_PATH to find compiled Message Builder programs to execute. It specifies directories to search for the program to execute. The format of the value of EDI_PATH is identical to the standard PATH variable. If the EDI_PATH variable does not exist, the interpreter searches for programs to execute in the current directory. The **P** command option can also be used to add directories to the list of directories to be searched for programs to execute. For UNIX and AS/400 systems, the Integration Broker normally sets this environment variable to:

```
.: $CORE_LOCAL/4edi/pgm:$CORE_ROOT/4edi/pgm
```

and for Windows systems it is set to:

```
.;%CORE_LOCAL%\4edi\pgm;%CORE_ROOT%\4edi\pgm
```

EDI_TRACE_SIGUSR1_FILE

This debug feature enables activating the -T option on-the-fly on non-Windows platforms. Use it without the -T and -N options. The value of the environment variable is the filename (with path) that would have been used with the -N option. The file will be opened but nothing will be written to it until a SIGUSR1 signal is received. Another signal stops writing to the file. This allows tracing a hanging program even when all the traces written up to the moment of the hang would otherwise fill up all available disk space.

R4EDI_OPTIONS

Before processing the command line options, the interpreter processes the options specified by this environment variable. A Bourne Shell example:

```
R4EDI_OPTIONS="-C0 -T"
```

The double quotes are needed by the Shell.

11.3.3 Server mode

If the **s** option is specified when the interpreter is started, the interpreter will run in server mode.

When the interpreter runs in server mode, it behaves in a different way than when it runs in non-server mode. Instead of immediately starting to execute the specified Message Builder program, it waits until a client connects to it. When this happens, the Message Builder program starts to execute. Several clients may make connections to the Message Builder server. One instance of the Message Builder program specified when the server was started, is started for each client.

11.4 Examples

The simplest case is when there is one source code file containing all the source code. Assume that the name of the source code file is *example.s4*. The command to compile this file is:

```
c4edi example.s4
```

This command produces the output file *example.x4*, which can be executed using the command:

```
r4edi example.x4
```

or

```
r4edi example.s4
```

Now, assume that an INCLUDE statement that includes the file *include.s4h* is added to the *example.s4* file. Furthermore, assume that the included file is located in the *include* directory and that the INCLUDE statement looks like:

```
INCLUDE "include.s4h";
```

The following command can be used to compile the *example.s4* file:

```
c4edi -Iinclude example.s4
```

To avoid specifying the **I** option every time the compiler is invoked, the environment variable EDI_INCLUDE can be set to point to the *include* directory. If the Bourne shell is used on a UNIX system, the command to do this is:

```
EDI_INCLUDE=include; export EDI_INCLUDE
```

and if the C shell is used the command is:

```
setenv EDI_INCLUDE include
```

On Windows systems, environment variables are set using the *Control Panel*.

Now the **I** option in the compile command can be left out.

Now assume that the *example.s4* file calls a user function named EXAMPLE_FUNCTION and a user statement named EXAMPLE_STATEMENT that are both declared in the *library.s4* file located in the *library* directory. The command to compile the *example.s4* file now becomes (assuming that the EDI_INCLUDE environment is set as described above):

```
c4edi -Llibrary example.s4
```

To make it possible for the compiler to locate the EXAMPLE_FUNCTION function and the EXAMPLE_STATEMENT statement, the *library* directory must contain a table file with the extension ".t4", for example, *library.t4*. The contents of the *library.t4* file should look something like:

```
Function  EXAMPLE_FUNCTION  library.s4
Statement EXAMPLE_STATEMENT library.s4
```

To avoid specifying the **L** option every time the compiler is invoked, the environment variable EDI_LIB can be set to point to the *library* directory.

12 Message Builder language reference

12.1 Presentation

This chapter lists all the statements, compound statements, and functions supported by the Message Builder language.

The SYNTAX sections describe the syntax of all the statements and functions. Words shown in uppercase letters and punctuation characters should be entered as they are. All parts shown in italics should be replaced by an expression, a variable, and so on. Square brackets ([]) are used to indicate optional items and curly brackets ({}), together with the vertical bar (|) are used to indicate that one item must be chosen.

The following basic terms are used:

- *statement-block*

One or more single (for example, PRINT) and/or a compound (for example, IF) statements, surrounded by { and }.

- *variable*

A Message Builder variable. This can be without array indices (for example, \$s) or with array indices (for example, \$v[5], \$v[\$s+1], \$a[\$i][\$j]), without or with data record fields (for example \$r.\$f) and/or without or with a sub-string operator (for example \$s[\$x,\$y]). A variable name must start with a dollar sign (\$) followed by one or more alphanumeric or underscore characters (only alphabetic characters in the range A – Z are valid). There is no limit on the length of variable names.

- *Expression*

An *expression* consists of variables, integer and floating-point constants, string constants, arithmetic, comparison and logical operators, the string concatenation and sub-string operators, and calls to functions. Parentheses may be used to change the operator priority. An example: `2 * $test / ("23" & "1")`

- *const-expression*

A *const-expression* is similar to an *expression*. The difference is that only variables declared constant and calls to most of the built-in functions can be used. The documentation shows the built-in functions that can be used in *const-expressions*.

The error codes listed in the ERRORS section are constant integer Message Builder variables. The declaration of these variables may be found in the *error.s4h* include file. Note that the codes listed in the ERRORS section need not necessarily be errors; they can also be warning or informational codes.

12.2 Language overview

This section contains an overview of all items presented in this chapter. The items have been divided into groups and a short description together with a page number is given for each item.

12.2.1 General statements

The majority of the statements in this group are used to control the program flow.

Name	Description
ASSIGNMENT	Assignment of variables.
BLOCK	Group statements into a statement block.
BREAK	Break out of a loop.
CASE	Conditional execution.
CONTINUE	Continue with the next iteration of a loop.
EXECUTE	Execute a Message Builder program.
EXIT	Exit and terminate a Message Builder program.
FOR	Loop control.
IF	Conditional execution.
INCLUDE	Include Message Builder program code from another file.
REPEAT	Loop control.
RETURN	Return from a user function or statement.
SUSPEND	Suspend execution.
SWITCH	Switch Message Builder program.
THROW	Exception handling.
TRY	Error and exception handling.
WHILE	Loop control.
WITH	Use variable aliases.

12.2.2 Declaration statements

These statements are used to declare variables, data records, user-written and external functions and statements, and modules.

Name	Description
DECLARE EXTERNAL FUNCTION	Declare an external user function.
DECLARE EXTERNAL STATEMENT	Declare an external user statement.
DECLARE FUNCTION	Declare a user function.
DECLARE MODULE	Declare a module.
DECLARE RECORD	Declare a data record.
DECLARE STATEMENT	Declare a user statement.
DECLARE VARIABLE	Declare a variable.

12.2.3 SQL statements

The Message Builder language supports SQL as an embedded language. All SQL statements are prefixed by EXEC SQL and use the syntax specified in the *X/Open Portability Guide Issue 3 – Data Management*. To make it possible to communicate data between the Message Builder and SQL languages, Message Builder variables can be used in SQL statements if they are preceded by a colon (:) character. For example, using the \$v Message Builder variable in an INSERT statement may look like:

```
EXEC SQL INSERT INTO xyz (name) VALUES (: $v);
```

In most places where it is valid to put a Message Builder variable (preceded by a colon), an indicator variable can also be specified. The indicator variable follows immediately after the first variable without any separator characters (except for white space). The above INSERT statement with \$i as an indicator variable would look like:

```
EXEC SQL INSERT INTO xyz (name) VALUES (: $v : $i);
```

When writing data to the database, if the indicator variable contains the value -1, the data will be a NULL value. For all other values of the indicator variable, the data is the value of the \$v variable. The syntax for using indicator variables in the INTO part of the SELECT statement is the same.

Note: The reserved SQL keywords may not be used as user defined names (for example table names).

Beside SQL type string constants, Message Builder string constants are also supported. See the CONSTANTS section, paragraph [12.4.19](#).

Name	Description
DATABASE CLOSE	Close a database connection.
DATABASE EXECUTE	Send a string to the RDBMS for execution.
DATABASE OPEN	Open a database connection.
EXEC SQL ALTER TABLE	Add new columns to an existing base table.
EXEC SQL BEGIN WORK	Start a transaction.
EXEC SQL COMMIT WORK	End and store a transaction.
EXEC SQL CREATE INDEX	Create an index on a base table.
EXEC SQL CREATE TABLE	Create a base table.
EXEC SQL CREATE VIEW	Create a viewed table.
EXEC SQL DELETE	Remove data from a table.
EXEC SQL DROP INDEX	Destroy an index.
EXEC SQL DROP TABLE	Destroy a table.
EXEC SQL DROP VIEW	Destroy a view.
EXEC SQL GRANT	Grant privileges on a table.
EXEC SQL INSERT	Insert data into a table.
EXEC SQL REVOKE	Revoke privileges on a table.
EXEC SQL ROLLBACK WORK	End and undo a transaction.
EXEC SQL SELECT	Read data from one or more tables.
EXEC SQL UPDATE	Update data in a table.

12.2.4 EDI statements

The EDI statements are used to parse and generate EDI messages.

Name	Description
BOUNDED LOOP	Start an X12 bounded loop.
COMPOSITE	Start a composite data element.
DATA ELEMENT (EDIFACT)	Read an EDIFACT EDI data element
DATA ELEMENT (X12)	Read an X12 EDI data element.
ESCAPE	Insert escape characters into a string.
PRINT	Print to the output file.
RELATION	Check X12 relations.
SCAN	Move the current position or search forward in the input file.
SEGMENT	Start a segment definition.
SEGMENT GROUP	Start a group of segments.
TRUNCATE (statement)	Enable truncation of EDI separators when printing.
UNBOUNDED LOOP	Start an X12 unbounded loop.

12.2.5 Interface statements

The statements in this group are used to interface to other parts of the Integration Broker and to other programs.

Name	Description
CALL	Execute an external program.
PIPE CLOSE	Close the input and output pipes.
PIPE OPEN	Open the input and output pipes.
PIPE READ	Read data from the input pipe.
PIPE WRITE	Write data to the output pipe.

12.2.6 File management statements

This group of statements is used to manipulate files in different ways.

Name	Description
CLOSE	Close the input or output file.
FILE APPEND	Append a file to another.
FILE COPY	Copy a file.
FILE DELETE	Delete a file.
FILE MOVE	Move (rename) a file.
IOSET CREATE	Create a new input/output set.
IOSET DELETE	Delete an input/output set.
IOSET SWITCH	Switch between input/output sets.
OPEN	Open the input or output file.
PRINT	Print to the output file.
READ	Read from the input file.

Name	Description
SCAN	Move the current position or search forward in the input file.
SPLIT CLOSE	Close the split input file.
SPLIT INTO	Split a file into several files.
SPLIT OPEN	Open the split input file.

12.2.7 String-handling functions and statements

The functions and statements in this group are used for string manipulations.

Name	Description
CHARACTER	Convert a character code to a string.
CODE	Return the code for a character.
DATE	Convert a DATE value to a string.
ESCAPE	Insert escape characters into a string.
FORMAT	Format a string.
REGEXP	Match a string against a regular expression.
STRBACKWARD	Search for a sub-string.
STRFORWARD	Search for a sub-string.
STRLEN	Get the length of a string.
STRREPEAT	Return a repeated string.
TRIMLEFT	Remove leading pad characters.
TRIMRIGHT	Remove trailing pad characters.
TRUNCATE (function)	Remove trailing empty EDI elements

12.2.8 Coroutine functions and statements

The functions and statements in this group are used to create, terminate and manipulate coroutines.

Name	Description
COROUTINE	Function that returns information about a coroutine.
COROUTINE KILL	Terminate the execution of another coroutine.
COROUTINE LOCK	Lock execution of a coroutine.
COROUTINE NEW	Create a new coroutine.
COROUTINE RESUME	Resume execution of another coroutine.
COROUTINE TERMINATE	Terminate the current coroutine.

12.2.9 Debug statements

The functions and statements in this group are mainly used for debug purposes.

Name	Description
ANTINOLOG	Undo the effect of NOLOG.
DEBUG	Control debug logging.

Name	Description
DUMP BACKTRACE	Output a call backtrace for the current coroutine.
DUMP COROUTINES	Output a list of all currently existing coroutines.
DUMP PROFILE	Write profiling information.
DUMP VARIABLES	Dump variable information.

12.2.10 Miscellaneous functions

This group contains the remaining functions supported by the Message Builder language.

Name	Description
ARGUMENT	Get a command line argument.
ARGUMENTCOUNT	Get the number of command line arguments.
ARRAYDIM	Get the number of dimensions for an array.
ARRAYSIZE	Get the size of an array.
CURRENTDATE	Get the current date and time.
FIDENT	Convert a function identifier to a string
FROM_ISO8859	Convert from a standard character set.
GETDATE	Return a field from a date value.
INTERP_VERSION	Return the version number of the interpreter.
IOSET	Get the identifier for the current input/output set.
PROGRAMINFO	Return program information.
SIDENT	Convert a statement identifier to a string.
SOURCEFILE	Get the name of the current Message Builder source file.
SOURCELINE	Get the current line number in the current Message Builder source file.
SOURCEMODULE	Get the name of the current module.
SOURCEPROCEDURE	Get the name of the current user declared function or statement.
SYSTEM	Return the system type.
TO_ISO8859	Convert to a standard character set.

12.2.11 Miscellaneous statements

This group contains the remaining statements supported by the Message Builder language.

Name	Description
ARRAY DELETE	Delete array elements.
ARRAY INSERT	Insert array elements.
EXPORT	Export a value to an environment variable.
IMPORT	Import the value of an environment variable.
LOADPGM	Load a Message Builder program.
LOG	Write a message to the log.
NOLOG	Disable logging.
PRAGMA	Control the compilation process in various ways.

Name	Description
PRINTERR	Print to the error file.
RELEASE	Release (delete) an environment variable.
SERIALIZE	Generate a string representation for a value.
SETDATE	Modify fields in a date variable.
SLEEP	Pause program execution for a specified number of seconds.
UNLOADPGM	Unload a Message Builder program.
UNSERIALIZE	Convert a string representation of a value to the value itself.
USE	Associate a function or statement name with an external function or statement.

12.2.12 Other topics

This group contains the remaining topics presented in this chapter.

Name	Description
COMMENTS	Comments in the Message Builder language.
CONSTANTS	Numeric and string constants in the Message Builder language.
EXPRESSIONS	Message Builder expressions.
RESERVED VARIABLES	Variables with special meaning in the Message Builder language.
SUBSTRING	Sub-string extraction operator.
VARIABLE TYPES	Variable types supported by the Message Builder language.

12.3 Evaluation order

When calling a function or executing a statement, the evaluation order for the arguments or the parameters is undefined. For example, if the NEW_NAME function modifies the `$fileName` variable, the effect of the statement:

```
FILE COPY $fileName TO NEW_NAME($fileName);
```

is undefined.

The order of evaluation for expressions is also undefined. For example, it is undefined which of the X or Y functions that will be called first in the assignment:

```
$a = X($b) + Y($c);
```

Programs speculating in the evaluation order may cease to work for future versions of Message Builder.

12.4 Reference pages

On the following pages, each built-in function and statement and other topics are described in detail.

12.4.1 ANTINOLOG

Name

ANTINOLOG – Undo the effect of NOLOG.

Syntax

ANTINOLOG statement-block.

Description

When *statement-block* is executed, the effect of all surrounding NOLOG statements is suppressed. Also, all NOLOG statements inside *statement-block* does not suppress log messages.

Examples

A typical usage for the ANTINOLOG statement is when it is desired to see log messages in an existing routine that normally suppresses log messages. Assume there exist a statement that deletes files without outputting any error messages in case of errors:

```
INCLUDE "exception.s4h";

DECLARE STATEMENT
SILENT_DELETE IN $filename PRIMITIVE
{
    NOLOG {
        TRY {
            FILE DELETE $filename;
        }
        CATCH $exception
        WHEN $Exception_Execute {}
    }
    RETURN;
}
```

If this statement is used from a program where some problems exist, the error message produced by the FILE DELETE statement can be seen if an ANTINOLOG statement surrounds the call to SILENT_DELETE:

```
.
.
.
ANTINOLOG {
    SILENT_DELETE $file;
}
.
.
.
```

See also

NOLOG, LOG.

Notes

This statement is intended for debug purposes and has limited value in production code.

12.4.2 ARGUMENT

Name

ARGUMENT – Function that returns a command line argument.

Syntax

ARGUMENT(*number*)

Description

The ARGUMENT function returns the command line argument specified by *number* as a string. The *number* argument is an *expression* that is converted to an integer.

If *number* is equal to 0, the path name of the Message Builder program file is returned, if it is equal to 1, the first argument following the Message Builder program file is returned, and so forth.

If *number* is greater than the number of command line arguments, an empty string is returned.

If *number* is less than 0, an interpreter command argument preceding the Message Builder program name is returned. Minus one returns the interpreter command argument preceding the Message Builder program name. The lowest negative value that does not produce an error returns the interpreter name.

Errors

The following values can be assigned to `$Error`:

`$Error_ArgumentNoSuchArgument`

The *number* argument is too small. It does not correspond to a command argument.

Examples

If the program:

```
DECLARE VARIABLE $index INTEGER;

IF ARGUMENTCOUNT() <> 3 {
    PRINT "Usage: r4edi test.s4 <text> <index> <offset>\n";
    EXIT 1;
}
$index = ARGUMENT(2);
PRINT "File: " & ARGUMENT(0) & "\n";
PRINT "Text: " & ARGUMENT(1) & "\n";
PRINT "Index: " & $index & "\n";
PRINT "Offset: " & ARGUMENT(3) & "\n";
```

is executed using the command:

```
r4edi test.s4 "Hello World!" 10 32
```

the following is written to the output file:

```
File: test.s4
Text: Hello World!
```

```
Index: 10
Offset: 32
```

The following program fragment finds the argument number for the interpreter name, that is, *r4edi*:

```
INCLUDE "error.s4h";
INCLUDE "exception.s4h";
DECLARE VARIABLE $argumentNumber INTEGER;
DECLARE VARIABLE $argument STRING;

$argumentNumber = 0;
TRY {
    NOLOG {
        WHILE 1 {
            $argumentNumber = $argumentNumber - 1;
            $argument = ARGUMENT($argumentNumber);
        }
    }
}
CATCH $exception
WHEN $Exception_Execute {}
$argumentNumber = $argumentNumber + 1;
PRINT "The interpreter name is " & ARGUMENT($argumentNumber) & "\n";
```

See also

ARGUMENTCOUNT.

Notes

The ARGUMENTCOUNT function can be used to obtain the number of command-line arguments.

12.4.3 ARGUMENTCOUNT

Name

ARGUMENTCOUNT – Function that returns the number of command-line arguments.

Syntax

ARGUMENTCOUNT()

Description

The ARGUMENTCOUNT function returns the number of command line arguments as an integer value.

Examples

If the program:

```
PRINT ARGUMENTCOUNT() & "\n";
```

is named *x.s4* and is started with the command:

```
r4edi x.s4 arg1 arg2 arg3
```

the following is written to the output file:

3

See also

ARGUMENT.

12.4.4 ARRAY DELETE

Name

ARRAY DELETE – Delete elements from an array.

Syntax

ARRAY DELETE *array* FROM *from* [COUNT *count*];

Description

The ARRAY DELETE statement deletes elements from an array, specified by *array*. The index for the first element to delete is specified by *from* and *count* specifies the number of elements to delete (the default is 1). All the elements with indices higher than the sum of *from* and *count* are shifted downwards *count* elements. A count of zero deletes zero elements.

Both *from* and *count* are *expressions* that are converted to integers.

Errors

The following values can be assigned to `$Error`:

- `$Error_ArrayDeleteNotAnArray`
The *array* parameter does not specify an array.
- `$Error_ArrayDeleteNonPositiveFrom`
The *from* parameter is less than or equal to zero.
- `$Error_ArrayDeleteNegativeCount`
The *count* parameter is negative.
- `$Error_ArrayDeleteTooBigFrom`
The *from* parameter is greater than the current size of *array*.
- `$Error_ArrayDeleteTooBigCount`
The sum of *from* and *count* plus one are greater than the current size of *array*.

Examples

Running the following example:

```
DECLARE VARIABLE $a[] INTEGER;
FOR $i = 1 TO 5 {
    $a[$i] = $i;
}
ARRAY DELETE $a FROM 3 COUNT 2;
FOR $i = 1 TO ARRAYSIZE($a) {
    PRINT " " & $a[$i];
}
PRINT "\n";
```

prints the following:

```
1 2 5
```

See also

ARRAY INSERT.

12.4.5 ARRAY INSERT

Name

ARRAY INSERT – Insert elements into an array.

Syntax

ARRAY INSERT *array* FROM *from* [COUNT *count*];

Description

The ARRAY INSERT statement inserts new elements into an array, specified by *array*. The index of the first inserted element is specified by *from* and *count* specifies the number of elements to insert (the default is 1). All the elements with indices higher than or equal to *from* are shifted upwards *count* elements. A count of zero inserts zero elements. The values of the inserted elements are zero for INTEGERS and FLOATs, empty strings for STRINGs, the computers base time for DATEs, and empty arrays for arrays.

Both *from* and *count* are *expressions* that are converted to integers.

Errors

The following values can be assigned to `$Error`:

- `$Error_ArrayInsertNotAnArray`
The *array* parameter does not specify an array.
- `$Error_ArrayInsertNonPositiveFrom`
The *from* parameter is less than or equal to zero.
- `$Error_ArrayInsertNegativeCount`
The *count* parameter is negative.
- `$Error_ArrayInsertTooBigFrom`
The *from* parameter is greater than the current size of *array*.

- `$Error_ArrayInsertTooBigCount`
The new size of the array would become too big. That is, the size would become greater than can be stored in a signed integer.

Examples

Running the following example:

```
DECLARE VARIABLE $a[] INTEGER;
FOR $i = 1 TO 5 {
    $a[$i] = $i;
}
ARRAY INSERT $a FROM 3 COUNT 2;
$a[3] = 30;
$a[4] = 40;
FOR $i = 1 TO ARRAYSIZE($a) {
    PRINT " " & $a[$i];
}
PRINT "\n";
```

prints the following:

```
1 2 30 40 3 4 5
```

See also

ARRAY DELETE.

12.4.6 ARRAYDIM

Name

ARRAYDIM – Function that returns the number of dimensions for an array.

Syntax

ARRAYDIM(*array*)

Description

The ARRAYDIM function returns the number of dimensions for an array as an integer value. For scalar variables, this function returns 0. The *array* argument can be an array variable, a function call or an *expression*. In case *array* is an *expression* or a call to a function returning a scalar, 0 will be returned.

The ARRAYDIM function is valid in *const-expressions* provided the compiler is able to determine the dimension for *array*.

Examples

The code fragment:

```
DECLARE VARIABLE $s INTEGER;
DECLARE VARIABLE $v[] INTEGER;
DECLARE VARIABLE $a[][] INTEGER;

$s = 2;
$v[3] = 0;
```

```

$a[2][7] = 1;
PRINT ARRAYDIM($s) & " ";
PRINT ARRAYDIM($v) & " ";
PRINT ARRAYDIM($v[1] & " ";
PRINT ARRAYDIM($a) & " ";
PRINT ARRAYDIM($a[2]) & " ";
PRINT ARRAYDIM($a[2][6]) & "\n";

```

will produce the output:

```
0 1 0 2 1 0
```

See also

ARRAYSIZE.

12.4.7 ARRAYSIZE

Name

ARRAYSIZE – Function that returns the size of an array.

Syntax

ARRAYSIZE(*array*)

Description

The ARRAYSIZE function returns the size of an array as an integer value. For scalar variables, this function returns -1. This makes it possible to check if a variable is an array or not. If ARRAYSIZE returns a value greater than or equal to 0, it is an array. The *array* argument can be an array variable, a function call or an *expression*. In case *array* is an *expression* or a call to a function returning a scalar, -1 will be returned.

The ARRAYSIZE function is valid in *const-expressions* provided the compiler is able to determine the size of *array*.

Examples

The code fragment:

```

DECLARE VARIABLE $s INTEGER;
DECLARE VARIABLE $v[] INTEGER;
DECLARE VARIABLE $a[][] INTEGER;

$s = 2;
$v[3] = 0;
$a[2][7] = 1;
PRINT ARRAYSIZE($s) & " ";
PRINT ARRAYSIZE($v) & " ";
PRINT ARRAYSIZE($v[1] & " ";
PRINT ARRAYSIZE($a) & " ";
PRINT ARRAYSIZE($a[1]) & " ";
PRINT ARRAYSIZE($a[2]) & " ";
PRINT ARRAYSIZE($a[2][6]) & "\n";

```

will produce the output:

```
-1 3 -1 2 0 7 -1
```

See also

ARRAYDIM.

12.4.8 ASSIGNMENT

Name

ASSIGNMENT – Assign a value to a variable.

Syntax

variable-reference = *expression*;

or

record-reference = {*record-reference* | *function-call*};

or

array-reference = {*array-reference* | *function-call*};

Description

Assignments are used to change the contents of a variable.

The left-hand side of an assignment is a scalar, a data record variable reference or an array variable reference. If the left hand side is a scalar, a sub-string operator (see the SUBSTRING operator, paragraph [12.4.121](#)) may also be present. The right hand side depends on the left-hand side. If the left-hand side is a scalar (*variable-reference*), the right hand side is an *expression*. If the left hand side is a data record variable (*record-reference*), the right hand side must be a data record variable or a function call that returns a data record value. If the left-hand side is an array (*array-reference*), the right hand side must be an array or a function call that returns an array.

When a data record variable is assigned a data record value, the types must be the same and when an array is assigned to another array, the number of dimensions of both the arrays must be the same. When assigning an array, if the elements are data records, both the left and right sides must be arrays of the same record type.

Arrays are automatically expanded if a value is assigned to a non-existing array element; for example, if the size of an array is 5 and a value is assigned to element number 10, the new size of the array will be 10. The values of the new elements will be zero for INTEGER or FLOAT variables, strings filled with spaces for CHAR variables, empty strings for STRING variables or the computer's base time for DATE variables. See the DECLARE VARIABLE statement, page 159, for an explanation of the computer's base time. For record variables, the fields will be as described above and for array variables, the value will be an empty array.

Examples

Various examples of assignments:

```
DECLARE RECORD DEMO {  
    DECLARE FIELD $f1 INTEGER;  
    DECLARE FIELD $f2 FLOAT;  
}  
DECLARE VARIABLE $s INTEGER;
```



```

DECLARE VARIABLE $v1[] INTEGER;
DECLARE VARIABLE $v2[] INTEGER;
DECLARE VARIABLE $a[][] INTEGER;
DECLARE VARIABLE $r1 RECORD DEMO;
DECLARE VARIABLE $r2 RECORD DEMO;

/* scalar variable assignments */
$s = 2 * 3;
$v1[3] = 20;
$s = 3 * ($v1[3] + 2);
$a[$s][1] = FUNC($s, 10) - 2;

/* array variable assignments /
$v2 = $v1;
$a[1] = $v2;

/* record field assignments /
$r1.$f1 = 57;
$r1.$f2 = 75.75;

/* data record assignments */
$r2 = $r1;

```

The following example demonstrates how the size of an array can be reset to 0:

```

DECLARE VARIABLE $a[] INTEGER;
DECLARE VARIABLE $empty[] = {} CONSTANT INTEGER;
.
.
.
$a = $empty;

```

The sub-string operator can be used to replace the contents of a part of a string. The following example:

```

DECLARE VARIABLE $s STRING;
$s = "12345";
$s[3,4] = "ABC";
PRINT $s & "\n";

```

produces the output:

```
12ABC5
```

See also

EXPRESSIONS.

12.4.9 BLOCK

Name

BLOCK – Group statements into a statement block.

Syntax

BLOCK *statement-block*

Description

The BLOCK compound statement groups a number of simple and compound statements into a statement block. The reason for this may be to avoid variable name conflicts, that is, a variable declared inside a statement block shadows a variable with the same name declared outside the block. See the DECLARE VARIABLE statement, paragraph [12.4.41](#), for further information.

Examples

Executing the following example:

```
DECLARE VARIABLE $x INTEGER;
DECLARE VARIABLE $y INTEGER;
DECLARE VARIABLE $temp INTEGER;

$x = 1;
$y = 2;
$temp = 3;
BLOCK {
    /* swap $x and $y */
    DECLARE VARIABLE $temp INTEGER;
    $temp = $x;
    $x = $y;
    $y = $temp;
}
PRINT $x & " " & $y & " " & $temp & "\n";
```

produces the following result:

```
2 1 3
```

12.4.10 BOUNDED LOOP

Name

BOUNDED LOOP – Start an X12 bounded loop.

Syntax

```
{MANDATORY | OPTIONAL}
  BOUNDED LOOP loopid-list [REPEAT [count]] statement-block
```

Description

The BOUNDED LOOP compound statement starts a bounded loop of segments and other loops. A bounded loop starts with an LS segment containing a loop ID and ends with an LE segment containing a matching loop ID. The BOUNDED LOOP statement automatically handles the LS and LE segments and checks that the loop IDs in them are correct.

The *loopid-list* parameter is a list of *expressions* separated by commas. Each element in the list is converted to a string.

The *count* parameter is an *expression* that is converted to an integer.

One of the loop IDs, specified by *loopid-list*, must equal the loop ID for the bounded loop.

If the OPTIONAL modifier is specified, the bounded loop does not have to exist in the input file. If the MANDATORY modifier is specified and the bounded loop does not exist in the input file, the Message Builder interpreter generates an EDI error.

If the bounded loop is allowed to be repeated, this can be specified using the REPEAT modifier. The maximum number of allowed repetitions is specified by *count*. If *count* is not specified, an unlimited number of repetitions are allowed. If the REPEAT modifier is not specified, the repeat count is set to one.

Nesting of bounded loops is allowed, that is, it is possible to define a bounded loop inside another bounded loop.

The `$EdiSegCnt` reserved variable is incremented two times: before *statement-block* is executed the first time and when the bounded loop is exited.

Errors

The following values can be assigned to `$Error`:

- `$Error_BoundedLoopMandatoryEof`
End-of-file encountered when expecting a mandatory bounded loop. This is an EDI error.
- `$Error_BoundedLoopReadError`
Read error on the input file.
- `$Error_BoundedLoopNoTag`
No segment tag found. This is an EDI error.
- `$Error_BoundedLoopWrongTag`
Wrong tag found when expecting mandatory bounded loop. This is an EDI error.
- `$Error_BoundedLoopExtraData`
Extra data at end of segment. This is an EDI error.
- `$Error_BoundedLoopIdEof`
End-of-file encountered when expecting loop ID. This is an EDI error.
- `$Error_BoundedLoopIdNone`
No data element found in the LS or LE segment. This is an EDI error.
- `$Error_BoundedLoopIdWrong`
The MANDATORY modifier was specified and the LS segment did not contain any of the loop IDs specified by *loopid-list*. This is an EDI error.
- `$Error_BoundedLoopTooMany`
The bounded loop contains too many repetitions. This is an EDI error.
- `$Error_BoundedLoopUnknownSegment`
An unexpected segment was found in the loop. This is an EDI error.
- `$Error_BoundedLoopCharacterError`
The read data contains a character that is not valid according to the `$EdiCharValid` reserved variable.
- `$Error_Pgmlogic_BoundedLoopInsideSegment`
BOUNDED LOOP statement executed inside SEGMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_BoundedLoopInsideComposite`
BOUNDED LOOP statement executed inside COMPOSITE statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_BoundedLoopInsideElement`
BOUNDED LOOP statement executed inside DATA ELEMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_BoundedLoopInsideRelation`

BOUNDED LOOP statement executed inside RELATION statement. This is a PGMLOGIC error.

Examples

The following program parses a bounded loop with loop ID B1:

```
DECLARE VARIABLE $loop INTEGER;

$EdiSegSep = "";
$EdiElemSep = "+";
$loop = 0;
OPTIONAL BOUNDED LOOP "B1" REPEAT 10 {
    $loop = $loop + 1;
    OPTIONAL SEGMENT "S1" {
        MANDATORY STRING 4 {
            PRINT "S1:" & $loop & "\n";
        }
    }
    OPTIONAL SEGMENT "S2" {
        MANDATORY STRING 4 {
            PRINT "S2:" & $loop & "\n";
        }
    }
    OPTIONAL SEGMENT "S3" {
        MANDATORY STRING 4 {
            PRINT "S3:" & $loop & "\n";
        }
    }
}
```

If the data

```
LS+B1'S1+1'S2+1'S3+1'S2+2'S1+3'S3+3'S3+4'LE+B1'
```

are used as input to the above program, the following is written to the output file:

```
S1:1
S2:1
S3:1
S2:2
S1:3
S3:3
S3:4
```

See also

COMPOSITE, DATA ELEMENT (X12), SEGMENT, SEGMENT GROUP, UNBOUNDED LOOP.

Notes

Specifying more than one loop ID in *loopid-list* should be avoided. This is not needed when parsing standard EDI messages.

The CONDITIONAL modifier can be used instead of OPTIONAL. The recommendation is to use OPTIONAL.

12.4.11 BREAK

Name

BREAK – Break out of a loop.

Syntax

BREAK;

Description

The BREAK statement breaks out of the innermost FOR (see paragraph [12.4.72](#)), REPEAT (see paragraph [12.4.72](#)), WHILE (see paragraph [12.4.137](#)), or EXEC SQL SELECT (see paragraph [12.4.61](#)) loop; that is, continue the execution with the first statement after the loop. The BREAK statement is only valid inside a loop.

Examples

Executing the following program:

```
DECLARE VARIABLE $i INTEGER;
DECLARE VARIABLE $j INTEGER;

$i = 1;
WHILE $i <= 5 {
    $j = 1;
    WHILE $j <= 5 {
        PRINT $j & " ";
        IF $j = $i {
            BREAK;
        }
        $j = $j + 1;
    }
    PRINT "\n";
    $i = $i + 1;
}
```

produces the following output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

See also

CONTINUE, EXEC SQL SELECT, FOR, REPEAT, WHILE.

12.4.12 CALL

Name

CALL – Execute an external program.

Syntax

CALL *command* [EXIT *exit-value*];

Description

The CALL statement executes *command* using the command interpreter (*sh* for UNIX systems, *cmd* for Windows systems, and *QCMD* for AS/400 systems). The *command* parameter is an *expression* that is converted to a string.

The *exit-value* output *variable* parameter receives the exit value of the executed program. It should be of integer type.

Errors

The following values can be assigned to `$Error`:

- `$Error_CallNoSupport`
The CALL statement is not supported.
- `$Error_CallSystemOverLoad`
System overload, for example, too many processes. This is a warning.
- `$Error_CallEmpty`
The *command* parameter is empty.
- `$Error_CallSystemFail`
Failed to execute *command*.

Examples

The following example first compiles and then executes the *another.s4* Message Builder program:

```
DECLARE VARIABLE $exitCode INTEGER;

CALL "c4edi another.s4" EXIT $exitCode;
IF $exitCode = 0 {
    /* no compilation errors */
    CALL "r4edi another.s4";
}
```

Notes

The Message Builder program will normally wait until the external program has finished executing. If this is not desired, on UNIX systems an `&` character can be used at the end of *command* to execute the program in the background. On Windows systems, the *start* command is used instead. When executing the program in the background, the value of *exit-value* is of no significance.

On UNIX and Windows machines, the CALL statement is equivalent to executing the commands

```
sh -c "command"
```

or

```
cmd /c "command"
```

respectively.

When using redirection on Windows systems, the *cmd* interpreter open files in non-shared mode. This means that each program needs its own file.

If *command* is the Message Builder interpreter, the **1** and **2** options can be used to open files in shared mode.

12.4.13 CASE

Name

CASE – Multi-way selection statement.

Syntax

```
CASE selector
WHEN const-expression-list-1 statement-block-1
WHEN const-expression-list-2 statement-block-2
.
.
.
WHEN const-expression-list-N statement-block-N
[WHEN OTHERS statement-block-others]
```

Description

The CASE statement executes one (or zero) of the statement blocks depending on the value of the *selector* parameter (an *expression*).

Each *const-expression-list-n* parameter is a list of *const-expressions* separated by commas.

Each *statement-block-n* parameter is a *statement-block*.

The statement block that has a constant expression in its expression list that equals the value of *selector* is executed. If no such constant expression exists and the WHEN OTHERS part is present, the *statement-block-others* statement block is executed. No statement block is executed if no constant expression equals *selector* and the WHEN OTHERS part is not present.

Examples

The code fragment:

```
DECLARE VARIABLE $selector INTEGER;

$selector = 2;
CASE 2 * $selector
WHEN 0 {
    PRINT "Zero\n";
}
WHEN 2 {
    PRINT "Two\n";
}
WHEN 4, 5 {
    PRINT "Four or Five\n";
}
WHEN OTHERS {
```

```
        PRINT "OTHERS\n";  
    }
```

will write the string `Four` or `Five` to the output file.

See also

IF.

12.4.14 CHARACTER

Name

CHARACTER – Function that converts a character code to a string.

Syntax

CHARACTER(*code*)

Description

The CHARACTER function returns a string of length one containing the character whose character code is *code*. The *code* argument is an *expression* that is converted to an integer. If the *code* argument is less than zero or greater than 255 the 8 least significant bits are used to form the returned character. For example, if *code* is 321, the returned character is "A".

The CHARACTER function is valid in *const-expressions* (in this case, *code* must be a *const-expression*).

Examples

The following program writes the decimal digits to the output file:

```
FOR $i = 0 TO 9 {  
    PRINT CHARACTER($i + CODE("0")) & "\n";  
}
```

See also

CODE.

12.4.15 CLOSE

Name

CLOSE – Close the input or output file.

Syntax

CLOSE {INPUT | OUTPUT [SYNCHRONIZE]};

Description

The CLOSE statement closes the input or output file, depending on whether the INPUT or OUTPUT modifier is specified, respectively.

The standard output is the default output file after a CLOSE OUTPUT statement and the standard input is the default input file after a CLOSE INPUT statement.

The SYNCHRONIZE modifier forces the file contents to be physically written to disk before the file is being closed.

Errors

The following values can be assigned to `$Error`:

- `$Error_CloseInputClosed`
The input file is already closed. This is a warning.
- `$Error_CloseOutputClosed`
The output file is already closed. This is a warning.
- `$Error_CloseSynchronizeFail`
Failed to physically write the file contents to disk.

Examples

Close the file with the incoming EDI message:

```
CLOSE INPUT;
```

Close the output file:

```
CLOSE OUTPUT;
```

See also

OPEN, PRINT, READ.

Notes

The input and output files are automatically closed when a Message Builder program terminates. However, all files that have been opened should also be closed explicitly.

12.4.16 CODE

Name

CODE – Function that returns the code for a character.

Syntax

```
CODE(character)
```

Description

The CODE function returns an integer representing the character code for the first character in the *character* argument. The *character* argument is an *expression* that is converted to a string. If *character* is an empty string, 0 is returned.

The CODE function is valid in *const-expressions* (in this case, *character* must be a *const-expression*).

Examples

The following example converts an uppercase letter in `$upper` to lowercase:

```
$lower = CHARACTER(CODE($upper) + (CODE("a") - CODE("A")));
```

See also

CHARACTER.

12.4.17 COMMENTS

Name

COMMENTS – Comments in the Message Builder language.

Description

Two types of comments can be used in a Message Builder program: C-style and rest-of-line comments.

C-style comments start with the sequence `/*` and end with `*/`. They cannot be nested; that is, the `*/` in

```
/* this is /* a complete comment */
```

ends the comment. (Note that the compiler issues a warning if the sequence `/*` is encountered inside a C-style comment.) A C-style comment may contain new-lines.

A rest-of-line comment starts with the `#` character and continues to the end of the line.

The `#` character is ignored in C-style comments and the `/*` and `*/` sequences are ignored in rest-of-line comments.

Examples

```
/*
 * Print the name and the age.
 */
PRINT "Name:" & $name & "\n";    # print the name
PRINT "Age:" & $age & "\n";      /* print the age */
```

12.4.18 COMPOSITE

Name

COMPOSITE – Start a composite data element.

Syntax

```
{CONDITIONAL | OPTIONAL | MANDATORY}
  COMPOSITE [STRING] statement-block
```

Description

Whenever a composite data element is expected, a COMPOSITE compound statement should be used. This statement checks if the next element in the input file is a composite data element, and, if it is, the *statement-block* is executed. If the next element is not a composite and the MANDATORY modifier is specified, an EDI error is generated.

Before *statement-block* is executed, the `$EdiElemCnt` reserved variable is incremented by 1 and `$EdiCompCnt` is set to 0.

Within a composite element the composite element separator specified by the `$EdiCompSep` reserved variable is used instead of the element separator specified by `$EdiElemSep`.

The STRING modifier tells the interpreter that the next data element is of string type. This is needed in order to properly parse multi-byte data. When parsing non-multi-byte data, the STRING modifier makes no difference.

Errors

The following values can be assigned to `$Error`:

- `$Error_CompositeEof`
An unexpected end-of-file was encountered. This is an EDI error.
- `$Error_CompositeReadError`
Read error on the input file.
- `$Error_CompositeNone`
No composite element found when one was expected. This error can only occur if the MANDATORY modifier was used. This is an EDI error.
- `$Error_CompositeExtraData`
Too many composite elements in composite. This is an EDI error.
- `$Error_CompositeCscError`
Error related to multi-byte handling.
- `$Error_CompositePartialError`
Incomplete multi-byte character.
- `$Error_CompositeCharacterError`
The read data contains a character that is not valid according to the `$EdiCharValid` reserved variable.
- `$Error_Pgmlogic_CompositeOutsideSegment`
COMPOSITE statement executed outside a SEGMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_CompositeInsideComposite`
COMPOSITE statement executed inside another COMPOSITE statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_CompositeInsideElement`
COMPOSITE statement executed inside DATA ELEMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_CompositeInsideRelation`
COMPOSITE statement executed inside a RELATION statement. This is a PGMLOGIC error.

Examples

```

CONDITIONAL COMPOSITE {
    MANDATORY FIXED STRING 6 VALIDATE "PRICAT";
    CONDITIONAL FIXED NUMBER 1;
    CONDITIONAL NUMBER 10;
}

```

See also

DATA ELEMENT (EDIFACT), DATA ELEMENT (X12), SEGMENT

Notes

Nested composite elements are not allowed. The Message Builder compiler attempts to check for these. However, if the *statement-block* contains calls to user functions or user statements, the compiler will not detect this. The error will instead be detected at run time and then be treated as a PGMLOGIC error; that is, the Message Builder interpreter will terminate if the error is not trapped using the TRY statement (see paragraph [12.4.131](#)).

The OPTIONAL modifier is equivalent to CONDITIONAL. Use CONDITIONAL when parsing EDIFACT messages and OPTIONAL when parsing X12 messages.

12.4.19 CONSTANTS

Name

CONSTANTS – Numeric and string constants in the Message Builder language.

Description

The Message Builder language supports integer, floating-point and string constants.

An integer constant consists of decimal digits preceded by an optional minus sign. Examples of integer constants are:

```

105
2
-57

```

Integer constants in hexadecimal notation are also supported. A hexadecimal number starts with the characters 0x or 0X followed by one or more hexadecimal digits. The digits 0 to 9 and the letters A to F (both upper and lowercase can be used) are used to represent the hexadecimal digits. Examples of hexadecimal integer constants are:

```

0x53e
0X1F

```

A floating-point constant consists of a mantissa and an optional exponent. The mantissa consists of an optional leading minus sign (-), decimal digits, and an optional decimal point (.). The exponent is separated from the mantissa by an "e" or "E" character and consists of an optional leading plus or minus sign (+ or -) followed by decimal digits. Examples of floating-point constants are:

```

15.5
2E3
-1.7e-4

```

A Message Builder string constant must be enclosed within double quotes ("). New line characters are not allowed in a string. Special characters can be specified in string constants using the following special character sequences:

```
\a Audible alarm (bell).
\b Backspace.
\f Form feed.
\n New line.
\r Carriage return.
\t Horizontal tab.
\v Vertical tab.
\" Double quote.
\\ Backslash.
\xnn Hexadecimal value. nn is one or two hexadecimal digits.
\nnn Octal value. nnn is one to three octal digits.
\c The character c (c is any character except a, b, f, n, r, t, v, x or
0 - 7).
```

Examples of string constants are:

```
"MB"
"Line 1\nLine 2\n"
```

SQL-style string constants are also supported. Such strings are enclosed within single quotes ('). Two single quotes in a row can be used to insert a single quote in a string. None of the special sequences described above is supported in SQL-type strings. An SQL-type string constant can be used wherever a Message Builder string constant can be used. Examples of SQL type string constants are:

```
'MB'
'An SQL type string with a quote (') character'
```

12.4.20 CONTINUE

Name

CONTINUE – Continue with the next iteration of a loop.

Syntax

CONTINUE;

Description

The CONTINUE statement causes the next iteration of the innermost loop to begin. It can be used in FOR (see paragraph [12.4.72](#)), REPEAT (see paragraph [12.4.100](#)), WHILE (see paragraph [12.4.137](#)), or EXEC SQL SELECT (see paragraph [12.4.61](#)) loops. The CONTINUE statement is only valid inside a loop.

Examples

Executing the following program:

```
FOR $i = 1 TO 5 {
  FOR $j = 1 TO 5 {
    IF $j = $i {
      CONTINUE;
    }
    PRINT $j & " ";
  }
}
```

```

    }
    PRINT "\n";
}

```

will produce the following output:

```

2 3 4 5
1 3 4 5
1 2 4 5
1 2 3 5
1 2 3 4

```

See also

BREAK, EXEC SQL SELECT, FOR, REPEAT, WHILE.

12.4.21 COROUTINE

Name

COROUTINE – Function that returns information about a coroutine.

Syntax

COROUTINE(*selector*)

Description

The COROUTINE function returns an integer value giving information about coroutines. The selector argument indicates which type of information to return:

- `$Coroutine_SelectorIdOfOurselves`
Return the coroutine ID for the calling coroutine.
- `$Coroutine_SelectorIdOfParent`
Return the coroutine ID for the parent coroutine (the coroutine that created the calling coroutine). If the calling coroutine has ID zero (the main coroutine), the return value is minus one. It is also minus one if the parent has terminated.
- `$Coroutine_SelectorIdForPrevious`
Return the coroutine ID for the previously executing coroutine. Minus one is returned if only coroutine zero (the main coroutine) does exist or if the current coroutine is executing because the previous one terminated.
- `$Coroutine_SelectorReasonForSwitch`
Return the reason for the last coroutine switch.
If selector is `$Coroutine_SelectorReasonForSwitch`, the return value can be any of the following:
 - `$Coroutine_SwitchNone`
No coroutine switch has occurred in this program.
 - `$Coroutine_SwitchResume`
A COROUTINE RESUME statement was executed.
 - `$Coroutine_SwitchTerminate`
A COROUTINE TERMINATE statement was executed.
 - `$Coroutine_SwitchReturn`
The previous coroutine terminated because a RETURN was executed from within the statement specified in COROUTINE NEW.
 - `$Coroutine_SwitchException`

The previous coroutine terminated because of an uncaught exception.

- `$Coroutine_SwitchAuto`
Automatic switch, that is, the number of instructions specified by the *count* parameter in COROUTINE NEW had been executed.

All of the above mnemonic variables are constants declared in the *coroutine.s4h* include file.

Errors

The following value can be assigned to `$Error`:

`$Error_CoroutineInvalidSelector`

The *selector* argument is invalid.

See also

COROUTINE KILL, COROUTINE LOCK, COROUTINE NEW, COROUTINE RESUME, COROUTINE TERMINATE.

12.4.22 COROUTINE KILL

Name

COROUTINE KILL – Terminate the execution of another coroutine.

Syntax

COROUTINE KILL *id*;

Description

The COROUTINE KILL statement terminates execution of the coroutine specified by the *id* parameter. The *id* parameter is an *expression* that is converted to an integer. It should have been returned from a call to COROUTINE NEW.

An error is generated if the *id* specifies the main coroutine (coroutine ID zero) or the currently executing coroutine.

Errors

The following value can be assigned to `$Error`:

- `$Error_CoroutineKillInvalidId`
Invalid coroutine ID.
- `$Error_CoroutineKillNoExistId`
The coroutine ID specified by *id* does not exist.
- `$Error_CoroutineKillMainNotAllowed`
Trying to kill the main coroutine, that is, *id* is zero.
- `$Error_CoroutineKillNotOurSelves`
Trying to kill ourselves, that is, *id* specifies the currently executing coroutine.

See also

COROUTINE, COROUTINE LOCK, COROUTINE NEW, COROUTINE RESUME, COROUTINE TERMINATE.

12.4.23 COROUTINE LOCK

Name

COROUTINE LOCK – Lock execution of a coroutine.

Syntax

COROUTINE LOCK *statement-block*

Description

The COROUTINE LOCK compound statement prevents the currently executing coroutine from being automatically interrupted by another coroutine before *statement-block* has finished executing. This can be useful if the coroutine was created with a *count* parameter greater than zero (see paragraph [12.4.24](#)). COROUTINE RESUME statements can still be used inside *statement-block* to switch to other coroutines.

Examples

Executing the following program:

```
DECLARE VARIABLE $id INTEGER;
DECLARE VARIABLE $message STRING;

COROUTINE NEW $id COUNT 1 CALL PRINT_MESSAGES;
FOR $i = 1 TO 20 {
    $message = "Message " & $i;
    COROUTINE RESUME $id;
}
EXIT;

DECLARE STATEMENT
PRINT_MESSAGES
{
    WHILE 1 {
        COROUTINE LOCK {
            IF $message <> "" {
                PRINT $message & "\n";
                $message = "";
            }
        }
    }
    RETURN;
}
```

will output something like:

```
Message 1
Message 3
Message 5
Message 7
Message 9
```



```
Message 11
Message 13
Message 15
Message 17
Message 19
```

Note: The exact output may differ depending on the compiler version. Running the same compiled program several times will however produce the same result.

See also

COROUTINE, COROUTINE KILL, COROUTINE NEW, COROUTINE RESUME, COROUTINE TERMINATE.

12.4.24 COROUTINE NEW

Name

COROUTINE NEW – Create a new coroutine.

Syntax

COROUTINE NEW *id* [COUNT *count*] CALL *statement*;

Description

The Message Builder interpreter is capable of simultaneously running multiple coroutines. A coroutine is similar to a thread with the difference that there is never more than one coroutine executing concurrently. The following things are local to a coroutine (everything else are global):

- The current position in the program.
- Variables local to user functions and statements.
- Set up exceptions (TRY statements).
- SEGMENT FLOATING code blocks.

Each coroutine is identified by a non-negative integer ID. The coroutine ID of the main coroutine (that is the single coroutine that is created when the program is started) is zero.

The *statement* “parameter” is a user statement call that shall be executed by the coroutine. The user statement may only take IN parameters. OUT and IN OUT parameters are not allowed. Copies of all the parameters are made; hence a change in any of the parameters after the coroutine has been created will not be visible in the created coroutine. A coroutine will not start executing immediately, instead it will continue executing the statement following the COROUTINE NEW statement in the coroutine that called COROUTINE NEW. A COROUTINE RESUME or COROUTINE TERMINATE with the coroutine ID of the newly created coroutine as parameter must be executed in order to start executing the new coroutine (this might not be true if the *count* parameter is specified and greater than zero).

The *id* parameter is an integer output *variable* parameter that receives the ID of the newly created coroutine.

By default a coroutine continues execution until it executes a COROUTINE RESUME statement or it terminates. If the *count* parameter is specified and greater than zero,

the coroutine will automatically switch to another coroutine after executing for a while. In such a case the *count* parameter specifies the number of pseudo code instructions to execute before switching to another coroutine. A larger value will cause the coroutine to execute for a longer time interval before doing the switch. A switch to another coroutine can be done earlier by executing one of the COROUTINE RESUME or COROUTINE TERMINATE statements.

A coroutine terminates when *statement* returns (because of a RETURN statement being executed or because of an uncaught exception), when it executes a COROUTINE TERMINATE statement, or when another coroutine executes a COROUTINE KILL statement with the coroutine's ID as parameter. When a coroutine terminates because of a return from *statement*, the parent coroutine (the one that created the terminated coroutine) resumes execution or, if the parent coroutine doesn't exist, the previously running coroutine.

The EXIT statement always terminates a program. A program also terminates if the coroutine with ID zero (the main coroutine) terminates for any reason.

Errors

The following value can be assigned to `$Error`:

```
$Error_CoroutineNewInvalidCount
```

The *count* parameter is negative.

Examples

If the following program is executed:

```
DECLARE VARIABLE $id INTEGER;

COROUTINE NEW $id CALL EXAMPLE "Other";
PRINT "Main coroutine 1\n";
COROUTINE RESUME $id;
PRINT "Main coroutine 2\n";
COROUTINE RESUME $id;
EXIT;

DECLARE STATEMENT
EXAMPLE IN $name PRIMITIVE
{
    DECLARE VARIABLE $i INTEGER;

    $i = 1;
    WHILE 1 {
        PRINT $name & " coroutine " & $i & "\n";
        COROUTINE RESUME 0;
        $i = $i + 1;
    }
    RETURN;
}
```

it will output the following:

```
Main coroutine 1
Other coroutine 1
Main coroutine 2
Other coroutine 2
```

See also

COROUTINE, COROUTINE KILL, COROUTINE LOCK, COROUTINE RESUME, COROUTINE TERMINATE.

12.4.25 COROUTINE RESUME

Name

COROUTINE RESUME – Resume execution of another coroutine.

Syntax

COROUTINE RESUME *id*;

Description

The COROUTINE RESUME statement stops execution of the currently executing coroutine and resumes execution of the coroutine specified by the *id* parameter. The *id* parameter is an *expression* that is converted to an integer. It should have been returned from a call to COROUTINE NEW or have the value zero (the main coroutine).

Errors

The following value can be assigned to `$Error`:

- `$Error_CoroutineResumeInvalidId`
Invalid coroutine ID.
- `$Error_CoroutineResumeNoExistId`
The coroutine ID specified by *id* does not exist.

Examples

See the COROUTINE NEW statement, page 113.

See also

COROUTINE, COROUTINE KILL, COROUTINE LOCK, COROUTINE NEW, COROUTINE TERMINATE.

Notes

If the *id* parameter specifies the same coroutine as the one already executing, the calling coroutine just continues executing.

12.4.26 COROUTINE TERMINATE

Name

COROUTINE TERMINATE – Terminate the execution of another coroutine.

Syntax

COROUTINE TERMINATE *id*;

Description

The COROUTINE TERMINATE statement terminates the currently executing coroutine and resumes execution of the coroutine specified by the *id* parameter. The *id* parameter is an *expression* that is converted to an integer. It should have been returned from a call to COROUTINE NEW or have the value zero (the main coroutine).

An error is generated if the statement is executed from within the main coroutine (coroutine ID zero) or if *id* specifies the currently executing coroutine.

Errors

The following values can be assigned to `$Error`:

- `$Error_CoroutineTerminateInvalidId`
Invalid coroutine ID.
- `$Error_CoroutineTerminateNoExistId`
The coroutine ID specified by *id* does not exist.
- `$Error_CoroutineTerminateMainNotAllowed`
Trying to terminate the main coroutine, that is, *id* is zero.
- `$Error_CoroutineTerminateNotOurselves`
Trying to terminate ourselves, that is, *id* specifies the currently executing coroutine.

Examples

If the following program is executed:

```
DECLARE VARIABLE $id INTEGER;

COROUTINE NEW $id CALL EXAMPLE "Other";
PRINT "Main coroutine 1\n";
COROUTINE RESUME $id;
PRINT "Main coroutine 2\n";
COROUTINE RESUME $id;
EXIT;

DECLARE STATEMENT
EXAMPLE IN $name PRIMITIVE
{
    PRINT $name & " coroutine 1\n";
    COROUTINE RESUME 0;
    PRINT $name & " coroutine 2\n";
    COROUTINE TERMINATE 0;
    RETURN;
}
```

it will output the following:

```
Main coroutine 1
Other coroutine 1
Main coroutine 2
Other coroutine 2
```

See also

COROUTINE, COROUTINE KILL, COROUTINE LOCK, COROUTINE NEW, COROUTINE RESUME.

12.4.27 CURRENTDATE

Name

CURRENTDATE – Function that returns the current date and time.

Syntax

CURRENTDATE()

Description

The CURRENTDATE function returns the local current date and time as a DATE value.

The CURRENTDATE function is valid in *const-expressions*. The value returned in such cases is the date and time when the program was compiled.

Examples

If the current date is February 1, 1999, the PRINT statement:

```
PRINT DATE(CURRENTDATE()), "%Y-%m-%d") & "\n";
```

will write the following to the output file:

```
1999-02-01
```

See also

DATE.

12.4.28 DATA ELEMENT (EDIFACT)

Name

DATA ELEMENT – Read an EDIFACT EDI data element from the input file.

Syntax

```
{CONDITIONAL | MANDATORY} [FIXED | VARIABLE] NUMBER  
  [min,] max [VALIDATE format] [SET destination [VERIFY verify]]  
  {; | statement-block}
```

```
{CONDITIONAL | MANDATORY} [FIXED | VARIABLE] STRING  
  [min,] max [VALIDATE format] [SET destination [VERIFY verify]]  
  {; | statement-block}
```

Description

The DATA ELEMENT compound statement reads an EDIFACT EDI data element from the input file and matches the data with the specification.

The NUMBER and STRING keywords specify the type of data element. The *min*, *max*, and *verify* parameters are *expressions* that are converted to integers, *format* is an *expression* that is converted to a string and *destination* is a *variable*.

If the VARIABLE modifier is used, the valid length of the element is specified by *min* and *max*. If *min* is omitted, all lengths less than or equal to *max* are valid. If the FIXED modifier is used, only *max* may be specified and the input data must be exactly *max* characters long. VARIABLE is the default.

If reading a STRING data element, the *min* and *max* parameters are specified in units of characters instead of bytes (this makes a difference when reading multi-byte character data). For the NUMBER element type, *min* and *max* are specified in units of bytes.

If *format* is specified, it is used to validate the input data. The contents of the input data must match the format specified by *format*. See the descriptions of the different data types for a description of how *format* is used.

If the SET modifier is used, the data element read from the input file will be assigned to the variable specified by *destination*. If an error occurs, the contents of *destination* are undefined.

The VERIFY modifier is used to specify an expression that must evaluate to a true (non-zero) value. The *destination* variable can be used in this expression. If the *verify* expression evaluates to 0, an EDI error is generated.

If the MANDATORY modifier is specified and the input data does not match the data element description, an EDI error is generated.

If the input data match the data element description, the *statement-block*, if supplied, is executed and the \$EdiElemCnt or \$EdiCompCnt reserved variable is incremented by one, depending on whether the element is outside or inside a composite element, respectively.

- NUMBER

The NUMBER data element statement reads an EDI element from the input file and checks whether the input is a decimal number. A valid number consists of characters in the range 0 – 9, an optional comma (,) or dot (.) representing the decimal point and an optional leading minus sign (–) denoting a negative number. No other characters are allowed. The length specification of the data element does not include the minus sign and the decimal point or comma. If *format* is specified, it should contain the number that the data element contains to be considered valid. Note that NUMBER data elements must start and end with characters in the range 0 – 9 (not counting the minus sign).

Examples of valid numbers are:

```
25.2   -123,2 6237316   0
```

- STRING

The STRING data element statement reads a string EDI element from the input file. If *format* is specified, it should contain the string that the data element should contain, in order to be considered valid.

Errors

The following values can be assigned to \$Error:

- `$Error_ElementMinTooSmall`
The min length must be greater than or equal to 1.
- `$Error_ElementMinGtMax`
The min length is greater than the max length.
- `$Error_ElementEof`
An unexpected end-of-file was encountered. This is an EDI error.
- `$Error_ElementReadError`
Read error on input file.
- `$Error_ElementNone`
No element found when expecting one. This is an EDI error.
- `$Error_ElementNoMandatory`
Mandatory element is missing. This is an EDI error.
- `$Error_ElementWrong`
The data element does not match the specification. This is an EDI error.
- `$Error_ElementCscError`
Error related to multi-byte character handling.
- `$Error_ElementPartialError`
Incomplete multi-byte character.
- `$Error_ElementCharacterError`
The read data contains a character that is not valid according to the `$EdiCharValid` reserved variable.
- `$Error_Pgmlogic_ElementOutsideSegment`
DATA ELEMENT statement executed outside a SEGMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_ElementInsideElement`
DATA ELEMENT statement executed inside another DATA ELEMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_ElementInsideRelation`
DATA ELEMENT statement executed inside a RELATION statement. This is a PGMLOGIC error.

Examples

A NUMBER data element example:

```
DECLARE VARIABLE $exchangeRate FLOAT;

CONDITIONAL NUMBER 2, 6 SET $exchangeRate
    VERIFY $exchangeRate < 2.0 {
    PRINT "Rate is " & FORMAT("%.2f", $exchangeRate) & "\n";
}
MANDATORY FIXED NUMBER 4;
```

A STRING data element example:

```
DECLARE VARIABLE $name STRING;

CONDITIONAL STRING 12 SET $name {
    PRINT "Name is " & $name & "\n";
}
MANDATORY FIXED STRING 6 VALIDATE "PRICAT";
```

See also

COMPOSITE, SEGMENT, SEGMENT GROUP.

Notes

The OPTIONAL modifier can be used instead of CONDITIONAL. Use CONDITIONAL when parsing EDIFACT messages and OPTIONAL when parsing X12 messages.

This statement can only be used inside segments, and inside composite elements.

The Message Builder compiler attempts to check for this. However, if user functions or statements are used, the compiler may not detect this. The error will instead be detected at run time and then be treated as a PGMLOGIC error; that is, the Message Builder interpreter will terminate if the error is not trapped using the TRY statement (see paragraph [12.4.131](#)).

12.4.29 DATA ELEMENT (X12)

Name

DATA ELEMENT – Read an X12 EDI data element from the input file.

Syntax

```
{OPTIONAL | MANDATORY} [FIXED | VARIABLE] BINARY
    [min,] max [SET [FILE] destination] {; | statement-block}

{OPTIONAL | MANDATORY} [FIXED | VARIABLE] DATE
    [min,] max [VALIDATE format] [SET destination [VERIFY verify]]
    {; | statement-block}

{OPTIONAL | MANDATORY} [FIXED | VARIABLE] DECIMAL
    [min,] max [VALIDATE format] [SET destination [VERIFY verify]]
    {; | statement-block}

{OPTIONAL | MANDATORY} [FIXED | VARIABLE] NUMERIC
    [min,] max [IMPLIED position] [VALIDATE format]
    [SET destination [VERIFY verify]] {; | statement-block}

{OPTIONAL | MANDATORY} [FIXED | VARIABLE] STRING
    [min,] max [VALIDATE format] [SET destination [VERIFY verify]]
    {; | statement-block}

{OPTIONAL | MANDATORY} [FIXED | VARIABLE] TIME
    [min,] max [VALIDATE format] [SET destination [VERIFY verify]]
    {; | statement-block}
```

Description

The DATA ELEMENT compound statement reads an X12 EDI data element from the input file and matches the data with the specification.

The BINARY, DATE, DECIMAL, NUMERIC, STRING, and TIME keywords specify the type of data element. The *min*, *max*, *position*, and *verify* parameters are *expressions* that are converted to integers, *format* is an *expression* that is converted to a string and *destination* is a *variable*.

If the VARIABLE modifier is used, the valid length of the element is specified by *min* and *max*. If *min* is omitted, all lengths less than or equal to *max* are valid. If the FIXED

modifier is used, only *max* may be specified and the input data must be exactly *max* characters long. VARIABLE is the default.

If reading a STRING data element, the *min* and *max* parameters are specified in units of characters instead of bytes (this makes a difference when reading multi-byte character data). For all the other element types, *min* and *max* are specified in units of bytes.

The IMPLIED modifier is used to specify the position for the implied decimal point in a NUMERIC data element. The *position* parameter specifies the number of implied decimal positions. The value put in *destination* has the implied decimal point inserted. Note that the IMPLIED modifier is valid only in the NUMERIC data element statement. The default value for *position* is 0.

If *format* is specified, it is used to validate the input data. The contents of the input data must match the format specified by *format*. See the descriptions of the different data types for a description of how *format* is used.

If the SET modifier is used, the data element read from the input file will be assigned to the variable specified by *destination*. When reading a binary data element, the FILE modifier may be used to specify that the data should be put in a file. In this case, the value assigned to *destination* will be the name of the temporary file containing the data. If an error occurs, the contents of *destination* are undefined.

The VERIFY modifier is used to specify an expression that must evaluate to a true (non-zero) value. The *destination* variable can be used in this expression. If the *verify* expression evaluates to 0, an EDI error is generated.

If the MANDATORY modifier is specified and the input data does not match the data element description, an EDI error is generated.

If the input data match the data element description, the *statement-block*, if supplied, is executed and the \$EdiElemCnt or \$EdiCompCnt reserved variable is incremented, depending on whether the element is outside or inside a composite element, respectively. The increment is two for binary data elements and one for the other data element types.

- BINARY

The BINARY data element statement covers two data elements in the input file. The two elements may, but need not be, a composite data element.

The first element must hold a number specifying the length of the second element. This length is checked and must be inside the range specified by *min* and *max*. The second element can hold any binary data, including separators, etc.

If the SET FILE modifier is specified, the binary data will be written to a temporary file and the filename will be assigned to the variable specified by *destination*.

- DATE

The DATE data element statement reads an EDI element from the input file and checks whether the input is a date string matching the format specified by *format*. If *format* is omitted, the X12 default format "%Y%m%d" is used. See the description of the \$DateFormat reserved variable in the RESERVED VARIABLES section, paragraph [12.4.101](#), for a description of the contents of *format*.

Examples of format strings and valid EDI data are:

```
"%Y-%m-%d"    1993-11-17
"%Y%m%d"      931117
```

- DECIMAL

The DECIMAL data element statement reads an EDI element from the input file and checks whether the input is a decimal number. A valid decimal number consists of characters in the range 0 – 9, an optional dot (.) representing the decimal point and an optional leading plus (+) or minus (–) sign denoting a positive or negative number, respectively. No other characters are allowed. The length specification for the data element does not include the sign and the decimal point. If *format* is specified, it should contain the number that the data element should contain to be considered valid. Unlike EDIFACT NUMBER data elements (see paragraph [12.4.28](#)), DECIMAL data elements need not start and end with numerical digits.

Examples of valid decimal numbers are:

```
25.2   -123.2 6237316   0
```

- **NUMERIC**

The NUMERIC data element statement reads an EDI element from the input file and checks whether the input is an integral number. A valid numeric consists of characters in the range 0 – 9 and an optional leading plus or minus sign denoting a positive or negative number, respectively. No other characters are allowed. The length specification for the data element does not include the sign. If *format* is specified, it should contain the numeric that the data element should contain to be considered valid.

Examples of valid numeric values are:

```
123-450
```

- **STRING**

The STRING data element statement reads a string EDI element from the input file. If *format* is specified, it should contain the string that the data element should contain in order to be considered valid.

- **TIME**

The TIME data element statement reads an EDI element from the input file and checks whether the input is a time string matching the format specified by *format*. If *format* is omitted, the X12 default format "%H%M%S%t" is used. See the description of the `$DateFormat` reserved variable in the RESERVED VARIABLES section, paragraph [12.4.101](#), for a description of the contents of *format*.

Examples of format strings and valid EDI data:

```
"%H%M%S%t"   15564097   155640 1556
"%h%M%S%p"   035640PM
```

Errors

The following values can be assigned to `$Error`:

- `$Error_ElementMinTooSmall`
The min length must be greater than or equal to 1.
- `$Error_ElementMinGtMax`
The min length is greater than the max length.
- `$Error_ElementEof`
An unexpected end-of-file was encountered. This is an EDI error.
- `$Error_ElementCreateError`
Failed to create temporary file. This can only happen with binary data elements for which the FILE modifier has been specified.
- `$Error_ElementReadError`
Read error on input file.
- `$Error_ElementWriteError`
Write error on output file. This can only happen with binary data elements for which the FILE modifier has been specified.

- `$Error_ElementNone`
No element found when expecting one. This is an EDI error.
- `$Error_ElementNoMandatory`
Mandatory element is missing. This is an EDI error.
- `$Error_ElementWrong`
The data element does not match the specification. This is an EDI error.
- `$Error_ElementIllegalLength`
Illegal length of binary data element. This is an EDI error.
- `$Error_ElementWrongLength`
Wrong length of binary data element. This is an EDI error.
- `$Error_ElementWrongSeparator`
Wrong separator in binary data element. This is an EDI error.
- `$Error_ElementSeparatorError`
Not a separator after a binary data element. This is an EDI error.
- `$Error_ElementFailTempFile`
Failed to generate a temporary file name. This can only happen with binary data elements for which the FILE modifier has been specified.
- `$Error_ElementCscError`
Error related to multi-byte character handling.
- `$Error_ElementPartialError`
Incomplete multi-byte character.
- `$Error_ElementCharacterError`
The read data contains a character that is not valid according to the `$EdiCharValid` reserved variable.
- `$Error_Pgmlogic_ElementOutsideSegment`
DATA ELEMENT statement executed outside a SEGMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_ElementInsideElement`
DATA ELEMENT statement executed inside another DATA ELEMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_ElementInsideRelation`
DATA ELEMENT statement executed inside a RELATION statement. This is a PGMLOGIC error.

Examples

A BINARY data element example:

```
DECLARE VARIABLE $fileName STRING;

OPTIONAL SEGMENT "BIN" {
    MANDATORY BINARY 10000 SET FILE $fileName {
        PRINT "Binary data is stored in " & $fileName & "\n";
    }
}
```

A DATE data element example:

```
DECLARE VARIABLE $date DATE;

OPTIONAL FIXED DATE 6 SET $date {
    PRINT "Date is " & DATE($date, "%Y-%m-%d") & "\n";
}
```

A DECIMAL data element example:

```

DECLARE VARIABLE $exchangeRate FLOAT;

OPTIONAL DECIMAL 2, 6 SET $exchangeRate {
    PRINT "Rate is " & FORMAT("%.2f", $exchangeRate) & "\n";
}
MANDATORY FIXED DECIMAL 4;

```

A NUMERIC data element example:

```

DECLARE VARIABLE $msgId INTEGER;

OPTIONAL NUMERIC 2, 6 SET $msgId VERIFY $msgId <> 0 {
    PRINT "Message id is " & $msgId & "\n";
}
MANDATORY FIXED NUMERIC 4;

```

A STRING data element example:

```

DECLARE VARIABLE $name STRING;

OPTIONAL STRING 12 SET $name {
    PRINT "Name is " & $name & "\n";
}
MANDATORY FIXED STRING 6 VALIDATE "PRICAT";

```

A TIME data element example:

```

DECLARE VARIABLE $time DATE;

OPTIONAL FIXED TIME 6 VALIDATE "%H%M%S" SET $time {
    PRINT "Time is " & DATE($time, "%H.%M.%S") & "\n";
}

```

See also

BOUNDED LOOP, COMPOSITE, SEGMENT, SEGMENT GROUP, UNBOUNDED LOOP.

Notes

The **CONDITIONAL** modifier can be used instead of **OPTIONAL**. Use **CONDITIONAL** when parsing EDIFACT messages and **OPTIONAL** when parsing X12 messages.

This statement can only be used inside segments and inside composite elements. The Message Builder compiler attempts to check for this. However, if user functions or statements are used, the compiler may not detect this. The error will instead be detected at run time and then be treated as a PGMLOGIC error; that is, the Message Builder interpreter will terminate if the error is not trapped using the **TRY** statement (see paragraph [12.4.131](#)).

12.4.30 DATABASE CLOSE

Name

DATABASE CLOSE – Close a database connection.

Syntax

DATABASE CLOSE [DBID *dbid*];

Description

The DATABASE CLOSE statement closes a database connection previously opened using the DATABASE OPEN statement.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

If a transaction is active when DATABASE CLOSE is executed, any changes to the database are automatically committed.

Errors

The following values can be assigned to `$Error`:

- `$Error_DbCloseNotOpen`
The database is not open.
- `$Error_DbCloseCloseError`
An error occurred while closing the database.

Examples

See the description of DATABASE OPEN (paragraph [12.4.32](#)) for an example.

See also

DATABASE OPEN.

12.4.31 DATABASE EXECUTE

Name

DATABASE EXECUTE – Send a string to the RDBMS for execution.

Syntax

DATABASE EXECUTE *database-command* [DBID *dbid*];

Description

The DATABASE EXECUTE statement sends the *database-command* string to the database engine for execution. The *database-command* parameter is an *expression* that is converted to a string.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Special commands handled locally (i.e. not sent to the database server)

- **AUTOCOMMIT ON**
Act as if EXEC SQL COMMIT WORK was added after each other SQL statement. Not the default. This feature is available only on DB2, ODBC, MySql and Oracle database interfaces.
- **AUTOCOMMIT OFF**
Do not commit the changes to the database until EXEC SQL COMMIT WORK is issued. This is the default.
- **SET 4EDI CODEPAGE=*codepagenumber***
Available only on Oracle interfaces. Declare the codepage in which strings sent to the database server are encoded, and request the codepage in which strings received from the database server should be encoded. The codepage numbers are not IBM or Windows codepage number but Oracle codepage numbers, available in Oracle documentation; for example utf8 is 873.
- **ASK 4EDI IF CONNECTION HAS ORACLE TRANSPARENT APPLICATION FAILOVER ENABLED**
This is a debug feature; it will log a message (in the Trace Viewer) indicating whether the Oracle client has successfully set up TAF after connecting to an Oracle RAC (Real Application Cluster). It has no other effect.
- **PING**
Available only on MySQL. Check if the database connection still works and attempt a reconnection if not.

All of these special statements are case sensitive.

Errors

The following values can be assigned to `$Error`:

- `$Error_DbExecuteNoSupport`
The DATABASE EXECUTE statement is not supported.
- `$Error_DbExecuteNotOpen`
The database is not open.
- `$Error_DbExecuteExecError`
Error when executing *database-command*.

Examples

Creating a table in an Oracle database may look like:

```
DATABASE EXECUTE "CREATE TABLE Employee " &
"(Name CHAR(30), Address CHAR(30))";
```

See also

DATABASE OPEN.

12.4.32 DATABASE OPEN

Name

DATABASE OPEN – Open a database connection.

Syntax

DATABASE OPEN *specification* [TYPE *type*] [DBID *dbid*];

Description

A DATABASE OPEN statement must be executed before any of the other DATABASE or SQL statements are executed.

The *specification* parameter is an *expression* that is converted to a string. The format of the string depends on the RDBMS being used:

- For Informix databases, the format of *specification* is *database-name@server-name*.
The *@server-name* part is optional. For Informix 7.x.x SE databases, *specification* can be augmented by the path for the database directory, that is, *pathname/database-name@server-name*.
- For MySQL drivers, the format of *specification* is *user/password/database/host/port/unix-socket*. All fields are optional.
- For DB2 and ODBC databases, the format of *specification* is *data-source/user/password/driver-name/driver-data*. */Driver-name* and */driver-data* are optional. If the slash character must be used in one of the five fields mentioned the syntax is DATABASE OPEN @sqlattrs(sepfirst) "#data-source#user#password#driver-name#driver-data"; where # can be replaced with any character not used in the fields.
- For Oracle databases, the format of *specification* is *user/password@hostname*. The *@hostname* part is optional. The hostname can be replaced with the name of an entry in the tnsnames.ora file.
WARNING: The Oracle database client might have many bugs depending on version; for example, until Oracle 11.2 log in attempts during the password expiration grace period cause a crash, in session pool mode, and either a crash or a failure to log in, instead of just a warning, in normal mode. In the latter case, DATABASE CLOSE prints an invalid handle error, and any EXEC SQL statements may repeat the warning as an error or output an invalid handle error. Also, memory may be corrupted by the Oracle client.
Therefore, always use the latest Oracle client from the paid Oracle Support site not the free Oracle Download site.
The Oracle Support site also contains a matrix showing which client versions can connect to which server versions.
- For Sybase databases, the format of *specification* is *user/password/database-name/server-name/character-set*. All fields except the *user* field are optional. Trailing slashes may be omitted.
- For the AS/400 Native database, the format of *specification* is *library-name/options*.
The *options* field is optional and a trailing slash may be omitted. The *library-name* field specifies the location of an SQL package object, which is created by the AS/400 SQL interface. The library name for the SQL package is usually specified as QTEMP. The *options* field is used to specify the database options. If this field is not specified, the options are taken from the DB_FORMATS environment variable. See the NOTES section for a description of the contents of the *options* field.

The *type* parameter is an *expression* that is converted to a string. It is used to tell the interpreter which database interface to use. Recognized values are *Db2_81* (DB2 8.x and later), *Informix_73* (Informix 7.3.x and later), *Informix_92* (Informix 9.2 and later, on certain platforms), *MySQL_41* (MySQL 4.1 native driver, on certain platforms), *MySQL_50* (MySQL 5.0 and later native driver, on certain platforms), *Odbc_2* (ODBC 2.x), *Odbc_3* (ODBC 3.x), *Odbc3_UnixODBC_post_2_2_14* (on 64-bit Unix platforms the UnixODBC

ABI has changed in version 2.2.14 and later – the DB2 drivers require the new ABI), *Oracle_81* (on Windows only, name kept to avoid breaking existing applications; it is actually an Oracle 9.2 or later interface), *Oracle_92* (Oracle 9.2.x), *Oracle_101* (Oracle 10.1.x and later, including Oracle 11 on platforms without a \$CORE_ROOT/4edi/dbload/oracle111.db; tested up to Oracle 12.1), *Oracle_111* (Oracle 11.1.x and later), *OS400_SQL* (AS/400 Native), *Sybase_111* (Sybase 11.1) and *Sybase_1251* (Sybase 12.5.1 and later).

The case does not matter. See the description of the EDI_DB environment variable (page 78) for further details.

The *dbid* parameter is a variable that receives an integer identifying the database connection established by the DATABASE OPEN statement. For most databases (the exception is the AS/400 native database), a Message Builder program can have multiple connections open simultaneously. The identifier returned in the *dbid* variable is used to identify the connection when executing DATABASE and SQL statements. If there were no database connections open before the DATABASE OPEN statement was executed, the *dbid* variable will receive a value of 0. Since the default value of the database identifier for all DATABASE and SQL statements is 0, the *dbid* parameter does not need to be used if a program only uses one simultaneous database connection.

Errors

The following values can be assigned to `$Error`:

- `$Error_DbOpenUnknownType`
There is no support for the specified database type.
- `$Error_DbOpenTooMany`
Trying to open too many connections to the database.
- `$Error_DbOpenInitError`
Error when initializing the database.
- `$Error_DbOpenOpenError`
Error when opening the connection to the database.
- `$Error_DbOpenOpenDirectory`
Failed to open a directory when searching for the requested database interface.
- `$Error_DbOpenReadDirectory`
Directory read error when searching for the requested database interface.
- `$Error_DbOpenLoadInterface`
Failed to load a database interface when searching for the requested interface.
- `$Error_DbOpenNoInit`
Found a file with a *.db* extension that is not a database interface.
- `$Error_DbOpenWrongVersion`
Found an old and no longer supported database interface.

Examples

Opening an Informix database may look like:

```
DATABASE OPEN "dbuser";
```

Opening an Oracle or Sybase database may look like:

```
DATABASE OPEN "dbuser/dbpasswd";
```

See also

DATABASE CLOSE.

Notes

The database interfaces are always supplied as dynamically loadable shared libraries (DLLs on Windows systems). On a standard Integration Broker, the database interfaces are located in the `$CORE_ROOT/4edi/dbload` directory.

Previous versions of the 4edi interpreter used to reuse database connections when several programs called DATABASE OPEN with the same specification and without calling DATABASE CLOSE. For reasons of preserving the state of database connections, this is no longer the behavior of the 4edi interpreter. If you want to cache connections to an Oracle database see the information on session pooling support below.

When using the ODBC database interface, the *specification* string does not depend on the database really used. For example, if ODBC is used together with an Oracle database, the *specification* string must be of the format specified for ODBC, not for Oracle.

Environment variables used with various database interfaces

All these environment variables are still applicable even if EXPORTed immediately before DATABASE OPEN; this allows, for example, different settings to be used for different Oracle session pools. The default value refers to the functionality with the environment variable unset.

When using *Odbc_30*, the EDI_CLOB_THRESHOLD can be used to support large strings. It must be set to the maximum string size of the underlying database (8000 for SQL Server, 32672 for remotely connecting to OS/400 databases). On SQL Server, you also need to use casts around large strings:

CAST(:\$big_string AS varchar(max)) for input parameters

SELECT CAST(looong AS varchar(max)) for output parameters.

The environment variable EDI_ODBC_API_CALLS_LOG can be set to "on" to enable logging of all calls to ODBC.

The Informix 7.x.x interface uses Informix-ESQL/C. If the EDI_INFORMIX7_EMBEDDED_SQL_LOG environment variable is set to "on", all executed embedded SQL statements are output as debug log messages.

The Oracle interfaces use the Oracle Call Interface. If the EDI_ORACLE_OCI_CALLS_LOG environment variable is set to "on", all OCI calls are output as debug log messages.

Setting EDI_ORACLE_OCI_EXT_CRED_IF_EMPTY=on will enable non-password authentication if DATABASE_OPEN is also used with the username and password fields both empty. This seems to be an obsolete feature provided by Oracle, since it does not work with session pooling.

When connecting to an Oracle RAC, you may want to enable aq_ha_notifications using DBMS_SERVICE.MODIFY_SERVICE, to ensure connections are closed and switch to a new node as soon as a node goes down and not upon issuing the first SQL request.

Oracle session pooling (as said before, this replaces the connection pooling of 4edi for the Oracle database interface):

- set EDI_ORACLE_OCI_SESSION_POOL_ENABLE=1 if you want to enable session pooling. None of the other variables will have any effect if session pooling is not enabled. The other variables are not required to enable

session pooling. One session pool is created for each user/password/database combination used with DATABASE OPEN, and then a session from the corresponding pool is returned for each call of DATABASE OPEN. The default is disabled.

- EDI_ORACLE_OCI_SESSION_POOL_MIN: number of sessions to always keep in one session pool even if none of them are active; the default is 0
- EDI_ORACLE_OCI_SESSION_POOL_INCR: number of new connections to create when all sessions in a pool are used and DATABASE OPEN is called; the minimum value and the default are 1
- EDI_ORACLE_OCI_SESSION_POOL_MAX: maximum number of sessions in one pool. This is a dangerous option to set since if the number of sessions is exceeded DATABASE OPEN will fail with an error. The default is unlimited.
- set EDI_ORACLE_OCI_SESSION_POOL_STMTCACHE=1 if you want to enable statement caching between the sessions in the pool; the default is disabled
- EDI_ORACLE_OCI_SESSION_POOL_STMTCACHE_SIZE: number of statements to cache if the previous environment variable is set to 1. When the Oracle client installed is version 9.2 this variable will not work (with a warning).
- EDI_ORACLE_OCI_SESSION_POOL_TIMEOUT: number of seconds of inactivity after which a session (on which DATABASE CLOSE was called) will be removed from then pool. The default is to only remove a session when the number of sessions is greater than the minimum. This is useful when a non-RAC server goes down (to clear the session pool faster), or to ensure the Oracle server is relieved after peak load.
- EDI_ORACLE_OCI_SESSION_POOL_RECYCLE_COUNT: this environment variable can be set to the number of DATABASE CLOSE statements for a particular session pool after which the session being closed by DATABASE CLOSE will be reconnected instead of being released to the pool. By default this feature is disabled. This is useful when connecting to Oracle RAC clusters and one of the nodes is restarted after a delay; and during that delay all the session pool sessions have closed with an error and then reconnected to the alive node thus defeating load balancing. An alternative is listener-side load balancing: see DBMS_SERVICE.MODIFY_SERVICE in Oracle documentation.

The Sybase interface uses the Open Client Client-Library (CT) and the CS-Library. If the EDI_SYBASE10_CT_CALLS_LOG environment variable is set to "on", all CT library calls are output as debug log messages and, if the EDI_SYBASE10_CS_CALLS_LOG environment variable is set to "on", all CS library calls are output as debug log messages. The EDI_SYBASE11_CS_IFILE, EDI_SYBASE125_CS_IFILE and EDI_SYBASE125 respectively can be set to indicate an alternate path for the Sybase interfaces file, and unlike the other environment variables these must be set before starting Integrator to be effective.

The MySQL native interface will log all its calls as debug log messages if the environment variable EDI_MYSQL_API_CALLS_LOG is set to "on".

The database options for the AS/400 Native database (according to the IBM SQL API QSQPRCED) are as follows:

- Options are given in the form of a string, 13 characters long. For details of each option, see the table below and the *IBM SQL Programmers Guide*.
- If commitment control is selected, only SQL tables, SQL views, and the SQL

package are affected. Files referred to by other Message Builder statements than SQL statements are not affected. Files must be journaled when commitment control is used.

The following table defines the contents of the option string:

Position	Possible Values	Description
1		Commitment control
	C	*CHG
	S	*CS
	A	*ALL
	N	*NONE
2-4		Date format
	USA	USA standard, mm.dd.yyyy
	ISO	ISO standard, yyyy-mm-dd
	EUR	European, dd.mm.yyyy
	JIS	Japanese, yyyy-mm-dd
	MDY	Mm/dd/yy
	DMY	Dd/mm/yy
	YMD	Yy/mm/dd
	JUL	Julian, yy/ddd
5	/ . , - blank	Date separator
6-8		Time format
	HMS	Hh:mm:ss
	USA	Hh:mm a.m., hh:mm p.m.
	ISO	Hh.mm.ss
	EUR	Hh.mm.ss
	JIS	Hh:mm:ss
9	: . , blank	Time separator
10-12	SYS, SQL	Naming convention (SYS is used for Integration Broker).
13	. ,	Decimal point

The commitment alternatives are (normally *NONE or *CHG are used):

Mode	Description
*CHG	Specifies the objects referred to in SQL ALTER, COMMENT ON, CREATE, DROP, GRANT, LABEL ON, and REVOKE statements and the rows updated, deleted, and inserted are locked until the end of the unit of work (transaction). Uncommitted changes in other jobs can be seen.
*CS	Specifies the objects referred to in SQL ALTER, COMMENT ON, CREATE, DROP, GRANT, LABEL ON, and REVOKE statements and the rows updated, deleted, and inserted are locked until the end of the unit of work (transaction). A row that is selected, but not updated, is locked until the next row is selected. Uncommitted changes in other jobs cannot be seen.
*ALL	Specifies the objects referred to in SQL ALTER, COMMENT ON, CREATE, DROP, GRANT, LABEL ON, and REVOKE statements and the rows selected, updated, deleted, and inserted are locked until the end of the unit of work (transaction). Uncommitted changes in other jobs cannot be seen.
*NONE	Specifies that commitment control is not used. Uncommitted changes in other jobs can be seen. If the SQL DROP COLLECTION statement is included in the program, *NONE must be used.

12.4.33 DATE

Name

DATE – Function that converts a date value to a string of the specified format.

Syntax

DATE(operand, format)

Description

The DATE function converts *operand* to a string of format *format*.

The *operand* argument is an *expression* that is converted to a DATE value and the *format* argument is an *expression* that is converted to a string.

The DATE value is then converted to a string using the *format* string as conversion specification. The result is returned. The format of the *format* string is the same as the format of the `$DateFormat` reserved variable. See the RESERVED VARIABLES section, page 233.

The DATE function is valid in *const-expressions*.

Examples

The PRINT statement:

```
PRINT DATE(CURRENTDATE(), "%Y-%m-%d %H.%M") & "\n";
```

may write the following to the output file:

```
2003-12-07 15.33
```

See also

CURRENTDATE, DATA ELEMENT (X12), RESERVED VARIABLES.

12.4.34 DEBUG

Name

DEBUG – Control the output of debug information.

Syntax

DEBUG {EDI | EXCEPTION | SQL | TRACE} {ON | OFF};

Description

The DEBUG statement enables (ON) or disables (OFF) the output of debug information.

The EDI modifier is used to control EDI parsing debug information. When EDI debug logging is enabled, information about all elements found in the input file is written to the debug log file.

The EXCEPTION modifier is used to control the output of exception debug information. When exception debug logging is enabled, information about all thrown and caught exceptions is written to the debug log file.

The SQL modifier is used to control SQL debug logging. When SQL debug logging is enabled, all DATABASE and SQL statements executed and all data read from the SQL database are written to the debug log.

The TRACE modifier is used to control trace information logging. Trace information is written to the trace log file (by default the standard error output) and it consists of the source code file names and line numbers corresponding to the code being executed. Note that trace only works if the Message Builder program has been compiled with the **T** option.

The debug information enabled by this statement is always output, even if the statement generating the output is inside a NOLOG statement block.

Examples

The first example demonstrates trace information logging: If the following program is stored in a file named *ex.s4*:

```
DEBUG TRACE ON;
FOR $i = 1 TO 3
{
    PRINT $i & "\n";
}
DEBUG TRACE OFF;
```

the following is written to the trace log file:

```
ex.s4 2 4 3 4 3 4 3 6
```

The second example demonstrates exception debug logging. If the following compiled program is stored in a file named *exc.x4*:

```
DEBUG EXCEPTION ON;
TRY {
    THROW "DEMO";
}
CATCH $exception
WHEN "DEMO" {}
DEBUG EXCEPTION OFF;
```

the following debug messages are written to the log:

```
DEBUG :20040130:13.50.05:r4edi(exc):DEBUG EXCEPTION: debug on
DEBUG :20040130:13.50.05:r4edi(exc):EXCEPTION: throw "DEMO"
DEBUG :20040130:13.50.05:r4edi(exc):EXCEPTION: caught "DEMO"
DEBUG :20040130:13.50.05:r4edi(exc):DEBUG EXCEPTION: debug off
```

The third example demonstrates SQL debug logging: If the following program is executed:

```
DECLARE VARIABLE $plus INTEGER;
DECLARE VARIABLE $minus INTEGER;
DECLARE VARIABLE $s STRING;
DECLARE VARIABLE $c1 INTEGER;
DECLARE VARIABLE $c2 INTEGER;
DECLARE VARIABLE $c3 STRING;
```

```

DEBUG SQL ON;
DATABASE OPEN "dbuser/dbpassword";
EXEC SQL CREATE TABLE DEMO (C1 INTEGER, C2 INTEGER, C3 CHAR(10));
EXEC SQL BEGIN WORK;
$plus = 1;
WHILE $plus <= 5 {
    $minus = -$plus;
    $s = $plus & $minus;
    EXEC SQL INSERT INTO DEMO (C1, C2, C3)
        VALUES (: $plus, : $minus, : $s);
    $plus = $plus + 1;
}
EXEC SQL COMMIT WORK;
EXEC SQL SELECT * INTO : $c1, : $c2, : $c3 FROM DEMO {
    PRINT $c1 & " " & $c2 & " " & $c3 & " " & "\n";
}
EXEC SQL DROP TABLE DEMO;
DATABASE CLOSE;
DEBUG SQL OFF;

```

the following debug messages are written to the log (note that all fields except the log message itself have been removed):

```

DEBUG SQL: debug on
DATABASE OPEN "dbuser/dbpassword"
EXEC SQL CREATE TABLE DEMO (C1 INTEGER,C2 INTEGER,C3 CHAR(10))
EXEC SQL BEGIN WORK
EXEC SQL INSERT INTO DEMO (C1,C2,C3) VALUES (:1,:2,:3) 1:0 -1:0 "1-1":0
EXEC SQL INSERT INTO DEMO (C1,C2,C3) VALUES (:1,:2,:3) 2:0 -2:0 "2-2":0
EXEC SQL INSERT INTO DEMO (C1,C2,C3) VALUES (:1,:2,:3) 3:0 -3:0 "3-3":0
EXEC SQL INSERT INTO DEMO (C1,C2,C3) VALUES (:1,:2,:3) 4:0 -4:0 "4-4":0
EXEC SQL INSERT INTO DEMO (C1,C2,C3) VALUES (:1,:2,:3) 5:0 -5:0 "5-5":0
EXEC SQL COMMIT WORK
EXEC SQL SELECT * FROM DEMO
EXEC SQL FETCH 1:0 -1:0 "1-1":0
EXEC SQL FETCH 2:0 -2:0 "2-2":0
EXEC SQL FETCH 3:0 -3:0 "3-3":0
EXEC SQL FETCH 4:0 -4:0 "4-4":0
EXEC SQL FETCH 5:0 -5:0 "5-5":0
EXEC SQL DROP TABLE DEMO
DATABASE CLOSE
DEBUG SQL: debug off

```

When a **DEBUG SQL ON** statement has been executed, all **DATABASE** and **EXEC SQL** statements executed are output to the log. Message Builder variables embedded in SQL statements are replaced by the `:n` construct. The first embedded variable is replaced by `:1`, the second by `:2`, and so forth. The values for the embedded variables follow after the SQL statement. Each value is of the form `v:i`, where `v` is the value and `i` the indicator value. An **EXEC SQL FETCH** line represents each row returned from the database as a result of a **SELECT** statement. The fetched values are output as `v:i`, where `v` is the fetched value and `i` the indicator value.

Notes

The debug information is by default written to the error log file that in turn, by default, is the standard error output. The **L** option to the Message Builder interpreter can be used to change this.

Trace information is by default written to the standard error output. The **N** option to the Message Builder interpreter can be used to select another output file.

The **E**, **S** and **T** options to the Message Builder interpreter can be used to initially enable EDI, SQL and TRACE debug logging, respectively, when starting a Message Builder program. If any of these options are given when the interpreter is invoked, the logging can be disabled using a DEBUG OFF statement in the program.

12.4.35 DECLARE EXTERNAL FUNCTION

Name

DECLARE EXTERNAL FUNCTION – Declare an external user function.

Syntax

DECLARE [PUBLIC] EXTERNAL FUNCTION *name* (*argument-list*) [*ret-type*];

Description

An external user function is a function, written using Message Builder code, which is loaded during run time using the LOADPGM statement. It is called in the same way as a built-in function, such as STRLEN.

The PUBLIC modifier only has meaning if the function is declared inside a module (see paragraph [12.4.38](#)).

The name of the function is *name* and *argument-list* is a list of zero or more arguments.

The function name, *name*, must start with an alphabetic character or an underscore character. The remaining characters may be alphabetic characters, decimal digits or underscore characters. There is no upper limit for the length of the name.

The list of formal arguments, *argument-list*, is a list of zero or more arguments that are the input values to the function. Commas separate the arguments. Each formal argument is of the format:

arg-var [*array-spec*] [= *const-expression*] [PRIMITIVE | *record-spec*]

The *arg-var* field is a variable name, which is used only for readability and does not have to be declared). A default value can be specified using the *const-expression* field. When calling a function, trailing arguments that have default values may be left out. These arguments will have the values specified by the default values when the function is entered. If an argument has a default value, all of the subsequent arguments must also have default values.

The default values specified in an external user function declaration override the default values specified in the declaration of the function actually being called.

By default, the formal argument types are determined by the type of the variables or expressions used when calling the external function. The *array-spec* field can be used to specify that an actual argument must be an array of a specific dimension. A number of [] pairs are used to specify the array dimension. The dimension equals the number of [] pairs. The PRIMITIVE keyword or the *record-spec* field is used to specify that the argument must be a scalar or a data record, respectively. The *record-spec* field is the keyword RECORD followed by a data record name. It is invalid to specify both the PRIMITIVE keyword and an *array-spec*. The compiler issues an error if the type of an actual argument does not match the specification. It is up to the programmer to make sure that the argument type specifications in the external function declaration are consistent with the original user function.

By default, the type of the value returned by the external function is determined at run time. The *ret-type* field can be used to inform the compiler about the type of the return value. The *ret-type* field has the following format:

```
{ PRIMITIVE | array-spec | record-spec | array-spec record-spec }
```

As before, the *array-spec* field is a number of [] pairs indicating the dimension of an array and the *record-spec* field is the keyword RECORD followed by a data record name. If the PRIMITIVE keyword is specified, the return value is a scalar. If *array-spec* and not *record-spec* is specified, the return value must be an array with non-record elements. If *record-spec* and not *array-spec* is specified, the return value is a record of the specified type. And finally, if both *record-spec* and *array-spec* are specified, the return value is an array of data records of the specified type.

The evaluation order for the arguments when an external user function is called is undefined.

External user functions do not have to be declared before they are called.

An external user function declaration is not allowed in a *statement-block* or in a user function or statement.

The DECLARE EXTERNAL FUNCTION declaration is used to inform the compiler on what the original function looks like, and therefore the order and number of arguments must match the original DECLARE FUNCTION declaration. For example, the external function declaration for the function:

```
DECLARE FUNCTION ABC($a, $b=0) { ... }
```

should look something like:

```
DECLARE EXTERNAL FUNCTION ABC($a, $b=0);
```

Examples

The following program loads an external Message Builder program named *ext.s4*, associates a function named EXT in the external program with the external function name ABC and calls the external function:

```
DECLARE EXTERNAL FUNCTION ABC($a);

LOADPGM "ext.s4";
USE FUNCTION "EXT" FROM "ext.s4" AS ABC;
PRINT ABC(5) & "\n";
```

If the contents of the *ext.s4* file are:

```
PRAGMA EXPORT FUNCTION EXT;
DECLARE PUBLIC FUNCTION EXT($x)
{
    DECLARE VARIABLE $x2 INTEGER;
    $x2 = $x * $x;
    RETURN $x2;
}
```

a value of 25 will be written to the output file.

In the next example, where the external function is located inside a module, the function name, in the USE and PRAGMA EXPORT statements, must be preceded by the module name:

```
DECLARE EXTERNAL FUNCTION ABC($a);

LOADPGM "ext.s4";
```



```
USE FUNCTION "MOD.EXT" FROM "ext.s4" AS ABC;
PRINT ABC(5) & "\n";
```

If the contents of the `ext.s4` file are:

```
PRAGMA EXPORT FUNCTION MOD.EXT;
DECLARE MODULE MOD {
    DECLARE PUBLIC FUNCTION EXT($x)
    {
        DECLARE VARIABLE $x2 INTEGER;
        $x2 = $x * $x;
        RETURN $x2;
    }
}
```

a value of 25 will be written to the output file.

By moving the `PRAGMA EXPORT` statement inside the module we do not have to prefix the function name with the module name (the calling program must however still use `MOD.EXT` in its `USE` statement):

```
DECLARE MODULE MOD {
    PRAGMA EXPORT FUNCTION EXT;
    DECLARE PUBLIC FUNCTION EXT($x)
    {
        DECLARE VARIABLE $x2 INTEGER;
        $x2 = $x * $x;
        RETURN $x2;
    }
}
```

The `PRAGMA` directive tells the compiler to save information that makes it possible to find the `EXT` or `MOD.EXT` function when calling it from another program. The **F** option to the Message Builder compiler or the `FIDENT` built-in function can be used instead of the `PRAGMA` directive.

The following external function declaration specifies that the return value is a scalar variable (`PRIMITIVE`) and that the first argument must be a one-dimensional array and the second a scalar:

```
DECLARE EXTERNAL FUNCTION EXTFUN($a[], $e = 1 PRIMITIVE) PRIMITIVE;
```

The following example shows how to use data record arguments and return values in external function declarations:

```
DECLARE RECORD ABC {
    DECLARE FIELD $a1 INTEGER;
    DECLARE FIELD $a2 INTEGER;
}
DECLARE RECORD DEF {
    DECLARE FIELD $d1 INTEGER;
    DECLARE FIELD $d2 INTEGER;
}
DECLARE EXTERNAL FUNCTION RECEXTFUN($a[] RECORD ABC) RECORD DEF;
```

The external function could also be declared as:

```
DECLARE EXTERNAL FUNCTION RECEXTFUN($a);
```

However, this prevents the compiler from doing type checks.

See also

DECLARE EXTERNAL STATEMENT, DECLARE FUNCTION, DECLARE RECORD, FIDENT, LOADPGM, PRAGMA, UNLOADPGM, USE.

Notes

In order to make it possible to check that the number of arguments is correct when an external function is called, the Message Builder compiler modifies the names of external functions. For example, the function name `FUNC` for the external declaration:

```
DECLARE EXTERNAL FUNCTION FUNC($a, $b);
```

is replaced by the name `F%FUNC:IN:IN`. The `F%` prefix indicates that it is a function and each `:IN` sequence indicates one (input) argument. The interpreter uses these modified names when referring to external functions, for example when a called external function cannot be found.

There are some restrictions on the name of an external user function; for example, an external function cannot have the same name as a built-in function, such as `STRLEN`. If an invalid name is used, the Message Builder compiler generates an error indicating that the external function name is not valid. See `DECLARE FUNCTION`, page 142, for a list of all names that are not valid as function names.

12.4.36 DECLARE EXTERNAL STATEMENT

Name

DECLARE EXTERNAL STATEMENT – Declare an external user statement.

Syntax

```
DECLARE [PUBLIC] EXTERNAL STATEMENT name parameter-list;
```

Description

An external user statement is a statement (procedure) written using Message Builder code, which is loaded during run time using the `LOADPGM` statement. It is called in the same way as a built-in statement, such as `EXPORT`.

The `PUBLIC` modifier only has meaning if the statement is declared inside a module (see paragraph [12.4.38](#)).

The name of the statement is *name* and *parameter-list* is a list of zero or more parameters.

The statement name, *name*, must start with an alphabetic character or an underscore character. The remaining characters may be alphabetic characters, decimal digits or underscore characters. There is no upper limit for the length of the name.

The list of formal parameters, *parameter-list*, is a list of zero or more parameters. Commas or noise words separate the parameters. A valid noise word is the same as a valid statement name. When calling an external user statement, the same noise words as the ones used in the external statement declaration must be used. If a comma was used in the declaration, a comma must be used in the call.

Each formal parameter can be either an IN, OUT, or IN OUT parameter. An IN parameter cannot be modified by the called external statement and looks like:

```
IN param-var [array-spec] [= const-expression] [PRIMITIVE | record-spec]
```

An OUT parameter is used to return values from the called statement and looks like:

```
OUT param-var [array-spec] [[= const-expression] DROP]  
[PRIMITIVE | record-spec]
```

Finally, an IN OUT parameter can be used both as an input and output parameter and it is specified as:

```
IN OUT param-var [array-spec] [= const-expression] [PRIMITIVE | record-spec]
```

The *param-var* fields are variable names, which are only used for readability and do not have to be declared. A default value can be specified for IN and IN OUT parameters using the *const-expression* field. The DROP modifier may be specified for an OUT variable. It causes the OUT value to be dropped if the caller does not supply a parameter.

When calling an external statement, parameters that have default values (an OUT parameter with DROP specified is considered to have a default value) may be left out. These parameters will have the values specified by the default values when the external user statement is entered. For OUT variables with DROP specified, the variables are by default of string type. The *const-expression* may be used to tell the Message Builder compiler what type the OUT parameter has when it is not specified in a call. Note that the value of *const-expression* does not matter, only the type matters.

The default values and DROP modifiers specified in an external user statement declaration override the default values and DROP modifiers specified in the declaration of the statement actually being called.

If the first formal parameter has a default value, all of the remaining parameters must also have default values. For the remaining parameters, if a parameter has a default value, all other parameters with the same noise word preceding them must also have default values.

Specifying a default value for an IN OUT parameter also means that the value is dropped when the default value is used.

When calling the external user statement, the parameters can be given in any order as long as the Message Builder compiler can uniquely determine the different parameters. More precisely, the rules are as follows:

- The parameter corresponding to the first formal parameter must always be given as the first parameter.
- For the remaining parameters, parameters that have equal noise words preceding them must be given in the same order as their corresponding formal parameters.

By default, the formal parameter types are determined by the type of the variables or expressions used when calling the external statement. The *array-spec* field can be used to specify that an actual parameter must be an array of a specific dimension. A number of [] pairs are used to specify the array dimension. The dimension equals the number of [] pairs. The PRIMITIVE keyword or the *record-spec* field is used to specify that the parameter must be a scalar or a data record, respectively. The *record-spec* field is the keyword RECORD followed by a data record name. It is invalid to specify both the PRIMITIVE keyword and an *array-spec*. The compiler issues an error if the type of an actual parameter does not match the specification. It is up to the programmer to make

sure that the parameter type specifications in the external statement declaration are consistent with the original user statement.

The evaluation order for the parameters when an external user statement is called is undefined.

External user statements need not be declared before they are called.

An external user statement declaration is not allowed in a *statement-block* or in a user statement or function.

The DECLARE EXTERNAL STATEMENT declaration is used to inform the compiler on what the original statement looks like, and therefore the order and number of parameters must match the original DECLARE STATEMENT declaration, except for the statement name and noise words. For example, the external statement declaration for the user statement:

```
DECLARE STATEMENT ABC IN $x1 X1 IN $x2 X2 IN $x3 { ... }
```

should look something like:

```
DECLARE EXTERNAL STATEMENT ABC IN $x1 X1 IN $x2 X2 IN $x3;
```

Examples

The following program loads an external Message Builder program named *ext.s4*, associates a statement named EXT in the external program with the external statement name ABC and calls the external statement:

```
DECLARE EXTERNAL STATEMENT ABC IN $p RESULT OUT $r;
DECLARE VARIABLE $result INTEGER;

LOADPGM "ext.s4";
USE STATEMENT "EXT" FROM "ext.s4" AS ABC;
ABC 5 RESULT $result;
PRINT $result & "\n";
```

If the contents of the *ext.s4* file are:

```
PRAGMA EXPORT STATEMENT EXT;
DECLARE STATEMENT EXT IN $x RES OUT $y
{
    $y = $x * $x;
    RETURN;
}
```

a value of 25 will be written to the output file.

In the next example, where the external statement is located inside a module, the statement name, in the USE and PRAGMA EXPORT statements, must be preceded by the module name:

```
DECLARE EXTERNAL STATEMENT ABC IN $p RESULT OUT $r;
DECLARE VARIABLE $result INTEGER;

LOADPGM "ext.s4";
USE STATEMENT "MOD.EXT" FROM "ext.s4" AS ABC;
ABC 5 RESULT $result;
PRINT $result & "\n";
```

If the contents of the *ext.s4* file are:

```
PRAGMA EXPORT STATEMENT MOD.EXT;
DECLARE MODULE MOD {
    DECLARE PUBLIC STATEMENT EXT IN $x RES OUT $y
```

```

    {
        $y = $x * $x;
        RETURN;
    }
}

```

a value of 25 will be written to the output file.

By moving the PRAGMA EXPORT statement inside the module we do not have to prefix the statement name with the module name (the calling program must however still use MOD.EXT in its USE statement):

```

DECLARE MODULE MOD {
    PRAGMA EXPORT STATEMENT EXT;
    DECLARE PUBLIC STATEMENT EXT IN $x RES OUT $y
    {
        $y = $x * $x;
        RETURN;
    }
}

```

The PRAGMA directive tells the compiler to save information that makes it possible to find the EXT or MOD.EXT statement when calling it from another program.

The **S** option to the Message Builder compiler or the SIDENT built-in function can be used instead of the PRAGMA directive.

The following external statement declaration specifies that the first parameter is a one-dimensional array and the second and third parameters are scalars:

```

DECLARE EXTERNAL STATEMENT EXTSTMNT IN $a[]
                                     ELEMENT IN $e = 1 PRIMITIVE
                                     OUTPUT OUT $out PRIMITIVE;

```

The following example shows how to use data record parameters in external statement declarations:

```

DECLARE RECORD ABC {
    DECLARE FIELD $a1 INTEGER;
    DECLARE FIELD $a2 INTEGER;
}
DECLARE RECORD DEF {
    DECLARE FIELD $d1 INTEGER;
    DECLARE FIELD $d2 INTEGER;
}
DECLARE EXTERNAL STATEMENT MUL2 IN $a RECORD ABC, OUT $b RECORD DEF;

```

The external statement could also be declared as:

```

DECLARE EXTERNAL STATEMENT MUL2 IN $a, OUT $b;

```

However, this prevents the compiler from doing type checks.

See also

DECLARE EXTERNAL FUNCTION, DECLARE STATEMENT, LOADPGM, PRAGMA, SIDENT, UNLOADPGM, USE.

Notes

In order to make it possible to check that the parameters are correct when an external statement is called, the Message Builder compiler modifies the names of external statements. For example, the statement name `STMNT` for the external declaration:

```

DECLARE EXTERNAL STATEMENT STMNT IN $a, OUT $b, IN OUT $c;

```

is replaced by the name `S%STMT:IN:OUT:INOUT`. The `S%` prefix indicates that it is a statement and the `:IN`, `:OUT` and `:INOUT` sequences specifies that there are three parameters. The first is an IN parameter, the second an OUT parameter, and the third an IN OUT parameter. The interpreter uses these modified names when referring to external statements, for example when a called external statement cannot be found.

When an error in a call to an external user statement is found, the parameter numbers in error messages from the Message Builder compiler correspond to the formal parameters in the external statement declaration.

There are some restrictions on the name of an external user statement; for example, an external user statement cannot have the same name as a built-in statement, such as `PRINT`. If an invalid name is used, the Message Builder compiler generates an error indicating that the statement name is not valid. See `DECLARE STATEMENT`, page 153, for a list of all names that are not valid as statement names.

The noise words separating the parameters do not have to be the same in the external user statement declaration and in the user statement declaration.

12.4.37 DECLARE FUNCTION

Name

`DECLARE FUNCTION` – Declare a user function.

Syntax

`DECLARE [PUBLIC] FUNCTION name (argument-list) [ret-type] statement-block`

Description

A user function is a function, written using Message Builder code, which can be used in the same way as a built-in function, such as `STRLEN`.

The `PUBLIC` modifier only has meaning if the function is declared inside a module (see paragraph [12.4.38](#)).

The name of the function is *name*, *argument-list* is a list of zero or more arguments and *statement-block* is the statements representing the code of the user function.

The function name, *name*, must start with an alphabetic character or an underscore character. The remaining characters may be alphabetic characters, decimal digits or underscore characters. There is no upper limit for the length of the name.

The list of formal arguments, *argument-list*, is a list of zero or more arguments that are the input values to the function. Commas separate the arguments. Each formal argument is of the format:

arg-var [*array-spec*] [= *const-expression*] [`PRIMITIVE` | *record-spec*]

The *arg-var* field is a variable name, which is automatically declared as a constant local variable to the function. It should not be declared separately. A default value can be specified using the *const-expression* field. When calling a function, trailing arguments that have default values may be left out. These arguments will have the values specified by the default values when the function is entered. If an argument has a default value, all of the subsequent arguments must also have default values.

By default, the formal argument types are determined by the type of the variables or expressions used when calling the function. The *array-spec* field can be used to specify that an actual argument must be an array of a specific dimension. A number of [] pairs are used to specify the array dimension. The dimension equals the number of [] pairs. The PRIMITIVE keyword or the *record-spec* field is used to specify that the argument must be a scalar or a data record, respectively. The *record-spec* field is the keyword RECORD followed by a data record name. It is invalid to specify both the PRIMITIVE keyword and an *array-spec*. The compiler issues an error if the type of an actual argument does not match the specification.

The evaluation order for the arguments when a user function is called is undefined.

A RETURN statement (see paragraph 12.4.102) may be placed anywhere in a function where another statement is valid. The last statement in a user function must be a RETURN, an EXIT, or a THROW. The RETURN statements must have one parameter, the return value. The type of the RETURN statement expression determines the type of value returned. If the *ret-type* field is specified, the compiler checks that the value returned is compatible with *ret-type*. The *ret-type* field has the following format:

```
{ PRIMITIVE | array-spec | record-spec | array-spec record-spec }
```

As before, the *array-spec* field is a number of [] pairs indicating the dimension of an array and the *record-spec* field is the keyword RECORD followed by a data record name. If the PRIMITIVE keyword is specified, the return value must be a scalar. If *array-spec* and not *record-spec* is specified, the return value must be an array with non-record elements. If *record-spec* and not *array-spec* is specified, the return value must be a record of the specified type. And finally, if both *record-spec* and *array-spec* are specified, the return value must be an array of data records of the specified type.

Note: In order to access fields of data records or arrays of data records supplied as arguments, the arguments must be declared to be data records or arrays of data records, respectively.

All variables declared in *statement-list* are local to the user function. The initial values when the function is entered are as follows:

- For variables with initial values, the initial value itself.
- For variables with no initial value specified:
 - Zero for INTEGER and FLOAT variables.
 - A string filled with spaces for CHAR variables.
 - An empty string for STRING variables.
 - The computer's base time for DATE variables. See the DECLARE VARIABLE statement, page 159, for an explanation of the computer's base time.
 - An empty array for array variables.
 - A record with all INTEGER and FLOAT fields set to zero, all CHAR fields set to strings filled with spaces, all STRING fields set to empty strings, all DATE fields set to the computer's base time, and all array fields set to empty arrays.

User functions need not be declared before they are called.

Recursion is supported, that is, a function may call itself directly or indirectly.

A user function declaration is not allowed in a *statement-block* or in another user function or statement.

Examples

A user function named EXAMPLE having two arguments may look like:

```

DECLARE FUNCTION EXAMPLE($a, $b = 0)
{
    DECLARE VARIABLE $tmp = 0 INTEGER;
    FOR $i = $a TO $b {
        $tmp = $tmp + $i;
    }
    RETURN $tmp;
}

```

Calling the above function with one or two arguments may look like:

```

$a = EXAMPLE(2 * $z);
$b = EXAMPLE(2 * $z, $y / 2);

```

The following function declaration specifies that the return value is a scalar variable (PRIMITIVE) and that the first argument must be a one-dimensional array and the second a scalar:

```

DECLARE FUNCTION EXTRACT_ELEMENT($a[], $e = 1 PRIMITIVE) PRIMITIVE
{
    RETURN $a[$e];
}

```

The following example shows how to use data record arguments and return values:

```

DECLARE RECORD ABC {
    DECLARE FIELD $a1 INTEGER;
    DECLARE FIELD $a2 INTEGER;
}
DECLARE RECORD DEF {
    DECLARE FIELD $d1 INTEGER;
    DECLARE FIELD $d2 INTEGER;
}

DECLARE FUNCTION MUL2($a RECORD ABC) RECORD DEF
{
    DECLARE VARIABLE $r RECORD DEF;
    $r.$d1 = 2 * $a.$a1;
    $r.$d2 = 2 * $a.$a2;
    RETURN $r;
}

DECLARE VARIABLE $a RECORD ABC;
DECLARE VARIABLE $b RECORD DEF;

$a.$a1 = 2;
$a.$a2 = 3;
$b = MUL2($a);
PRINT $b.$d1 & " " & $b.$d2 & "\n";

```

The above example writes the following string to the output file:

```
4 6
```

See also

DECLARE RECORD, DECLARE STATEMENT, RETURN.

Notes

Since arguments to user functions do not have to be typed, it is not always possible for the Message Builder compiler to check if the code is valid or not (actually, the compiler

never checks the usage of formal arguments without an *array-spec* or *record-spec* field or a PRIMITIVE keyword). For example, the code fragment:

```
DECLARE VARIABLE $a INTEGER;
DECLARE FUNCTION ABC($x)
{
    $a = $x;
    RETURN 0;
}

DECLARE VARIABLE $v[] INTEGER;
DECLARE VARIABLE $z INTEGER;
$v[10] = 15;
$z = ABC($v);
```

will pass the compiler with no errors, but a run time error is obtained when the assignment:

```
$a = $x;
```

is executed, because an array is assigned to a scalar variable.

User function arguments are passed by reference. Since the function arguments are read-only inside a function, they cannot be changed by the function directly. However, strange things might happen when, for example, the actual argument to a function is a global variable and the global variable is directly modified by the function. The following code demonstrates this:

```
DECLARE VARIABLE $global INTEGER;
DECLARE FUNCTION XYZ($x)
{
    PRINT $x & "\n";
    $global = 16;
    PRINT $x & "\n";
    RETURN 0;
}
DECLARE VARIABLE $temp INTEGER;
$global = 11;
$temp = XYZ($global);
```

The above example outputs the following to the output file:

```
11
16
```

There are restrictions on the names of user functions. A user function cannot have the same name as a Message Builder keyword or built-in function. If an invalid name is used, the Message Builder compiler generates an error indicating that the user-function name is invalid.

The keywords are:

- AND, ANTINOLOG, ARRAY,
- BAND, BLOCK, BNOT, BOR, BOUNDED, BREAK, BSHIFT, BXOR,
- CALL, CASE, CLOSE, CONDITIONAL, CONTINUE, COROUTINE,
- DATABASE, DEBUG, DECLARE, DROP,
- ELSE, EXCLUSION, EXEC, EXECUTE, EXIT, EXPORT,
- FILE, FLOATING, FOR, GROUP,
- IF, IMPORT, IN, INCLUDE,
- LIST_CONDITIONAL, LOAD, LOADPGM, LOG,
- MANDATORY, NOLOG, NOT,

- ON, OPEN, OPTIONAL, OR, OTHERS, OUT,
- PAIRED, PIPE, PRAGMA, PRINT,
- READ, REGEXP, RELEASE, REPEAT, REQUIRED, RETURN,
- SCAN, SERIALIZE, SET, SETDATE, SLEEP, SPLIT, SUSPEND, SWITCH,
- THROW, TRUNCATE, TRY, TYPE,
- UNLOADPGM, UNSERIALIZE, USE,
- WHEN, WHILE, WITH, XOR

The built-in functions are:

- ARGUMENT, ARGUMENTCOUNT, ARRAYDIM, ARRAYSIZE,
- CHARACTER, CODE, CURRENTDATE,
- DATE, ESCAPE,
- FORMAT, FROM_ISO8859,
- GETDATE, INTERP_VERSION, PROGRAMINFO,
- STRBACKWARD, STRFORWARD, STRLEN, STRREPEAT, SYSTEM,
- TO_ISO8859, TRIMLEFT, TRIMRIGHT, TRUNCATE,
- SOURCEFILE, SOURCELINE, SOURCEMODULE, SOURCEPROCEDURE

If the **O** option is not specified when a Message Builder program is compiled, some optimizations are performed:

- Unused functions that aren't exported, for example by PRAGMA EXPORT, are removed.
- Calls to functions that consist of only one RETURN statement are replaced by the expression in the RETURN statement, for example,

```
DECLARE FUNCTION FUNC($a, $b, $c)
{
    RETURN $a & $b & $c;
}
PRINT FUNC("A", "B", "C");
```

is converted to:

```
PRINT "A" & "B" & "C";
```

12.4.38 DECLARE MODULE

Name

DECLARE MODULE – Declare a module.

Syntax

DECLARE [PUBLIC] MODULE [INTERFACE] *name declarations*

Description

The DECLARE MODULE statement declares a module. A module encapsulates a number of declarations. Declarations in a module can also be hidden; that is, they are not visible from outside the module.

The name of the module is given by *name*. It must start with an alphabetic character or an underscore character. The remaining characters may be alphabetic characters,

decimal digits or underscore characters. There is no upper limit for the length of the name.

The PUBLIC modifier may be specified. It has no effect and is allowed only because of consistency reasons.

The *declarations* part is zero or more declarations surrounded by { and } characters. Valid declarations are variable, data record, user function, user statement, external user function, and external user statement declarations. A module declaration is not allowed inside another module. INCLUDE and PRAGMA directives may also be present in a module. When all INCLUDE directives have been processed, only variable declarations, user function and statement declarations, external user function and statement declarations, and PRAGMA directives may remain.

Declarations are visible outside the module if the PUBLIC modifier is specified after the DECLARE keyword. Otherwise, they are not.

A module can be divided into an interface and an implementation part. The interface part is declared when the INTERFACE keyword is present and the implementation part is declared otherwise. Only zero or one module interface may be declared for each module implementation and it must be located before the implementation. If a module interface is declared, the module implementation must also be declared. A module interface may only contain variable and record declarations and PRAGMA and INCLUDE directives. The purpose of module interfaces is to make it possible to divide a module into an interface part that can be put in an include file and an implementation part that can be put in a library file.

When referring to a variable, a data record, a function or a statement declared inside a module, the module name followed by a dot (.) must be added in front of the name; for example, referring to the variable \$a and the function FUNC in the module ABC looks like:

```
ABC.$a
ABC.FUNC()
```

When referring to a variable, data record, function or statement declared in the same module, the module name does not have to be specified.

The user statements MODULE_INIT and MODULE_CLEAN declared in a module have special meaning. All of the MODULE_INIT statements are automatically executed before anything else, and all of the MODULE_CLEAN statements are automatically executed before an EXIT or SWITCH statement, or before the end of the program is reached (unless a run-time error occurs). If a MODULE_CLEAN statement, directly or indirectly, executes an EXIT or SWITCH statement, a recursive call to MODULE_CLEAN is not performed. The execution order of the different MODULE_INIT and MODULE_CLEAN statements is undefined. Parameters to the MODULE_INIT or MODULE_CLEAN statements may not be specified.

For programs dynamically loaded using the LOADPGM statement, all MODULE_INIT statements are automatically executed when the program is loaded and all MODULE_CLEAN statements are automatically executed when the program is unloaded using the UNLOADPGM statement. Note that the MODULE_CLEAN statements in dynamically loaded programs are not executed when an EXIT or SWITCH statement is executed.

Examples

The following example implements a module that manages a last-in, first-out stack of strings:

```

DECLARE MODULE STACK {
    DECLARE VARIABLE $Stack[] STRING;
    DECLARE VARIABLE $StackSize INTEGER;

    DECLARE STATEMENT MODULE_INIT
    {
        $StackSize = 1;
        RETURN;
    }

    DECLARE PUBLIC STATEMENT PUSH IN $s
    {
        $Stack[$StackSize] = $s;
        $StackSize = $StackSize + 1;
        RETURN;
    }

    DECLARE PUBLIC FUNCTION POP()
    {
        $StackSize = $StackSize - 1;
        RETURN $Stack[$StackSize];
    }
}

```

The `MODULE_INIT` statement could be replaced by a direct initialization of the `$StackSize` variable, but it is included in order to demonstrate how to use it. The advantage of using the above version, compared to one where modules are not used, is that all the details of the implementation are not visible outside the module. It is not possible to directly read or to modify the `$Stack` and `$StackSize` variables from outside the module, therefore, it is possible to write programs that are easier to maintain.

Given the `STACK` module, execution of the program,

```

STACK.PUSH "String 1";
STACK.PUSH "String 2";
STACK.PUSH "String 3";
PRINT STACK.POP() & "\n";
PRINT STACK.POP() & "\n";
PRINT STACK.POP() & "\n";

```

will cause the following to be written to the output file:

```

String 3
String 2
String 1

```

The next example demonstrates how module interfaces are used. The include file *example.s4h* contains the module interface:

```

DECLARE MODULE INTERFACE EXAMPLE {
    DECLARE PUBLIC RECORD R {
        DECLARE FIELD $f1 INTEGER;
        DECLARE FIELD $f2 STRING;
    }
}

```

The library file *example.s4m* contains the module implementation:

```

DECLARE MODULE EXAMPLE {
    DECLARE PUBLIC FUNCTION
    GET_RECORD() RECORD R
    {
        DECLARE VARIABLE $r RECORD R;
        $r.$f1 = 5;
    }
}

```

```

        $r.$f2 = "string";
        RETURN $r;
    }

    DECLARE PUBLIC STATEMENT
    PRINT_RECORD IN $r RECORD R
    {
        PRINT $r.$f1 & "\n";
        PRINT $r.$f2 & "\n";
        RETURN;
    }
}

```

A program that uses the EXAMPLE module might look something like

```

INCLUDE "example.s4h";
DECLARE VARIABLE $x RECORD EXAMPLE.R;
$x = EXAMPLE.GET_RECORD();
EXAMPLE.PRINT_RECORD $x;

```

Notes

The extension functions and statements described in the *Extender* manual are also valid in modules. The MODULE_INIT and/or MODULE_CLEAN statements may also be implemented as extension statements.

There are some restrictions on the name of a module. If an invalid name is used, the Message Builder compiler generates an error indicating that the module name is not valid. The following is a list of all names that are not valid as module names:

- AND, ANTINOLOG, ARRAY,
- BAND, BLOCK, BNOT, BOR, BOUNDED, BREAK, BSHIFT, BXOR,
- CALL, CASE, CLOSE, CONDITIONAL, CONTINUE, COROUTINE,
- DATABASE, DEBUG, DECLARE,
- ELSE, EXCLUSION, EXEC, EXECUTE, EXIT, EXPORT,
- FILE, FLOATING, FOR, GROUP,
- IF, IMPORT, INCLUDE,
- LIST_CONDITIONAL, LOAD, LOADPGM, LOG,
- MANDATORY, NOLOG, NOT,
- ON, OPEN, OPTIONAL, OR, OTHERS,
- PAIRED, PIPE, PRAGMA, PRINT,
- READ, REGEXP, RELEASE, REPEAT, REQUIRED, RETURN,
- SCAN, SERIALIZE, SET, SETDATE, SLEEP, SPLIT, SUSPEND, SWITCH,
- THROW, TRUNCATE, TRY, TYPE,
- UNLOADPGM, UNSERIALIZE, USE,
- WHEN, WHILE, WITH, XOR

12.4.39 DECLARE RECORD

Name

DECLARE RECORD – Declare a data record.

Syntax

DECLARE [PUBLIC] RECORD *name declarations*

or

DECLARE [PUBLIC] RECORD *new-name IS old-name;*

Description

The DECLARE RECORD statement declares a data record. A data record is a user defined data type consisting of an aggregate of other variables, called data record fields.

The first version of DECLARE RECORD declares a new record. The second version gives the existing record name, *old-name*, the alias *new-name*. The first version is described first.

The name of the data record is given by *name*. It must start with an alphabetic character or an underscore character. The remaining characters may be alphabetic characters, decimal digits or underscore characters. There is no upper limit for the length of the name.

The PUBLIC modifier only has meaning if the data record is declared inside a module (see paragraph [12.4.38](#)).

The *declarations* part is one or more data record field declarations surrounded by { and } characters. INCLUDE and PRAGMA directives may also be present in a data record declaration. When all INCLUDE directives have been processed, only data record field declarations and PRAGMA directives may remain. A valid data record field declaration is identical to a variable declaration with the optional VARIABLE keyword replaced by FIELD:

```
DECLARE [PUBLIC] [FIELD] field type;
```

The name of the data record field is *field* and *type* is the type of the field. All field names must be unique in a data record declaration. See the DECLARE VARIABLE directive (paragraph [12.4.41](#)) for a description of *type*. Note that if *type* is another data record type, it must already have been declared. A record cannot have fields of the same type as the record itself. The CONSTANT and ONCE keywords are not allowed and no initial value can be specified. If the record is declared inside a module and the PUBLIC modifier is specified for a field, the field is visible outside the module. Otherwise, it is not.

It is also possible to use TYPEOF (see paragraph [12.4.41](#)) to declare record fields. Since a record cannot be declared inside a user function or statement, the type of the declared variable is always known at compile time.

The following is an example of a record declaration:

```
DECLARE RECORD DEMO {  
  DECLARE FIELD $a INTEGER;  
  DECLARE FIELD $b STRING;  
  DECLARE FIELD $c INTEGER;  
}
```

A data record variable is declared by specifying a record type as the *type* field (see DECLARE VARIABLE, paragraph [12.4.41](#)). For example:

```
DECLARE VARIABLE $r RECORD DEMO;
```

Specifying the record variable followed by a dot and the field name references the individual fields in a record variable. For example, referencing the fields in the DEMO record would look something like:

```
$r.$a  
$r.$b  
$r.$c
```

The second version of the DECLARE RECORD statement makes it possible to create aliases for records. A record variable of type *new-name* can be used on all places where a record variable of type *old-name* is valid and vice versa. This will for example, be a way to "import" a data record into a module. See the EXAMPLES section for an example.

A data record declaration is not allowed inside a *statement-block* or in a user function or statement.

Examples

Example 1

The following example:

```
DECLARE RECORD RR {  
    DECLARE FIELD $i INTEGER;  
    DECLARE FIELD $f FLOAT;  
}  
DECLARE VARIABLE $r RECORD RR;  
  
$r.$i = 5;  
$r.$f = 7.9;  
PRINT $r.$i & " " & $r.$f & "\n";
```

outputs the following to the output file:

```
5 7.900000
```

Example 2

The following example declares a record with a field of record type:

```
DECLARE RECORD ONE {  
    DECLARE FIELD $i INTEGER;  
    DECLARE FIELD $f FLOAT;  
}  
DECLARE RECORD TWO {  
    DECLARE FIELD $int INTEGER;  
    DECLARE FIELD $val RECORD ONE;  
}  
DECLARE VARIABLE $r RECORD TWO;  
  
$r.$int = 5;  
$r.$val.$i = 8;  
$r.$val.$f = -56.9;  
PRINT $r.$int & " " & $r.$val.$i & " " & $r.$val.$f & "\n";
```

The following is written to the output file:

```
5 8 -56.900000
```

Example 3

The following example demonstrates how record fields can be made private to a module although the record itself is public:

```

DECLARE MODULE MOD {
    DECLARE PUBLIC RECORD REC {
        DECLARE FIELD $i INTEGER;
        DECLARE FIELD $f FLOAT;
    }

    DECLARE PUBLIC FUNCTION
    GETREC($a PRIMITIVE, $b PRIMITIVE) RECORD REC
    {
        DECLARE VARIABLE $r RECORD REC;
        $r.$i = $a;
        $r.$f = $b;
        RETURN $r;
    }

    DECLARE PUBLIC STATEMENT
    PRINTREC IN $r RECORD REC
    {
        PRINT $r.$i & " " & $r.$f & "\n";
        RETURN;
    }
}

DECLARE VARIABLE $rec RECORD MOD.REC;
$rec = MOD.GETREC(10,20.0);
/* note that we can't directly access the record fields here */
MOD.PRINTREC $rec;

```

Example 4

The following example demonstrates how the second version of the DECLARE RECORD statement can be used to "import" a record from another module:

```

DECLARE MODULE MOD1 {
    DECLARE PUBLIC RECORD REC {
        DECLARE PUBLIC FIELD $i INTEGER;
        DECLARE PUBLIC FIELD $f FLOAT;
    }
}

DECLARE MODULE MOD2 {
    DECLARE PUBLIC RECORD RECIMPORT IS MOD1.REC;
}

DECLARE VARIABLE $r RECORD MOD2.RECIMPORT;
$r.$i = 99;
$r.$f = 18.97;
.
.
.

```

See also

DECLARE VARIABLE.

Notes

Variables and data record fields of anonymous record types may be declared. In other words, the record declaration can be done directly in the variable or field declaration. See DECLARE VARIABLE, paragraph [12.4.41](#), for further information.

There are some restrictions on the name of a data record. If an invalid name is used, the Message Builder compiler generates an error indicating that the record name is not valid. The following is a list of all names that are not valid as data record names:

- AND, ANTINOLOG, ARRAY,
- BAND, BLOCK, BNOT, BOR, BOUNDED, BREAK, BSHIFT, BXOR,
- CALL, CASE, CLOSE, CONDITIONAL, CONTINUE, COROUTINE,
- DATABASE, DEBUG, DECLARE,
- ELSE, EXCLUSION, EXEC, EXECUTE, EXIT, EXPORT,
- FILE, FLOATING, FOR, GROUP, IF,
- IMPORT, INCLUDE,
- LIST_CONDITIONAL, LOAD, LOADPGM, LOG,
- MANDATORY, NOLOG, NOT,
- ON, OPEN, OPTIONAL, OR, OTHERS,
- PAIRED, PIPE, PRAGMA, PRINT,
- READ, REGEXP, RELEASE, REPEAT, REQUIRED, RETURN,
- SCAN, SERIALIZE, SET, SETDATE, SLEEP, SPLIT, SUSPEND, SWITCH,
- THROW, TRUNCATE, TRY, TYPE,
- UNLOADPGM, UNSERIALIZE, USE,
- WHEN, WHILE, WITH, XOR

12.4.40 DECLARE STATEMENT

Name

DECLARE STATEMENT – Declare a user statement.

Syntax

DECLARE [PUBLIC] STATEMENT name parameter-list statement-block

Description

A user statement is a statement (procedure), written using Message Builder code that can be used in the same way as a built-in statement, such as EXPORT.

The PUBLIC modifier only has meaning if the statement is declared inside a module (see paragraph [12.4.38](#)).

The name of the statement is *name*, *parameter-list* is a list of zero or more parameters and *statement-list* is the list of statements representing the code of the user statement.

The statement name, *name*, must start with an alphabetic character or an underscore character. The remaining characters may be alphabetic characters, decimal digits or underscore characters. There is no upper limit for the length of the name.

The list of formal parameters, *parameter-list*, is a list of zero or more parameters. Commas or noise words separate the parameters. A valid noise word is the same as a valid statement name. When calling a user statement, the same noise words as the ones used in the declaration of the user statement must be used. If a comma was used in the declaration, a comma must be used in the call.

Each formal parameter can be an IN, OUT or IN OUT parameter. An IN parameter cannot be modified and looks like:

```
IN param-var [array-spec] [= const-expression] [PRIMITIVE | record-spec]
```

An OUT parameter is used to return values from the called statement and looks like:

```
OUT param-var [array-spec] [[= const-expression] DROP]  
[PRIMITIVE | record-spec]
```

Finally, an IN OUT parameter can be used both as an input and output parameter and it is specified as:

```
IN OUT param-var [array-spec] [= const-expression] [PRIMITIVE | record-spec]
```

The *param-var* fields are variable names, which are automatically declared as local variables to the statement and should not be declared separately. A default value can be specified for IN and IN OUT parameters using the *const-expression* field. The DROP modifier may be specified for an OUT variable. It causes the OUT value to be dropped if the caller does not supply a parameter.

When calling a statement, parameters that have default values (an OUT parameter with DROP specified is considered to have a default value) may be left out. These parameters will have the values specified by the default values when the user statement is entered. For OUT variables with DROP specified, the variables are by default of string type. The *const-expression* may be used to tell the Message Builder compiler what type the out parameter shall have when it is not specified in a call. Note that the value of *const-expression* does not matter, only the type.

If the first formal parameter has a default value, all of the remaining parameters must also have default values. For the remaining parameters, if a parameter has a default value, all other parameters with the same noise word preceding them must also have default values.

Specifying a default value for an IN OUT parameter also means that the value is dropped when the default value is used.

When calling the user statement, the parameters can be given in any order as long as the Message Builder compiler can uniquely determine the different parameters. More precisely, the rules are as follows:

- The parameter corresponding to the first formal parameter must always be given as the first parameter.
- For the remaining parameters, parameters that have equal noise words preceding them must be given in the same order as their corresponding formal parameters.

By default, the formal parameter types are determined by the type of the variables or expressions used when calling the statement. The *array-spec* field can be used to specify that an actual parameter must be an array of a specific dimension. A number of [] pairs are used to specify the array dimension. The dimension equals the number of [] pairs. The PRIMITIVE keyword or the *record-spec* field is used to specify that the parameter must be a scalar or a data record, respectively. The *record-spec* field is the keyword RECORD followed by a data record name. It is invalid to specify both the PRIMITIVE keyword and an *array-spec*. The compiler issues an error if the type of an actual parameter does not match the specification.

Note: In order to access fields of data records or arrays of data records supplied as parameters, the parameters must be declared to be data records or arrays of data records, respectively.

The evaluation order for the parameters when a user statement is called is undefined.

A RETURN statement (see paragraph [12.4.102](#)) may be placed anywhere in a user statement where another statement is valid. The last statement in a user statement must be RETURN, EXIT or THROW. The RETURN statements must have no parameters.

All variables declared in *statement-list* are local to the user statement. The initial values when the statement is entered are as follows:

- For variables with initial values, the initial value itself.
- For variables with no initial value specified:
 - Zero for INTEGER and FLOAT variables.
 - A string filled with spaces for CHAR variables.
 - An empty string for STRING variables.
 - The computer's base time for DATE variables. See the DECLARE VARIABLE statement, paragraph [12.4.41](#), for an explanation of the computer's base time.
 - An empty array for array variables.
 - A record with all INTEGER and FLOAT fields set to zero, all CHAR fields set to strings filled with spaces, all STRING fields set to empty strings, all DATE fields set to the computer's base time, and all array fields set to empty arrays.

User statements need not be declared before they are called.

Recursion is supported; that is, a statement may call itself directly or indirectly.

A user statement declaration is not allowed in a *statement-block* or in another user statement or function.

Examples

A user statement, named EXAMPLE, having three parameters, may look like:

```
DECLARE STATEMENT EXAMPLE IN $a
                                TO IN $b = 0
                                SET OUT $c DROP
{
    DECLARE VARIABLE $tmp = 0 INTEGER;
    FOR $i = $a TO $b {
        $tmp = $tmp + $i;
    }
    $c = $tmp;
    RETURN;
}
```

Calling the above statement with one, two, or three parameters may look like:

```
EXAMPLE 2 * $z;
EXAMPLE 2 * $z TO $y / 2;
EXAMPLE 2 * $z TO $y / 2 SET $x;
```

The following statement declaration specifies that the first parameter must be a one-dimensional array and the second and third parameters must be scalars:

```
DECLARE STATEMENT EXTRACT_ELEMENT IN $a[]
                                ELEMENT IN $e = 1 PRIMITIVE
                                OUTPUT OUT $out PRIMITIVE
{
    $out = $a[$e];
    RETURN;
}
```

The following example shows how to use data record parameters:

```

DECLARE RECORD ABC {
    DECLARE FIELD $a1 INTEGER;
    DECLARE FIELD $a2 INTEGER;
}
DECLARE RECORD DEF {
    DECLARE FIELD $d1 INTEGER;
    DECLARE FIELD $d2 INTEGER;
}

DECLARE STATEMENT MUL2 IN $a RECORD ABC, OUT $b RECORD DEF
{
    $b.$d1 = 2 * $a.$a1;
    $b.$d2 = 2 * $a.$a2;
    RETURN;
}

DECLARE VARIABLE $a RECORD ABC;
DECLARE VARIABLE $b RECORD DEF;

$a.$a1 = 2;
$a.$a2 = 3;
MUL2 $a, $b;
PRINT $b.$d1 & " " & $b.$d2 & "\n";

```

The above example writes the following string to the output file:

```
4 6
```

See also

DECLARE FUNCTION, DECLARE RECORD, RETURN.

Notes

When an error in a call to a user statement is found, the parameter number in the error message output by the Message Builder compiler corresponds to the formal parameter in the statement declaration.

Since parameters to user statements do not have to be typed, it is not always possible for the Message Builder compiler to check whether the code is valid or not (actually, the compiler never checks the usage of formal parameters without an *array-spec* or *record-spec* field or a PRIMITIVE keyword). For example, the code fragment

```

DECLARE VARIABLE $a INTEGER;
DECLARE STATEMENT ABC IN $x
{
    $a = $x;
    RETURN;
}

DECLARE VARIABLE $v[] INTEGER;
$v[10] = 15;
ABC $v;

```

will pass the compiler with no errors, but a run time error is obtained when the assignment

```
$a = $x;
```

is executed, because an array is assigned to a scalar variable.

User statement input parameters are passed by reference. Since the statement parameters are read-only inside a statement, the statement cannot change them directly. However, strange things might happen when, for example, the actual input parameter to a statement is a global variable and the statement directly modifies the global variable. The following code demonstrates this:

```
DECLARE VARIABLE $global INTEGER;
DECLARE STATEMENT XYZ IN $x
{
    PRINT $x & "\n";
    $global = 16;
    PRINT $x & "\n";
    RETURN;
}
$global = 11;
XYZ $global;
```

The above example outputs the following to the output file:

```
11
16
```

There are some restrictions on the name of a user statement; for example, a user statement cannot have the same name as a built-in statement, such as PRINT. If an invalid name is used, the Message Builder compiler generates an error indicating that the user statement name is not valid. The following is a list of all names that are not valid as user statement names:

- AND, ANTINOLOG, ARRAY,
- BAND, BLOCK, BNOT, BOR, BOUNDED, BREAK, BSHIFT, BXOR,
- CALL, CASE, CLOSE, CONDITIONAL, CONTINUE, COROUTINE,
- DATABASE, DEBUG, DECLARE, ELSE, EXCLUSION, EXEC, EXECUTE, EXIT, EXPORT, FILE, FLOATING, FOR,
- GROUP, IF, IMPORT, INCLUDE,
- LIST_CONDITIONAL, LOAD, LOADPGM, LOG,
- MANDATORY, NOLOG, NOT,
- ON, OPEN, OPTIONAL, OR, OTHERS,
- PAIRED, PIPE, PRAGMA, PRINT,
- READ, REGEXP, RELEASE, REPEAT, REQUIRED, RETURN,
- SCAN, SERIALIZE, SET, SETDATE, SLEEP, SPLIT, SUSPEND, SWITCH,
- THROW, TRUNCATE, TRY, TYPE,
- UNLOADPGM, UNSERIALIZE, USE,
- WHEN, WHILE, WITH, XOR

If the **O** option is not specified when a Message Builder program is compiled, some optimizations are performed:

- Unused statements that aren't exported, for example by PRAGMA EXPORT, are removed.
- Calls to statements that only contain one single non-compound statement plus a RETURN statement are replaced by the single non-compound statement.
For example:

```
DECLARE STATEMENT
STMNT IN $a SEP1 IN $b SEP2 IN $c
{
    PRINT $a & $b & $c;
    RETURN;
}
STMNT "A" SEP1 "B" SEP2 "C";
```

is converted to:

```
PRINT "A" & "B" & "C";
```

- Calls to empty statements, i.e. statements that consist of a single RETURN statement, are removed.

12.4.41 DECLARE VARIABLE

Name

DECLARE VARIABLE – Declare a variable.

Syntax

```
DECLARE [PUBLIC] [VARIABLE] decl-var [= const-expression]  
[CONSTANT | ONCE] type;
```

Description

The DECLARE VARIABLE statement declares a variable. The VARIABLE keyword is optional. *Decl-var* is the name of the variable, *const-expression* is an optional initial value, and *type* is the variable type. The CONSTANT and ONCE modifiers indicate that the variable is read-only. The Message Builder compiler will issue an error if a CONSTANT or ONCE declared variable is assigned. Another effect of the ONCE modifier is that the same variable may be declared multiple times (see below).

A variable name must start with a dollar sign (\$) and be followed by one or more alphanumeric or underscore characters (only alphabetic characters in the range A – Z are valid). There is no limit to the length of variable names.

The variable type is INTEGER, FLOAT, STRING, CHAR, DATE, a data record declared using the DECLARE RECORD declaration directive, or the same type as another variable (see the “TYPEOF” section below). If the type is CHAR, a size specification must be given, in parenthesis, after the CHAR keyword, for example:

```
DECLARE VARIABLE $c CHAR(15+2);
```

The size specification should be a *const-expression*. If the type is STRING, a size may also be given, but it is optional. If the type is a data record, *type* is the keyword RECORD followed by the name of the record, for example:

```
DECLARE VARIABLE $r1 RECORD REC1;  
/* record REC2 is declared inside module MOD */  
DECLARE VARIABLE $r2 RECORD MOD.REC2;
```

See the VARIABLE TYPES section, paragraph [12.4.136](#), for more information about variable types.

Default Initial Values

Type	Default Initial Value
INTEGER	0
FLOAT	0.000000
CHAR	A string filled with spaces
STRING	An empty string
DATE	The computer's base time. The definition of the base time is different for different computers. For UNIX machines it is the time in seconds since January 1, 1970.
Array	An empty array
Data Record	Each field of a record has a default initial value according to its type. For example, an INTEGER field has a default initial value of 0.

Initialization

A variable can optionally be initialized within the DECLARE statement. For example:

```
DECLARE VARIABLE $i = 0 INTEGER;
```

declares an integer variable `$i` and initializes it to the value 0. The “Arrays and Records” section below gives examples of initialization of arrays and records within DECLARE statements.

Due to backward compatibility reasons, it is recommended that DATE variables aren’t initialized within DECLARE statements. Instead initialize them using an assignment statement.

Arrays and Records

The declared variable is either a single scalar or data record or an array. If an array is declared, *decl-var* also contains a number of left and right bracket pairs. The number of such pairs indicates the number of dimensions for the array. For example:

```
DECLARE VARIABLE $array[][] INTEGER;
DECLARE VARIABLE $recordArray[] RECORD REC3;
```

declares an integer array with two dimensions and a record array with one dimension. An initial value can be specified using a comma-separated list surrounded by curly brackets, for example:

```
DECLARE VARIABLE $ia1[] = { 1, 2, 3 } INTEGER;
```

The initial value of the `$ia1` array has the size three. If the number of dimensions is greater than one, the comma-separated list can consist of other comma-separated lists surrounded by curly brackets, for example:

```
DECLARE VARIABLE $ia2[][] = {{ 8, 7 }, {}, { 3 }} INTEGER;
```

The size of `$ia2` initially will be three. The size of the three elements will be two, zero and one.

The initial value for a data record and a data record array may be specified as follows:

```
DECLARE RECORD REC4 {
    DECLARE FIELD $i INTEGER;
    DECLARE FIELD $s STRING;
}
DECLARE VARIABLE $r = { 2, "ABC" } RECORD REC4;
DECLARE VARIABLE $ra[] = {{ 5, "abc" }, { 7, "xyz" }} RECORD REC4;
```

The type of a variable may also be an anonymous data record type. In such a case, the record name is replaced by the *declarations* part of the record declaration (see DECLARE RECORD, paragraph [12.4.39](#)). For example:

```
DECLARE VARIABLE $anonymous RECORD {
    DECLARE FIELD $i INTEGER;
    DECLARE FIELD $f FLOAT;
}
```

TYPEOF

A variable can also be declared to be of the same type as another variable:

```
DECLARE VARIABLE $x TYPEOF $y;
```

The `$x` variable is declared to be of the same type as the `$y` variable. If the type of `$y` is not known (that is, it is a formal argument or parameter), the type of `$x` is determined at run-time. No initial value, array dimension, or CONSTANT or ONCE modifier can be specified for a TYPEOF declared variable.

Other Points

Variables must be declared before they are used. Normally a variable must only be declared once. However, a variable declared inside a statement block is only visible inside the block and it shadows a variable with the same name declared outside it. Variables declared inside a user function or statement is local to the function or statement and shadow variables with the same name outside the function or statement.

Another situation when the same variable may be declared multiple times is when the ONCE modifier is used. Besides making the variable constant, the ONCE modifier also makes it possible to declare it multiple times provided that, except for the *const-expression* value, the declarations are identical and the variable is not used before any of the declarations. Only the first declaration is significant, all the other ones are ignored. This feature makes it more convenient to use "configuration" variables in a program. A default value can be given somewhere in the program using a ONCE declared variable. The user who runs the compiler can then change the default value by supplying another ONCE declaration, for example, on the command line.

Note: The initialization of variables declared inside a statement block is performed only when user functions or statements are entered or, for code outside functions and statements, when the program is started.

The PUBLIC modifier only has meaning if the variable is declared inside a module (see paragraph [12.4.38](#)). The PUBLIC keyword may not be used for variables declared inside statement blocks or for variables local to user functions or statements.

Examples

Declare some variables:

```
DECLARE VARIABLE $ArtOrdNum INTEGER;
DECLARE VARIABLE $ArtName STRING;
DECLARE VARIABLE $Value CHAR(100);
DECLARE VARIABLE $Record RECORD EXAMPLE1;
DECLARE VARIABLE $array[][] STRING;
DECLARE VARIABLE $recordArray[][][] RECORD EXAMPLE2
```

The following example will cause the Message Builder compiler to output an error message indicating that the \$s variable is multiple declared:

```
DECLARE VARIABLE $s STRING;
$s = ARGUMENT(1);
PRINT $s;
DECLARE VARIABLE $s STRING;
IF $s = "-a" {
    $s = ARGUMENT(2);
    PRINT $s;
}
```

If the second declaration of \$s is moved into the IF statement block, the second declaration just shadows the first one and no error will be issued:

```
DECLARE VARIABLE $s STRING;
$s = ARGUMENT(1);
PRINT $s;
IF $s = "-a" {
    DECLARE VARIABLE $s STRING;
    $s = ARGUMENT(2);
    PRINT $s;
}
/* here, the value of $s equals ARGUMENT(1) */
```

Example for when variables that are local to a statement block are only initialized when the program is started or when user functions or statements are entered:

```
FOR $i=1 TO 5 {  
    DECLARE VARIABLE $data = 1 INTEGER;  
  
    $data = $data * $i;  
    PRINT $data & "  ";  
}  
PRINT "\n";
```

will produce the following output

```
1  2  6  24  120
```

The following is a generic statement for swapping two variables (for this to work properly, the statement parameters should be of the same type):

```
DECLARE STATEMENT  
SWAP IN OUT $a, IN OUT $b  
{  
    DECLARE VARIABLE $t TYPEOF $a;  
    $t = $a;  
    $a = $b;  
    $b = $t;  
    RETURN;  
}
```

See also

DECLARE RECORD, VARIABLE TYPES.

Notes

Specifying a size for a STRING variable is meaningful only when using it as a destination variable in an EXEC SQL SELECT statement. In this case, the size is the size of the destination buffer when requesting data from the RDBMS. If no size was given when the variable was declared, the buffer size 255 is used.

If the **O** option is not specified when a Message Builder program is compiled, unused variables are removed; that is, they do not occupy any space in the compiled program.

12.4.42 DUMP BACKTRACE

Name

DUMP BACKTRACE – Output a call backtrace for the current coroutine.

Syntax

```
DUMP BACKTRACE [filename];
```

Description

The DUMP BACKTRACE statement outputs a call backtrace for the current coroutine. The call backtrace is appended to the file specified by the *filename* parameter. If *filename* isn't specified or is an empty string, the backtrace is written to the standard output.

The *filename* parameter is an *expression* that is converted to a string.

If the executing program was compiled using the **T** compiler option, each called function or statement produces one line of the form:

```
Call from address nn, program "name"
```

where *nn* represents the pseudo code address and *name* the name of the program from which the call was made.

If the executing program instead was compiled using the **T** compiler option, each called function or statement produces one line of the form:

```
Call from line nn, file "name"
```

where *nn* represents the source code line number and *name* the name of the source code file from which the call was made.

Errors

The following value can be assigned to `$Error`:

```
$Error_DumpBacktraceOpenFail
```

Failed to open the *filename* file.

Examples

If the following program is named *backtrace.s4* (the line numbers are added for convenience):

```
1  STATEMENT1;
2
3  DECLARE STATEMENT STATEMENT1
4  {
5      STATEMENT2;
6      PRINT "Exit STATEMENT1\n";
7      RETURN;
8  }
9
10 DECLARE STATEMENT STATEMENT2
11 {
12     STATEMENT3;
13     PRINT "Exit STATEMENT2\n";
14     RETURN;
15 }
16
17 DECLARE STATEMENT STATEMENT3
18 {
19     DUMP BACKTRACE "output.txt";
20     PRINT "Exit STATEMENT3\n";
21     RETURN;
22 }
```

and compiled using the command:

```
c4edi backtrace.s4
```

When executed, the following will be written to the *output.txt* file (the addresses might differ):

```
Call from address 11, program "backtrace.x4"
Call from address 4, program "backtrace.x4"
Call from address 26, program "backtrace.x4"
```

If it instead is compiled using the command:

```
c4edi -T backtrace.s4
```

the result becomes:

```
Call from line 12, file "backtrace.s4"  
Call from line 5, file "backtrace.s4"  
Call from line 1, file "backtrace.s4"
```

See also

DUMP COROUTINES.

12.4.43 DUMP COROUTINES

Name

DUMP COROUTINES – Output a list of all currently existing coroutines.

Syntax

DUMP COROUTINES [*filename*];

Description

The DUMP COROUTINES statement outputs a list of all currently existing coroutines. The list is appended to the file specified by the *filename* parameter. If *filename* isn't specified, the list is written to the standard output.

The *filename* parameter is an *expression* that is converted to a string.

For each coroutine, one line of information and a backtrace is output. The line of information has the following format:

```
Coroutine n1: parent = n2, variable stack size = n3, call stack size = n4
```

Where *n1* represents the coroutine ID, *n2* is the ID of the parent coroutine, *n3* is the size of the variable stack, and *n4* is the size of the function and statements call stack.

See the DUMP BACKTRACE statement (paragraph [12.4.42](#)) for a description of a backtrace.

Errors

The following value can be assigned to `$Error`:

```
$Error_DumpCoroutinesOpenFail
```

Failed to open the *filename* file.

Examples

If the following program is named *example.s4*:

```
INCLUDE "coroutine.s4h";  
  
DECLARE VARIABLE $id1 INTEGER;  
DECLARE VARIABLE $id2 INTEGER;
```

```

COROUTINE NEW $id1 CALL One;
COROUTINE NEW $id2 CALL Two;
COROUTINE RESUME $id1;
COROUTINE RESUME $id2;
DUMP COROUTINES;

DECLARE STATEMENT ONE
{
    ONE2;
    RETURN;
}

DECLARE STATEMENT ONE2
{
    COROUTINE RESUME COROUTINE($Coroutine_SelectorIdForPrevious);
    PRINT "In statement ONE2\n";
    RETURN;
}

DECLARE STATEMENT TWO
{
    TWO2;
    RETURN;
}

DECLARE STATEMENT TWO2
{
    COROUTINE RESUME COROUTINE($Coroutine_SelectorIdForPrevious);
    PRINT "In statement TWO2\n";
    RETURN;
}

```

and compiled using the command:

```
c4edi -T example.s4
```

The following will be written to the standard output when executed:

```

Coroutine 0: parent = -1, variable stack size = 0, call stack size = 0
Coroutine 1: parent = 0, variable stack size = 0, call stack size = 1
    call from line 14, file "example.s4"
Coroutine 2: parent = 0, variable stack size = 0, call stack size = 1
    call from line 27, file "example.s4"

```

See also

DUMP BACKTRACE.

12.4.44 DUMP PROFILE

Name

DUMP PROFILE – Dump profile information.

Syntax

DUMP PROFILE [*information*];

Description

Normally profiling information is output when a Message Builder program terminates. The DUMP PROFILE statement forces profiling information to be written to the profile information file. After the profiling information has been output, it is reset.

If the *information* parameter is specified, it is output together with the program names in the profile information file. It can be used to identify different DUMP PROFILE invocations. The *information* parameter is an *expression* that is converted to a string.

See the description of the **d** interpreter option (paragraph [11.3.1](#)) for more information about profiling.

Examples

If the following program:

```
FOR $i = 1 TO 10000 {
    PRINT_INTEGER $i;
}
DUMP PROFILE "First dump";
FOR $i = 10001 TO 15000 {
    PRINT_INTEGER $i;
}

DECLARE STATEMENT
PRINT_INTEGER IN $integer PRIMITIVE
{
    PRINT $integer;
    PRINT "\n";
    RETURN;
}
```

is named *prog.s4* and compiled using the command:

```
c4edi -T prog.s4
```

and then executed using the command:

```
r4edi -dprof prog.x4
```

something like the following will be appended to the *prof* file (see the description of the **d** interpreter option, paragraph [11.3.1](#), for more information about the format):

```
PROGRAM prog.x4 "First dump"
FILE prog.s4
1 10001 0.110122
2 10000 0.199350 0.514516
4 1 0.016948
12 10000 0.194657
13 10000 0.117704
14 10000 0.204377
PROGRAM prog.x4
FILE prog.s4
5 5001 0.085836
6 5000 0.143261 0.295070
12 5000 0.082315
13 5000 0.069334
14 5000 0.152676
```

12.4.45 DUMP VARIABLES

Name

DUMP VARIABLES – Dump variables.

Syntax

DUMP VARIABLES [*filename*];

Description

The DUMP VARIABLES statement writes the contents of all variables to the file specified by *filename*. If *filename* is not specified, the variable dump is written to the standard output. The *filename* parameter is an expression that is converted to a string.

For each loaded program (programs loaded using the LOADPGM statement) and the main program (the program specified when the interpreter was started), all global variables are output. The output for each program starts with a line of the form:

```
GLOBAL VARIABLES Program="program-name"
```

Next the global variables are output.

Next the total number of bytes used and allocated by global variables is output:

```
TOTAL GLOBAL VARIABLE MEMORY USAGE: Used=ucount, Allocated=acount
```

Next the following line is output:

```
STACK VARIABLES:
```

Next the variables on the currently executing coroutine's variable stack are output.

The last line outputs the total number of bytes used and allocated for variables:

```
TOTAL MEMORY USAGE: Used=ucount, Allocated=acount
```

Each variable is output on a line on its own. The first item output for each variable is the variable name (if it has a name and the name is known). Variable names are only output if the program was compiled using the **v** compiler option. After the variable name follows the type. It is one of INTEGER, DATE, FLOAT, CHAR, STRING, RECORD, ARRAY INTEGER, ARRAY DATE, ARRAY FLOAT, ARRAY CHAR, ARRAY STRING or ARRAY RECORD.

If the variable type is CHAR, STRING, ARRAY CHAR or ARRAY STRING, the variable type is followed, in parenthesis, by the declared size for the CHAR or STRING.

Then, if the variable is an array, the number of dimensions is indicated by a corresponding number of [] pairs.

If the variable is of CHAR or STRING type, the length of the string value (as returned by STRLEN) and the size of the allocated string buffer are indicated as:

```
Len=length, Sz=size
```

If the variable is an array, the length of the array (the value returned by ARRAYSIZE) and the allocated number of elements is output as:

```
Len=length, Sz=size
```

If the variable is of ARRAY CHAR or ARRAY STRING type, the total length and allocated string buffer size for all CHAR or STRING variables in the array are output as:

```
StrLen=length, StrSz=size
```

If the variable is not an array or a record, the last thing output on the line is the value (for strings, only the first 20 characters are output):

```
Val=value
```

For arrays and records, the individual array elements and fields are output on indented lines following the variable.

Errors

The following value can be assigned to `$Error`:

```
$Error_DumpVariablesOpenFail
```

Failed to open the *filename* file.

Examples

If the following program:

```
DECLARE VARIABLE $array[] STRING;

FOR $i = 1 TO 10 {
    $array[$i] = "string " & $i;
}
DUMP VARIABLES "vardump";
FOR $i = 1 TO ARRAYSIZE($array) {
    PRINT $array[$i] & "\n";
}
```

is named *prog.s4* and compiled using the command:

```
c4edi -v prog.s4
```

The following will be written to the *vardump* file when it is executed:

```
GLOBAL VARIABLES Program="x.x4":
  $i INTEGER Val=11
  $i INTEGER Val=0
  INTEGER Val=10
  INTEGER Val=1
  STRING Len=7, Sz=0, Val="vardump"
  STRING Len=7, Sz=0, Val="string "
  STRING Len=1, Sz=0, Val="\n"
  $array ARRAY STRING[] Len=10, Sz=16, StrLen=81, StrSz=640
    STRING Len=8, Sz=64, Val="string 1"
    STRING Len=8, Sz=64, Val="string 2"
    STRING Len=8, Sz=64, Val="string 3"
    STRING Len=8, Sz=64, Val="string 4"
    STRING Len=8, Sz=64, Val="string 5"
    STRING Len=8, Sz=64, Val="string 6"
    STRING Len=8, Sz=64, Val="string 7"
    STRING Len=8, Sz=64, Val="string 8"
    STRING Len=8, Sz=64, Val="string 9"
    STRING Len=9, Sz=64, Val="string 10"
  TOTAL GLOBAL VARIABLE MEMORY USAGE: Used=929, Allocated=1488
STACK VARIABLES:
TOTAL MEMORY USAGE: Used=929, Allocated=1488
```


12.4.46 ESCAPE

Name

ESCAPE – Function that inserts escape characters into a string.

Syntax

ESCAPE(string[, escch[, escape]])

Description

The ESCAPE function inserts escape characters into a string. The string to insert escape characters into is *string*. The character to use as escape character is the first character in *escch* and the characters to escape are all the characters in the string *escape*. The modified string is returned. All arguments are *expressions* that are converted to strings.

The default for the *escch* argument is the current EDI release character. If the release character is undefined (`$EdiRelChar = " "`), no characters are inserted before the characters to be escaped. Specifying an empty string also causes the EDI release character to be used.

The default value for *escape* is an empty string which is equivalent to a string containing all the EDI separator characters specified by the following reserved variables (reserved variables set to a space do not cause the *escch* character to be inserted):

- Segment separator `$EdiSegSep`
- Element separator `$EdiElemSep`
- Component element separator `$EdiCompSep`
- Segment code separator `$EdiSegCodeSep`
- Pad character `$EdiPadChar`
- Release character `$EdiRelChar`

The ESCAPE function can be used in *const-expressions* provided all arguments are *const-expressions* and both the *escch* and *escape* arguments are specified and not empty strings.

Examples

Executing the statement:

```
PRINT ESCAPE("A:B.C", "\\ ", " :. ") & "\n";
```

produces the output:

```
A\:B\.C
```

12.4.47 EXEC SQL ALTER TABLE

Name

EXEC SQL ALTER TABLE – Add columns to an existing base table.

Syntax

```
EXEC SQL [DBID dbid] ALTER TABLE base-table-name
    {ADD column-identifier data-type |
    ADD (column-identifier data-type [, column-identifier data-type] ... ) };
```

Description

The EXEC SQL ALTER TABLE statement adds one or more columns to *base-table-name*.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

See EXEC SQL CREATE TABLE, paragraph [12.4.51](#), for a discussion of possible values for the *data-type* parameter.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_AlterTableNoSupport`
The EXEC SQL ALTER TABLE statement is not supported.
- `$Error_AlterTableNotOpen`
The database is not open.
- `$Error_AlterTableExecError`
An error occurred when executing the EXEC SQL ALTER TABLE statement.
- `$Error_AlterTableIllegal`
The EXEC SQL ALTER TABLE statement was illegal.

Examples

```
EXEC SQL ALTER TABLE BaseTableName ADD Column REAL;
```

Notes

The Sybase database does not support the

`ADD (column-identifier data-type [, column-identifier data-type] ... )` variant.

12.4.48 EXEC SQL BEGIN WORK

Name

EXEC SQL BEGIN WORK – Start a transaction.

Syntax

```
EXEC SQL [DBID dbid] BEGIN WORK;
```

Description

The EXEC SQL BEGIN WORK statement starts a transaction, that is, a sequence of database operations that are terminated by the EXEC SQL COMMIT WORK or EXEC SQL ROLLBACK WORK statements.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to \$Error:

- \$Error_BeginWorkNoSupport
The EXEC SQL BEGIN WORK statement is not supported.
- \$Error_BeginWorkNotOpen
The database is not open.
- \$Error_BeginWorkBeginError
The execution of the EXEC SQL BEGIN WORK statement caused an error.

Examples

See EXEC SQL COMMIT WORK, paragraph [12.4.49](#), for an example.

See also

EXEC SQL COMMIT WORK, EXEC SQL ROLLBACK WORK.

Notes

This statement is not part of the X/Open standard.

In Oracle and MySQL, the EXEC SQL BEGIN WORK statement is redundant and has no effect. A new transaction is always started when connecting to the database and when the previous transaction is terminated.

For the AS/400 Native database interface, the EXEC SQL BEGIN WORK statement is redundant. The statement is allowed, but will have no effect because the start of a transaction is implicit on the AS/400. If the database is opened using commitment control, a new transaction is automatically started at the first SQL statement after the DATABASE OPEN and each subsequent EXEC SQL COMMIT WORK. No EXEC SQL BEGIN WORK statement is required. Therefore, if the database is opened using commitment control, any updates will not be committed unless the EXEC SQL COMMIT WORK statement is used, even if no EXEC SQL BEGIN WORK was issued.

12.4.49 EXEC SQL COMMIT WORK

Name

EXEC SQL COMMIT WORK – Terminate the current transaction.

Syntax

EXEC SQL [DBID *dbid*] COMMIT WORK;

Description

The EXEC SQL COMMIT WORK statement terminates the current transaction. Any changes to the database that were made by the terminated transaction are committed.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_CommitNoSupport`
The EXEC SQL COMMIT WORK statement is not supported.
- `$Error_CommitNotOpen`
The database is not open.
- `$Error_CommitCommitError`
An error occurred when executing the EXEC SQL COMMIT WORK statement.

Examples

```
EXEC SQL BEGIN WORK;  
EXEC SQL INSERT INTO tbl (c1, c2) VALUES (:c1, :c2);  
EXEC SQL COMMIT WORK;
```

See also

EXEC SQL BEGIN WORK, EXEC SQL ROLLBACK WORK.

Notes

For the AS/400 Native database interface, the SQL API used does not support cursors with hold. This means that it is not possible to issue an EXEC SQL COMMIT WORK inside an EXEC SQL SELECT statement block.

12.4.50 EXEC SQL CREATE INDEX

Name

EXEC SQL CREATE INDEX – Create an index on a base table.

Syntax

```
EXEC SQL [DBID dbid] CREATE [UNIQUE] INDEX index-name  
ON base-table-name  
(column-identifier [ASC | DESC] [, column-identifier [ASC | DESC]] ... );
```

Description

The EXEC SQL CREATE INDEX statement creates an index named *index-name* on the existing base table *base-table-name*.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_CreateIndexNoSupport`
The EXEC SQL CREATE INDEX statement is not supported.
- `$Error_CreateIndexNotOpen`
The database is not open.
- `$Error_CreateIndexExecError`
An error occurred when executing the EXEC SQL CREATE INDEX statement.
- `$Error_CreateIndexIllegal`
The EXEC SQL CREATE INDEX statement was illegal.

Examples

```
EXEC SQL CREATE INDEX IndexName ON BaseTableName
                        (Column1, Column2 ASC, Column3 DESC);
```

See also

EXEC SQL DROP INDEX.

Notes

The Sybase database does not support the ASC or DESC keywords.

12.4.51 EXEC SQL CREATE TABLE

Name

EXEC SQL CREATE TABLE – Create a base table.

Syntax

```
EXEC SQL [DBID dbid] CREATE TABLE base-table-name
        (column-identifier data-type [NOT NULL]
        [, column-identifier data-type [NOT NULL]] ... );
```

Description

The EXEC SQL CREATE TABLE statement creates a base table named *base-table-name*.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The *data-type* parameter can be one of:

CHARACTER[(*n*)] or CHAR[(*n*)]
CHARACTER VARYING(*n*) or VARCHAR(*n*)
DECIMAL[(*p* [, *s*))] or DEC[(*p* [, *s*))]
DOUBLE PRECISION
FLOAT[(*p*)]
INTEGER or INT
NUMERIC[(*p* [, *s*))]
REAL
SMALLINT

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_CreateTableNoSupport`
The EXEC SQL CREATE TABLE statement is not supported.
- `$Error_CreateTableNotOpen`
The database is not open.
- `$Error_CreateTableExecError`
An error occurred when executing the EXEC SQL CREATE TABLE statement.
- `$Error_CreateTableIllegal`
The EXEC SQL CREATE TABLE statement was illegal.

Examples

```
EXEC SQL CREATE TABLE BaseTableName
                        (Column1 CHARACTER, Column2 INTEGER);
```

See also

EXEC SQL DROP TABLE.

Notes

The Informix-SE database does not support the CHARACTER VARYING (or VARCHAR) data type.

The Sybase database does not support the DECIMAL, DOUBLE PRECISION or NUMERIC data types and no precision can be specified for FLOAT.

For the AS/400 Native database interface, the statement:

```
EXEC SQL CREATE TABLE tablename;
```

creates the table *tablename* in the current library (CURLIB) of the job running the Message Builder program. This library name can be changed using the CHGCURLIB command.

12.4.52 EXEC SQL CREATE VIEW

Name

EXEC SQL CREATE VIEW – Create a viewed table.

Syntax

```
EXEC SQL [DBID dbid] CREATE VIEW viewed-table-name
    [(column-identifier [, column-identifier] ... )]
AS query-specification [WITH CHECK OPTION];
```

Description

The EXEC SQL CREATE VIEW statement creates a viewed table named *viewed-table-name* that is the result of evaluating *query-specification*.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_CreateViewNoSupport`
The EXEC SQL CREATE VIEW statement is not supported.
- `$Error_CreateViewNotOpen`
The database is not open.
- `$Error_CreateViewExecError`
An error occurred when executing the EXEC SQL CREATE VIEW statement.
- `$Error_CreateViewIllegal`
The EXEC SQL CREATE VIEW statement was illegal.

Examples

```
EXEC SQL CREATE VIEW ViewedTableName
    AS SELECT Age FROM Persons WHERE Age = 19;
```

See also

EXEC SQL DROP VIEW.

12.4.53 EXEC SQL DELETE

Name

EXEC SQL DELETE – Delete rows from a table.

Syntax

```
EXEC SQL [DBID dbid] DELETE FROM table-name  
[WHERE search-condition];
```

Description

The EXEC SQL DELETE statement deletes zero or more rows from the table identified by *table-name*.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_DeleteNoSupport`
The EXEC SQL DELETE statement is not supported.
 - `$Error_DeleteNotOpen`
The database is not open.
 - `$Error_DeleteExecError`
An error occurred when executing the EXEC SQL DELETE statement.
 - `$Error_DeleteIllegal`
The EXEC SQL DELETE statement was illegal.
-

Examples

```
EXEC SQL DELETE FROM Employee WHERE Name = 'Brown';  
EXEC SQL DELETE FROM Employee WHERE Name = :$Name;
```

12.4.54 EXEC SQL DROP INDEX

Name

EXEC SQL DROP INDEX – Remove an index.

Syntax

```
EXEC SQL [DBID dbid] DROP INDEX index-name;
```

Description

The EXEC SQL DROP INDEX statement removes the index named *index-name* from the database.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_DropIndexNoSupport`
The EXEC SQL DROP INDEX statement is not supported.
- `$Error_DropIndexNotOpen`
The database is not open.
- `$Error_DropIndexExecError`
An error occurred when executing the EXEC SQL DROP INDEX statement.
- `$Error_DropIndexIllegal`
The EXEC SQL DROP INDEX statement was illegal.

Examples

```
EXEC SQL DROP INDEX IndexName;
```

See also

EXEC SQL CREATE INDEX.

12.4.55 EXEC SQL DROP TABLE

Name

EXEC SQL DROP TABLE – Destroy a table.

Syntax

```
EXEC SQL [DBID dbid] DROP TABLE base-table-name  
[CASCADE | RESTRICT];
```

Description

The base table named *base-table-name* is removed from the database.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_DropTableNoSupport`
The EXEC SQL DROP TABLE statement is not supported.
- `$Error_DropTableNotOpen`
The database is not open.
- `$Error_DropTableExecError`

An error occurred when executing the EXEC SQL DROP TABLE statement.

- `$Error_DropTableIllegal`
The EXEC SQL DROP TABLE statement was illegal.

Examples

```
EXEC SQL DROP TABLE BaseTableName;
```

See also

EXEC SQL CREATE TABLE.

Notes

Oracle and Sybase databases do not support the CASCADE and RESTRICT keywords.

12.4.56 EXEC SQL DROP VIEW

Name

EXEC SQL DROP VIEW – Destroy a view.

Syntax

```
EXEC SQL [DBID dbid] DROP VIEW viewed-table-name  
[CASCADE | RESTRICT];
```

Description

The viewed table named *viewed-table-name* is removed from the database.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_DropViewNoSupport`
The EXEC SQL DROP VIEW statement is not supported.
- `$Error_DropViewNotOpen`
The database is not open.
- `$Error_DropViewExecError`
An error occurred when executing the EXEC SQL DROP VIEW statement.
- `$Error_DropViewIllegal`
The EXEC SQL DROP VIEW statement was illegal.

Examples

```
EXEC SQL DROP VIEW ViewedTableName;
```

See also

EXEC SQL CREATE VIEW.

Notes

Oracle and Sybase databases do not support the CASCADE and RESTRICT keywords.

12.4.57 EXEC SQL GRANT

Name

EXEC SQL GRANT – Grant privileges on a table.

Syntax

```
EXEC SQL [DBID dbid] GRANT {ALL [PRIVILEGES] |  
    grant-privilege [, grant-privilege] ... }  
    ON table-name  
    TO {PUBLIC | user-name [, user-name] ... }  
    [WITH GRANT OPTION];
```

Description

The GRANT statement grants privileges on *table-name* to one or more users.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_GrantNoSupport`
The EXEC SQL GRANT statement is not supported.
- `$Error_GrantNotOpen`
The database is not open.
- `$Error_GrantExecError`
An error occurred when executing the EXEC SQL GRANT statement.
- `$Error_GrantIllegal`
The EXEC SQL GRANT statement was illegal.

Examples

```
EXEC SQL GRANT ALL ON TableName TO PUBLIC;
```

See also

EXEC SQL REVOKE.

Notes

ODBC databases do not support the PRIVILEGES keyword.

12.4.58 EXEC SQL INSERT

Name

EXEC SQL INSERT – Insert rows into a table.

Syntax

```
EXEC SQL [DBID dbid] INSERT INTO table-name
    [(column-identifier [, column-identifier] ...)]
    {query-specification | VALUES (insert-value [, insert-value] ...)};
```

Description

Zero or more rows are inserted into the table identified by *table-name*.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_InsertNoSupport`
The EXEC SQL INSERT statement is not supported.
- `$Error_InsertNotOpen`
The database is not open.
- `$Error_InsertExecError`
An error occurred when executing the EXEC SQL INSERT statement.
- `$Error_InsertIllegal`
The EXEC SQL INSERT statement was illegal.

Examples

The following example inserts one row into the `persons` table:

```
EXEC SQL INSERT INTO persons (name, age, phone_no)
    VALUES ('Susan', :$age, '36 12 42');
```

The next example demonstrates how a null value can be inserted into the `age` column:

```
$ind = -1;
EXEC SQL INSERT INTO persons (name, age, phone_no)
    VALUES ('Susan', :$age :$ind, '36 12 42');
```

12.4.59 EXEC SQL REVOKE

Name

EXEC SQL REVOKE – Revoke privileges on a table.

Syntax

```
EXEC SQL [DBID dbid] REVOKE
    {ALL [PRIVILEGES] | revoke-privilege [, revoke-privilege] ... }
    ON table-name
    FROM {PUBLIC | user-name [, user-name] ... }
    [CASCADE | RESTRICT];
```

Description

The EXEC SQL REVOKE statement revokes privileges on *table-name* that are held by one or more of the specified users.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_RevokeNoSupport`
The EXEC SQL REVOKE statement is not supported.
- `$Error_RevokeNotOpen`
The database is not open.
- `$Error_RevokeExecError`
An error occurred when executing the EXEC SQL REVOKE statement.
- `$Error_RevokeIllegal`
The EXEC SQL REVOKE statement was illegal.

Examples

```
EXEC SQL REVOKE ALL ON TableName FROM PUBLIC;
```

See also

EXEC SQL GRANT.

Notes

Informix-SE and Oracle databases do not support the CASCADE or RESTRICT keywords. Sybase databases do not support the RESTRICT keyword.

ODBC databases do not support the PRIVILEGES keyword.

12.4.60 EXEC SQL ROLLBACK WORK

Name

EXEC SQL ROLLBACK WORK – Cancel the current transaction.

Syntax

EXEC SQL [DBID *dbid*] ROLLBACK WORK;

Description

The current transaction is terminated. Any changes to the database that were made by the terminated transaction are canceled.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_RollbackNoSupport`
The EXEC SQL ROLLBACK WORK statement is not supported.
- `$Error_RollbackNotOpen`
The database is not open.
- `$Error_RollbackRollbackError`
An error occurred when executing the EXEC SQL ROLLBACK WORK statement.

Examples

```
EXEC SQL ROLLBACK WORK;
```

See also

EXEC SQL BEGIN WORK, EXEC SQL COMMIT WORK.

Notes

For the AS/400 Native database interface, the SQL API used does not support cursors with hold. This means that it is not possible to issue an EXEC SQL ROLLBACK WORK inside an EXEC SQL SELECT statement block.

12.4.61 EXEC SQL SELECT

Name

EXEC SQL SELECT – Retrieve data from a database.

Syntax

```
EXEC SQL [DBID dbid] SELECT [ALL | DISTINCT] select-list
      INTO variable-list
      FROM table-list
      [WHERE sql-search]
      [GROUP BY group-list]
      [HAVING sql-search]
      [ORDER BY order-by-list]
      statement-block
```

Description

This statement extends the X/Open SQL standard with an attached *statement-block*. This *statement-block* is repeated for each row returned by the EXEC SQL SELECT statement.

The INTO clause specifies the Message Builder destination variables (host variables) where the selected information will be put. For each variable in the INTO clause it is possible to specify an indicator variable, preferably of type INTEGER. The value of an indicator variable indicates a null value or truncated data. A null value is indicated by minus one and a truncated value by a positive value indicating the length of the non-truncated data. The indicator variable is 0 if the value is not null and is not truncated. When a null value is read, the destination variable is set to 0 for INTEGER and FLOAT variables, a string filled with spaces for CHAR variables, an empty string for STRING variables and the computer's base time for DATE variables. See the DECLARE VARIABLE statement, paragraph [12.4.41](#), for an explanation of the computer's base time.

When the select loop terminates, the INTO clause destination variables contain the values of the last read row. If the EXEC SQL SELECT statement did not return any rows or if an error occurred, the values of all the INTO variables are undefined.

In order to ensure compatibility between all supported database interfaces, trailing spaces are always truncated in strings returned from the database.

Remember that many databases, such as recent Oracle or Microsoft SQL Server, do not preserve the order of inserted rows, even if no rows have ever been deleted from that database table.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_SelectNoSupport`
The EXEC SQL SELECT statement is not supported.
- `$Error_SelectNotOpen`
The database is not open.
- `$Error_SelectExecError`
An error occurred when executing the EXEC SQL SELECT statement.
- `$Error_SelectFetchError`
An error occurred when fetching selected data.

- `$Error_SelectIllegal`
The EXEC SQL SELECT statement was illegal.

Examples

The following EXEC SQL SELECT loop prints a list of all employees older than 60 years:

```
DECLARE VARIABLE $Name STRING;
DECLARE VARIABLE $Age INTEGER;

EXEC SQL SELECT Name, Age
        INTO :$Name, :$Age
        FROM Employee
        WHERE Age > 60
        ORDER BY Name {
    PRINT "Name: " & $Name & ", Age: " & $Age & "\n";
}
```

The following example is similar to the previous one. The difference is that an indicator variable is used to determine whether the contents of the name column are null:

```
DECLARE VARIABLE $Name STRING;
DECLARE VARIABLE $Age INTEGER;
DECLARE VARIABLE $NameInd INTEGER;

EXEC SQL SELECT Name, Age
        INTO :$Name :$NameInd, :$Age
        FROM Employee
        WHERE Age > 60
        ORDER BY Name {
    IF $NameInd <> -1 {
        PRINT "Name: " & $Name & ", Age: " & $Age & "\n";
    }
}
```

Notes

EXEC SQL COMMIT WORK and EXEC SQL ROLLBACK WORK statements should not be used inside select loops.

If a destination variable is of CHAR or STRING data type, the size specification given when the variable was declared is used as the size of the destination buffer when requesting data from the RDBMS. If a destination variable is of STRING type and no size was specified when it was declared, the destination buffer size 255 is used. See the DECLARE VARIABLE statement, paragraph [12.4.41](#), for further information.

If an array is used to store the result of an EXEC SQL SELECT statement, such as in the following example:

```
DECLARE VARIABLE $Name[] STRING;
DECLARE VARIABLE $Empty[] CONSTANT STRING;
DECLARE VARIABLE $i INTEGER;

$Name = $Empty;      /* make the array empty */
$i = 1;
EXEC SQL SELECT Name
        INTO :$Name[$i]
        FROM Employee
        WHERE Age > 60
        ORDER BY Name {
    $i = $i + 1;
}
```


the size of the `$Name` array will be one greater than the number of rows read from the table. The reason is that the size of the array is expanded before each attempt to fetch data from the database and the last attempt that does not return any data will also expand the array. The extra element will contain an empty string if it is a string variable, or a value of zero otherwise.

When the select loop terminates because all the rows have been read, the `$$sqlCode` reserved variable is set to 0.

For the AS/400 Native database interface, the following syntax applies for accessing tables:

```
EXEC SQL SELECT FROM "libraryname/tablename"
```

In this case, the table *tablename* in the specified library *libraryname* will be used.

The statement:

```
EXEC SQL SELECT FROM tablename;
```

uses the table *tablename* in the library list (LIBL) of the job running the Message Builder program. This library list is normally set in the job description of a job, and can be changed using the ADDLIB, CHGLIB, CHGSYSLIB, EDTLIB, and RMVLIB commands.

For the AS/400 Native database interface, extra care must be taken when doing a comparison in which the LIKE clause and a variable are used. The reason is that tables have fixed width columns. The fixed width columns that contains character and string values are padded with blanks to fill out any space not taken up by the value itself, and these blanks are significant when LIKE comparisons are made. For example, a column value of:

```
"Text      "
```

will not match a variable value of:

```
"Text"
```

because the trailing blanks are significant. However, it will match a variable value of:

```
"Text*"
```

because the wildcard allows for all subsequent characters.

For the AS/400 Native database interface, the SQL API used does not support cursors with hold. This means that it is not possible to issue an EXEC SQL COMMIT WORK or an EXEC SQL ROLLBACK WORK inside an EXEC SQL SELECT statement block.

12.4.62 EXEC SQL UPDATE

Name

EXEC SQL UPDATE – Update values of columns in a table.

Syntax

```
EXEC SQL [DBID dbid] UPDATE table-name
      SET column-identifier = update-value [, column-identifier = update-value] ...
      WHERE search-condition;
```

Description

Zero or more rows of the table identified by *table-name* are updated.

The *dbid* parameter is an *expression* that is converted to an integer. It is used to identify the database connection and has previously been returned by the DATABASE OPEN statement. This parameter does not have to be specified by Message Builder programs that only use one simultaneous database connection.

The DATABASE OPEN statement must be executed before this statement is used.

Errors

The following values can be assigned to `$Error`:

- `$Error_UpdateNoSupport`
The EXEC SQL UPDATE statement is not supported.
- `$Error_UpdateNotOpen`
The database is not open.
- `$Error_UpdateExecError`
An error occurred when executing the EXEC SQL UPDATE statement.
- `$Error_UpdateIllegal`
The EXEC SQL UPDATE statement was illegal.

Examples

```
EXEC SQL UPDATE Employee SET Address = :$newAddress
WHERE Name = :$name AND Address = :$oldAddress;
```

12.4.63 EXECUTE

Name

EXECUTE – Execute a Message Builder program.

Syntax

EXECUTE program-name ARGUMENTS arguments;

Description

The EXECUTE statement executes a Message Builder program. The executed program will run in parallel with all of the already executing programs.

The *program-name* parameter is the file name of the Message Builder program to execute. It is an *expression* that is converted to a string. The *arguments* parameter is a string vector (one-dimensional array) containing all of the arguments, that is, the strings that will be returned by the ARGUMENT function (see paragraph [12.4.2](#)) in the newly executed program. Note that the first element in this array is the program name, that is, the string returned when the ARGUMENT function is called with a 0 as argument.

The Message Builder interpreter uses the same search path to find the program file specified in the EXECUTE statement as it does for the program file specified on the Message Builder interpreter command line. If *program-name* has the extension ".s4", it is replaced by ".x4".

Errors

The following values can be assigned to `$Error`:

- `$Error_ProgramOpenError`
Failed to open the executable Message Builder program file.
- `$Error_ProgramEmpty`
The Message Builder program file is empty.
- `$Error_ProgramReadError`
Error when reading the Message Builder program file.

Examples

The following program is put in the file `ex1.s4`:

```
DECLARE VARIABLE $a[] STRING;  
PRINT "Program 1, first output\n";  
$a[1] = "ex2.s4";  
EXECUTE "ex2.s4" ARGUMENTS $a;  
SUSPEND 0;  
PRINT "Program 1, second output\n";
```

And the following program is put in `ex2.s4`:

```
PRINT "Program 2, first output\n";  
SUSPEND 0;  
PRINT "Program 2, second output\n";
```

If the above programs are compiled and executed using the commands:

```
c4edi ex1.s4  
c4edi ex2.s4  
r4edi ex1.s4
```

the following is written to the output file:

```
Program 1, first output  
Program 2, first output  
Program 1, second output  
Program 2, second output
```

See also

SUSPEND.

12.4.64 EXIT

Name

EXIT – Exit the Message Builder program.

Syntax

EXIT [*value* [FORCE]];

Description

The EXIT statement closes all open files and external interfaces and then terminates the Message Builder process with exit status *value*. The *value* parameter is an *expression* that is converted to an integer. The default for *value* is 0.

When the interpreter runs in server mode (see paragraph [11.3.3](#)), only the program that executes that EXIT statement terminates, not the interpreter itself. By specifying the FORCE modifier all the programs running in the interpreter and the interpreter itself are terminated.

Examples

Exit with good status:

```
EXIT;
```

Exit with good status:

```
EXIT 0;
```

Exit with bad status:

```
EXIT 1;
```

Exit with the status specified by the \$exitValue variable:

```
EXIT $ExitValue;
```

Notes

On UNIX, Windows and AS/400 systems, the convention is that an exit status of zero indicates success. A non-zero value (a value of 1 is most common) normally indicates some type of error.

Reaching the end of a Message Builder program is equivalent to executing the statement:

```
EXIT 0;
```

12.4.65 EXPORT

Name

EXPORT – Export a value to the environment.

Syntax

```
EXPORT value AS environ-var;
```

Description

The EXPORT statement assigns a value to the specified environment variable.

The *value* parameter is an *expression* that is converted to a string and used as the value of the environment variable. The *environ-var* parameter is also an *expression* that is converted to a string and used as the environment variable name.

The EXPORT statement is typically used to transfer information to another Message Builder program when using the EXECUTE or SWITCH statements or when executing an external program using the CALL or PIPE OPEN statements.

Errors

The following value can be assigned to `$Error`:

```
$Error_ExportNameEmpty
```

The *environ-var* expression evaluated to an empty string.

Examples

If the *module1.s4* program exports an environment variable named AGE:

```
DECLARE VARIABLE $MyAge INTEGER;
$MyAge = 35;
EXPORT $MyAge AS "AGE";
SWITCH "module2.s4";
```

the *module2.s4* program can import the AGE environment variable:

```
DECLARE VARIABLE $HisAge FLOAT;
IMPORT "AGE" INTO $HisAge;
```

See also

CALL, EXECUTE, IMPORT, PIPE OPEN, RELEASE, SWITCH.

Notes

Changes of environment variables are local to the Message Builder interpreter process that modifies them. If another program is started using the CALL statement and that program modifies the environment variables, the changes will not be visible to the calling Message Builder program when the CALL statement terminates. However, changes made by the calling Message Builder program are visible to the called program.

When multiple Message Builder programs run in parallel inside the same interpreter (see the EXECUTE statement, paragraph [12.4.63](#)), environment variables can be used for communication between the programs. All exported variables are visible to all the programs.

All variables, except the reserved ones, are reset and forgotten when changing program using the SWITCH statement. The EXPORT statement together with the IMPORT statement provides a way to share information between different Message Builder programs.

For UNIX and AS/400 systems, the name of environment variables and the contents of them are case sensitive; for example, the environment variable names ENV and Env refer to two different variables. For Windows systems, the name of environment variables is not case sensitive.

Only variables that are valid environment variable names are exported to the environment. Invalid environment variable names can only be used to communicate between Message Builder programs run by the same interpreter process. A valid environment variable names starts with an alphabetic character and the remaining characters are alphanumeric or underscore characters.

When an interpreter is started, the initial set of environment variables is inherited from the environment.

12.4.66 EXPRESSIONS

Name

EXPRESSIONS – Message Builder expressions.

Description

A Message Builder expression consists of *variables*, integer and floating-point constants, string constants, arithmetic, comparison, logical, and bitwise operators, the string concatenation and sub-string operators and calls to built-in and user functions. Parentheses may be used to change the operator precedence.

If an array with index is used in an expression, the index must be greater than or equal to 1 and less than or equal to the size of the array.

Otherwise, an AGGREGATE exception is generated and, if the exception is not caught using the TRY statement (see paragraph [12.4.131](#)), the Message Builder interpreter terminates.

Constants and *variables* of different types may be freely intermixed in an expression. Conversions to appropriate types are performed automatically. The operator determines which type of conversion is performed (see "OPERATORS" below).

When converting a STRING to a FLOAT Message Builder scans the string from the first character and extracts the longest sub-string that is a valid string representation of a FLOAT. If there is no such sub-string, the STRING converts to 0.000000. Similarly, when converting a STRING to an INTEGER Message Builder scans the string from the first character and extracts the longest sub-string that is a valid string representation of an INTEGER. If there is no such sub-string, the STRING converts to 0.

The following table gives some examples of type conversion.

Conversion Type	Example
STRING to FLOAT	"12.52" converts to 12.520000
STRING to FLOAT	"12,52" converts to 12.000000
STRING to FLOAT	"AA12.52" converts to 0.000000
STRING to INTEGER	"12.52" converts to 12
STRING to INTEGER	"12,52" converts to 12
STRING to INTEGER	"AA12.52" converts to 0

Refer to the description of the `$DateFormat` reserved variable on page 235 for details of type conversion between STRING and DATE types.

The parts of an expression are not guaranteed to be evaluated in any particular order. All parts of an expression are always evaluated, regardless of whether it is necessary or not (for example, AND and OR).

The following unary and binary operators can be used in an expression:

Operators	Meaning
-	Unary minus arithmetic operator. The operand is converted to a FLOAT value and the result is a FLOAT.

Operators	Meaning
*	Multiply arithmetic operator. The operands are converted to FLOAT values and the result is a FLOAT.
/	Divide arithmetic operator. The operands are converted to FLOAT values and the result is a FLOAT.
%	Modulus arithmetic operator. The operands are converted to FLOAT values and the result is a FLOAT.
+	Add arithmetic operator. The operands are converted to FLOAT values and the result is a FLOAT.
-	Subtract arithmetic operator. The operands are converted to FLOAT values and the result is a FLOAT.
<	Less than comparison operator. If both operands are strings, the operands are compared lexically. Otherwise the operands are converted to FLOAT values and the comparison is made arithmetically. The result is an INTEGER with value 1 if the comparison is true or 0 if it is false.
<=	Less than or equal to comparison operator. The description of the operands and the result is the same as for the less than operator.
>	Greater than comparison operator. The description of the operands and the result is the same as for the less than operator.
>=	Greater than or equal to comparison operator. The description of the operands and the result is the same as for the less than operator.
=	Equal to comparison operator. The description of the operands and the result is the same as for the less than operator.
<>	Not equal to comparison operator. The description of the operands and the result is the same as for the less than operator.
NOT	Logical not operator. The operand is converted to a FLOAT value and the result is an INTEGER with value 1 if the operand is 0, or 0 if the operand is non-zero.
AND	Logical and operator. The operands are converted to FLOAT values and the result is an INTEGER with value 1 if both operands are non-zero, or 0 otherwise.
OR	Logical OR operator. The operands are converted to FLOAT values and the result is an INTEGER with value 1 if at least one of the operands is non-zero, or 0 otherwise.
XOR	Logical EXCLUSIVE-OR operator. The operands are converted to FLOAT values and the result is an INTEGER with value 1 if exactly one of the operands is non-zero, or 0 otherwise.
BNOT	Bitwise NOT operator. The operand is converted to an INTEGER value and the result is an INTEGER. If a bit in the argument is zero, the corresponding bit in the result becomes one and vice versa.
BAND	Bitwise AND operator. The operands are converted to INTEGER values and the result is an INTEGER. A bit in the result becomes one if both of the corresponding bits in the arguments are one. Otherwise it becomes zero.
BOR	Bitwise OR operator. The operands are converted to INTEGER values and the result is an INTEGER. A bit in the result becomes zero if both of the corresponding bits in the arguments are zero. Otherwise it becomes one.
BXOR	Bitwise EXCLUSIVE-OR operator. The operands are converted to INTEGER values and the result is an INTEGER. A bit in the result becomes zero if the corresponding bits in the arguments are equal. Otherwise it becomes one.
BSHIFT	Shift operator. The operands are converted to INTEGER values and the result is an INTEGER. If the value of the right argument is positive, the left argument is shifted left the number of positions specified by the right argument. If the value of the right argument is negative, the left argument is shifted right the number of positions specified by the absolute value of the right argument.
&	String concatenation operator. Both operands are converted to string values and the result is of type STRING.

The sub-string extraction and array indexing operators are also supported (see paragraph [12.4.121](#) for a description of the sub-string operator).

The following table lists the unary and binary operators in decreasing precedence order. Operators listed on the same line have equal precedence. Note that the precedence order can be changed using parentheses:

- - (unary minus)
- * / %
- + - (binary minus)
- BNOT
- BAND
- BOR BXOR
- BSHIFT
- &
- = <> < <= > >=
- NOT
- AND
- OR XOR

Errors

The following values can be assigned to `$Error`:

- `$Error_DivideByZero`
Division by zero error when using one of the divide or modulus operators.
- `$Error_Aggregate_NotAnAggregate`
An array index is specified for a non-array variable or too many indices are specified for an array variable. This is an AGGREGATE error.
- `$Error_Aggregate_AggregateInvalid`
Illegal usage of array or record variable. This is an AGGREGATE error.
- `$Error_Aggregate_OutsideArray`
Array index out of bounds. This is an AGGREGATE error.

Examples

The example

```
DECLARE VARIABLE $a INTEGER;  
$a = 4;  
PRINT $a & "\n";
```

will produce the output

4

while the following example

```
DECLARE VARIABLE $a INTEGER;  
$a = 1;  
PRINT 2 * ($a + 1) & "\n";
```

produces the output

4.000000

The reason is that the first PRINT statement outputs an integer, while the second outputs a floating-point number.

See also

ASSIGNMENT, CONSTANTS, VARIABLE TYPES.

12.4.67 FIDENT

Name

FIDENT – Function that converts a function identifier to a string.

Syntax

FIDENT(identifier)

Description

Given a function name identifier, the FIDENT function returns a string containing the function name. It also exports the function (see the PRAGMA directive, paragraph [12.4.92](#)).

The *identifier* argument is the function name identifier. Note that it should not be surrounded by double quotes.

If the function specified by *identifier* is located inside a module, the returned string includes a module prefix. All alphabetic letters in the returned string are uppercase letters.

Examples

The assignment:

```
$s = FIDENT(DEMOFUNC);
```

is equivalent to:

```
PRAGMA EXPORT FUNCTION DEMOFUNC;  
$s = "DEMOFUNC";
```

See also

SIDENT.

Notes

The FIDENT function is convenient to use together with the USE statement.

12.4.68 FILE APPEND

Name

FILE APPEND – Append a file to another file.

Syntax

FILE APPEND *srcname* TO *dstname*;

Description

The file specified by *srcname* is appended to the file specified by *dstname*. If *dstname* does not exist, it is created. Both *srcname* and *dstname* (if it exists) must be regular files.

The *srcname* and *dstname* parameters are both *expressions* that are converted to strings.

Errors

The following values can be assigned to `$Error`:

- `$Error_FappendNoSource`
The source file name is empty.
- `$Error_FappendNoDest`
The destination file name is empty.
- `$Error_FappendSrcMissing`
The source file does not exist.
- `$Error_FappendSrcNotRegular`
The source is not a regular file.
- `$Error_FappendDstNotRegular`
The destination file does exist and it is not a regular file.
- `$Error_FappendSameFile`
The source and destination files are the same.
- `$Error_FappendCopyFail`
An error occurred while copying the file specified by *srcname*.

Examples

A UNIX and AS/400 example:

```
/* get the Integration Broker home directory */
DECLARE VARIABLE $root STRING;
IMPORT "CORE_ROOT" INTO $root;
/* get file name command line argument */
DECLARE VARIABLE $srcFile STRING;
$srcFile = ARGUMENT(1);
/* append the file */
FILE APPEND $srcFile TO $root & "/append.tmp";
```

A Windows example:

```
/* get the Integration Broker home directory */
DECLARE VARIABLE $root STRING;
IMPORT "CORE_ROOT" INTO $root;
/* get file name command line argument */
DECLARE VARIABLE $srcFile STRING;
$srcFile = ARGUMENT(1);
/* append the file */
FILE APPEND $srcFile TO $root & "\\append.tmp";
```

See also

FILE COPY, FILE DELETE, FILE MOVE.

12.4.69 FILE COPY

Name

FILE COPY – Copy a file.

Syntax

FILE COPY *srcname* TO *dstname*;

Description

A copy named *dstname* is made of the file specified by *srcname*. If the file named *dstname* already exists, it is overwritten. Both *srcname* and *dstname* (if it exists) must be regular files.

The *srcname* and *dstname* parameters are both *expressions* that are converted to strings.

Errors

The following values can be assigned to `$Error`:

- `$Error_FcopyNoSource`
The source file name is empty.
- `$Error_FcopyNoDest`
The destination file name is empty.
- `$Error_FcopySrcMissing`
The source file does not exist.
- `$Error_FcopySrcNotRegular`
The source is not a regular file.
- `$Error_FcopyDstNotRegular`
The destination file does exist and it is not a regular file.
- `$Error_FcopySameFile`
The source and destination files are the same.
- `$Error_FcopyCopyFail`
An error occurred while copying the file.

Examples

A UNIX and AS/400 example:

```
/* get the Integration Broker home directory */
DECLARE VARIABLE $root STRING;
IMPORT "CORE_ROOT" INTO $root;
/* get file name command line argument */
DECLARE VARIABLE $srcFile STRING;
$srcFile = ARGUMENT(1);
/* copy the file */
FILE COPY $srcFile TO $root & "/copy.tmp";
```

A Windows example:

```
/* get the Integration Broker home directory */
DECLARE VARIABLE $root STRING;
IMPORT "CORE_ROOT" INTO $root;
```

```
/* get file name command line argument */
DECLARE VARIABLE $srcFile STRING;
$srcFile = ARGUMENT(1);
/* copy the file */
FILE COPY $srcFile TO $root & "\\copy.tmp";
```

See also

FILE APPEND, FILE DELETE, FILE MOVE.

12.4.70 FILE DELETE

Name

FILE DELETE – Delete a file.

Syntax

FILE DELETE *filename*;

Description

The file specified by *filename* is deleted. It must be a regular file.

The *filename* parameter is an *expression* that is converted to a string.

Errors

The following values can be assigned to \$Error:

- \$Error_FdeleteNoName
The name of the file to delete is empty.
- \$Error_FdeleteMissing
The file to delete does not exist.
- \$Error_FdeleteNotRegular
The file is not a regular file.
- \$Error_FdeleteRemoveFail
Failed to delete the file.

Examples

```
/* get file name command line argument */
DECLARE VARIABLE $file STRING;
$file = ARGUMENT(1);
/* delete the file */
FILE DELETE $file;
```

See also

FILE APPEND, FILE COPY, FILE MOVE.

12.4.71 FILE MOVE

Name

FILE MOVE – Move and/or rename a file.

Syntax

FILE MOVE *oldname* TO *newname*;

Description

The file named *oldname* is renamed to *newname*. The *oldname* and *newname* parameters are both *expressions* that are converted to strings.

This statement can be used to change the name of a file, to move a file from one directory to another, or both. Both *oldname* and *newname* (if it exists) must be regular files.

If *newname* does exist, it is first removed.

Errors

The following values can be assigned to `$Error`:

- `$Error_FmoveNoSource`
The old file name is empty.
- `$Error_FmoveNoDest`
The new file name is empty.
- `$Error_FmoveSrcMissing`
The old file does not exist.
- `$Error_FmoveSrcNotRegular`
The old file is not a regular file.
- `$Error_FmoveDstNotRegular`
A file with name *newname* does exist and it is not a regular file.
- `$Error_FmoveSameFile`
The old and new file names specify the same file.
- `$Error_FmoveRenameFail`
Failed to rename the file.
- `$Error_FmoveCopyFail`
Failed to copy the file. This may happen if *oldname* and *newname* specifies files on different file systems.
- `$Error_FmoveRemoveFail`
Failed to remove the old file. This may happen if *oldname* and *newname* specifies files on different file systems. Both the *oldname* and *newname* files will exist.

Examples

A UNIX and AS/400 example:

```
/* get the Integration Broker home directory */
DECLARE VARIABLE $root STRING;
IMPORT "CORE_ROOT" INTO $root;
/* get file name command line argument */
```

```

DECLARE VARIABLE $oldFile STRING;
$oldFile = ARGUMENT(1);
/* rename/move the file */
FILE MOVE $oldFile TO $root & "/move.tmp";

```

A Windows example:

```

/* get the Integration Broker home directory */
DECLARE VARIABLE $root STRING;
IMPORT "CORE_ROOT" INTO $root;
/* get file name command line argument */
DECLARE VARIABLE $oldFile STRING;
$oldFile = ARGUMENT(1);
/* rename/move the file */
FILE MOVE $oldFile TO $root & "\\move.tmp";

```

See also

FILE APPEND, FILE COPY, FILE DELETE.

Notes

If the FILE MOVE statement is used to rename/move a file within the same file system, the operation is guaranteed to be atomic, that is, either the operation succeed or otherwise no change occur.

12.4.72 FOR

Name

FOR – Loop control.

Syntax

FOR loop-var = start {TO | DOWNT0} end [STEP step] statement-block

Description

The FOR statement executes a statement block once for each value of a loop variable.

Loop-var is the loop variable, *start* is the initial value of the loop variable, *end* is the terminating value and *step* is the loop increment/decrement value (the default is 1). The statements in *statement-block* are executed once for each value of the loop variable.

The *start*, *end* and *step* parameters are all *expressions* that are converted to integers. The *loop-var* variable is a scalar variable that is automatically declared as an integer in the *statement-block*. It cannot be modified and shadows variables with the same name declared outside the *statement-block*.

If the TO keyword is specified, *statement-block* will be executed as long as *loop-var* is less than or equal to *end*, and *step* will be added to the loop variable after each execution of *statement-block*. If *start* is greater than *end*, *statement-block* will not be executed.

If the DOWNT0 keyword is used, the loop will be executed as long as *loop-var* is greater than or equal to *end*, and *step* will be subtracted from the loop variable after each

execution of *statement-block*. If *start* is less than *end*, *statement-block* will not be executed.

The *start*, *end* and *step* parameters are all computed once, when the FOR loop is entered.

Examples

The following simple example

```
/* note that the $i variable isn't declared */
FOR $i = 1 TO 5 {
    PRINT "." & $i;
}
PRINT "\n";
```

will produce the following output

```
.1.2.3.4.5
```

The following code fragment exemplifies how the loop variable shadows a variable with the same name declared outside the FOR loop:

```
DECLARE VARIABLE $i INTEGER;
$i = 2;
FOR $i = 1 TO 2 * $i {
    PRINT $i & " ";
}
PRINT $i & "\n";
```

The above example writes the following to the output file:

```
1 2 3 4 2
```

Finally, the following code fragment exemplifies that the *end expression* is computed before the loop is entered:

```
DECLARE VARIABLE $a INTEGER;
$a = 5;
FOR $i = 1 TO $a {
    $a = 1;
    PRINT $i & " ";
}
PRINT $a & "\n";
```

The above example writes the following to the output file:

```
1 2 3 4 5 1
```

See also

BREAK, CONTINUE, REPEAT, WHILE.

12.4.73 FORMAT

Name

FORMAT – Function that formats a string.

Syntax

FORMAT(*format* [, *argument*, ...])

Description

The `FORMAT` function formats a string, using a format string (*format*) to control the result, and returns the formatted string. Except for a number of special sequences, the contents of the format string are copied as is to the returned string.

The *format* argument is an *expression* that is converted to a string. The remaining arguments are *expressions* that are converted to appropriate types.

The format string contains two types of objects: plain characters that are copied to the returned string and conversion specifications, each of which results in fetching zero or more values from the list of arguments following the format string. Excess arguments are ignored.

Each conversion specification is introduced by the character `%` followed by the following items (in the specified order):

- Zero or more flags that modify the meaning of the conversion specification.
- An optional string of decimal digits to specify a minimum field width. If the converted value has fewer characters than the field width, it is padded on the left (or right, if the left-adjustment flag, described below, has been given) to the field width. If a zero precedes the field width, the string is right adjusted with zero padding on the left (see the leading-zero flag described below).
- A precision that gives the minimum number of digits to appear for the `d`, `i`, `o`, `u`, `x` or `X` conversions, the number of digits to appear after the decimal point for the `e` and `f` conversions, the maximum number of significant digits for the `g` conversion, or the maximum number of characters to be printed from a string in the `s` conversion. The precision takes the form of a period (`.`) followed by a decimal digit string; a null digit string is treated as zero
- A conversion character that indicates the type of conversion to be applied.

A field width or precision can be indicated by an asterisk (`*`) instead of a digit string. In this case, an integer argument supplies the field width or precision. The arguments specifying field width, or precision, or both must appear in that order before the argument to be converted.

The flag characters and their meanings are:

Flag characters	Meaning
<code>-</code>	The resulting conversion is left justified within the field.
<code>+</code>	The resulting signed conversion always begins with a sign (<code>+</code> or <code>-</code>).
<code>space</code>	If the first character of a signed conversion is not a sign, a space is prefixed to the result.
<code>#</code>	This flag specifies that the value is converted to an alternate form. For <code>c</code> , <code>d</code> , <code>i</code> , <code>s</code> and <code>u</code> conversions, the flag has no effect. For <code>o</code> conversions, it forces the first digit of the result to be a <code>0</code> . For <code>x</code> and <code>X</code> conversions, a non-zero result has <code>0x</code> or <code>0X</code> prefixed to it. For <code>e</code> , <code>E</code> , <code>f</code> , <code>g</code> , and <code>G</code> conversions, the result always contains a decimal point, even if no digits follow it. For <code>g</code> and <code>G</code> conversions, trailing zeroes are not removed from the result.
<code>0</code>	Leading zeroes (following any indication of sign or base) are used to pad to the field width for all conversion characters. No space padding is performed. If both the <code>0</code> and <code>-</code> appear, the <code>0</code> flag is ignored. For <code>d</code> , <code>i</code> , <code>o</code> , <code>u</code> , <code>x</code> and <code>X</code> conversions, if a precision is specified, the <code>0</code> flag is ignored.

The conversion characters and their meaning are:

Conversion characters	Meaning
d, i	The integer argument is converted to signed decimal notation. The precision specifies the minimum number of digits to appear. The default precision is 1.
o	The integer argument is converted to unsigned octal notation. The precision specifies the minimum number of digits to appear. The default precision is 1.
u	The integer argument is converted to unsigned decimal notation. The precision specifies the minimum number of digits to appear. The default precision is 1.
x, X	The integer argument is converted to unsigned hexadecimal notation. The letters <code>abcdef</code> are used for <code>x</code> conversions and <code>ABCDEF</code> for <code>X</code> conversions. The precision specifies the minimum number of digits to appear. The default precision is 1.
f	The floating-point argument is converted to decimal notation in the style <code>[-]ddd.ddd</code> . The number of digits after the decimal point is equal to the precision. If the precision is missing, six digits are output. If the precision is explicitly 0, no decimal point is output.
e, E	The floating-point argument is converted in the style <code>[-]d.ddde±ddd</code> . There is one digit before the decimal point and the number of digits after it is equal to the precision. When the precision is missing, six digits are produced. If the precision is 0, no decimal point appears. The <code>E</code> format code produces a number with <code>E</code> instead of <code>e</code> introducing the exponent. The exponent always contains at least two digits.
g, G	The floating-point argument is converted in style <code>f</code> or <code>e</code> (or in style <code>E</code> in case of a <code>G</code> format code), with the precision specifying the number of significant digits. The style used depends on the value converted: style <code>e</code> or <code>E</code> is used only if the exponent resulting from the conversion is less than minus four or greater than or equal to the precision. Trailing zeroes are removed from the fractional part of the result. A decimal point appears only if a digit follows it.
c	The integer argument is converted to a character. The resulting character has the same character code as the integer value.
s	The argument is converted to a string and characters from the string are used until the string ends or the number of characters indicated by the precision specification is reached. If the precision is missing, it is taken to be infinite.
%	Output a <code>%</code> . No argument is used.

In no case does a nonexistent or small field width cause truncation of a field. If the result of a conversion is wider than the field width, the field is expanded to contain the conversion result.

The `FORMAT` function is valid in *const-expressions* provided *format* and all the *argument* arguments are *const-expressions*.

Errors

The following value can be assigned to `$Error`:

```
$Error_FormatTooFewArguments
```

The format string requires more arguments.

Examples

The `FTOS` user function converts a floating-point number to a string with two decimal digits:

```
DECLARE FUNCTION FTOS($f)
{
    RETURN FORMAT("%.2f", $f);
}
```

Calling the FTOS function with a value of 1.7 returns the string:

1.70

The ITOS user function converts an integer to a string of length 4 (leading zeroes are added until the length becomes 4):

```
DECLARE FUNCTION ITOS($i)
{
    RETURN FORMAT("%.04d", $i);
}
```

Note: If the *\$i* argument is greater than or equal to 10000, the length of the returned string is actually longer than 4 characters.

Calling the ITOS function with a value of 15 returns the string:

0015

Notes

The FORMAT function is implemented using the standard C library function *sprintf*, hence the similarity.

12.4.74 FROM_ISO8859

Name

FROM_ISO8859 – Function that converts from a standard character set.

Syntax

FROM_ISO8859(*string*)

Description

The FROM_ISO8859 function converts the characters in *string* from the ISO8859-1 character set to the character set used by the machine running the Message Builder interpreter. The converted string is returned.

The *string* argument is an *expression* that is converted to a string.

See also

TO_ISO8859.

Notes

For machines that use the ISO8859-1 character set (UNIX and Windows machines), the returned string equals *string*.

12.4.75 GETDATE

Name

GETDATE – Function that returns a field from a date value.

Syntax

GETDATE(date, field-selector)

Description

The GETDATE function extracts and returns the field specified by *field-selector* from the date value *date*.

The *date* argument is an *expression* that is converted to a date value.

The *field-selector* argument is an integer selector:

- \$DateField_Year Return the year field.
- \$DateField_Month Return the month number field. The return value is in the range 0 – 11.
- \$DateField_Monthday Return the month day number field. The return value is in the range 1 – 31.
- \$DateField_Hour Return the hours field. The return value is in the range 0 – 23.
- \$DateField_Minute Return the minutes field. The return value is in the range 0 – 59.
- \$DateField_Second Return the seconds field. The return value is in the range 0 – 59.
- \$DateField_Fraction Return the fractions of a second field. The return value is in the range 0 – 99.

The above constant variables are declared in the *datefld.s4h* include file.

Errors

The following value can be assigned to \$Error:

```
$Error_GetdateInvalidField
```

The *field-selector* argument is invalid.

Examples

The following example outputs the current date and time:

```
INCLUDE "datefld.s4h";
DECLARE VARIABLE $date DATE;

$date = CURRENTDATE();
PRINT "Year: " & GETDATE($date, $DateField_Year) & "\n";
PRINT "Month: " & GETDATE($date, $DateField_Month) & "\n";
PRINT "Month Day: " & GETDATE($date, $DateField_Monthday) & "\n";
PRINT "Hour: " & GETDATE($date, $DateField_Hour) & "\n";
PRINT "Minute: " & GETDATE($date, $DateField_Minute) & "\n";
PRINT "Second: " & GETDATE($date, $DateField_Second) & "\n";
PRINT "Fraction: " & GETDATE($date, $DateField_Fraction) & "\n";
```

See also

SETDATE.

12.4.76 IF

Name

IF – Conditional execution.

Syntax

```
IF test-expr1 statement-block1
[ELSE IF test-expr2 statement-block2
[ELSE IF test-expr3 statement-block3]]
...
[ELSE statement-blockN]
```

Description

The IF compound statement selects a block of statements for execution, depending on the result of the evaluation of a number of test expressions. If *test-expr1* evaluates to true (that is, the result is non-zero), *statement-block1* is executed. Otherwise, if *test-expr2* evaluates to true, *statement-block2* is executed and so forth. If none of the test expressions evaluates to true the ELSE statement block, if present, is executed. Nothing is executed if none of the test expressions evaluates to true and the ELSE part is not present.

All the test expressions are *expressions* that are converted to floating-point values.

Examples

```
DECLARE VARIABLE $a = 10 INTEGER;
DECLARE VARIABLE $b = 20 INTEGER;
DECLARE VARIABLE $s = "Hello" STRING;

IF $a < $b {
    PRINT "$a is less than $b\n";
}
ELSE IF $a = $b {
    PRINT "$a is equal to $b\n";
}
ELSE {
    PRINT "$a is greater than $b\n";
}

IF $s = "Hello" {
    PRINT "$s is equal to 'Hello'\n";
}

IF $a < $b AND $s = "Hello" {
    .
    .
    .
}
```

Notes

If the **O** option is not specified when a Message Builder program is compiled, code that is never executed and tests that always are true are removed. For example, the following code fragment

```

DECLARE VARIABLE $Config_1 = 0 CONSTANT INTEGER;
DECLARE VARIABLE $Config_2 = 1 CONSTANT INTEGER;

IF $Config_1 {
    PRINT "Text 1\n";
}
IF $Config_2 {
    PRINT "Text 2\n";
}

```

is converted to

```

DECLARE VARIABLE $Config_1 = 0 CONSTANT INTEGER;
DECLARE VARIABLE $Config_2 = 1 CONSTANT INTEGER;

PRINT "Text 2\n";

```

12.4.77 IMPORT

Name

IMPORT – Import the value of an environment variable.

Syntax

IMPORT environ-var INTO destination;

Description

The IMPORT statement assigns the value of the environment variable specified by *environ-var* to the Message Builder variable specified by *destination*. The *environ-var* parameter is an *expression* that is converted to a string and *destination* is a *variable*.

If an error occurs, the contents of the *destination* variable are undefined.

Errors

The following values can be assigned to `$Error`:

- `$Error_ImportNameEmpty`
The *environ-var* expression evaluated to an empty string.
- `$Error_ImportNameMissing`
The environment variable does not exist. This is a warning.

Examples

If the *module1.s4* program exports an environment variable named AGE

```

DECLARE VARIABLE $MyAge INTEGER;
$MyAge = 35;
EXPORT $MyAge AS "AGE";
SWITCH "module2.s4";

```

the *module2.s4* program can import the AGE environment variable

```

DECLARE VARIABLE $HisAge FLOAT;
IMPORT "AGE" INTO $HisAge;

```

The following UNIX and AS/400 example imports and uses the value of the CORE_ROOT environment variable:

```
DECLARE VARIABLE $root STRING;
IMPORT "CORE_ROOT" INTO $root;
OPEN FILE OUTPUT $root & "/" & $filename;
```

On Windows the above example would look like:

```
DECLARE VARIABLE $root STRING;
IMPORT "CORE_ROOT" INTO $root;
OPEN FILE OUTPUT $root & "\\\" & $filename;
```

The following example checks if the environment variable COUNT does exist and if not it creates it:

```
INCLUDE "error.s4h";
DECLARE VARIABLE $Count STRING;
$error = 0;
IMPORT "COUNT" INTO $Count;
IF $Error = $Error_ImportNameMissing {
    EXPORT "0" AS "COUNT";
    $Count = 0;
}
```

See also

EXECUTE, EXPORT, RELEASE, SWITCH.

Notes

All variables, except the reserved ones (see RESERVED VARIABLES, paragraph [12.4.101](#)), are reset and forgotten when changing the program using the SWITCH statement. The EXPORT statement together with IMPORT provides a way to share information between different Message Builder programs.

12.4.78 INCLUDE

Name

INCLUDE – Include Message Builder program code from another file.

Syntax

INCLUDE *filename* [ONCE];

Description

The INCLUDE statement includes Message Builder program code from the file whose file name is *filename*. The *filename* parameter is a *const-expression* that is converted to a string.

The ONCE modifier can be used to prevent a file from being included multiple times. An INCLUDE statement, not located in a module, with the ONCE modifier specified, does not include *filename* if *filename* already has been included from a position outside any modules. In the same way, an INCLUDE statement located in a module does not include *filename* if *filename* already has been included from the same module.

The Message Builder compiler uses the directories specified by the EDI_INCLUDE environment variable and the directories specified by the **I** compiler command option when searching for the file to include.

When end-of-file is reached in the included file, input is again taken from the original file.

Examples

```
INCLUDE "variables.s4h";
```

Notes

Inclusion is typically used to declare variables. Inclusion of code should normally not be used. Use user functions, user statements and modules, combined with the library feature (the EDI_LIB environment variable) of the Message Builder compiler, instead.

There are no restrictions on the file names in the INCLUDE statement. The header files provided with Integration Broker use the ".s4h" suffix.

The number of include nesting levels is limited only by the number of concurrent files that the Message Builder compiler can have open.

12.4.79 INTERP_VERSION

Name

INTERP_VERSION – Function that returns the version number of the interpreter.

Syntax

```
INTERP_VERSION()
```

Description

The INTERP_VERSION function returns the version number of the Message Builder interpreter as a string. The string is of the format:

n1.n2.n3

The *n1*, *n2* and *n3* fields are decimal numbers.

Examples

Executing the following program with the 6.0.4 version of the Message Builder interpreter:

```
PRINT INTERP_VERSION() & "\n";
```

produces the following output:

```
6.0.4
```

12.4.80 IOSET

Name

IOSET – Function that returns the identifier for the current input/output set.

Syntax

IOSET()

Description

The IOSET function returns an integer containing the identifier for the current input/output set.

See also

IOSET CREATE, IOSET DELETE, IOSET SWITCH.

12.4.81 IOSET CREATE

Name

IOSET CREATE – Create a new input/output set.

Syntax

IOSET CREATE *id*;

Description

Input/output sets make it possible to have several sets of input/output files open at the same time and switch between them. The following statements always work with the current input/output set:

- BOUNDED LOOP
- CLOSE
- COMPOSITE
- DATA ELEMENT
- OPEN
- PRINT
- READ
- RELATION
- SCAN
- SEGMENT
- SEGMENT GROUP
- TRUNCATE
- UNBOUNDED LOOP

In these statements, it is not possible to specify which file to read or write and, hence without input/output sets, it would only be possible to have one input and one output file open at a time. Input/output sets solve this problem and make it possible to use multiple input/output files at the same time.

The following are private to an input/output set:

- Input file/string opened using OPEN FILE INPUT or OPEN STRING INPUT.
- Output file/string opened using OPEN FILE OUTPUT or OPEN STRING OUTPUT.
- The \$EdiPos, \$EdiSegCnt, \$EdiElemCnt, \$EdiCompCnt, \$EdiSeg, \$EdiFile, \$EdiSegCodeSep, \$EdiRelChar, \$EdiPadChar, \$EdiSegSep, \$EdiElemSep, \$EdiCompSep, \$EdiCharSet, \$EdiInitMode, \$EdiResetFlag, \$EdiPreFlag, \$EdiByteFlag and \$EdiCharValid reserved variables.

A non-negative integer identifier identifies each input/output set. Initially, an input/output set with identifier zero does exist. New input/output sets can be created with the IOSET CREATE statement.

The *id* parameter is an output *variable* parameter that receives the identifier of the newly created input/output set. It should be of integer type.

The input file for an input/output set is initially the standard input and the output file is the standard output.

All the reserved variables that are private to an input/output set initially have the same values as these reserved variables have for input/output set zero when the interpreter is started.

Examples

If the following program is executed:

```
DECLARE VARIABLE $id INTEGER;

IOSET CREATE $id;
PRINT "First I/O set\n";
IOSET SWITCH $id;
OPEN FILE OUTPUT "example.txt";
PRINT "Second I/O set\n";
IOSET SWITCH 0;
PRINT "First I/O set again\n";
```

the following two lines are written to the standard output:

```
First I/O set
First I/O set again
```

and the *example.txt* file will contain the following line:

```
Second I/O set
```

See also

IOSET, IOSET DELETE, IOSET SWITCH.

12.4.82 IOSET DELETE

Name

IOSET DELETE – Delete an input/output set.

Syntax

IOSET DELETE *id*;

Description

The IOSET DELETE statement deletes the input/output set specified by the *id* parameter. The *id* parameter is an *expression* that is converted to an integer. It should have been returned from an earlier call to IOSET CREATE.

It is not allowed to delete the current input/output set or the input/output set with identifier zero.

Input/output sets that are no longer needed should be deleted in order to preserve resources.

Errors

The following values can be assigned to `$Error`:

- `$Error_IosetDeleteInvalidIdentifier`
The *id* parameter is negative.
- `$Error_IosetDeleteUnknownIdentifier`
The input/output identifier specified by *id* does not exist.
- `$Error_IosetDeleteZeroIdentifier`
Trying to delete the input/output set with identifier zero.
- `$Error_IosetDeleteCurrentIdentifier`
Trying to delete the current input/output set.

Examples

The example from the IOSET CREATE reference page can be extended with an IOSET DELETE statement:

```
DECLARE VARIABLE $id INTEGER;

IOSET CREATE $id;
PRINT "First I/O set\n";
IOSET SWITCH $id;
OPEN FILE OUTPUT "example.txt";
PRINT "Second I/O set\n";
IOSET SWITCH 0;
IOSET DELETE $id;
PRINT "First I/O set again\n";
```

The output will be the same.

See also

IOSET, IOSET CREATE, IOSET SWITCH.

12.4.83 IOSET SWITCH

Name

IOSET SWITCH – Switch between input/output set.

Syntax

IOSET SWITCH [*id*];

Description

The IOSET SWITCH statement makes another input/output set the current input/output set. The *id* parameter specifies the input/output set to become the current input/output set. It is an *expression* that is converted to an integer. It should be zero or have been returned from an earlier call to IOSET CREATE. The default for *id* is zero.

The output buffer for the old input/output set is flushed before the switch is done. If *id* specifies the current input/output set, the only effect is that the output buffer is flushed.

Errors

The following value can be assigned to `$Error`:

- `$Error_IosetSwitchInvalidIdentifier`
The *id* parameter is negative.
- `$Error_IosetSwitchUnknownIdentifier`
The input/output identifier specified by *id* does not exist.
- `$Error_IosetSwitchFlush`
Failed to flush the output buffer for the old input/output set.

Examples

See the IOSET CREATE (page 208) and IOSET DELETE (page 209) statements.

See also

IOSET, IOSET CREATE, IOSET DELETE.

12.4.84 LOADPGM

Name

LOADPGM – Load a Message Builder program.

Syntax

LOADPGM program-name [ID *id*];

Description

The LOADPGM statement loads a Message Builder program. This makes it possible to dynamically load compiled Message Builder programs and execute user functions and statements in them.

The *program-name* parameter is an *expression* that is converted to a string and used as the file name of the Message Builder program to load.

The *id* parameter is an *expression* that is converted to a string. It is used to distinguish between different instances of the same program. The default value for *id* is an empty string.

Normally it is an error to try to load the same program multiple times with identical *ids*. However, in order to be backward compatible with older versions of Message Builder, the same program may be loaded multiple times if *id* is an empty string. All attempts, except the first one, are ignored.

The Message Builder interpreter uses the EDI_PATH environment variable to find the program files to load. If the EDI_PATH variable does not exist, the interpreter searches for files to load in the current directory. The **P** interpreter option can also be used to add directories to be searched for files to load.

Errors

The following values can be assigned to `$Error`:

- `$Error_LoadpgmEmptyName`
The *program-name* parameter is an empty string.
- `$Error_LoadpgmAlreadyLoaded`
The *program-name* program with ID *id* has already been loaded.
- `$Error_LoadpgmProgramOpenError`
Failed to open the Message Builder program file specified by *program-name*.
- `$Error_LoadpgmProgramSeekError`
Seek error on *program-name*.
- `$Error_LoadpgmProgramEmpty`
The file specified by the *program-name* parameter is empty.
- `$Error_LoadpgmProgramReadError`
Read error on *program-name*.
- `$Error_LoadpgmProgramReadSize`
Could not read the expected number of bytes from *program-name*.
- `$Error_LoadpgmNotExecutable`
The *program-name* file is not an executable Message Builder program.
- `$Error_LoadpgmWrongVersion`
Compiled Message Builder programs contain a version number. The version number found in *program-name* is not the same as the one expected by the Message Builder interpreter.
- `$Error_LoadpgmIllegalFormat`
The format of the compiled Message Builder program specified by *program-name* is illegal.

Examples

See DECLARE EXTERNAL FUNCTION, page 135, and DECLARE EXTERNAL STATEMENT, page 138.

See also

DECLARE EXTERNAL FUNCTION, DECLARE EXTERNAL STATEMENT, PRAGMA, UNLOADPGM, USE.

Notes

All `MODULE_INIT` statements in the loaded program are automatically executed when it is loaded (see the `DECLARE MODULE` statement, paragraph [12.4.38](#)).

The ID for a program not loaded by `LOADPGM` (for example the main program specified on the interpreter command line) is the empty string.

12.4.85 LOG

Name

LOG – Write a message to the log.

Syntax

LOG *message* [TYPE *type*];

Description

The LOG statement writes the message specified by *message* to the log. The type of the message is specified by *type*, which can be one of "INFO", "DEBUG", "WARNING", "ERROR", "FATAL" or any other non-standard string (in which case the type field will contain the non-standard string). The default for *type* is "ERROR".

Both *message* and *type* are *expressions* that are converted to strings.

The message written to the log is of the following format:

type:date:time:name:message

The *date* and *time* fields are the date and time of day, respectively, for when the message was written and *name* is the name of the interpreter (normally *r4edi*) followed by the name of the Message Builder program inside parentheses.

If the Message Builder program was compiled with the trace option (**T**) specified two extra fields are inserted between the *name* and *message* fields. These extra fields are the name of the source code file and the source code line number, respectively, for the LOG statement.

A new-line character should not terminate the message string, because one is automatically added.

Examples

Write an error to the log:

```
LOG SOURCEPROCEDURE() & ": failed to open file " & $file;
```

Write a warning to the log:

```
LOG SOURCEPROCEDURE() & ": file delete failed: " & $file TYPE  
"WARNING";
```

See also

NOLOG, PRINTERR.

Notes

Non-standard log type strings should be avoided.

The **I** option to the Message Builder interpreter also affects messages written with the LOG statement.

LOG messages are formatted according to the Trace Log format and, in a standard configuration of Integration Broker, the LOG messages are written to the Trace Log. For a description of the Trace Log, see the *System and Administration Manager* manual.

12.4.86 NOLOG

Name

NOLOG – Suppress log information.

Syntax

NOLOG *statement-block*

Description

When *statement-block* is executed, nothing will be written to the log.

NOLOG compound statements may be nested. Logging will be enabled as soon as the outermost NOLOG *statement-block* has terminated.

Messages written using the LOG statement are also suppressed.

Examples

The following user statement avoids end-of-file messages when executing the SCAN statement:

```
DECLARE STATEMENT
SILENT_SCAN IN $ScanCount
{
    NOLOG {
        SCAN COUNT $ScanCount;
    }
    RETURN;
}
```

See also

ANTINOLOG, LOG.

Notes

If a user function or statement is called from the NOLOG *statement-block*, log messages generated in it are also suppressed.

12.4.87 OPEN

Name

OPEN – Open the input or output file.

Syntax

```
OPEN {FILE | STRING} {INPUT | OUTPUT | APPEND} parameter
    [START start] [SIZE size] [TEXT | BINARY];
```

Description

The OPEN statement opens the file or string specified by the *parameter* parameter. It is an *expression* that is converted to a string.

If the INPUT modifier is specified, the input file is opened. The `$EdiFile` reserved variable is set to *parameter*, `$EdiSeg` is set to an empty string, and `$EdiSegCnt`, `$EdiCompCnt`, `$EdiElemCnt` and `$EdiPos` are set to 0. The input file is used by the READ and SCAN statements and all the EDI parsing statements.

If the OUTPUT modifier is specified, the output file is opened. The PRINT statement uses the output file. If the file specified by *parameter* does not exist, it is created. If the file does exist, it is truncated to zero length. If this is not desired, the APPEND modifier can be used instead. If so, data will be appended to the end of the file.

If OPEN FILE is specified, *parameter* is the name of the file to open.

If OPEN STRING INPUT is specified, the contents of the *parameter* parameter will be the data returned when reading data from the input file.

If OPEN STRING OUTPUT is specified, *parameter* must be a *variable* (note that it is not allowed to specify a sub-string operator in this case, see SUBSTRING, paragraph [12.4.121](#)). When the output file is closed (using CLOSE), the data written to the output file will be assigned to *parameter*. If APPEND is specified instead of OUTPUT, the written data will be appended to the value the *parameter* variable had when the OPEN statement was executed. Note that the *parameter* variable is modified when the output file is closed. If the *parameter* variable ceases to exist before the output file is closed (that is, it is a local variable to a user function or statement), the written data is thrown away. It is not valid to use a reserved variable as *parameter*.

The *start* and *size* parameters are only valid when an input file or string is opened. They are used to specify a part of the file or string to read. The start position will be *start* and the size will be *size*. If *size* is zero, the part starting at *start* and continuing to the end of the file will be opened for reading. The default values for both *start* and *size* are 0 and both are *expressions* that are converted to integers.

The TEXT and BINARY modifiers (the default is BINARY) can be used to specify whether the input or output file is a text or a binary file, respectively:

- These modifiers are of no significance on UNIX machines.
- On Windows, when a text file is read, a carriage return/linefeed (CR/LF) sequence is converted to a single linefeed character, and when a text file is written a linefeed character is converted to a CR/LF sequence. When a text file is read, if the file contains Ctrl+Z characters, the first Ctrl+Z character in the file is taken to mark end-of-file.
- On AS/400, the TEXT modifier, when writing, creates a file of CCSID 819 containing

ASCII data. It also causes a linefeed character to be converted to a CR/LF sequence.

The BINARY modifier, as the default, creates a file of CCSID 500, containing EBCDIC data. The TEXT modifier, when reading, causes data conversion to take place from the CCSID of the file to the CCSID of the process (in Integration Broker this is always CCSID 500). It also causes a CR/LF sequence to be converted to a linefeed character. The BINARY modifier, when reading, does no data conversion.

The TEXT modifier is not valid in OPEN STRING statements.

The default output file is the standard output and the default input file is the standard input.

Errors

The following values can be assigned to `$Error`:

- `$Error_OpenEmptyName`
The *parameter* expression evaluated to an empty string.
- `$Error_OpenOpenFail`
Failed to open the file.
- `$Error_OpenSeekFail`
Failed to set the start position for the file opened for reading. This error can only occur when *start* is non-zero.
- `$Error_OpenInvalidStart`
The *start* parameter is invalid, i.e. negative.
- `$Error_OpenInvalidSize`
The *size* parameter is invalid, i.e. negative.
- `$Error_OpenIllegalStart`
The specified start position is beyond the end of the file.
- `$Error_OpenIllegalSize`
The specified end position (*start* plus *size*) is beyond the end of the file.

Examples

The following example opens the file *outfile.dat* as the output file and writes some data to it:

```
OPEN FILE OUTPUT "outfile.dat";
PRINT "Hello outfile\n";
CLOSE OUTPUT;
```

The next example appends new data to the end of the file specified by the `$OutPutFile` variable:

```
OPEN FILE APPEND $OutPutFile;
PRINT "This Is The Last Row In This File.\n";
CLOSE OUTPUT;
```

The last example opens a string as the input file:

```
OPEN STRING INPUT "Data to be returned\n";
READ $s UNTIL "\n";
PRINT $s & "\n";
CLOSE INPUT;
```

The output produced by the above example is:

```
Data to be returned
```

See also

CLOSE, READ, PRINT, SCAN.

Notes

The main purpose of the OPEN statement is to open EDI files and strings for reading and writing.

Only one input and one output file may be opened at the same time. If a file is being opened when another one is already open, the already opened file is closed. However, the input/output sets feature (see the IOSET CREATE statement, paragraph [12.4.81](#)) can be used to circumvent this.

If the output file is to be used by another program, in order to flush the file buffers, the file should be closed before the program is started.

Instead of opening the input file with OPEN FILE INPUT, the input file can be opened using the **e** option when invoking the Message Builder interpreter.

12.4.88 PIPE CLOSE

Name

PIPE CLOSE – Close the input and output pipes.

Syntax

PIPE CLOSE;

Description

The PIPE CLOSE statement closes the input and output pipes.

Errors

The following value can be assigned to `$Error`:

`$Error_PipeCloseClosed`

The input and output pipes are already closed. This is a warning.

Examples

See the PIPE OPEN statement, paragraph [12.4.89](#).

See also

PIPE OPEN, PIPE READ, PIPE WRITE.

Notes

The PIPE CLOSE statement is not supported on the AS/400 machine.

12.4.89 PIPE OPEN

Name

PIPE OPEN – Open the input and output pipes.

Syntax

PIPE OPEN *command* [TEXT | BINARY];

Description

The PIPE OPEN statement creates pipes to the command specified by *command*. The *command* parameter is an *expression* that is converted to a string.

The command specified by *command* is executed and input and output pipes to the executing program are created. The input and output pipes are used by the PIPE READ and PIPE WRITE statements respectively.

The TEXT and BINARY modifiers can be used to specify whether the input and output pipes is text or binary pipes, respectively. The default is BINARY. These modifiers are of no significance on UNIX machines. On Windows:

- When reading from a text pipe, a CR/LF sequence is converted to a linefeed character.
- When writing to a text pipe, a linefeed character is converted to a CR/LF sequence.

Errors

The following values can be assigned to `$Error`:

- `$Error_PipeOpenEmptyCommand`
The *command* expression evaluated to an empty string.
- `$Error_PipeOpenNoSupport`

Pipes are not supported.

- `$Error_PipeOpenOpenFail`
Failed to open pipe.

Examples

```
DECLARE VARIABLE $reply STRING;
PIPE OPEN "my_prog";
PIPE WRITE $command;
PIPE READ $reply UNTIL "\n";
PIPE CLOSE;
```

See also

PIPE CLOSE, PIPE READ, PIPE WRITE.

Notes

Only one input and one output pipe may be opened at the same time. If pipes already are open when a PIPE OPEN is executed, they are closed.

The PIPE OPEN statement is not supported on the AS/400 machine.

12.4.90 PIPE READ

Name

PIPE READ – Read from the input pipe.

Syntax

PIPE READ *destination* [COUNT *length*] [UNTIL *separator*];

Description

The PIPE READ statement reads data from the input pipe. The read data is put in the *variable* specified by *destination*. If *destination* is not of string type, the data is first converted to the appropriate type.

If *length* is specified and non-zero, the PIPE READ statement tries to read *length* bytes of data. The *length* parameter is an *expression* that is converted to an integer.

If *separator* is specified and is a non-empty string, the PIPE READ statement tries to read until a character that is equal to the first character in *separator* is found. The separating character is excluded from the read data. The *separator* parameter is an *expression* that is converted to a string.

If both *length* and *separator* are specified data is read until one of the conditions is fulfilled, whichever comes first.

If end-of-file is encountered before *length* number of bytes has been read or the *separator* character has been found, the characters read so far are returned in *destination* and `$Error` is set to `$Error_PipeReadEndOfFile`.

If none of the *length* or *separator* parameters are specified or if they are set to zero and an empty string, respectively, all data until end-of-file is encountered is read. Note that the `$Error` variable is not set in this case.

The contents of *destination* are undefined if an error occurs.

Errors

The following values can be assigned to `$Error`:

- `$Error_PipeReadEndOfFile`
End-of-file encountered. This is an informational code, not an error.
- `$Error_PipeReadCountTooSmall`
The *length* parameter is less than 0.
- `$Error_PipeReadNotOpen`
The input pipe is not open.
- `$Error_PipeReadReadError`
Read error.
- `$Error_PipeCscError`
Error related to multi-byte character handling.

Examples

See the PIPE OPEN statement, paragraph [12.4.89](#).

See also

PIPE CLOSE, PIPE OPEN, PIPE WRITE.

Notes

When *separator*, but not *length*, is specified, the PIPE READ statement may read a huge amount of data if the *separator* character is not found in the input. This can be prevented by also using *length* to limit the number of characters to be read.

Very large *length* values should be avoided because the Message Builder interpreter allocates *length* bytes of memory also when the amount actually read is small.

The PIPE READ statement is not supported on the AS/400 machine.

12.4.91 PIPE WRITE

Name

PIPE WRITE – Write data to the output pipe.

Syntax

PIPE WRITE *data*;

Description

The PIPE WRITE statement writes *data* to the output pipe.

The *data* parameter is an *expression* that is converted to a string.

For performance reasons, the Message Builder interpreter normally buffers data written to the output pipe. The result is that the data is not written to the output pipe until the buffer becomes full, data is read from the input pipe, the output pipe is closed, or the program terminates.

Errors

The following values can be assigned to `$Error`:

- `$Error_PipeWriteWriteFail`
Error when writing to the output pipe.
- `$Error_PipeWriteNotOpen`
The output pipe is not open.

Examples

See the PIPE OPEN statement, paragraph [12.4.89](#).

See also

PIPE CLOSE, PIPE OPEN, PIPE READ.

Notes

The PIPE WRITE statement is not supported on the AS/400 machine.

12.4.92 PRAGMA

Name

PRAGMA – Directive to control the compilation process in various ways.

Syntax

```
PRAGMA EXPORT {FUNCTION | STATEMENT} name;  
    or  
PRAGMA FILE filename;  
    or  
PRAGMA LINE line-number;  
    or  
PRAGMA TRACE {ON | OFF};  
    or  
PRAGMA [MAIN] VERSION name, version[, program];  
    or  
PRAGMA WARNINGS {ON | OFF};
```

Description

The PRAGMA directives are used to control various aspects of the compilation. They always take effect when the program is compiled, and it is the position of the directive that determines when it has effect; for example, it does not matter if it is located in a statement block that never can be executed.

The PRAGMA EXPORT directive is used to tell the compiler to output information so that it will be possible to call the *name* function or statement from another program. See the LOADPGM and USE statements for more details. Alternatively, the **F** and **S** options to the Message Builder compiler or the FIDENT (see paragraph 12.4.67) and SIDENT (see paragraph 12.4.108) functions can be used. Note that if a function or statement that is declared inside a module is exported and the PRAGMA EXPORT directive is located outside the module, the module name and a dot must prefix the function or statement name.

The PRAGMA FILE and PRAGMA LINE directives are mainly intended to be used when Message Builder source code is generated automatically.

The PRAGMA FILE directive is used to set the name of the current input Message Builder source code file. The *filename* parameter is a *const-expression* that is converted to a string. When the Message Builder compiler starts reading a new source code file, the current input source code file name is set to the name of the new file. The PRAGMA FILE directive can be used to change the name to *filename*. The current input source code file name is used by the compiler when it outputs error, warning and informational messages. It is also returned by the SOURCEFILE function and used when trace information logging is enabled (see the DEBUG statement, paragraph 12.4.34).

The PRAGMA LINE directive is used to set the current line number for the next line in the input Message Builder source code file. The *line-number* parameter is a *const-expression* that is converted to an integer. When the Message Builder compiler starts reading a new source code file, the current line number is set to 1, and then it is incremented by 1 for each new-line found in the input file. The PRAGMA LINE directive can be used to change the line number for the next line in the input source code file to *line-number*. The value of *line-number* must be greater than 0. As for the current input source code file name, the current line number is used when the compiler outputs error, warning and informational messages and when trace information logging is enabled. It is also returned by the SOURCELINE function.

The PRAGMA TRACE directive only has effect when the **T** option to the Message Builder compiler is specified. It is used to tell the compiler to stop (PRAGMA TRACE OFF) or start (PRAGMA TRACE ON) generating code for source code trace. In this way, it is possible to output trace information only for selected parts of a program; for example, functions and statements that are known to work can be surrounded by PRAGMA TRACE OFF/PRAGMA TRACE ON directives in order to limit the amount of trace information. PRAGMA TRACE directives may be nested.

The PRAGMA VERSION directive is used to specify the version number of the source code. It is typically used to set the version numbers for the different components used to build a program. The *name* parameter is the name of the part of the program whose version number is *version*. The name of the program is *program*. The *program* parameter may only be specified if the MAIN modifier is present and only one PRAGMA VERSION with the MAIN modifier may be present in a program. Multiple PRAGMA VERSION directives with the same *name* parameter may be specified, provided all the *version* parameters are equal. The interpreter can output the version numbers set by PRAGMA VERSION directives by specifying the **V** option and the name of the compiled program (see the EXAMPLES section for an example).

The PRAGMA WARNINGS directive is used to turn warnings off and on for selected parts of the source code. This directive does only have effect when the **w** compiler option is used. All warnings between a PRAGMA WARNINGS OFF and a matching PRAGMA WARNINGS ON are suppressed. PRAGMA WARNINGS directives may be nested.

Examples

Assume that the following program is put in a file name *pragma.s4*:

```
PRINT SOURCEFILE() & "\n";
PRAGMA FILE "OtherName";
PRINT SOURCEFILE() & "\n";
PRINT SOURCELINE() & "\n";
PRAGMA LINE 100;
PRINT SOURCELINE() & "\n";
```

Running the above program will cause the following to be written to the output file:

```
pragma.s4
OtherName
4
100
```

If a program containing the following three PRAGMA VERSION directives:

```
PRAGMA MAIN VERSION "main", "1.0.3";
.
.
.
PRAGMA VERSION "sub1", "3.1.2";
.
.
.
PRAGMA VERSION "sub2", "2.3.4";
```

is run using the command (it is assumed that the name of the compiled program is *prog.x4*):

```
r4edi -V prog.x4
```

the following output will be produced:

```
Program Unit Revision: main 1.0.3
Program Unit Revision: sub1 3.1.2
Program Unit Revision: sub2 2.3.4
```

If instead the first directive contains the *program* parameter:

```
PRAGMA MAIN VERSION "main", "1.0.3", "prog";
.
.
.
PRAGMA VERSION "sub1", "3.1.2";
.
.
.
PRAGMA VERSION "sub2", "2.3.4";
```

the output will be:

```
Program Unit Revision: main 1.0.3 : prog
Program Unit Revision: sub1 3.1.2
Program Unit Revision: sub2 2.3.4
```

12.4.93 PRINT

Name

PRINT – Print data to the current output file.

Syntax

PRINT *data*;

Description

The PRINT statement writes the data in *data* to the current output file. If "truncate mode" is enabled (see the TRUNCATE statement, paragraph [12.4.130](#)), separator characters and segment tags may be suppressed.

The *data* parameter is an *expression* that is converted to a string.

For performance reasons, the Message Builder interpreter normally buffers data printed to the output file. This has the effect that the printed data is not written to the output file until the buffer becomes full, the output file is closed, or the program terminates. If the buffering is a problem, it can be turned off using the **b** option when the Message Builder interpreter is invoked.

Errors

The following value can be assigned to \$Error:

```
$Error_PrintOutputWriteFail
```

Error when writing to the output file.

Examples

The following example creates an EDIFACT UNH segment:

```
PRINT "UNH" & $EdiElemSep;  
PRINT ESCAPE($RefNo) & $EdiElemSep;  
PRINT "PRICAT" & $EdiCompSep;  
PRINT "0" & $EdiCompSep & "1" & $EdiCompSep;  
PRINT "UN" & $EdiCompSep;  
PRINT "EAN002" & $EdiSegSep;          /* segment truncated */
```

See also

OPEN, PRINTERR, TRUNCATE (statement).

Notes

When generating EDI messages, use the reserved variables \$EdiSegSep, \$EdiElemSep, \$EdiCompSep and \$EdiSegCodeSep to print the separator characters. This makes it easier to change the separator characters, if needed.

12.4.94 PRINTERR

Name

PRINTERR – Print data to the error file.

Syntax

PRINTERR *data*;

Description

The PRINTERR statement writes the data in *data* to the error file (the standard error output).

Errors

The following value can be assigned to `$Error`:

`$Error_PrintErrorWriteFail`

Error when writing to the error file.

See also

LOG, PRINT.

Notes

Error messages should normally be output using the LOG statement (see paragraph [12.4.85](#)). The PRINTERR statement is normally only used by programs that are started manually from a terminal, and as a debugging aid when developing Message Builder programs.

12.4.95 PROGRAMINFO

Name

PROGRAMINFO – Function that returns program information.

Syntax

PROGRAMINFO([*selector*])

Description

The PROGRAMINFO function returns information about the currently running program. The *selector* argument specifies what type of information to return. It is an *expression* that is converted to an integer and the default value is `$ProgramInfo_ProgramName`. The *proginfo.s4h* header file contains all the valid *selector* values:

Selector values	Meaning
<code>\$ProgramInfo_ProgramName</code>	The name of the program file for the currently running program is returned.
<code>\$ProgramInfo_ProgramId</code>	The ID for the currently running program is returned as a string. The ID was specified when the program was loaded using LOADPGM. If the currently running program was not loaded using LOADPGM, the ID is an empty string.
<code>\$ProgramInfo_InputType</code>	The type of the input "file" is returned: <code>\$ProgramInfo_TypeStandard</code> is returned if the input is the standard input, <code>\$ProgramInfo_TypeString</code> is returned if the input is a string, and <code>\$ProgramInfo_TypeFile</code> is returned if the input is a file. The <code>\$ProgramInfo_TypeStandard</code> , <code>\$ProgramInfo_TypeString</code> and <code>\$ProgramInfo_TypeFile</code> constants are declared in the <i>proginfo.s4h</i> file.
<code>\$ProgramInfo_InputFileName</code>	The name of the current input file is returned. An empty string is returned if the input is the standard input or a string.
<code>\$ProgramInfo_OutputType</code>	The type of the output "file" is returned: <code>\$ProgramInfo_TypeStandard</code> is returned if the output is the standard output, <code>\$ProgramInfo_TypeString</code> is returned if the output is a string variable, and <code>\$ProgramInfo_TypeFile</code> is returned if the output is a file.
<code>\$ProgramInfo_OutputFileName</code>	The name of the current output file is returned. An empty string is returned if the output is the standard output or a string variable.

Errors

The following value can be assigned to `$Error`:

```
$Error_PrograminfoInvalidSelector
```

The *selector* argument is invalid.

Examples

If the following one-line program:

```
PRINT PROGRAMINFO() & "\n";
```

is named *info.s4* and compiled and executed, the following is written to the output file:

```
info.x4
```

See also

LOADPGM.

12.4.96 READ

Name

READ – Read from the input file.

Syntax

READ *destination* [COUNT *length*] [UNTIL *separator*];

Description

The READ statement reads data from the input file. The read data is put in the *variable* specified by *destination*. If *destination* is not of string type, the data is first converted to the appropriate type.

If *length* is specified and non-zero, the READ statement tries to read *length* characters of data. The *length* parameter is an *expression* that is converted to an integer.

If *separator* is specified and is a non-empty string, the READ statement tries to read until a character that is equal to the first character in *separator* is found. The separating character is excluded from the read data. The *separator* parameter is an *expression* that is converted to a string.

If both *length* and *separator* are specified data is read until one of the conditions is fulfilled, whichever comes first.

If end-of-file is encountered before *length* number of characters has been read or the *separator* character has been found, the characters read so far are returned in *destination* and `$Error` is set to `$Error_ReadEndOfFile`.

If none of the *length* or *separator* parameters are specified or if they are set to zero and an empty string, respectively, all the data to the end of the file is read. Note that the `$Error` variable is not set in this case.

If the `$EdiCharSet` reserved variable is set to specify a multi-byte character set, the *length* parameter specifies the number of multi-byte characters to read and the *separator* parameter should contain a multi-byte separator character.

The contents of *destination* are undefined if an error occurs.

Errors

The following values can be assigned to `$Error`:

- `$Error_ReadEndOfFile`
End-of-file encountered. This is an informational code, not an error.
- `$Error_ReadCountTooSmall`
The *length* parameter is less than 0.
- `$Error_ReadReadError`
Read error.
- `$Error_ReadCscError`
Error related to multi-byte character handling.
- `$Error_ReadPartialError`
Incomplete multi-byte character.

Examples

```
INCLUDE "error.s4h";
OPEN FILE INPUT "flatfile.dat";
$Error = 0;
READ $line UNTIL "\n"; /* read until linefeed */
WHILE $Error <> $Error_ReadEndOfFile {
    PRINT "Length is " & STRLEN($line) & "\n";
    READ $line UNTIL "\n";
}
```

See also

OPEN.

Notes

When *separator*, but not *length*, is specified, the READ statement may read a huge amount of data if the *separator* character is not found in the input. This can be prevented by also using *length* to limit the number of characters to be read.

On Windows, when a text file is read, if the file contains Ctrl+Z characters, the first Ctrl+Z character in the file is taken to mark end-of-file.

12.4.97 REGEXP

Name

REGEXP – Match a string against a regular expression.

Syntax

REGEXP *regexp* STRING *string* [START *start*] [END *end*];

Description

The REGEXP statement searches for the regular expression *regexp* in the string *string*. The *start* and *end* output variables receive the positions for the first and last character, respectively, of the matching sub-string. If no match is found, *start* and *end* are set to 0.

The *regexp* and *string* parameters are *expressions* that are converted to strings. The *start* and *end* parameters should be integer *variables*.

A regular expression is zero or more *branches*, separated by vertical bar characters (|). It matches anything that matches one of the branches.

A branch is zero or more *pieces*, concatenated. It matches a match for the first, followed by a match for the second, etc.

A piece is an *atom*, possibly followed by "*", "+", or "?". An atom followed by "*" matches a sequence of zero or more matches of the atom. An atom followed by "+" matches a sequence of one or more matches of the atom. An atom followed by "?" matches a match of the atom, or the null string.

An atom is a regular expression in parentheses (matching a match for the regular expression), a *range* (see below), "." (matching any single character), "^" (matching

the null string at the beginning of *string*), "\$" (matching the null string at the end of *string*), a "\" followed by a single character (matching that character), or a single character with no other significance (matching that character).

A *range* is a sequence of characters enclosed in "[]". It normally matches any single character from the sequence. If the sequence begins with "^", it matches any single character not from the rest of the sequence. If two characters in the sequence are separated by "-", this is shorthand for the full list of characters between them (for example, "[0-9]" matches any decimal digit). To include a literal "]" in the sequence, make it the first character (following a possible "^"). To include a literal "-", make it the first or last character.

Errors

The following values can be assigned to `$Error`:

- `$Error_RegexpCompileError`
Failed to compile the regular expression.
- `$Error_RegexpSearchError`
Error when searching for the regular expression.

Examples

If the following program is executed

```
DECLARE VARIABLE $start INTEGER;  
DECLARE VARIABLE $end INTEGER;  
REGEXP "0[xX][0-9a-fA-F]+" STRING "abc0x9abcJ" START $start END $end;  
PRINT "Start = " & $start & ", End = " & $end & "\n";
```

The following is written to the output file

```
Start = 4, End = 9
```

12.4.98 RELATION

Name

RELATION – Check X12 relations.

Syntax

```
{PAIRED | REQUIRED | EXCLUSION | CONDITIONAL | LIST_CONDITIONAL}  
RELATION element-list {; | statement-block}
```

Description

The RELATION compound statement checks relations between data elements within a segment, or between elements within a composite element.

The relation check is based on the presence or absence of data elements. A data element is considered to be present when it is not empty.

The list of elements to check is specified by *element-list*, which is a list of *expressions* separated by commas. The elements in the list are converted to integers.

Each data element within a segment or a composite element is referenced using an index number. The index for the first element is 1. A composite element is counted as one element.

If the relation is fulfilled, the optional *statement-block* is executed, if supplied. The following relations can be checked:

- **PAIRED**
If any elements in *element-list* are present, all of them must be present. If not, an EDI error is generated. If all elements in *element-list* exist, the *statement-block* is executed.
- **REQUIRED**
At least one of the elements in *element-list* must be present, and if so, the *statement-block* is executed. If not, an EDI error is generated.
- **EXCLUSION**
Not more than one of the elements in *element-list* must be present. If more than one element is present, an EDI error is generated. If exactly one element is present, the *statement-block* is executed.
- **CONDITIONAL**
If the first element in *element-list* is present, all other elements must also be present. If not present, an EDI error is generated. However, if the first element is not present, any of the remaining elements in *element-list* are allowed to be present. The *statement-block* is executed if all elements in *element-list* are present.
- **LIST_CONDITIONAL**
If the first element in *element-list* is present, then at least one of the remaining elements must be present. If not present, an EDI error is generated. However, if the first element is not present, any of the remaining elements in *element-list* are allowed to be present. The *statement-block* is executed if the first element exists and at least one of the remaining elements also exists.

Errors

The following values can be assigned to `$Error`:

- `$Error_RelationPaired`
The elements do not match a paired relation. This is an EDI error.
- `$Error_RelationRequired`
The elements do not match a required relation. This is an EDI error.
- `$Error_RelationExclusion`
The elements do not match an exclusion relation. This is an EDI error.
- `$Error_RelationConditional`
The elements do not match a conditional relation. This is an EDI error.
- `$Error_RelationListConditional`
The elements do not match a list conditional relation. This is an EDI error.
- `$Error_Pgmlogic_RelationOutsideSegment`
RELATION statement executed outside a SEGMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_RelationInsideRelation`
RELATION statement executed inside another RELATION statement. This is a PGMLOGIC error.

Examples

```
DECLARE VARIABLE $RelationFulfilled INTEGER;

MANDATORY SEGMENT "ABC" {
    OPTIONAL STRING 100;
    OPTIONAL STRING 100;
    OPTIONAL STRING 100;
    OPTIONAL STRING 100;

    $RelationFulfilled = 0;
    PAIRED RELATION 1, 2, 3 {
        $RelationFulfilled = 1;
        PRINT "The paired relation was fulfilled\n";
    }
    IF $RelationFulfilled = 0 {
        PRINT "The paired relation was not fulfilled\n";
    }

    REQUIRED RELATION 1, 2, 3;
    EXCLUSION RELATION 1, 2, 3;
    CONDITIONAL RELATION 1, 2;
    LIST_CONDITIONAL RELATION 3, 4;
}
```

See also

COMPOSITE, DATA ELEMENT (X12), SEGMENT.

Notes

This statement can only be used inside segments or as a part of a composite element. The Message Builder compiler tries to check this. However, if user functions or statements are used, the compiler may not detect this. The error will instead be detected at run time and then be treated as a PGMLOGIC error, that is, the Message Builder interpreter will terminate if the error is not trapped using the TRY statement (see paragraph [12.4.131](#)).

12.4.99 RELEASE

Name

RELEASE – Release (remove) an environment variable.

Syntax

```
RELEASE [environ-var];
```

Description

The RELEASE statement removes the specified environment variable or, if no variable is specified, all previously exported ones.

The *environ-var* parameter is an *expression* that is converted to a string and used as the name of the environment variable to remove.

If the *environ-var* parameter is not specified, all variables that have been exported since the Message Builder interpreter was started are removed. To remove an environment variable that existed when the interpreter was started, the name of the variable must be given explicitly.

Errors

The following values can be assigned to `$Error`:

- `$Error_ReleaseNameEmpty`
The *environ-var* parameter evaluated to an empty string.
- `$Error_ReleaseNameMissing`
The environment variable does not exist.

Examples

Program 1:

```
DECLARE VARIABLE $MyAge INTEGER;  
$MyAge = 35;  
EXPORT $MyAge AS "AGE";  
EXPORT "Main Street" AS "ADDRESS";
```

Program 2 (called from program 1):

```
DECLARE VARIABLE $HisAge INTEGER;  
DECLARE VARIABLE $HisAddress STRING;  
IMPORT "AGE" INTO $HisAge;  
RELEASE "AGE";  
IMPORT "ADDRESS" INTO $HisAddress;  
RELEASE "ADDRESS";
```

See also

EXPORT, IMPORT.

Notes

All variables, except the reserved ones, are reset and forgotten when changing program using the SWITCH statement. The EXPORT statement, together with the IMPORT statement, provides a way to share information between different Message Builder programs.

12.4.100 REPEAT

Name

REPEAT – Loop control.

Syntax

REPEAT *statement-block* WHILE *test-expression*;

Description

The REPEAT compound statement executes the statements in *statement-block* as long as *test-expression* evaluates to true (non-zero). The *statement-block* is executed at least once.

The *test-expression* is an *expression* that is converted to a floating-point value.

Examples

The following code fragment

```
DECLARE VARIABLE $loop INTEGER;

$loop = -5;
REPEAT {
    IF $loop < 0 {
        $loop = -$loop;
    }
    PRINT $loop & " ";
    $loop = $loop - 1;
} WHILE $loop > 0;
PRINT "\n";
```

will output the following to the output file

```
5 4 3 2 1
```

See also

BREAK, CONTINUE, FOR, WHILE.

12.4.101 RESERVED VARIABLES

Name

RESERVED VARIABLES – Variables with special meaning in the Message Builder language.

Description

All of the Message Builder reserved variables are global variables that have special meaning in the language. The variable names are reserved and should not be declared.

When a reserved variable holding an EDI separator or special character is set to a space, the value is undefined or a default value is used.

Name	Type	Description
\$Error	INTEGER	The last error number. Note that this variable is not automatically reset to 0 when executing a statement without errors.
\$LogString	STRING	The last log message. Log messages suppressed by the NOLOG statement or the I interpreter option are also put in this variable. Note that the value of this variable is not changed when executing a statement without errors.
\$SqlCode	INTEGER	The status of the last executed SQL statement. A value different than 0 indicates an error. The \$Error and \$LogString variables are also set to appropriate values when an SQL error occurs.

Name	Type	Description
\$DateFormat	STRING	Specifies the format used when type converting a DATE value to and from a string. See below for a description of the format of the value of this variable. The default is "%y%m%d".
\$EdiSegSep	CHAR(1)	The EDI segment separator. The last character in a segment. The default value is "\x1c" (IS4).
\$EdiSegCodeSep	CHAR(1)	The EDI segment code separator. Separates the segment name from the first data element in the segment. If this separator is not defined, the separator specified by \$EdiElemSep is used as segment code separator. This variable does not have a default value. Only used by TRADACOMS, which specifies the value to be "=".
\$EdiElemSep	CHAR(1)	The EDI element separator. Separates two data elements. The default value is "\x1d" (IS3).
\$EdiCompSep	CHAR(1)	The EDI composite element separator. Separates data elements inside a composite data element. The default value is "\x1f" (IS1).
\$EdiRelChar	CHAR(1)	The EDI release character. Indicates that the following character is valid data, not a separator or special character. The default value is undefined.
\$EdiPadChar	CHAR(1)	The EDI pad character. Pad characters found in the input file are removed. Only used by TRADACOMS, which specifies the value to be "^" (caret). This variable does not have a default value.
\$EdiSegCnt	INTEGER	The number of the current segment. The first segment in the input file is segment number 1.
\$EdiElemCnt	INTEGER	The number of the current data element (or composite) within a segment. The first data element in a segment is element number 1.
\$EdiCompCnt	INTEGER	The number of the current element within a composite. The first element in a composite is element number 1.
\$EdiPos	INTEGER	The current position in the input file. Before any characters have been read, the position is 0.
\$EdiSeg	STRING	The name of the current EDI segment.
\$EdiFile	STRING	The name of the current input file.
\$EdiCharSet	STRING	The name of the character set that the input file contains. This is the name of the CSC used to decode the characters. The default value is an empty string, that is, no CSC will be used when reading the input file (all characters are one byte long).
\$EdiInitMode	INTEGER	Code that tells the CSC specified by the \$EdiCharSet reserved variable how to initialize itself. The recognized codes and the meaning of them is CSC dependent. The default value is zero. The value of this reserved variable is only significant when reading multi-byte character data.
\$EdiResetFlag	INTEGER	If the value of this reserved variable is non-zero, the mode of the CSC is reset to its initial value when starting to read a new element. If the value is zero, the mode will be unchanged, and thus depends on the mode the CSC is in after finishing the former element. The default value is zero. The value of this reserved variable is only significant when reading multi-byte character data.
\$EdiPreFlag	INTEGER	Some multi-byte character set encodings use escape sequences to switch between different interpretation modes of succeeding bytes. If set to a non-zero value, the corresponding escape sequence (if one exists) will be prefixed to all characters returned. The default value is zero. The value of this reserved variable is only significant when reading multi-byte character data.
\$EdiByteFlag	INTEGER	If this flag is non-zero, the length of EDI string elements are counted in bytes, not characters. This makes a difference when reading multi-byte characters. The default value is zero.

Name	Type	Description
\$EdiCharValid	STRING	Indicates which characters are valid when reading data from the input file using any of the BOUNDED LOOP, COMPOSITE, DATA ELEMENT, SEGMENT and UNBOUNDED LOOP statements. The value of this variable is handled as a bit string. A character is valid if the bit corresponding to the character is set. The least significant bit in the first byte of this variable indicates if the character with code zero is valid or not, the next bit indicates if the character with code one is valid, and so on. Character codes that don't have a corresponding bit in this string (because the string is too short) are considered invalid. If the value of this reserved variable is an empty string, all characters are considered valid. The default value is the empty string.
\$Serial	INTEGER	When using an Informix database interface, this variable contains the serial number of the last inserted row in the database. For other database interfaces, this variable has no meaning, Oracle uses ".NEXTVAL" and ".CURVAL" instead, and Sybase does not support this concept.

The \$DateFormat Reserved Variable Format

The \$DateFormat reserved variable is used to control the conversion between date and string values. The \$DateFormat variable may contain a number of two-character sequences with special meaning. These sequences always start with a % character. When converting from a date to a string value, all characters, except the special sequences, are copied *as is*, and the special sequences are replaced by the values indicated below. When converting from a string to a date value, all characters except the special sequences must match character by character, and the special sequences are used to extract the date value.

The following special sequences may be used both when converting from a date to a string and when converting from a string to a date value:

Values	Meaning
%%	The % character.
%a	Abbreviated weekday name (<i>Sun, Mon, Tue, Wed, Thu, Fri</i> or <i>Sat</i>). When converting from a string to a date value, the case does not matter.
%A	Full weekday name (<i>Sunday, Monday, Tuesday, Wednesday, Thursday, Friday</i> or <i>Saturday</i>). When converting from a string to a date value, the case does not matter.
%b	Abbreviated month name (<i>Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov</i> or <i>Dec</i>). When converting from a string to a date value, the case does not matter.
%B	Full month name (<i>January, February, March, April, May, June, July, August, September, October, November</i> or <i>December</i>). When converting from a string to a date value, the case does not matter.
%d	Day of month represented as two digits (01 – 31).
%e	Day of month represented as one space and one digit, or, two digits (" 1" – "31").
%f	Fractions of a second represented as two digits.
%H	Hour represented as two digits (00 – 23).
%I	Hour represented as two digits (00 – 11).
%j	Day number of year represented as three digits (001 – 366).
%m	Month number represented as two digits (01 – 12).
%M	Minute represented as two digits (00 – 59).
%p	Ante meridian or post meridian (AM or PM). When converting from a string to a date value, the case does not matter.
%S	Seconds represented as two digits (00 – 59).

Values	Meaning
%U	Week number of the year represented as two digits (01 – 53). Sunday is the first day of the week.
%w	Weekday number represented as one digit (0 – 6). Sunday is weekday number 0.
%W	Week number of the year represented as two digits (01 – 53). Monday is the first day of the week.
%y	Year within the 20th century represented as two digits (00 – 99), that is, the two-digit year yy is interpreted as 19yy. Due to backward compatibility reasons the interpretation of two-digit years cannot be changed. It is recommended that two-digit years are avoided.
%Y	Year including century represented as four digits, for example, 2001. The year must be 1900 or later.

When converting from string to date, there are three different ways to specify the day of the year in the `$DateFormat` reserved variable:

- Month and month day (%b, %B, %m and %d, %e).
- Week number and week day (%U, %W and %a, %A, %w).
- Year day (%j).

All other combinations will result in a date conversion error.

The %s and %t special sequences have slightly different meaning, depending on whether the conversion is from a date to a string or vice versa. First conversion from date to string is presented:

- %s Seconds represented as two digits (00 – 59).
- %t Fractions of a second represented as zero to two digits. Trailing zeroes are suppressed.

When converting from string to date the meanings are:

- %s Optional seconds represented as two digits (00 – 59).
- %t Optional fractions of a second represented as one or more digits. Two fractional digits are significant.

The DATE function (see paragraph [12.4.33](#)) can also be used to convert a date value to a string.

Notes

Except for the following variables, the reserved variables should not be assigned:

- `$Error` can be set to any appropriate error code.
- `$LogString` can be set to any appropriate log message.
- `$DateFormat` can be set to specify a new format string.
- `$EdiSegSep`, `$EdiSegCodeSep`, `$EdiElemSep`, `$EdiCompSep`, `$EdiRelChar` and `$EdiPadChar` should be set to correct values for each application.
- `$EdiCharSet`, `$EdiInitMode`, `$EdiResetFlag`, `$EdiPreFlag` and `$EdiByteFlag` should be set to appropriate values when reading data containing multi-byte characters.
- `$EdiCharValid` should be set to indicate which characters are valid.

Before Message Builder can read an EDI file, it must know which separators and special characters are used in the file. Assigning the appropriate reserved variables described above sets this up. The default setup supports the EDIFACT character set B and the X12 recommendation.

When writing EDI messages, the reserved variables holding EDI separator characters should be used in the PRINT statements instead of string constants. This increases the flexibility since it makes it easier to switch to other separators.

12.4.102 RETURN

Name

RETURN – Return from a user function or statement.

Syntax

RETURN [*value*];

Description

The RETURN statement returns control from a user function or statement.

When returning from a user function, the *value* parameter must be specified. It is the function return value. This parameter is an *expression*.

No return value can be specified when returning from a user statement.

RETURN statements are not allowed outside user functions or statements.

A user function or statement may contain multiple RETURN statements.

The last statement in user functions and statements must be a RETURN, an EXIT, or a THROW statement.

See also

DECLARE FUNCTION, DECLARE STATEMENT.

12.4.103 SCAN

Name

SCAN – Move the current position or search forward in the input file.

Syntax

SCAN {SEGMENT | TEXT | COUNT} *operand*;

Description

The SCAN statement moves the current position or searches forward, from the current file position, in the input file.

The *operand* parameter is an *expression* that is converted to a string or an integer, depending on the modifier:

If the SEGMENT modifier is specified, the SCAN statement searches for the next occurrence of the EDI segment tag specified by *operand* (the *operand* parameter is

converted to a string). If *operand* is an empty string, the SCAN statement searches for the next segment tag. For the search to be successful, the segment tag must be preceded by a segment separator and followed by an element or segment separator (or a segment code separator if it is defined). If the SCAN SEGMENT statement is successful, the new file position is set to the start of the segment tag. If it is unsuccessful, the file position is not changed. Note that if *operand* is not an empty string and the current position, before SCAN is executed, is at the beginning of the specified segment tag, the position is not moved. If *operand* is an empty string, the position is moved even if the current position is at the beginning of a segment tag.

If the TEXT modifier is specified, the SCAN statement searches for the next occurrence of the character string specified by *operand* (the *operand* parameter is converted to a string). If the SCAN TEXT statement is successful, the new file position is set to the position of the first character of the found string. If it is unsuccessful, the file position is not changed. When the current position matches the specified character string, the file position is not moved.

If the COUNT modifier is specified, the SCAN statement moves the file position *operand* bytes forward or backward, depending on whether *operand* is positive or negative. The *operand* parameter is converted to an integer. If the SCAN COUNT statement fails, that is, the end or the beginning of the file is reached before the position has been moved the specified number of bytes, the new file position is set to the end or the beginning of the file.

On Windows, avoid using SCAN COUNT on text files that contain the carriage return/linefeed (CR/LF) sequence or the Ctrl+Z character. Doing so may cause the current file position to become wrong.

After the position has moved using the COUNT or TEXT modifier, the state for the input file is set as it was when the file was opened; that is, it is assumed the previous character was a segment separator character.

Errors

The following values can be assigned to `$Error`:

- `$Error_ScanEndOfFile`
End-of-file encountered. This is an informational code, not an error.
- `$Error_ScanTextEmpty`
The text string to search for is empty.
- `$Error_ScanReadError`
Read error on the input file.
- `$Error_ScanSeekError`
Seek error on the input file.

Examples

The following example searches for all Message Reference Numbers in an EDIFACT interchange:

```
$Error = 0;
SCAN SEGMENT "UNH";
WHILE $Error = 0 {
    MANDATORY SEGMENT "UNH" {
        MANDATORY STRING 14 SET $MsgRef;
        PRINT $MsgRef & "\n";
        /* skip to the end of this segment */
        SCAN TEXT $EdiSegSep;
    }
}
```

```

    }
    $Error = 0;
    SCAN SEGMENT "UNH";
}

```

The next example skips the first 42 bytes of a file and then reads 20 bytes:

```

OPEN FILE INPUT $filename;
SCAN COUNT 42;
READ $data COUNT 20;
CLOSE INPUT;

```

Notes

The *operand* parameter in a SCAN COUNT statement is given in units of bytes, not characters (this makes a difference when reading a multi-byte character file).

The Message Builder interpreter may not be able to resynchronize and continue EDI parsing after a SCAN COUNT or a SCAN TEXT statement if the new file position is in the middle of an EDI data element or segment tag.

The SEGMENT modifier requires that the reserved variables holding the EDI separator characters are set to appropriate values, and that the input file follows the EDIFACT, X12, or TRADACOMS syntax rules.

To force the current position to be moved to the next segment tag when using the SEGMENT modifier, precede the SCAN statement with a SCAN SEGMENT statement with an empty tag.

12.4.104 SEGMENT

Name

SEGMENT – Start a segment.

Syntax

```

{MANDATORY | CONDITIONAL | OPTIONAL | FLOATING}
  SEGMENT name-list [REPEAT [count]] statement-block

```

Description

The SEGMENT compound statement reads an EDI segment tag from the input file and checks that the segment tag equals one of the names in *name-list*.

The *name-list* parameter is a list of *expressions* separated by commas. Each element in the list is converted to a string.

The *count* parameter is an *expression* that is converted to an integer. If the segment is allowed to be repeated in the message, this can be specified using the REPEAT modifier. The maximum number of allowed repetitions is specified by *count*. If *count* is not specified, an unlimited number of repetitions are allowed. If the REPEAT modifier is not specified, the repeat count is set to one.

The *statement-block* is executed for each occurrence of the segment in the input file.

If the **CONDITIONAL** or **OPTIONAL** modifier is specified, the segment does not have to exist in the input file. If the **MANDATORY** modifier is specified and the segment does not exist, the Message Builder interpreter generates an EDI error.

Before *statement-block* is executed, the `$EdiSeg` reserved variable is set to the matching segment name, the `$EdiSegCnt` variable is incremented by one, and `$EdiCompCnt` and `$EdiElemCnt` are set to 0.

The **FLOATING** modifier indicates that the segment may occur anywhere in the input file, not only at the current file position. The *name-list* parameter must contain exactly one element when the **FLOATING** modifier is used. When (and if) the Message Builder interpreter finds the specified segment tag in the input file, the *statement-block* is executed before the normal program flow continues. No local variables or formal arguments or parameters may be used in floating segments. Floating segments is an X12 feature.

A floating segment statement must be executed before the floating segment occurs in the input file.

If a floating segment is executed inside a user function or statement, the effect of the floating segment statement stops when control returns to caller. Otherwise, if it is executed outside any function or statement, the effect remains until the program terminates or another **SEGMENT FLOATING** with the same *name-list* is executed. If multiple floating segment statements with the same segment name have been executed, the last one overrides the previous ones. Thus, the action taken may change during the execution of a Message Builder program or a user function or statement.

Errors

The following values can be assigned to `$Error`:

- `$Error_SegmentMandatoryEof`
End-of-file encountered when expecting a mandatory segment. This is an EDI error.
- `$Error_SegmentReadError`
Read error on the input file.
- `$Error_SegmentNoTag`
No segment tag found. This is an EDI error.
- `$Error_SegmentWrongTag`
Wrong tag found when expecting mandatory segment. This is an EDI error.
Note that this value is also assigned to `$Error`, when (1) the number of repeated segments in the input file exceeds the value specified in the *count* parameter, and (2) the segment statement is followed by a segment statement that has a different segment tag. When both these conditions are met, `$Error_SegmentWrongTag` is assigned to `$Error`. Note that this value is assigned at the line containing the segment with a different tag, because the error is detected there.
- `$Error_SegmentExtraData`
Extra data at end of segment. This is an EDI error.
- `$Error_SegmentCharacterError`
The read data contains a character that is not valid according to the `$EdiCharValid` reserved variable.
- `$Error_Pgmlogic_SegmentInsideSegment`
SEGMENT statement executed inside another SEGMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_SegmentInsideComposite`
SEGMENT statement executed inside COMPOSITE statement. This is a PGMLOGIC error.

- `$Error_Pgmlogic_SegmentInsideElement`
SEGMENT statement executed inside DATA ELEMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_SegmentInsideRelation`
SEGMENT statement executed inside RELATION statement. This is a PGMLOGIC error.

Examples

A fictive EDIFACT example:

```
MANDATORY SEGMENT "ABC" REPEAT {
  MANDATORY STRING 6 {
    PRINT "Inside segment ABC\n";
  }
}
MANDATORY SEGMENT "DEF" {
  MANDATORY STRING 6 {
    PRINT "Inside segment DEF\n";
  }
}
```

If the contents of the input file are:

```
ABC+abc'DEF+def'
```

the following is written to the output file:

```
Inside segment ABC
Inside segment DEF
```

A fictive X12 example similar to the above example but with a floating segment added:

```
FLOATING SEGMENT "NTE" {
  MANDATORY STRING 60 {
    PRINT "Inside segment NTE\n";
  }
}

MANDATORY SEGMENT "ABC" REPEAT {
  MANDATORY STRING 6 {
    PRINT "Inside segment ABC\n";
  }
}
MANDATORY SEGMENT "DEF" {
  MANDATORY STRING 6 {
    PRINT "Inside segment DEF\n";
  }
}
```

If the contents of the input file are:

```
ABC+abc'NTE+nte1'NTE+nte2'DEF+def'
```

the following is written to the output file:

```
Inside segment ABC
Inside segment NTE
Inside segment NTE
Inside segment DEF
```

See also

BOUNDED LOOP, COMPOSITE, DATA ELEMENT (EDIFACT), DATA ELEMENT (X12), SEGMENT GROUP, UNBOUNDED LOOP.

Notes

The OPTIONAL modifier is equivalent to CONDITIONAL. The recommendation is to use CONDITIONAL when parsing EDIFACT messages, and OPTIONAL when parsing X12 messages.

Avoid specifying more than one segment name in *name-list*; it is not needed when parsing standard EDI messages. The possibility to have multiple segment names in *name-list* was added in order to be able to handle some non-standard EDI messages.

The SEGMENT, DATA ELEMENT and COMPOSITE statements in the *statement-block* serves two purposes, the Message Builder interpreter both reads the EDI message and performs a syntax check.

Nesting of SEGMENT statements is not allowed. The Message Builder compiler tries to check for these. However, if user functions or statements are used, the compiler may not detect this. The error will instead be detected at run time and then be treated as a PGMLOGIC error, that is, the Message Builder interpreter will terminate if the error is not trapped using the TRY statement (see paragraph [12.4.131](#)).

12.4.105 SEGMENT GROUP

Name

SEGMENT GROUP – Start a group of segments.

Syntax

```
{MANDATORY | CONDITIONAL}  
  SEGMENT GROUP name-list [REPEAT [count]] statement-block
```

Description

The SEGMENT GROUP compound statement starts a group of segments. One of the names, specified by *name-list*, must equal the name of the first segment in the group.

The *name-list* parameter is a list of *expressions* separated by commas. Each element in the list is converted to a string.

The *count* parameter is an *expression* that is converted to an integer. If the segment group is allowed to be repeated, this can be specified using the REPEAT modifier. The maximum number of allowed repetitions is specified by *count*. If *count* is not specified, an unlimited number of repetitions are allowed. If the REPEAT modifier is not specified, the repeat count is set to one.

If the CONDITIONAL modifier is specified, the segment group does not have to exist in the input file. If the MANDATORY modifier is specified and the segment group does not exist in the input file, the interpreter generates an EDI error.

Nesting of segment groups is allowed; that is, it is possible to define a segment group inside another segment group.

Errors

See the SEGMENT statement, paragraph [12.4.104](#).

Examples

```
/* a PRICAT message */
CONDITIONAL SEGMENT GROUP "PGI" REPEAT 1000 {
    MANDATORY SEGMENT "PGI" {
        .
        .
        .
    }

    CONDITIONAL SEGMENT GROUP "PIT" REPEAT 9999 {
        MANDATORY SEGMENT "PIT" {
            .
            .
            .
        }
        CONDITIONAL SEGMENT "PIA" REPEAT 10 {
            .
            .
            .
        }
        CONDITIONAL SEGMENT "IMD" REPEAT 10 {
            .
            .
            .
        }
    }
    .
    .
    .
}
}
```

See also

COMPOSITE, DATA ELEMENT (EDIFACT), SEGMENT.

Notes

The SEGMENT_GROUP keyword can be used as a synonym for SEGMENT GROUP.

The SEGMENT GROUP statement is almost identical to the UNBOUNDED LOOP statement. Use SEGMENT GROUP when parsing EDIFACT messages, and UNBOUNDED LOOP when parsing X12.

The OPTIONAL modifier can be used instead of CONDITIONAL. The recommendation is to use CONDITIONAL.

Avoid specifying more than one segment name in *name-list*; it is not needed when parsing standard EDI messages. The possibility to have multiple segment names in *name-list* was added in order to be able to handle some non-standard EDI messages.

12.4.106 SERIALIZE

Name

SERIALIZE – Generate a string representation for a value.

Syntax

SERIALIZE *value* INTO *dest*;

Description

The SERIALIZE statement converts a value to a string representing the value. The *value* parameter is an *expression* of any type, including arrays and records. The *dest* parameter receives the generated string. It is a *variable* and should be of string type.

Examples

The following program:

```
DECLARE RECORD R {
    DECLARE FIELD $i INTEGER;
    DECLARE FIELD $f FLOAT;
}

DECLARE VARIABLE $out STRING;
DECLARE VARIABLE $i INTEGER;
DECLARE VARIABLE $f FLOAT;
DECLARE VARIABLE $s STRING;
DECLARE VARIABLE $c CHAR(10);
DECLARE VARIABLE $d DATE;
DECLARE VARIABLE $a1[] INTEGER;
DECLARE VARIABLE $a2[] INTEGER;
DECLARE VARIABLE $r RECORD R;

$i = 5;
$f = 15.6;
$s = "test string\n";
$c = "abc";
$d = CURRENTDATE();
$a1[1] = 2;
$a1[2] = 3;
$a1[3] = 4;
$r.$i = 105;
$r.$f = .5;

SERIALIZE $i INTO $out;
PRINT $out & "\n";
SERIALIZE $f INTO $out;
PRINT $out & "\n";
SERIALIZE $s INTO $out;
PRINT $out & "\n";
SERIALIZE $c INTO $out;
PRINT $out & "\n";
SERIALIZE $d INTO $out;
PRINT $out & "\n";
SERIALIZE $a1 INTO $out;
PRINT $out & "\n";
```

```
SERIALIZE $a2 INTO $out;
PRINT $out & "\n";
SERIALIZE $r INTO $out;
PRINT $out & "\n";
```

produces the following output:

```
5
15.6
"test string\n"
"abc      "
D1999060213090707
{2,3,4}
{}
{105,0.5}
```

See also

UNSERIALIZE.

Notes

The UNSERIALIZE statement (see paragraph [12.4.134](#)) can be used to perform the inverse operation.

STRING and CHAR values are surrounded by double quote characters. Double quote characters and backslash characters inside strings are escaped by prefixing them with a backslash character. Non-printable characters are replaced by special sequences starting with a backslash character:

- \n Linefeed character.
- \t Tab character.
- \v Vertical tab character.
- \b Backspace character.
- \r Carriage return character.
- \f Formfeed character.
- \a Audible alert character.
- \nnn All the remaining non-printable characters (*nnn* is a three digit octal number).

DATE values are prefixed by a D character followed by the year (*yyyy*), month (*mm*), month day (*dd*), hour (*HH*), minute (*MM*), second (*SS*), and fractions of a second (*FF*):

yyyymmddHHMMSSFF

Arrays and records are surrounded by { and } characters and the elements are separated by comma characters.

12.4.107 SETDATE

Name

SETDATE – Modify fields in a date variable.

Syntax

SETDATE *date* FIELD *field-selector* VALUE *new-field*;

Description

The SETDATE statement modifies the specified field in a date variable. The variable to modify is *date*, the field to modify is specified by *field-selector*, and the new value of the field is *new-field*.

The *date* parameter is a *variable* that should be of date type.

The *new-field* parameter is an *expression* that is converted to an integer.

The *field-selector* parameter is an integer selector:

- \$DateField_Year Modify the year field. The *new-field* parameter should include the century.
- \$DateField_Month Modify the month number field. A month number field is in the range 0 – 11.
- \$DateField_Monthday Modify the month day number field. A month day number field is in the range 1 – 31.
- \$DateField_Hour Modify the hours field. An hours field is in the range 0 – 23.
- \$DateField_Minute Modify the minutes field. A minutes field is in the range 0 – 59.
- \$DateField_Second Modify the seconds field. A seconds field is in the range 0 – 59.
- \$DateField_Fraction Modify the fractions of a second field. A fractions of a second field is in the range 0 – 99.

The above constant variables are declared in the *datefld.s4h* include file.

The *new-field* value may be outside the valid range for a field. The interpreter adjusts both the value of the new field and other fields so all fields are within valid ranges. For example, if the hours field is set to 25, the month day field is incremented by one and the hours field is set to one. If the hours field is set to -1, the month day field is decremented by one and the hours field is set to 23. If the month day field in turn becomes invalid, it is also adjusted, and so on until all fields have valid values.

Errors

The following value can be assigned to \$Error:

```
$Error_SetdateInvalidField
```

The *field-selector* parameter is invalid.

Examples

The following example prints the current date and time plus two hours:

```
INCLUDE "datefld.s4h";
DECLARE VARIABLE $date DATE;

$date = CURRENTDATE();
SETDATE $date FIELD $DateField_Hour
      VALUE GETDATE($date, $DateField_Hour) + 2;
```

```
PRINT "Year: " & GETDATE($date, $DateField_Year) & "\n";
PRINT "Month: " & GETDATE($date, $DateField_Month) & "\n";
PRINT "Month Day: " & GETDATE($date, $DateField_Monthday) & "\n";
PRINT "Hour: " & GETDATE($date, $DateField_Hour) & "\n";
PRINT "Minute: " & GETDATE($date, $DateField_Minute) & "\n";
PRINT "Second: " & GETDATE($date, $DateField_Second) & "\n";
PRINT "Fraction: " & GETDATE($date, $DateField_Fraction) & "\n";
```

See also

GETDATE.

12.4.108 SIDENT

Name

SIDENT – Function that converts a statement identifier to a string.

Syntax

SIDENT(*identifier*)

Description

Given a statement name identifier, the SIDENT function returns a string containing the statement name. It also exports the statement (see the PRAGMA directive, paragraph [12.4.92](#)).

The *identifier* argument is the statement name identifier. Note that it should not be surrounded by double quotes.

If the statement specified by *identifier* is located inside a module, the returned string includes a module prefix. All alphabetic letters in the returned string are uppercase letters.

Examples

The assignment:

```
$s = SIDENT(DEMOSTMNT);
```

is equivalent to:

```
PRAGMA EXPORT STATEMENT DEMOSTMNT;
$s = "DEMOSTMNT";
```

See also

FIDENT.

Notes

The SIDENT function is convenient to use together with the USE statement.

12.4.109 SLEEP

Name

SLEEP – Pause program execution for a specified number of seconds.

Syntax

SLEEP *time*;

Description

The SLEEP statement stops execution of a program for a certain amount of time. The time is specified by the *time* parameter as the number of seconds to sleep.

The *time* parameter is an *expression* that is converted to an integer.

Examples

```
SLEEP 10;
```

12.4.110 SOURCEFILE

Name

SOURCEFILE – Function that returns the name of the Message Builder source code file.

Syntax

SOURCEFILE()

Description

The SOURCEFILE function returns the name of the Message Builder source code file from where the call is made. The returned value is a string.

The SOURCEFILE function is valid in *const-expressions*.

Examples

```
PRINTERR "Error in file " & SOURCEFILE() & "\n";
```

See also

SOURCELINE, SOURCEMODULE, SOURCEPROCEDURE.

Notes

The Message Builder compiler replaces the SOURCEFILE function call with a string containing the name of the file.

12.4.111 SOURCELINE

Name

SOURCELINE – Function that returns the source code line number.

Syntax

SOURCELINE()

Description

The SOURCELINE function returns the source code line number from where the call is made. The returned value is an integer.

The SOURCELINE function is valid in *const-expressions*.

Examples

```
PRINTERR "Error at line " & SOURCELINE() & "\n";
```

See also

SOURCEFILE, SOURCEMODULE, SOURCEPROCEDURE.

Notes

The Message Builder compiler replaces the SOURCELINE function call with an integer constant.

12.4.112 SOURCEMODULE

Name

SOURCEMODULE – Function that returns the name of the Message Builder module.

Syntax

SOURCEMODULE()

Description

The SOURCEMODULE function returns the name of the Message Builder module from where the call is made. The returned value is a string where all alphabetic letters are uppercase letters. If the call is made outside a module, an empty string is returned.

The SOURCEMODULE function is valid in *const-expressions*.

Examples

```
PRINTERR "Error in function " & SOURCEMODULE() &
      "." & SOURCEPROCEDURE() & "\n";
```

See also

SOURCEFILE, SOURCELINE, SOURCEPROCEDURE.

Notes

The Message Builder compiler replaces the SOURCEMODULE function call with a string containing the name of the module.

12.4.113 SOURCEPROCEDURE

Name

SOURCEPROCEDURE – Function that returns the name of the Message Builder procedure.

Syntax

SOURCEPROCEDURE()

Description

The SOURCEPROCEDURE function returns the name of the Message Builder user function or statement from where the call is made. The returned value is a string where all alphabetic letters are uppercase letters. If the call is made outside a user function or statement, an empty string is returned.

If the user function or statement is located inside a module, the module name is not included in the name returned by the SOURCEPROCEDURE function.

The SOURCEPROCEDURE function is valid in *const-expressions*.

Examples

```
PRINTERR "Error in function " & SOURCEPROCEDURE() & "\n";
```

See also

SOURCEFILE, SOURCELINE, SOURCEMODULE.

Notes

The Message Builder compiler replaces the SOURCEPROCEDURE function call with a string containing the name of the user function or statement.

12.4.114 SPLIT CLOSE

Name

SPLIT CLOSE – Close the split input file.

Syntax

SPLIT CLOSE;

Description

The SPLIT CLOSE statement closes the split input file opened using the SPLIT OPEN statement.

Errors

The following value can be assigned to `$Error`:

`$Error_SplitCloseNoInputFile`

No split input file open.

Examples

See the SPLIT INTO statement, page 251.

See also

SPLIT INTO, SPLIT OPEN.

12.4.115 SPLIT INTO

Name

SPLIT INTO – Split a file into several files.

Syntax

SPLIT INTO *outfile* [{ON SEGMENT | AFTER | BEFORE} *operands* [SET *index*]];

Description

The SPLIT INTO statement extracts a part of a file, opened using the SPLIT OPEN statement, up to a specified search string. By using the SPLIT INTO statement repeatedly, it is possible to split a file into several files.

The data extracted from the input file are stored in the file specified by *outfile*. If the output file does not exist, it is created. If it does exist, it is overwritten. The *outfile* parameter is an *expression* that is converted to a string.

The search strings that will indicate break points for the split operation are specified using the *operands* parameter. It is a list of one or more *expressions* separated by commas. Each *expression* in the list is converted to a string. If none of the search strings is found, the end of file is used as break point. If *operands* is a single empty string or it is not specified, all the data to the end of the file is extracted.

The ON SEGMENT modifier indicates that the *operands* parameter specifies segment names; that is, for the search to be successful the data in the input file must be preceded by a segment separator and followed by an element separator (or a segment

code separator if one has been defined). The file is split immediately before the matching segment name.

The AFTER and BEFORE modifiers indicate that the *operands* parameter specifies text strings. The BEFORE modifier indicates that the break point for the split operation is immediately before the text string. The AFTER modifier indicates that the break point is immediately after the text string.

The *index* parameter is a *variable* that is set to the string in *operands* that were used to split the file. If the split operation used the end of file as break point, the *index* variable is set to an empty string.

Errors

The following values can be assigned to `$Error`:

- `$Error_SplitOutFileEmpty`
The *outfile* parameter is an empty string.
- `$Error_SplitInputFileNotOpen`
No input file open. The SPLIT OPEN statement must be executed before SPLIT INTO.
- `$Error_SplitOutputFileOpen`
Failed to create the output file.
- `$Error_SplitInputReadError`
Read error on the split input file.
- `$Error_SplitOutputWriteError`
Write error on the output file.
- `$Error_SplitInputSeekError`
Seek error on the split input file.

Examples

The following example splits an interchange into individual message files:

```
DECLARE VARIABLE $index STRING;
DECLARE VARIABLE $count = 0 INTEGER;
DECLARE VARIABLE $fileName STRING;

/* get the file name from the command line */
$fileName = ARGUMENT(1);

/* open the file to split */
SPLIT OPEN $fileName;

/* extract data until the first message header */
/* (or interchange trailer if no messages) */
SPLIT INTO $fileName & ".UNB" ON SEGMENT "UNH", "UNZ"
    SET $index;

/* extract data until the next UNH segment (or */
/* interchange trailer), repeat for all messages */
WHILE $index = "UNH" {
    $count = $count + 1;
    SPLIT INTO $fileName & "." & $count
        ON SEGMENT "UNH", "UNZ" SET $index;
}

/* extract the interchange trailer (until EOF) */
```

```

IF $index = "UNZ" {
    SPLIT INTO $fileName & ".UNZ";
}

/* close the split input file */
SPLIT CLOSE;

```

An example that splits a file after each row:

```

DECLARE VARIABLE $index STRING;
DECLARE VARIABLE $count = 0 INTEGER;
DECLARE VARIABLE $destFile STRING;
DECLARE VARIABLE $srcFile STRING;

$srcFile = ARGUMENT(1);

/* open the file to split */
SPLIT OPEN $srcFile;

/* extract lines until EOF */
REPEAT {
    $count = $count + 1;
    $destFile = $srcFile & "." & $count;
    SPLIT INTO $destFile AFTER "\n" SET $index;
} WHILE $index <> "";

/* close the split input file */
SPLIT CLOSE;

```

See also

SPLIT CLOSE, SPLIT OPEN.

Notes

The SPLIT INTO statement is typically used to extract EDI messages from a file containing an interchange/transaction set.

12.4.116 SPLIT OPEN

Name

SPLIT OPEN – Open the split input file.

Syntax

SPLIT OPEN inputfile;

Description

The SPLIT OPEN statement opens the input file used by the SPLIT INTO statement. The file to split is specified by *inputfile*. The *inputfile* parameter is an *expression* that is converted to a string.

If a split input file already is open when SPLIT OPEN is executed, it is automatically closed as soon as the new split input file has been successfully opened.

Errors

The following values can be assigned to `$Error`:

- `$Error_SplitOpenEmptyInputFile`
The *inputfile* parameter is an empty string.
- `$Error_SplitOpenOpenFail`
Failed to open the file specified by the *inputfile* parameter.

Examples

See the SPLIT INTO statement, paragraph [12.4.115](#).

See also

SPLIT CLOSE, SPLIT INTO.

12.4.117 STRBACKWARD

Name

STRBACKWARD – Function that searches for a sub-string.

Syntax

STRBACKWARD(*string*, *search*[, *start*])

Description

The STRBACKWARD function searches a string for a sub-string. The search starts at the position specified by *start* or, if *start* is not specified, at the end of the string. The *string* argument is the string to search and *search* is the sub-string to search for. The *string* and *search* arguments are *expressions* that are converted to strings and *start* is an *expression* that is converted to an integer.

If *start* is zero or not specified, it is equivalent to the *start* value:

```
STRLEN($string) - STRLEN($search) + 1
```

If the *search* string is found, the index for the start of it is returned. Otherwise, 0 is returned. The index for the first character in a string is 1.

The STRBACKWARD function is valid in *const-expressions* (*string*, *search* and *start* must in such a case be *const-expressions*).

Errors

The following values can be assigned to `$Error`:

- `$Error_StrbackwardEmptySearch`
The *search* argument is an empty string.
- `$Error_StrbackwardIllegalStart`
The *start* argument is illegal.

Examples

The following example replaces the last comma character in the `$s` string with a period:

```
DECLARE VARIABLE $s STRING;
DECLARE VARIABLE $index INTEGER;
.
.
.
$index = STRBACKWARD($s, ",");
IF $index > 0 {
    $s[$index,$index] = ".";
}
```

See also

REGEXP, STRFORWARD.

12.4.118 STRFORWARD

Name

STRFORWARD – Function that searches for a sub-string.

Syntax

STRFORWARD(*string*, *search*[, *start*])

Description

The STRFORWARD function searches a string for a sub-string. The search starts at the position specified by *start*. If *start* is not specified, it starts at the beginning of the string. The *string* argument is the string to search and *search* is the sub-string to search for. The *string* and *search* arguments are *expressions* that are converted to strings and *start* is an *expression* that is converted to an integer.

If the *search* string is found, the index for the start of it is returned. Otherwise, 0 is returned. The index for the first character in a string is 1.

The STRFORWARD function is valid in *const-expressions* (*string*, *search* and *start* must in such a case be *const-expressions*).

Errors

The following values can be assigned to `$Error`:

- `$Error_StrforwardEmptySearch`
The *search* argument is an empty string.
- `$Error_StrforwardIllegalStart`
The *start* argument is illegal.

Examples

The following example replaces the first comma character in the `$s` string with a period:

```
DECLARE VARIABLE $s STRING;
DECLARE VARIABLE $index INTEGER;
```

```
.  
.   
.   
$index = STRFORWARD($s, ",");  
IF $index > 0 {  
    $s[$index,$index] = ".";  
}
```

See also

REGEXP, STRBACKWARD.

12.4.119 STRLEN

Name

STRLEN – Function that returns the length of a string.

Syntax

STRLEN(*operand*)

Description

The STRLEN function returns the length, in characters, of *operand*. The *operand* argument is an *expression* that is converted to a string.

The STRLEN function is valid in *const-expressions* (*operand* must in such a case be a *const-expression*).

Examples

The following example:

```
DECLARE VARIABLE $length INTEGER;
DECLARE VARIABLE $data STRING;
$data = "Hello";
$length = STRLEN($data) + STRLEN(" World!");
PRINT $length & "\n";
$length = STRLEN($data & " World!");
PRINT $length & "\n";
PRINT STRLEN($length) & "\n";
```

produces the following output:

```
12
12
2
```

Notes

A string may contain null characters, that is, a character with character code zero. The STRLEN function returns the length of the string including null characters.

12.4.120 STRREPEAT

Name

STRREPEAT – Function that returns a repeated string.

Syntax

STRREPEAT(*string*, *count*)

Description

The STRREPEAT function builds a string containing the *string* argument repeated *count* times and returns the result. The *string* argument is an *expression* that is converted to a string, and *count* is an *expression* that is converted to an integer.

The STRREPEAT function is valid in *const-expressions* (*string* and *count* must in such a case be *const-expressions*).

Errors

The following value can be assigned to `$Error`:

```
$Error_StrrepeatIllegalCount
```

The *count* argument is less than 0.

Examples

The following example right justifies the strings in the `$s` array. The field width is 20.

```
FOR $i = 1 TO ARRAYSIZE($s) {  
    $s[$i] = STRREPEAT(" ", 20 - STRLEN($s[$i])) & $s[$i];  
}
```

12.4.121 SUBSTRING

Name

SUBSTRING – Sub-string extraction operator.

Syntax

`string[from, to]`

Description

The sub-string extraction operator extracts a sub-string from a string.

The string to extract the sub-string from is *string*. It should be a scalar value that is either a simple variable reference, an array element or a record field.

The *from* and *to* arguments are *expressions* that are converted to integers. The index for the start of the sub-string to extract is indicated by *from*, and the index for the last character is indicated by *to*. The index for the first character in a string is 1.

The sub-string operator may also be used as the left hand side in an assignment. In such a case the sub-string is replaced by the right hand side of the assignment.

The sub-string operator is valid in *const-expressions* (*from* and *to* must in such a case be *const-expressions* and *string* must represent a constant value).

Errors

The following values can be assigned to `$Error`:

- `$Error_SubstrTooBig`
The *to* argument is greater than the string length. This is a warning.
- `$Error_SubstrFromTooSmall`
The *from* argument is less than or equal to 0.
- `$Error_SubstrTooSmall`
The *to* argument is less than or equal to 0.
- `$Error_SubstrFromGtTo`
The *from* argument is greater than *to*.
- `$Error_SubstrFromTooBig`
The *from* argument is greater than the string length.

Examples

The following program:

```
DECLARE VARIABLE $string STRING;
DECLARE VARIABLE $array[] STRING;

$string = "Thu Nov 21 1993";
$array[1] = $string;
/* extract the month name /
PRINT $string[5, 5 + 2] & "\n";
/* extract the year */
PRINT $array[1][12, 12 + 3] & "\n";
```

produces the following result:

```
Nov
1993
```

12.4.122 SUSPEND

Name

SUSPEND – Suspend execution and make it possible for another Message Builder program to execute.

Syntax

SUSPEND [*code*];

Description

The SUSPEND statement suspends execution and makes it possible for another Message Builder program to execute.

The *code* parameter is an expression that is converted to an integer.

If the *code* parameter is 0 or the Message Builder interpreter is not running in server mode (see paragraph [11.3.3](#)), the execution of the program that calls SUSPEND will continue as soon as all other programs have had a chance to execute.

If the *code* parameter is non-zero or is not specified and the Message Builder interpreter is running in server mode (see paragraph [11.3.3](#)), the execution of the calling program will continue as soon as there is an event waiting; that is, a call to the S4EDI.READEVENT statement (see the *Message Builder Library* manual) is guaranteed to return an event.

Examples

See the EXECUTE statement, paragraph [12.4.63](#), for an example.

See also

EXECUTE, S4EDI.READEVENT in the *Message Builder Library* manual.

12.4.123 SWITCH

Name

SWITCH – Switch Message Builder program.

Syntax

SWITCH *filename*;

Description

The SWITCH statement switches to another Message Builder program and starts executing it. The program to switch to is specified by the *filename* parameter. It is an *expression* that is converted to a string.

The values of all Message Builder variables in the program that contains the SWITCH statement, except the reserved ones, are lost but all open files remain open.

The Message Builder interpreter uses the same search path to find the program file specified in the SWITCH statement as the program file specified on the Message Builder interpreter command line. If *filename* has the extension ".s4", it is replaced by ".x4".

Errors

The following values can be assigned to `$Error`:

- `$Error_SwitchEmptyFileName`
The *filename* parameter is an empty string.
- `$Error_ProgramOpenError`
Failed to open the executable Message Builder program file.
- `$Error_ProgramEmpty`
The Message Builder program file is empty.
- `$Error_ProgramReadError`
Error when reading the Message Builder program file.

Examples

The following statement switches to the Message Builder program *main.s4*:

```
SWITCH "main.s4";
```

The following example switches to alternative Message Builder programs, according to the type of the received message.

```
DECLARE VARIABLE $type STRING;
DECLARE VARIABLE $newfile STRING;

$newfile = "error.s4";
MANDATORY SEGMENT "UNH" {
  MANDATORY STRING 14;
  MANDATORY COMPOSITE {
    CONDITIONAL STRING 6 SET $type {
      IF $type = "PERQRY" {
        $newfile = "perqry.s4";
      }
      IF $type = "PRICAT" {
        $newfile = "pricat.s4";
      }
    }
    MANDATORY NUMBER 3;
    CONDITIONAL NUMBER 3;
    CONDITIONAL STRING 8;
    CONDITIONAL STRING 6;
  }
  CONDITIONAL STRING 35;
  CONDITIONAL COMPOSITE {
    MANDATORY NUMBER 2;
    CONDITIONAL FIXED STRING 1;
  }
}
SWITCH $newfile;
```

See also

EXPORT, IMPORT.

Notes

In most cases, it is better to use a user function or statement rather than the SWITCH statement.

The EXPORT and IMPORT statements can be used to transfer variable values between Message Builder programs.

12.4.124 SYSTEM

Name

SYSTEM – Function that returns the system type.

Syntax

SYSTEM()

Description

The SYSTEM function returns an integer value indicating the type of system the Message Builder interpreter is running on. Possible system types are:

- `$System_Posix` POSIX systems, for example, UNIX.
- `$System_Win32` Some flavor of the WIN32 API.
- `$System_Os400` The OS/400 operating system (IBM AS/400).

The above mnemonic variables are constants declared in the *sysinfo.s4h* include file.

Examples

```
CASE SYSTEM()
WHEN $System_Posix {
    $fileName = "/tmp/abc";
}
WHEN $System_Win32 {
    $fileName = "C:\\TEMP\\ABC";
}
WHEN $System_Os400 {
    $fileName = "QTEMP/TMP(ABC)";
}
```

12.4.125 THROW

Name

THROW – Generate an exception.

Syntax

THROW [*exception*];

Description

The THROW statement generates an exception/error. The *exception* parameter is the type of the exception. It is an *expression* that is converted to a string. The default value for *exception* is "EXECUTE".

If the exception string is "EXECUTE", "EDI", "AGGREGATE" or "PGMLOGIC" the effect will be the same as when an execution, EDI parsing, aggregate, or program logic error occurs, respectively. Other exception strings can also be specified. They are intended to be used together with the TRY statement (see paragraph [12.4.131](#)).

The *exception.s4h* include file contains mnemonic constants representing the "EXECUTE", "EDI", "AGGREGATE" and "PGMLOGIC" exception strings. The names of these mnemonic constants are `$Exception_Execute`, `$Exception_Edi`, `$Exception_Aggregate` and `$Exception_Pgmlogic`, respectively.

When a THROW statement is executed, the control goes to the CATCH part of a surrounding TRY statement or, if no TRY block surrounds the THROW statement, the Message Builder program terminates. Hence, the execution can never continue with a statement following a THROW statement.

Examples

See the TRY statement, paragraph [12.4.131](#).

See also

TRY.

12.4.126 TO_ISO8859

Name

TO_ISO8859 – Function that converts to a standard character set.

Syntax

TO_ISO8859(*string*)

Description

The TO_ISO8859 function converts the characters in *string* from the character set used by the machine running the Message Builder interpreter to the ISO8859-1 character set. The converted string is returned.

The *string* argument is an *expression* that is converted to a string.

See also

FROM_ISO8859.

Notes

For machines that use the ISO8859-1 character set (UNIX and Windows machines), the returned string equals *string*.

12.4.127 TRIMLEFT

Name

TRIMLEFT – Function that removes leading pad characters.

Syntax

TRIMLEFT(*string*[, *pad*])

Description

The TRIMLEFT function removes leading pad characters from the *string* argument and returns the result. The pad characters are specified by the *pad* argument. Note that *pad* may contain more than one character. All leading characters that exist in *pad* are removed from *string*. The default pad character is a space.

Both the *string* and *pad* arguments are *expressions* that are converted to strings.

The TRIMLEFT function is valid in *const-expressions* (*string* and *pad* must in such a case be *const-expressions*).

Examples

Write the contents of the `$s` string, with leading space and tab characters removed, to the output file:

```
PRINT TRIMLEFT($s, " \t");
```

See also

TRIMRIGHT.

12.4.128 TRIMRIGHT

Name

TRIMRIGHT – Function that removes trailing pad characters.

Syntax

TRIMRIGHT(*string*[, *pad*])

Description

The TRIMRIGHT function removes trailing pad characters from the *string* argument and returns the result. The pad characters are specified by the *pad* argument. Note that *pad* may contain more than one character. All trailing characters that exist in *pad* are removed from *string*. The default pad character is a space.

Both the *string* and *pad* arguments are *expressions* that are converted to strings.

The TRIMRIGHT function is valid in *const-expressions* (*string* and *pad* must in such a case be *const-expressions*).

Examples

Write the contents of the `$s` string, with trailing space and tab characters removed, to the output file:

```
PRINT TRIMRIGHT($s, " \t");
```

See also

TRIMLEFT.

12.4.129 TRUNCATE (function)

Name

TRUNCATE – Function that removes trailing empty EDI elements.

Syntax

TRUNCATE(*string*)

Description

The TRUNCATE function removes all trailing EDI element separators that correspond to empty elements in the *string* argument. In the same way, trailing composite element separators are also removed. If only the segment tag remains when trailing element and composite separators have been removed, the segment tag and the segment separator are also removed. The truncated string is returned.

The *string* argument is an *expression* that is converted to a string.

The TRUNCATE function performs the truncation at the end of segments and composites. If the supplied string ends in the middle of a segment or composite, this partial segment or composite is not modified.

The TRUNCATE function considers all characters between a segment separator and an element separator as segment tags (if the segment code separator is defined, all characters between segment separators and segment code separators are treated as segment tags). Release characters can be used to insert element and segment code separators (or other separators) into a segment tag.

The TRUNCATE function assumes that the *string* argument starts with a segment tag and, hence, it inserts an implicit segment separator character before the first character in *string*.

Examples

The following program:

```
DECLARE VARIABLE $unh STRING;

/* set EDIFACT separators */
$EdiSegSep = " ";
$EdiSegCodeSep = " ";
$EdiElemSep = "+";
$EdiCompSep = ":";
$EdiRelChar = "?";
$EdiPadChar = " ";

/* create UNH segment (using empty elements) */
$unh = "UNH" & $EdiElemSep &
      "MSGREF12345" & $EdiElemSep &
      "" & $EdiCompSep &
      "" & $EdiCompSep &
      "" & $EdiCompSep &
      "" & $EdiCompSep &
      "" & $EdiElemSep &
      "" & $EdiElemSep &
      "" & $EdiCompSep &
      "" & $EdiSegSep;
PRINT $unh & "\n";
PRINT TRUNCATE($unh) & "\n";
```

produces the following result:

```
UNH+MSGREF12345+:::++:'
UNH+MSGREF12345'
```

See also

TRUNCATE (statement).

12.4.130 TRUNCATE (statement)

Name

TRUNCATE – Enable or disable truncation of separators when printing.

Syntax

TRUNCATE {ON | OFF};

Description

The TRUNCATE statement turns "truncate mode" on or off. The "truncate mode" affects the result of the PRINT statement. In "truncate mode," the PRINT statement truncates (removes) all trailing element separators that correspond to empty elements. Trailing composite element separators are also truncated in the same way. If only the segment tag remains when trailing element and composite separators have been removed, the segment tag and the segment separator are also removed.

The TRUNCATE statement considers all characters between a segment separator and an element separator as segment tags (if the segment code separator is defined, all characters between segment separators and segment code separators are treated as segment tags). Release characters can be used to insert element and segment code separators (or other separators) into a segment tag.

In order to make the segment tag truncation work for the first segment tag, the previous character is assumed to be a segment separator when truncation is enabled or a new file is opened.

The default when Message Builder is invoked is TRUNCATE OFF.

Examples

The program:

```
/* set EDIFACT separators */
$EdiSegSep = "";
$EdiSegCodeSep = " ";
$EdiElemSep = "+";
$EdiCompSep = ":";
$EdiRelChar = "?";
$EdiPadChar = " ";

/* create UNH segment (using empty variables) */
PRINT "UNH" & $EdiElemSep;
PRINT "MSGREF12345" & $EdiElemSep;
PRINT $MsgType & $EdiCompSep;
PRINT $VerNum & $EdiCompSep;
PRINT $RelNum & $EdiCompSep;
PRINT $Agency & $EdiCompSep;
PRINT $AssocCode & $EdiElemSep;
PRINT $Access & $EdiElemSep;
```

```
PRINT $SeqNum & $EdiCompSep;  
PRINT $Ind & $EdiSegSep;
```

results in the following being written to the output file:

```
UNH+MSGREF12345+:::++:'
```

By adding "TRUNCATE ON;" at the beginning of the program, the output will change to:

```
UNH+MSGREF12345'
```

See also

TRUNCATE (function), PRINT.

Notes

"Truncate mode" is used to avoid printing empty elements at the end of segments (and at the end of composites).

12.4.131 TRY

Name

TRY – Error and exception handling.

Syntax

```
TRY guarded-statement-block  
CATCH exception-var  
WHEN const-expr-list-1 statement-block-1  
WHEN const-expr-list-2 statement-block-2  
.  
.  
.  
WHEN const-expr-list-N statement-block-N  
[WHEN OTHERS statement-block-others]
```

Description

The TRY compound statement takes care of errors that occur when a Message Builder program is executed. It is also used to trap exceptions generated by the THROW statement. When an error or exception occurs while a *guarded-statement-block* is being executed, the execution will continue at the point specified by the CATCH keyword. A string indicating the type of the exception is put in the *exception-var* variable, and then the CATCH part behaves in the same way as a CASE statement with *exception-var* as selector.

Const-expr-list-n are lists of *const-expressions* separated by commas, and *statement-block-n* and *guarded-statement-block* are *statement-blocks*.

The exception string is "EXECUTE", "EDI", "AGGREGATE" or "PGMLOGIC" when an execution, EDI parsing, aggregate, or program logic error has occurred, respectively. Other exception strings may also show up if the THROW statement is used. If no WHEN OTHERS part is present and the exception string does not equal any of the choices in the WHEN lists, the exception is propagated to the next TRY construct surrounding the current one, if any. If there are no such constructs, the program will terminate.

The *exception.s4h* header file contains constant variable declarations for the "standard" exception strings. The names of these constant variables are `$Exception_Execute`, `$Exception_Edi`, `$Exception_Aggregate`, and `$Exception_Pgmlogic`, respectively.

The *exception-var* variable is automatically declared as a string scalar variable in all the WHEN statement blocks. It shadows variables with the same name declared outside these blocks.

Examples

The following example implements a user-declared statement that creates a file, writes some data to it, and closes the file while taking care of all possible errors:

```
INCLUDE "exception.s4h";

DECLARE STATEMENT
CREATE_FILE IN $filename PRIMITIVE DATA IN $data PRIMITIVE
{
    TRY {
        IF $fileName = "*" {
            THROW;
        }
        NOLOG {
            OPEN FILE OUTPUT $fileName;
        }
    }
    CATCH $exception
    WHEN $Exception_Execute {
        LOG "CREATE_FILE: failed to create file '" & $fileName & "'";
        THROW;
    }

    TRY {
        NOLOG {
            PRINT $data;
        }
    }
    CATCH $exception
    WHEN $Exception_Execute {
        LOG "CREATE_FILE: failed to write data to file '" &
            $fileName & "'";
        TRY { /* ignore FILE DELETE errors */
            FILE DELETE $fileName;
        }
        CATCH $exception
        WHEN $Exception_Execute {}
        THROW;
    }

    TRY {
        NOLOG {
            CLOSE OUTPUT;
        }
    }
    CATCH $exception
    WHEN $Exception_Execute {
        LOG "CREATE_FILE: failed to close file";
        FILE DELETE $fileName;
        THROW;
    }
}
```

```

        RETURN;
    }

```

The following is an example that demonstrates how TRY can be used to handle errors when parsing EDI messages:

```

INCLUDE "exception.s4h";
DECLARE VARIABLE $String STRING;

TRY {
    MANDATORY SEGMENT "AAA" {
        CONDITIONAL VARIABLE STRING 3 SET $String {
            PRINT "AAA:" & $String & "\n";
        }
    }
    MANDATORY SEGMENT "REP" REPEAT 3 {
        CONDITIONAL VARIABLE STRING 3 SET $String {
            PRINT "REP:" & $String & "\n";
        }
    }
}
CATCH $exception
WHEN $Exception_Edi {
    PRINT "Error when parsing EDI - resynchronizing\n";
    SCAN SEGMENT "BBB";
}

/* no error handling for the next segment */
MANDATORY SEGMENT "BBB" {
    CONDITIONAL VARIABLE STRING 3 SET $String {
        PRINT "BBB:" & $String & "\n";
    }
}

```

See also

CASE, THROW.

Notes

If a user function or statement is called from *guarded-statement-block* and an error occurs there, the execution will continue at the point specified by the CATCH keyword.

12.4.132 UNBOUNDED LOOP

Name

UNBOUNDED LOOP – Start an X12 unbounded loop.

Syntax

```

{MANDATORY | OPTIONAL}
    UNBOUNDED LOOP name-list [REPEAT [count]] statement-block

```

Description

The UNBOUNDED LOOP compound statement starts a loop of segments and other loops.

The *name-list* parameter is a list of *expressions* separated by commas. Each element in the list is converted to a string.

The *count* parameter is an *expression* that is converted to an integer.

One of the names, specified by *name-list*, must equal the name of the first segment in the loop.

If the OPTIONAL modifier is specified, the unbounded loop need not exist in the input file. If the MANDATORY modifier is specified and the unbounded loop does not exist in the input file, the Message Builder interpreter generates an EDI error.

Repetition of the unbounded loop can be specified using the REPEAT modifier. The maximum number of allowed repetitions is specified by *count*. If *count* is not specified, an unlimited number of repetitions are allowed. If the REPEAT modifier is not specified, the repeat count is set to 1.

Nesting of unbounded loops is allowed; that is, it is possible to define an unbounded loop inside another unbounded loop.

Errors

The following values can be assigned to `$Error`:

- `$Error_UnboundedLoopMandatoryEof`
End-of-file encountered when expecting a mandatory unbounded loop. This is an EDI error.
- `$Error_UnboundedLoopReadError`
Read error on the input file.
- `$Error_UnboundedLoopNoTag`
No segment tag found. This is an EDI error.
- `$Error_UnboundedLoopWrongTag`
Wrong tag found when expecting a mandatory unbounded loop.

This is an EDI error. Note that this value is also assigned to `$Error`, when (1) the number of repeated unbounded loops in the input file exceeds the value specified in the *count* parameter, and (2) the unbounded loop statement is followed by an unbounded loop statement that has a different loop tag. When both these conditions are met, `$Error_UnboundedLoopWrongTag` is assigned to `$Error`. Note that this value is assigned at the line containing the unbounded loop with a different tag, because the error is detected there.
- `$Error_UnboundedLoopExtraData`
Extra data at end of segment. This is an EDI error.
- `$Error_UnboundedLoopCharacterError`
The read data contains a character that is not valid according to the `$EdiCharValid` reserved variable.
- `$Error_Pgmlogic_UnboundedLoopInsideSegment`
UNBOUNDED LOOP statement executed inside SEGMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_UnboundedLoopInsideComposite`
UNBOUNDED LOOP statement executed inside COMPOSITE statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_UnboundedLoopInsideElement`
UNBOUNDED LOOP statement executed inside DATA ELEMENT statement. This is a PGMLOGIC error.
- `$Error_Pgmlogic_UnboundedLoopInsideRelation`
UNBOUNDED LOOP statement executed inside RELATION statement. This is a

PGMLOGIC error.

Examples

In the following example, the first segment in the unbounded loop can be specified as OPTIONAL or MANDATORY. The reason is that it is the occurrence of the N1 segment that determines whether the loop exists or not.

```
OPTIONAL UNBOUNDED LOOP "N1" REPEAT 5 {
  OPTIONAL SEGMENT "N1" {
    .
    .
    .
  }
  OPTIONAL SEGMENT "N2" {
    .
    .
    .
  }
  OPTIONAL SEGMENT "N3" {
    .
    .
    .
  }
  OPTIONAL SEGMENT "N4" {
    .
    .
    .
  }
  OPTIONAL SEGMENT "G61" {
    .
    .
    .
  }
  OPTIONAL SEGMENT "REF" {
    .
    .
    .
  }
}
```

See also

BOUNDED LOOP, COMPOSITE, DATA ELEMENT (X12), SEGMENT.

Notes

The UNBOUNDED LOOP statement is almost identical to the SEGMENT GROUP statement. Use UNBOUNDED LOOP when parsing X12 messages, and SEGMENT GROUP when parsing EDIFACT.

The CONDITIONAL modifier can be used instead of OPTIONAL. The recommendation is to use OPTIONAL.

Avoid specifying more than one segment name in *name-list*; it is not needed when parsing standard EDI messages. The possibility to have multiple segment names in *name-list* was added in order to be able to handle some non-standard EDI messages.

12.4.133 UNLOADPGM

Name

UNLOADPGM – Unload a Message Builder program.

Syntax

UNLOADPGM *program-name* [ID *id*];

Description

The UNLOADPGM statement unloads a Message Builder program previously loaded using the LOADPGM statement.

The *program-name* parameter is an *expression* that is converted to a string and used as the file name of the Message Builder program to unload.

The ID for the program to unload is specified by the *id* parameter. The same ID as the one specified in the corresponding LOADPGM statement should be used. The *id* parameter is an *expression* that is converted to a string and the default value is an empty string.

Errors

The following values can be assigned to `$Error`:

- `$Error_UnloadpgmEmptyName`
The *program-name* parameter is an empty string.
- `$Error_UnloadpgmNotLoaded`
The Message Builder program specified by *program-name* and *id* is not loaded.
- `$Error_UnloadpgmProgramCalled`
Cannot unload the program because some code in it is currently used.
- `$Error_UnloadpgmVariableReferred`
Cannot unload the program because some variables in it are currently used.

Examples

See DECLARE EXTERNAL FUNCTION, paragraph [12.4.35](#), and DECLARE EXTERNAL STATEMENT, paragraph [12.4.36](#).

See also

LOADPGM.

Notes

All MODULE_CLEAN statements are automatically executed when the program is unloaded.

For performance reasons, the Message Builder interpreter keeps internal copies of loaded programs after they have been unloaded. This means that if a program is unloaded, modified, and then loaded again, the interpreter will continue to use the old version of the program. To avoid this, restart the interpreter.

12.4.134 UNSERIALIZE

Name

UNSERIALIZE – Convert a string representation of a value to the value itself.

Syntax

UNSERIALIZE *string* INTO *dest*;

Description

The UNSERIALIZE statement converts a string representation of a value to the value itself. It performs the reverse operation of the SERIALIZE statement. The string to convert is *string* and *dest* receives the value. The *string* parameter is an *expression* that is converted to a string and *dest* is a *variable* that should be of a type compatible with the value represented by *string*.

Errors

The following value can be assigned to \$Error:

```
$Error_UnserializeStringNotMatch
```

The *string* parameter and the type of the destination variable are not compatible.

Examples

Assume that two programs want to communicate integer array values using the EXPORT and IMPORT statements. This can be accomplished using the following two programs:

```
/* program 1 */
DECLARE VARIABLE $a[] INTEGER;
DECLARE VARIABLE $out STRING;
$a[1] = 23;
$a[2] = 34;
$a[3] = 45;
SERIALIZE $a INTO $out;
EXPORT $out AS "ARRAY_VALUE";

/* program 2 */
DECLARE VARIABLE $in STRING;
DECLARE VARIABLE $a[] INTEGER;
IMPORT "ARRAY_VALUE" INTO $in;
UNSERIALIZE $in INTO $a;
FOR $i = 1 TO ARRAYSIZE($a) {
    PRINT $a[$i] & "\n";
}
```

See also

SERIALIZE.

Notes

The SERIALIZE statement (see paragraph [12.4.106](#)) has probably been used to produce the *string* parameter.

See the documentation for the SERIALIZE statement for a description of the *string* format expected by UNSERIALIZE.

12.4.135 USE

Name

USE – Associate a function or statement name with an external function or statement.

Syntax

```
USE {FUNCTION | STATEMENT} external-name [FROM program-name]  
    [ID id] AS name;
```

Description

The USE statement associates the external declared function or statement *name* with an external function or statement from the *program-name* Message Builder program. The *id* parameter is used to distinguish between multiple copies of the same program (see the LOADPGM statement, paragraph [12.4.84](#)). The name of the function or statement in *program-name* is *external-name*. The *program-name* program has previously been loaded using the LOADPGM statement or it is the main program (that is, the Message Builder program specified when the interpreter was started). If *program-name* is not specified or is an empty string, the currently executing program is used. The default value for *id* is an empty string.

The *external-name*, *program-name* and *id* parameters are *expressions* that are converted to strings.

The Message Builder interpreter converts all lowercase letters in *external-name* to uppercase.

If the *external-name* function or statement is located inside a module declaration, the module name must be added in front of the function or statement name (for example, "MOD.FUNC").

The *name* parameter is a function or statement name that must be declared using DECLARE EXTERNAL FUNCTION or DECLARE EXTERNAL STATEMENT. If the external declaration is located inside a module declaration and the USE statement is not inside that module, the *name* parameter must have the module name as prefix (for example, MOD.FUNC).

Errors

The following values can be assigned to `$Error`:

- `$Error_UseEmptySubroutineName`
The *external-name* parameter is an empty string.
- `$Error_UseEmptyAssociatedName`
If this error code is obtained, the code generated by the Message Builder compiler is not correct. This should never happen.

- `$Error_UseUnknowProgram`

The Message Builder program specified by *program-name* is not loaded.

- `$Error_UseUnknownSubroutine`

The name of the external function or statement specified by *external-name* cannot be found in *program-name*.

Note that the compiler must be told to output information about all functions and statements that can be called from other programs. This is done either by using the PRAGMA EXPORT directive, by using the **F** and **S** options to the Message Builder compiler, or by using the FIDENT (see paragraph 12.4.67) and SIDENT (see paragraph 12.4.108) functions.

Examples

See DECLARE EXTERNAL FUNCTION, paragraph 12.4.35, and DECLARE EXTERNAL STATEMENT, paragraph 12.4.36.

See also

DECLARE EXTERNAL FUNCTION, DECLARE EXTERNAL STATEMENT, FIDENT, LOADPGM, PRAGMA, SIDENT, UNLOADPGM.

12.4.136 VARIABLE TYPES

Name

VARIABLE TYPES – Variable types supported by the Message Builder language.

Description

Message Builder supports the following built-in variable types:

- INTEGER

An integral data type. The precision of this data type corresponds to the signed long integer C data type. The precision of this type depends on the target machine. 32 bits are usually used to store an integer value, but some machines use 64 bit. Less than 32 bits are never used.

- FLOAT

A floating-point data type. The precision of this data type corresponds to the double C data type, which usually is a 64 bit IEEE-754 floating-point number.

- STRING or STRING(size)

A data type for character strings that has a dynamic storage size; memory is allocated when needed. There is no upper limit for the length of strings. The *size* specification is a *const-expression* that is converted to an integer. It is ignored except when using the variable in an EXEC SQL SELECT statement. Note that the value of a string variable may become longer than the *size* specified when it was declared.

A STRING variable can hold any character, including the null character.

Converting a STRING value to an INTEGER or FLOAT never fails. Leading spaces are ignored, and then all characters up to the first invalid one are used. The remaining characters are ignored. If there are no valid characters, the result is zero. For example, converting the string "123abc" to an integer gives the result 123. When converting a STRING to a DATE value, an error is generated if the string does not

contain a valid date (that is, the format of the string is not compatible with the `$DateFormat` reserved variable, see paragraph [12.4.101](#)).

- **CHAR(size)**

A data type for character strings with a fixed storage size of *size* characters. The *size* parameter is a *const-expression* that is converted to an integer. When a CHAR variable is assigned, it is padded with trailing spaces up to *size* characters or, if it is longer than *size*, it is truncated to a length of *size* characters.

A CHAR variable can hold any character, including the null character.

The behavior when converting a CHAR value to an INTEGER, FLOAT or DATE value is the same as when converting from a STRING.

- **DATE**

A data type representing a date and a time in hours, minutes, seconds, and fractions of a second. The precision is one-hundredth of a second.

When converting a DATE variable to a string value, for example, when printing a DATE variable or converting it to a STRING or CHAR variable, the format of the result is determined by the reserved variable `$DateFormat` (see paragraph [12.4.101](#)). A DATE variable can also be converted to a string using the DATE function (see paragraph [12.4.33](#)).

When a DATE variable is converted to an INTEGER or FLOAT value, the date is represented as the number of seconds since the computer's base time. The definition of the base time is different for different computers. For UNIX machines, it is the time since January 1, 1970.

A variable can also be a data record. A data record is a user defined data type that consists of one or more fields where the fields are of one of the above built-in types, another data record, or an array. The DECLARE RECORD directive (see paragraph [12.4.39](#)) is used to declare a data record.

Arrays of variables of all the types listed above are also supported. The number of dimensions for an array is the number of left and right bracket pairs specified when the array was declared (see DECLARE VARIABLE, paragraph [12.4.41](#)). For example:

```
DECLARE VARIABLE $array[][] INTEGER;
```

declares an integer array with two dimensions. Note that no sizes are given, and that arrays are automatically expanded if a non-existing element is assigned. However, an error is generated if a non-existing element is read (for example, referred to in an *expression*).

A two-dimensional array is actually an array of arrays, for example,

```
DECLARE VARIABLE $v[] INTEGER;
```

declares an array of integers while

```
DECLARE VARIABLE $a[][] INTEGER;
```

declares an array of arrays of integers. This means that a two-dimensional array does not have to be rectangular. For example, the `$a[3]` array of integers may contain five elements and `$a[4]` two.

The index for the first element of an array is 1.

Errors

The following value can be assigned to `$Error`:

```
$Error_StringToDateConvert
```

Error when converting a string to a DATE value.

See also

DECLARE VARIABLE.

12.4.137 WHILE

Name

WHILE – Loop control.

Syntax

WHILE *test-condition statement-block*

Description

The WHILE compound statement executes the statements in *statement-block* as long as *test-condition* evaluates to true (non-zero). If *test-condition* initially evaluates to false, *statement-block* is not executed.

The *test-condition* parameter is an *expression* that is converted to a floating-point value.

Examples

Loop forever:

```
DECLARE VARIABLE $True = 1 CONSTANT INTEGER;
WHILE $True {
    .
    .
    .
}
```

Loop 100 times or until \$quit is set to non-zero:

```
$quit = 0;
$count = 0;
WHILE $count < 100 AND $quit = 0 {
    .
    .
    .
    $count = $count + 1;
}
```

See also

BREAK, CONTINUE, FOR, REPEAT.

12.4.138 WITH

Name

WITH – Use variable aliases.

Syntax

WITH *aliases statement-block*

Description

The WITH compound statement is used to create aliases for expressions. An alias looks like a variable, but is actually just a reference to the expression that it is an alias for. If the expression that the variable is an alias for is a valid left value, it is valid to make assignments to the alias variable. Reasons for using WITH statements are to make the code more readable and also to make the code more efficient. The reason that the code can become more efficient is that the same expression doesn't have to be evaluated multiple times.

The *aliases* part consists of a comma-separated list of alias definitions. Each alias definition has the form:

variable AS expression [EXPAND]

where *variable* is a simple variable (without indices or record references) and *expression* is an expression that *variable* shall be an alias for. The *variable* variable is automatically declared and its type becomes the same as the type of *expression*. All the *variables* are local to *statement-block* and they do not exist on the right side of the AS keywords.

When *statement-block* is executed, all the *variables* can be used as normal variables inside it. If the corresponding *expression* is a valid left value, *variable* can also be assigned (with one exception: if *expression* is a sub-string expression). If *expression* is a reference to a reserved variable and *variable* is assigned in *statement-block*, the reserved variable will be set when *statement-block* is exited.

If the EXPAND modifier is specified, arrays in *expression* are expanded as required. It is not valid to specify EXPAND when *expression* is not a proper left value or a sub-string expression. If *expression* is a reference to an array element that does not exist and the EXPAND modifier isn't specified, an error will be obtained during runtime.

Examples

The following example illustrates how the usage of WITH can make code more efficient:

```
DECLARE VARIABLE $a[][] INTEGER;

FOR $i = 1 TO 100 {
    WITH $x AS $a[$i] EXPAND {
        FOR $j = 1 TO 100 {
            $x[$j] = $i * $j;
        }
    }
}

FOR $i = 1 TO 3 {
    FOR $j = 1 TO 10 {
        PRINT " " & $a[$i][$j];
    }
}
```

```
    }  
    PRINT "\n";  
}
```

The second double FOR loop could also have used the WITH statement, but it was written like this for illustrative purposes. Running the above program will output the following:

```
1 2 3 4 5 6 7 8 9 10  
2 4 6 8 10 12 14 16 18 20  
3 6 9 12 15 18 21 24 27 30
```

13 Appendix A: EDIFACT examples

13.1 Presentation

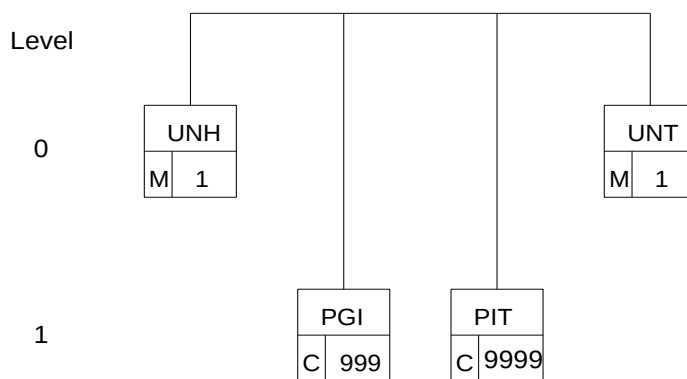
This appendix contains examples that read and write EDIFACT messages. All the messages used in this appendix are based on the EDIFACT message syntax. To fully understand the following examples, familiarity with EDIFACT concepts, such as branching diagrams and segment layout specifications, is necessary.

Most of the message specifications certified by the United Nations covers very large messages. In order to make the examples less complicated, a non-standard message specification – ARTINF – has been invented.

13.2 Syntax check

The purpose of the ARTINF message is to transfer article information. It consists of four types of standard segments.

The following figure contains the branching diagram for ARTINF:



13.2.1 Segment specification

The following table presents the UNH segment (the message header). Its purpose is to head, identify, and specify a message.

Ref	Repr.		Name	Remarks
0062	an..14	M	MESSAGE REFERENCE NUMBER	Sender's unique message reference
S009		M	MESSAGE IDENTIFIER	
0065	an..6	M	Message type identifier	ARTINF
0052	an..3	M	Message type version number	
0054	an..3	M	Message type release number	
0051	an..2	M	Controlling agency, coded	
0057	an..6	C	Association assigned code	
0068	an..35	C	COMMON ACCESS REFERENCE	
S010		C	STATUS OF THE TRANSFER	
0070	n..2	M	Sequence message transfer number	
0073	a1	C	First/last sequence message transfer ind.	
The next table presents the PGI (Product Group Information) segment. It specifies a group of articles.				
5709	an..3	M	PRICE/TARIFF TYPE, CODED	4 = Catalogue
C288		C	PRICING GROUP	
5711	an..3	C	Pricing group, coded	
4774	an..35	C	Textual equivalent of the associated coded repr	
The next table presents the PIT (Price ITem line) segment. It specifies an individual item on which information is wanted.				
1735	an..3	M	RELATIONAL QUALIFIER	1 = Line item
C511		C	ITEM IDENTIFICATION	
7721	an..3	M	Item qualifier	1 = Article identification
7722	an..35	C	Item number	
C509		C	PRICE INFORMATION	
5785	an..3	M	Price qualifier	
5782	n..15	C	Price	
5375	an..3	C	Price type, coded	
5387	an..3	C	Price type qualifier	
5284	n..9	C	Unit price basis	
6411	an..3	C	Measure unit qualifier	
1229	an..3	C	ACTION REQUEST, CODED	
C292		C	PRICE CHANGE INFORMATION	
5707	an..3	M	Price change indicator, coded	
1131	an..3	C	Code list qualifier	
7701	an..3	C	ARTICLE AVAILABILITY, CODED	
The next table presents the UNT (message trailer) segment. It is used to end and check the completeness of a message.				
0074	n..6	M	NUMBER OF SEGMENTS IN A MESSAGE	Number including UNH and UNT
0062	an..14	M	MESSAGE REFERENCE NUMBER	Number identical to same element in header segment UNH

Below is an example of a Message Builder program that reads an ARTINF message and performs syntax checks (data extraction will be discussed later):

```

/*
 * UNH - Message header.
 */
MANDATORY SEGMENT "UNH" {
    /* message reference number */
    MANDATORY STRING 14;

```

```

/* message identifier */
MANDATORY COMPOSITE {
    /* message type identifier */
    MANDATORY STRING 6 VALIDATE "ARTINF";
    /* message type version number */
    MANDATORY STRING 3;
    /* message type release number */
    MANDATORY STRING 3;
    /* controlling agency */
    MANDATORY STRING 2;
    /* association assigned code */
    CONDITIONAL STRING 6;
}
/* common access reference */
CONDITIONAL STRING 35;
/* status of the transfer */
CONDITIONAL COMPOSITE {
    /* sequence message transfer number */
    MANDATORY NUMBER 2;
    /* first/last sequence message transfer... */
    CONDITIONAL FIXED STRING 1;
}
} /* UNH */

/*
 * PGI segment - Product group information.
 */
MANDATORY SEGMENT "PGI" REPEAT 999 {
    /* price/tariff type */
    MANDATORY STRING 3;
    /* pricing group */
    CONDITIONAL COMPOSITE {
        CONDITIONAL STRING 3;
        CONDITIONAL STRING 35;
    }
} /* PGI */

/*
 * PIT segment - Price item line.
 */
/* price item line */
MANDATORY SEGMENT "PIT" REPEAT 9999 {
    /* relational qualifier */
    MANDATORY STRING 3;
    /* item identifier */
    CONDITIONAL COMPOSITE {
        /* item qualifier */
        MANDATORY STRING 3;
        /* item (article) number */
        CONDITIONAL STRING 35;
    }
    /* price information */
    CONDITIONAL COMPOSITE {
        /* price qualifier */
        MANDATORY STRING 3;
        /* price */
        CONDITIONAL NUMBER 15;
        /* price type, coded */
        CONDITIONAL STRING 3;
        /* price type qualifier */
        CONDITIONAL STRING 3;
    }
}

```

```

        /* unit price basis */
        CONDITIONAL NUMBER 9;
        /* measure unit qualifier */
        CONDITIONAL STRING 3;
    }
    /* action request */
    CONDITIONAL STRING 3;
    /* price change information */
    CONDITIONAL COMPOSITE {
        /* price change indicator */
        MANDATORY STRING 3;
        /* code list identifier */
        CONDITIONAL STRING 3;
    }
    /* article availability */
    CONDITIONAL STRING 3;
} /* PIT */

/*
 * UNT - Message trailer.
 */
MANDATORY SEGMENT "UNT" {
    /* number of segments in a message */
    MANDATORY NUMBER 6;
    /* message reference number /
    MANDATORY STRING 14;
} /* UNT */

```

13.3 Data extraction

The next step is to use the data in the EDI message. In the data element definition, it is possible to add a SET modifier that instructs Message Builder to assign the value of the read data element to the specified variable. It is also possible to add a statement block after each data element statement. This block of statements is executed if, and when, Message Builder finds the corresponding data element in the input file.

In the following example, the item identity part of the PIT segment of the above message specification has been broken out. A statement block has been added and the read data is assigned to the `$ArtNo` variable. The statement block contains code to print the read data.

```

.
.
.
/* item identifier */
CONDITIONAL COMPOSITE {
    /* item qualifier */
    MANDATORY STRING 3;
    /* item (article) number */
    CONDITIONAL STRING 35 SET $ArtNo {
        PRINT "Article number:" & $ArtNo & "\n";
    }
}
.
.
.

```

13.3.1 Storing extracted data in an SQL database

After extracting several data elements into variables, another typical task is to insert the data into a database. The SQL statements are used for this purpose:

```
DATABASE OPEN "register";
.
.
.
MANDATORY NUMBER 14 SET $ArtNo;
CONDITIONAL STRING 40 SET $Text;
MANDATORY NUMBER 8 SET $Price;
EXEC SQL INSERT INTO ArtCat
        VALUES (: $ArtNo, : $Text, : $Price);
```

13.3.2 Storing extracted data in a conventional text file

Data can be stored in any file format. Assume that the data from the previous example shall be stored as one line of text in a file instead of in a database. To be able to separate the different fields a comma (,) is used.

The DATABASE OPEN statement in the above example is replaced with an OPEN statement:

```
OPEN FILE OUTPUT "dest.txt";
```

and the EXEC SQL INSERT statement is replaced with a PRINT:

```
PRINT $ArtNo & "," & $Text & "," & $Price & "\n";
```

The result may be:

```
244150,Chili Sauce,12.55
7017703450,Ceylon Breakfast Tea,17.75
```

13.4 Creating an EDI message

When reading an EDI message, all the permutations of the message layout allowed by the standard must be considered. When creating a message, the data forms the layout of the message. Therefore it is a less complex task to create a message than to read one. This difference in complexity also shows in the Message Builder language. There are several statements designed to make it easier to read an EDI message. However, to create a message, there is basically only the need for the PRINT statement.

Values for the segment separator, the element separator and the component element separator are available in the reserved variables \$EdiSegSep, \$EdiElemSep and \$EdiCompSep, respectively.

13.4.1 Using data from an SQL database

In the following example, an ARTINF message is created. Only the Message Builder code for the PGI segments is shown.

```
DECLARE VARIABLE $Num INTEGER;
DECLARE VARIABLE $Text STRING;
DECLARE VARIABLE $TariffType STRING;

/* let Message Builder remove redundant separators */
TRUNCATE ON;
```

```

/* open/create the output file */
OPEN FILE OUTPUT "message.out";

/* set tariff type to 'Catalog' */
$TariffType = "4";
.
.
.
/* create PGI segments */
EXEC SQL SELECT GrpNumber, TextLine INTO :$Num, :$Text
          FROM ProductGroup {
  /* this code is repeated for each found product group */
  PRINT "PGI" & $EdiElemSep;
  PRINT $TariffType & $EdiElemSep;
  PRINT $Num & $EdiCompSep;
  PRINT ESCAPE($Text);
  PRINT $EdiSegSep;
}
.
.
.

```

The result may be:

```
PGI+4+310:Food'PGI+4+322:Clothes'
```

13.4.2 Using data from a conventional text file

In a previous section, an EDI message was read and the information was stored in a text file. Now the reverse operation will be performed - read the text file and create an EDIFACT message.

The text file created had the following format:

```
244150,Chili Sauce,12.55
7017703450,Ceylon Breakfast Tea,17.75
```

Below follows an example containing code fragments demonstrating how the READ statement can be used to read the file:

```

INCLUDE "error.s4h";
.
.
.
/* let Message Builder remove redundant separators */
TRUNCATE ON;

OPEN FILE INPUT "dest.txt";
OPEN FILE OUTPUT "edifact.msg";

/* create UNH and other segments */
.
.
.
/* read price information from the text file */
/* and store it in the EDIFACT message */
READ $ArtNo UNTIL ",";
READ $Text UNTIL ",";
READ $Price UNTIL "\n";

$Error = 0;
WHILE $Error <> $Error_ReadEndOfFile {
  PRINT $SegName & $EdiElemSep; /* start of segment */
  PRINT $ArtNo & $EdiElemSep;
  PRINT $Text & $EdiElemSep;
  PRINT $Price;
  PRINT $EdiSegSep; /* end of segment */
}

```

```
    READ $ArtNo UNTIL ",";
    READ $Text UNTIL ",";
    READ $Price UNTIL "\n";
}
.
.
.
```

14 Appendix B: An X12 example

14.1 Presentation

In the Message Builder language, the differences between working with EDIFACT and X12 messages are small. The differences that do exist are limited to the syntax check of the messages.

In order to make the following example less complicated, a non-standard message is invented. The new message is called "Article Information" and it consists of four types of standard X12 segments.

The specification of the message and the descriptions of the four segments are as follows:

000 Article Information This example for exercise use contains the format and establishes the data content of an Article Information Transaction Set (000) for use within the context of an Electronic Data Interchange (EDI) environment. The transaction set is a fictitious example, constructed by Frontec AMT AB.					
Table 1					
POS.NO.	SEG.ID	NAME	REG.DES.	MAX USE	LOOP REPEAT
010	ST	Transaction Set Header	M	1	
	LOOP ID - LIN				10000
020	LIN	Item identification	M	1	
030	CTP	Pricing Information	O	1	
040	SE	Transaction Set Trailer	M	1	

ST Transaction Set Header

To indicate the start of a transaction set and to assign a control number

ST *	ST01	143	*	ST02	329	N / L
	TS ID Code			TS Control Number		
	MIZ	ID	3/3	M	AN	4/9

SE Transaction Set Trailer

To indicate the end of the transaction set and provide the count of the transmitted segments (including the ST and SE segments).

SE *	SE01	96	*	SE02	329	N / L
	Number of Inc Segs			TS Control Number		
	M	NO	1/10	M	AN	4/9

LIN Item Identification

To specify basic item identification data..

LIN *	<table><tr><td>LIN01</td><td>350</td></tr><tr><td>Assigned ID</td><td></td></tr><tr><td>OIZ AN 1/11</td><td></td></tr></table>	LIN01	350	Assigned ID		OIZ AN 1/11		*	<table><tr><td>LIN02</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>M ID 2/2</td><td></td></tr></table>	LIN02	235	Prod/Serv ID Qual		M ID 2/2		*	<table><tr><td>LIN03</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>M AN 1/30</td><td></td></tr></table>	LIN03	234	Product/Service ID		M AN 1/30		*	<table><tr><td>LIN04</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN04	235	Prod/Serv ID Qual		O ID 2/2		*	<table><tr><td>LIN05</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN05	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN06</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN06	235	Prod/Serv ID Qual		O ID 2/2	
	LIN01	350																																													
	Assigned ID																																														
	OIZ AN 1/11																																														
	LIN02	235																																													
Prod/Serv ID Qual																																															
M ID 2/2																																															
LIN03	234																																														
Product/Service ID																																															
M AN 1/30																																															
LIN04	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
LIN05	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN06	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
*	<table><tr><td>LIN07</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN07	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN08</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN08	235	Prod/Serv ID Qual		O ID 2/2		*	<table><tr><td>LIN09</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN09	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN10</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN10	235	Prod/Serv ID Qual		O ID 2/2		*	<table><tr><td>LIN11</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN11	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN12</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN12	235	Prod/Serv ID Qual		O ID 2/2	
LIN07	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN08	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
LIN09	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN10	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
LIN11	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN12	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
*	<table><tr><td>LIN13</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN13	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN14</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN14	235	Prod/Serv ID Qual		O ID 2/2		*	<table><tr><td>LIN15</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN15	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN16</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN16	235	Prod/Serv ID Qual		O ID 2/2		*	<table><tr><td>LIN17</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN17	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN18</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN18	235	Prod/Serv ID Qual		O ID 2/2	
LIN13	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN14	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
LIN15	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN16	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
LIN17	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN18	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
*	<table><tr><td>LIN19</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN19	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN20</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN20	235	Prod/Serv ID Qual		O ID 2/2		*	<table><tr><td>LIN21</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN21	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN22</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN22	235	Prod/Serv ID Qual		O ID 2/2		*	<table><tr><td>LIN23</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN23	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN24</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN24	235	Prod/Serv ID Qual		O ID 2/2	
LIN19	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN20	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
LIN21	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN22	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
LIN23	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN24	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
*	<table><tr><td>LIN25</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN25	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN26</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN26	235	Prod/Serv ID Qual		O ID 2/2		*	<table><tr><td>LIN27</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN27	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN28</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN28	235	Prod/Serv ID Qual		O ID 2/2		*	<table><tr><td>LIN29</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN29	234	Product/Service ID		X AN 1/30		*	<table><tr><td>LIN30</td><td>235</td></tr><tr><td>Prod/Serv ID Qual</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	LIN30	235	Prod/Serv ID Qual		O ID 2/2	
LIN25	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN26	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
LIN27	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN28	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
LIN29	234																																														
Product/Service ID																																															
X AN 1/30																																															
LIN30	235																																														
Prod/Serv ID Qual																																															
O ID 2/2																																															
*	<table><tr><td>LIN31</td><td>234</td></tr><tr><td>Product/Service ID</td><td></td></tr><tr><td>X AN 1/30</td><td></td></tr></table>	LIN31	234	Product/Service ID		X AN 1/30		N / L																																							
LIN31	234																																														
Product/Service ID																																															
X AN 1/30																																															

SYNTAX NOTES

04 C0405 - If LIN04 is present, then LIN05 is required.
06 C0607 - If LIN06 is present, then LIN07 is required.
08 C0809 - If LIN08 is present, then LIN09 is required.
10 C1011 - If LIN10 is present, then LIN11 is required.
12 C1213 - If LIN12 is present, then LIN13 is required.
14 C1415 - If LIN14 is present, then LIN15 is required.
16 C1617 - If LIN16 is present, then LIN17 is required.
18 C1819 - If LIN18 is present, then LIN19 is required.
20 C2021 - If LIN20 is present, then LIN21 is required.
22 C2223 - If LIN22 is present, then LIN23 is required.
24 C2425 - If LIN24 is present, then LIN25 is required.
26 C2627 - If LIN26 is present, then LIN27 is required.
28 C2829 - If LIN28 is present, then LIN29 is required.
30 C3031 - If LIN30 is present, then LIN31 is required.

SEMANTIC NOTES

01 LIN01 is the line item identification.

COMMENTS

00 See the Data Dictionary for a complete list of ID's.
02 LIN02 through LIN31 provide for fifteen(15) different product/service ID's for each item. For Example:
Case, Color, Drawing No., UPC No., ISBN No., Model No., SKU.

CTP Pricing Information

To specify pricing information.

CTP*	<table><tr><td>CTP01</td><td>687</td></tr><tr><td>Class of Trade Code</td><td></td></tr><tr><td>O ID 2/2</td><td></td></tr></table>	CTP01	687	Class of Trade Code		O ID 2/2		*	<table><tr><td>CTP02</td><td>236</td></tr><tr><td>Price ID Code</td><td></td></tr><tr><td>O ID 3/3</td><td></td></tr></table>	CTP02	236	Price ID Code		O ID 3/3		*	<table><tr><td>CTP03</td><td>212</td></tr><tr><td>Unit Price</td><td></td></tr><tr><td>O R 1/14</td><td></td></tr></table>	CTP03	212	Unit Price		O R 1/14		*	<table><tr><td>CTP04</td><td>380</td></tr><tr><td>Quantity</td><td></td></tr><tr><td>O R 1/15</td><td></td></tr></table>	CTP04	380	Quantity		O R 1/15		*	<table><tr><td>CTP05</td><td>355</td></tr><tr><td>Unit/Basis Meas Code</td><td></td></tr><tr><td>X ID 2/2</td><td></td></tr></table>	CTP05	355	Unit/Basis Meas Code		X ID 2/2		*	<table><tr><td>CTP06</td><td>648</td></tr><tr><td>Price Mult Qualifier</td><td></td></tr><tr><td>O ID 3/3</td><td></td></tr></table>	CTP06	648	Price Mult Qualifier		O ID 3/3	
	CTP01	687																																													
	Class of Trade Code																																														
O ID 2/2																																															
CTP02	236																																														
Price ID Code																																															
O ID 3/3																																															
CTP03	212																																														
Unit Price																																															
O R 1/14																																															
CTP04	380																																														
Quantity																																															
O R 1/15																																															
CTP05	355																																														
Unit/Basis Meas Code																																															
X ID 2/2																																															
CTP06	648																																														
Price Mult Qualifier																																															
O ID 3/3																																															
	<table><tr><td>CTP07</td><td>649</td></tr><tr><td>Multiplier</td><td></td></tr><tr><td>XIZ R 1/10</td><td></td></tr></table>	CTP07	649	Multiplier		XIZ R 1/10		*	<table><tr><td>CTP08</td><td>762</td></tr><tr><td>Monetary Amount</td><td></td></tr><tr><td>OIZ R 1/15</td><td></td></tr></table>	CTP08	762	Monetary Amount		OIZ R 1/15		N / L																															
CTP07	649																																														
Multiplier																																															
XIZ R 1/10																																															
CTP08	762																																														
Monetary Amount																																															
OIZ R 1/15																																															

SYNTAX NOTES

04 C0405 - If CTP04 is present, then CTP05 is required.
06 C0607 - If CTP06 is present, then CTP07 is required.

SEMANTIC NOTES

07 CTP07 is a multiplier factor to arrive at a final discounted price. A multiplier of .90 would be the factor if a 10% discount is given.
08 CTP08 is the rebate amount.

COMMENTS

03 See Figures Appendix for an example detailing the use of CTP03 and CTP04.
03 See Figures Appendix for an example detailing the use of CTP03, CTP04 and CTP07.

The X12 statements in the Message Builder language are built on keywords used in the X12 message specifications; mandatory, optional, relation, unbounded loop and so on. The statements handle two tasks; they check the syntax of the received message and read data from it (into variables).

This manual does not explain how to interpret X12 message specifications. For more information about this, see the documents describing the X12 standard.

Below, an example of a Message Builder program that reads an "Article Information" message and performs a syntax check follows.


```

/*
 * ST - Transaction Set Header.
 */
MANDATORY SEGMENT "ST" {
    /* TS ID Code */
    MANDATORY STRING 3, 3;
    /* TS Control Number */
    MANDATORY STRING 4, 9;
}

/*
 * Loop ID - LIN.
 */
MANDATORY UNBOUNDED LOOP "LIN" REPEAT 10000 {
    /*
     * LIN - Item Identification.
     */
    MANDATORY SEGMENT "LIN" {
        /* Assigned ID */
        OPTIONAL STRING 1, 11;
        /* Prod/Serv ID Qual */
        MANDATORY STRING 2, 2;
        /* Product/Service ID */
        MANDATORY STRING 1, 30;

        /* Prod/Serv ID Qual */
        OPTIONAL STRING 2, 2;
        /* Product/Service ID */
        OPTIONAL STRING 1, 30;

        /* Prod/Serv ID Qual */
        OPTIONAL STRING 2, 2;
        /* Product/Service ID */
        OPTIONAL STRING 1, 30;

        /* Prod/Serv ID Qual */
        OPTIONAL STRING 2, 2;
        /* Product/Service ID */
        OPTIONAL STRING 1, 30;

        /* 10 more Product/Service IDs */
        .
        .
        .
        /* Prod/Serv ID Qual */
        OPTIONAL STRING 2, 2;
        /* Product/Service ID */
        OPTIONAL STRING 1, 30;

        /* check relations between elements */
        CONDITIONAL RELATION 4, 5;
        CONDITIONAL RELATION 6, 7;
        CONDITIONAL RELATION 8, 9;
        .
        .
        .
        CONDITIONAL RELATION 30, 31;
    }

    /*
     * CTP - Pricing Information.

```

```

    */
    OPTIONAL SEGMENT "CTP" {
        /* Class of Trade Code */
        OPTIONAL STRING 2, 2;
        /* Price ID Code */
        OPTIONAL STRING 3, 3;
        /* Unit Price */
        OPTIONAL NUMBER 1, 14;
        /* Quantity */
        OPTIONAL NUMBER 1, 15;
        /* Unit/Basis Meas Code */
        OPTIONAL STRING 2, 2;
        /* Price Mult Qualifier */
        OPTIONAL STRING 3, 3;
        /* Multiplier */
        OPTIONAL NUMBER 1, 10;
        /* Monetary Amount */
        OPTIONAL NUMBER 1, 15;

        /* check relations between elements */
        CONDITIONAL RELATION 4, 5;
        CONDITIONAL RELATION 6, 7;
    }
} /* Loop ID - LIN */

/*
 * SE - Transaction Set Trailer.
 */
MANDATORY SEGMENT "SE" {
    /* Number of Inc Segs */
    MANDATORY NUMERIC 1, 10;
    /* TS Control Number */
    MANDATORY STRING 4, 9;

```

15 Appendix C: Obsolete functions and statements

15.1 Presentation

This appendix contains manual pages for old and obsolete Message Builder functions and statements. These are still supported by Message Builder or Message Builder written replacements are supplied. New programs should not use them.

15.2 Reference Pages

On the following pages, all obsolete functions and statements are described.

15.2.1 BINARY

Name

BINARY – Function that converts a string to binary data.

Syntax

BINARY(*operand*)

Description

The *operand* argument is an *expression* that is converted to a string. Special sequences in the string are replaced by their binary counterparts (see the CONSTANTS section, paragraph [12.4.19](#)), and the modified string is returned.

Example

If the following program is executed:

```
DECLARE VARIABLE $notbin STRING;

$notbin = "\\x4C\\x5A";
PRINT $notbin & "\n";
PRINT BINARY($notbin) & "\n";
```

the following is written to the output file:

\x4C\x5A
LZ

Notes

The BINARY function is implemented as a Message Builder-written user function.

15.2.2 CENTER

Name

CENTER – Function that centers a string.

Syntax

CENTER(*operand*, *size* [, *pad*])

Description

The CENTER function returns the *operand* string centered within a field whose width is *size* characters. When needed, the returned string is padded with the first character in the string specified by *pad*. The default pad character is space. The CENTER function converts the *operand* and *pad expressions* to strings and the *size expression* to an integer.

If the *operand* string after removal of leading and trailing space characters is longer than *size* characters, the end of the string is truncated.

If the total number of pad characters is odd, the number of pad characters after the centered string will be one more than before it.

Examples

The following program:

```
DECLARE VARIABLE $value INTEGER;
DECLARE VARIABLE $line STRING;

$value = 4 * 109;
$line = CENTER("", 10, "-");
PRINT $line & "\n";
PRINT CENTER($value, 10) & "\n";
PRINT CENTER(NDEC(3 * 254, 0), 10) & "\n";
PRINT $line & "\n";
```

produces the following result:

```
-----
  436
  762
-----
```

See also

LEFT, RIGHT.

Notes

The CENTER function is implemented as a Message Builder-written user function.

15.2.3 CONVERT

Name

CONVERT – Map the characters in a file.

Syntax

CONVERT *filename* USING *table*;

Description

The CONVERT statement maps the characters in the file whose name is *filename* using the map table *table*. Each character in the original file is replaced by the character in *table* whose position equals the code of the original character. The index for the first character in *table* is 0. Both the *filename* and the *table* parameters are *expressions* that are converted to strings.

If a character code after mapping becomes 255, the file to convert is considered to contain invalid characters. In this case, the file is left unchanged and a warning is generated.

If the character code for a character in the file to convert is greater than or equal to the length of *table*, an error is generated and the file is left unchanged.

Errors

The following values can be assigned to `$Error`:

- `$Error_ConvertNoFile`
The *filename* parameter is empty.
- `$Error_ConvertOpenError`
Failed to open the file to convert.
- `$Error_ConvertReadError`
Read error on the file to convert.
- `$Error_ConvertTableTooShort`
A character code in the file to convert is greater than or equal to the length of *table*.
- `$Error_ConvertIllChar`
Illegal character in the file to convert. This is a warning.
- `$Error_ConvertSeekError`
Seek error on the file to convert.
- `$Error_ConvertTempName`
Failed to generate a temporary file name.
- `$Error_ConvertOpenTempError`
Failed to open the temporary file.
- `$Error_ConvertWriteError`
Write error on the converted file.
- `$Error_ConvertDeleteError`

Failed to delete the original file.

- `$Error_ConvertLinkError`
Failed to rename the temporary file to *filename*.
- `$Error_ConvertCopyError`
Failed to copy the temporary file to *filename*.

Examples

The following example converts all lowercase letters in a file to uppercase:

```
DECLARE VARIABLE $StrUpper STRING;
$StrUpper = "                " /* 00-1F */
& " !\" %&'()*+,-./0123456789:;<=>?" /* 20-3F */
& "@ABCDEFGHIJKLMNOPQRSTUVWXYZ" /* 40-5F */
& "@ABCDEFGHIJKLMNOPQRSTUVWXYZ" /* 60-7F */
& "                " /* 80-9F */
& "                " /* A0-BF */
& "                " /* C0-DF */
& "                "; /* E0-FF */

CONVERT "file.txt" USING $StrUpper;
```

See also

STRCNV.

Notes

The CONVERT statement can, for example, be used to convert a file from EBCDIC to ISO8859, or vice versa.

When converting large files, a temporary file is used. The name of this file is obtained using the *tempnam* C library function. The location of the temporary file is the current directory for the Message Builder interpreter.

15.2.4 EDI

Name

EDI – Function that adds EDI release characters to a string.

Syntax

EDI(*operand*)

Description

The EDI function inserts a release character before all special EDI characters found in *operand*. Trailing spaces are removed. The modified string is returned. The *operand* argument is an *expression* that is converted to a string.

If the release character is undefined (`$EdiRelChar = " "`), no characters are inserted before special EDI characters.

The following special EDI characters are searched for (reserved variables set to a space do not cause release characters to be inserted):

• Segment separator	\$EdiSegSep
Element separator	\$EdiElemSep
Component element separator	\$EdiCompSep
Segment code separator	\$EdiSegCodeSep
Pad character	\$EdiPadChar
Release character	\$EdiRelChar

Examples

Executing the program:

```
/* set EDIFACT separators */
$EdiSegSep = "'";
$EdiSegCodeSep = " ";
$EdiElemSep = "+";
$EdiCompSep = ":";
$EdiRelChar = "?";
$EdiPadChar = " ";

PRINT "XXX" & $EdiElemSep;
PRINT EDI("12:50") & $EdiSegSep;
```

causes the following to be written to the output file:

```
XXX+12?:50'
```

Notes

The main usage of the EDI function is for printing data elements in an EDI message.

The EDI function is implemented as a Message Builder-written user function.

Calling the EDI function is equivalent with calling the ESCAPE function (see paragraph [12.4.46](#)) with only one argument, except that trailing spaces are not removed by the ESCAPE function.

15.2.5 LEFT

Name

LEFT – Function that returns a left-justified string.

Syntax

LEFT(*operand* [, *size* [, *pad*]])

Description

The LEFT function returns the *operand* string left justified; that is, leading space characters are removed, and if the *size* argument is specified, the string is padded with trailing pad characters, so that the total length becomes *size*. The first character of the *pad* argument is the pad character. The default is a space.

The *operand* and *pad* arguments are *expressions* that are converted to strings, and *size* is an *expression* that is converted to an integer.

If the *operand* string after removal of leading space characters is longer than *size* characters, the end of the string is truncated.

Examples

The following example:

```
PRINT LEFT("    Hello ") & "World!\n";
PRINT LEFT("    Hello ", 10) & "World!\n";
PRINT LEFT("    Hello World!!!!", 10) & "\n";
```

results in the following being written to the output file:

```
Hello World!
Hello      World!
Hello Worl
```

The code fragment:

```
PRINT LEFT("Socks", 8, ".") & RIGHT(213, 5, ".") & "\n";
PRINT LEFT("Shorts", 8, ".") & RIGHT(7152, 5, ".") & "\n";
PRINT LEFT("Shirts", 8, ".") & RIGHT(64, 5, ".") & "\n";
```

writes the following table to the output file:

```
Socks.....213
Shorts...7152
Shirts.....64
```

See also

CENTER, RIGHT, TRUNC.

Notes

The LEFT function is implemented as a Message Builder-written user function.

15.2.6 NDEC

Name

NDEC – Function that returns a string with the specified number of fractional digits.

Syntax

NDEC(*operand*, *frac*)

Description

The NDEC function converts the floating-point number specified by the *operand* argument to a string with *frac* number of fractional digits. The resulting string is returned.

The *operand* argument is an *expression* that is converted to a floating-point number and *frac* is an *expression* that is converted to an integer.

Examples

The following one-line program:

```
PRINT NDEC(12.3, 2) & "\n";
```

produces the following output:

```
12.30
```

See also

FORMAT.

Notes

The NDEC function is implemented as a Message Builder-written user function.

15.2.7 ON EDI ERROR

Name

ON EDI ERROR – Set up an EDI error handling Message Builder program.

Syntax

ON EDI ERROR *filename*;

Description

The ON EDI ERROR statement tells the Message Builder interpreter which Message Builder program to switch to when an EDI syntax error occurs. The *filename* parameter is an *expression* that is converted to a string and used as the file name of the program to switch to when an EDI error occurs.

When the Message Builder interpreter switches to the EDI error handling program, the values of all variables (except the reserved ones) are forgotten, but all open files remain open.

The Message Builder interpreter uses the same search path to find the program file specified in the ON EDI ERROR statement as the program file specified on the Message Builder interpreter command line. If *filename* has the extension ".s4", it is replaced by ".x4".

Errors

The following values can be assigned to `$Error`:

- `$Error_ProgramOpenError`
Failed to open the executable Message Builder program file.
- `$Error_ProgramEmpty`
The Message Builder program file is empty.
- `Error_ProgramReadError`
Error when reading the Message Builder program file.

Examples

```
ON EDI ERROR "error.s4";
```

See also

ON EXECUTE ERROR, SWITCH.

15.2.8 ON EXECUTE ERROR

Name

ON EXECUTE ERROR – Specify the action to be taken when an execution error occurs.

Syntax

```
ON EXECUTE ERROR [CONTINUE | EXIT];
```

Description

This statement specifies whether the Message Builder interpreter continues the execution of a program when an execution error occurs.

When CONTINUE is specified:

- If an ERROR occurs, it is down rated to a WARNING and the interpreter continues executing the program.
- If a FATAL ERROR occurs, the interpreter will halt the execution.

When EXIT (the default) is specified, the Message Builder interpreter will halt the execution for both ERRORS and FATAL ERRORS.

The last executed ON EXECUTE ERROR statement overrides previous ones. The ON EXECUTE ERROR statement does not override the ON EDI ERROR statement.

The reserved variable `$Error` is updated when an error occurs, whether an ON EXECUTE ERROR statement has been executed or not.

Examples

Try to read a file that might or might not exist:

```
DECLARE VARIABLE $answer STRING;
DECLARE VARIABLE $config_file = "config" STRING;

ON EXECUTE ERROR CONTINUE;
$error = 0;      /* clear the old error */
OPEN FILE INPUT $config_file;
ON EXECUTE ERROR EXIT;
IF $Error <> 0 {
    CLOSE INPUT;
    PRINT "The file doesn't exist!\n";
    PRINT "Do you wish to create the file?\n";
    READ $answer UNTIL "\n";
    IF $answer = "Y" OR $answer = "y" {
        /* create the file */
        OPEN FILE OUTPUT $config_file;
```

```

        CLOSE OUTPUT;
    }
}

```

See also

ON EDI ERROR.

15.2.9 READTAG

Name

READTAG – Read a tag value from a parameter file.

Syntax

READTAG *tagname* FILE *filename* INTO *destination*;

Description

The READTAG statement reads a tag value from a parameter file and assigns the value to a variable.

The name of the parameter file containing the tag is specified by *filename*, and the name of the tag to read is specified by *tagname*. The tag value is assigned to the *variable* specified by *destination*. The *tagname* and *filename* parameters are *expressions* that are converted to strings.

If an execution error occurs, the contents of the *destination* variable are undefined.

Errors

The following values can be assigned to `$Error`:

- `$Error_ReadtagEmptyFilename`
The *filename* parameter is an empty string.
- `$Error_ReadtagEmptyTagname`
The *tagname* parameter is an empty string.
- `$Error_ReadtagOpenfail`
Failed to open the parameter file.
- `$Error_ReadtagNotag`
The tag name does not exist in the parameter file.
- `$Error_ReadtagReadfail`
Read error when reading the parameter file.

Examples

The following example reads the tag named *TestTag* from the *pfile* file:

```

READTAG "TestTag" FILE "pfile" INTO $tagval;
PRINT "Tag value is '" & $tagval & "'\n";

```

A problem with the above example is that if the *TestTag* tag does not exist, an execution error is generated. The following example uses the TRY statement to take care of execution errors:

```

INCLUDE "exception.s4h";
DECLARE VARIABLE $tagval STRING;

TRY {
    READTAG "TestTag" FILE "pfile" INTO $tagval;
    PRINT "Tag value is '" & $tagval & "'\n";
}
CATCH $exception
WHEN $Exception_Execute {
    PRINT "Error when reading tag\n";
}

```

See also

WRITETAG.

Notes

A parameter file is a text file containing tags and values. A tag is a case-sensitive label used to identify a value. A value is any text. A tag and its value are separated by an equal character (=). All characters between the start of a line and the equal character are the tag name. The value is all the characters between the equal character and the end of the line. A value may continue on the next line if the last character of the line is a backslash character (\). The backslash and the new line characters are removed. The following special character sequences may be used in the value:

- \a Audible alarm (bell).
- \b Backspace.
- \f Form feed.
- \r Carriage return.
- \t Horizontal tab.
- \v Vertical tab.
- \xnn Hexadecimal value. *nn* is one or two hexadecimal digits.
- \0 Null byte.

Empty lines and lines starting with a semicolon character (;) are ignored.

The READTAG statement is implemented as a Message Builder-written user statement.

15.2.10 RIGHT

Name

RIGHT – Function that returns a right-justified string.

Syntax

RIGHT(*operand*, *size* [, *pad*])

Description

The RIGHT function returns the *operand* string right justified; that is, trailing space characters are removed, and the string is padded with leading pad characters, so that

the total length becomes *size*. The first character of the *pad* argument is the pad character. The default is a space.

The *operand* and *pad* arguments are *expressions* that are converted to strings, and *size* is an *expression* that is converted to an integer.

If the *operand* string after removal of trailing space characters is longer than *size* characters, the end of the string is truncated.

Examples

The program:

```
PRINT RIGHT(24, 5) & "\n";
PRINT RIGHT(3735, 5) & "\n";
PRINT RIGHT(9, 5, "0") & "\n";
```

results in the following being written to the output file:

```
  24
3735
00009
```

See also

CENTER, LEFT.

Notes

The RIGHT function is implemented as a Message Builder-written user function.

15.2.11 STRCNV

Name

STRCNV – Function that maps the characters in a string.

Syntax

STRCNV(*string*, *table*)

Description

The STRCNV function maps the characters in *string* using the map table *table* and returns the new string. Both *string* and *table* are *expressions* that are converted to strings.

Each character in *string* is replaced by the character in *table* whose position equals the code of the original character. The index for the first character in *table* is 0.

If a character code after mapping becomes 255, the *string* argument is considered to contain invalid characters and a warning is generated. In this case, the return value equals *string*.

If the character code for a character in the string to convert is greater than or equal to the length of *table*, an error is generated.

Errors

The following values can be assigned to `$Error`:

- `$Error_StrcnvTableTooShort`
The *table* string is too short.
- `$Error_StrcnvIllChar`
The string to convert contains an illegal character. This is a warning.

Examples

The following program:

```
DECLARE VARIABLE $StrUpper STRING;
$StrUpper = "                " /* 00-1F */
& " !\" %&'()*+,-./0123456789:;<=>?" /* 20-3F */
& "@ABCDEFGHIJKLMNPOQRSTUVWXYZ" /* 40-5F */
& "@ABCDEFGHIJKLMNPOQRSTUVWXYZ" /* 60-7F */
& "                " /* 80-9F */
& "                " /* A0-BF */
& "                " /* C0-DF */
& "                "; /* E0-FF */

PRINT STRCNV("Hello!", $StrUpper) & "\n";
```

produces the output:

```
HELLO!
```

See also

CONVERT.

15.2.12 STRMID

Name

STRMID – Function that returns a part of a string.

Syntax

STRMID(*source*, *from*, *to*)

Description

The STRMID function extracts a sub-string from a string.

The *source* argument is the string to extract a sub-string from. It is an *expression* that is converted to a string. The *from* and *to* arguments are *expressions* that are converted to integers. The index for the start of the sub-string to extract is indicated by *from*, and the index for the last character is indicated by *to*.

The index for the first character in a string is 1.

Errors

The following values can be assigned to `$Error`:

- `$Error_StrmidTooBig`
The *to* argument is greater than the string length. This is a warning.
- `$Error_StrmidFromTooSmall`
The *from* argument is less than or equal to 0.
- `$Error_StrmidTooSmall`
The *to* argument is less than or equal to 0.
- `$Error_StrmidFromGtTo`
The *from* argument is greater than *to*.
- `$Error_StrmidFromTooBig`
The *from* argument is greater than the string length.

Examples

The following example:

```
/* extract month name */
PRINT STRMID("Thu Nov 21 1993", 5, 5 + 2);
```

results in the following being written to the output file:

```
Nov
```

See also

STRLEN.

Notes

Use the sub-string operator instead of the STRMID function. For example, the expression:

```
STRMID($string, $from, $to)
```

is equivalent to:

```
$string[$from, $to]
```

The STRMID function is implemented as a Message Builder-written user function.

15.2.13 TRUNC

Name

TRUNC – Function that removes trailing spaces.

Syntax

```
TRUNC(operand)
```

Description

The TRUNC function removes trailing spaces from the *operand* argument and returns the result.

The *operand* argument is an *expression* that is converted to a string.

Examples

The following example:

```
PRINT TRUNC("Hello      ") & "World!\n";
```

results in the following being written to the output file:

```
HelloWorld!
```

See also

LEFT.

Notes

The TRUNC function is implemented as a Message Builder-written user function.

15.2.14 UNIQUE_ID

Name

UNIQUE_ID – Function that returns a unique identity of the specified type.

Syntax

```
UNIQUE_ID(type[, filename])
```

Description

The UNIQUE_ID function returns the next identity for the identity type specified by *type*. The *type* argument is an *expression* that is converted to a string. This function is replaced by the UNIQUE extension. For more on the UNIQUE extension, refer to *Message Builder Library* manual.

By default, on UNIX and AS/400 machines, the identities are stored in the `$CORE_DATA/data/unique/uniqueid` file, and on Windows machines in the `%CORE_DATA%\data\unique\uniqueid` file. If the *filename* argument is specified, it specifies the name of the unique identities file. It is an *expression* that is converted to a string.

If the specified identity type is not found, the default identity type DEFAULT is used. To indicate this, the Message Builder interpreter sets the value of `$Error` to `$Error_UniqueIdUseDefault`.

Errors

The following values can be assigned to `$Error`:

- `$Error_UniqueIdUseDefault`
Failed to find the specified identity; the default is used instead. This is an informational code, not an error.
- `$Error_UniqueIdFail`
An error occurred while creating the unique identity.

Examples

```
INCLUDE "error.s4h";
DECLARE VARIABLE $id STRING;
$Error = 0;
$id = UNIQUE_ID("MsgRef");
IF $Error = $Error_UniqueIdUseDefault {
    PRINT "MsgRef not available as an identity type.\n";
    PRINT "We got the default instead!\n";
}
```

Notes

The UNIQUE_ID function is useful when creating the "message reference number" in a UNH segment.

A unique identities file consists of lines containing four fields:

type id-len num-base used-id

The fields follow immediately after each other; no separator or space character separates them. The *type* field is a 10-character string containing the identity type. This field is case sensitive. If it contains trailing spaces, these spaces do not have to be given when referring to the identity. The *id-len* field specifies the length of the *used-id* field. The *num-base* field specifies the base used in *used-id*. It must be in the range 2 - 36. Capital letters are used to represent "digits" in the range 10 - 35. It is typically 10 (a decimal number) or 36 (the characters 0 - 9 and A - Z). Both the *id-len* and *num-base* fields are decimal numbers of length 2 (a leading 0 must be present if the value is less than 10). The *used-id* field is the current unique identity and its initial value is normally zero. As an example, the `FileName` identity type normally is:

`FileName 053600000`

Note that a unique identities file must not contain empty lines.

The UNIQUE_ID function is implemented as a Message Builder-written user function.

15.2.15 UNIQUE_NAME

Name

UNIQUE_NAME – Function that returns a unique file name.

Syntax

UNIQUE_NAME([*dir*])

Description

The UNIQUE_NAME function creates unique file names.

The directory prefix for the unique file name is specified by the *dir* argument. The *dir* argument is an *expression* that is converted to a string.

If *dir* is not specified or is empty, a machine-dependent default directory prefix is used.

Errors

The following value can be assigned to `$Error`:

```
$Error_UniqueNameFail
```

Failed to create a unique file name.

Examples

Create a unique file name using the default directory:

```
$file1 = UNIQUE_NAME();
```

Specify a directory prefix on a UNIX or AS/400 system:

```
$file2 = UNIQUE_NAME("/usr/tradesync/tmp");
```

Specify a directory prefix on a Windows system:

```
$file2 = UNIQUE_NAME("C:\\users\\tradesync\\tmp");
```

Notes

The functionality of `UNIQUE_NAME` is based on the *tempnam* C library function.

The `UNIQUE_NAME` function creates file names that are unique in the specified directory. If `UNIQUE_NAME` is called multiple times with different directories, some of the returned file names may be equal (if the directory part is removed). If it is essential that all file names be unique, the `UNIQUE_ID` function may be used instead. One disadvantage with the `UNIQUE_ID` function is that it does not check whether the returned identity equals an existing file name.

The `UNIQUE_NAME` function is implemented as a Message Builder-written user function.

15.2.16 WRITETAG

Name

WRITETAG – Write a tag value to a parameter file.

Syntax

```
WRITETAG tagname FROM value FILE filename;
```

Description

The WRITETAG statement writes a tag value to a parameter file. If the tag does not exist, it is created. If the parameter file does not exist, it is created.

The name of the parameter file is specified by *filename*, the name of the tag by *tagname* and the value by *value*.

The *tagname*, *value* and *filename* parameters are all expressions that are converted to strings.

Errors

The following values can be assigned to `$Error`:

- `$Error_WritetagEmptyFilename`
The *filename* parameter is an empty string.
- `$Error_WritetagEmptyTagname`
The *tagname* parameter is an empty string.
- `$Error_WritetagOpenFail`
Failed to open the parameter file.
- `$Error_WritetagWriteFail`
Write error when writing to the parameter file.

Examples

```
DECLARE VARIABLE $tagval STRING;  
$tagval = "Hello World!";  
WRITETAG "TestTag" FROM $tagval FILE "pfile";
```

See also

READTAG.

Notes

See the READTAG statement, page 299, for a description of a parameter file.

The WRITETAG statement is implemented as a Message Builder-written user statement.

16 The Message Builder grammar

16.1 Presentation

This appendix contains the YACC grammar for the Message Builder compiler. Terminal symbols are written using uppercase letters, and non-terminals are written using a mix of lower- and uppercase letters. The terminal symbols are the keywords themselves.

Note that Message Builder code that appears to be valid according to the grammar is not always. Some errors are detected when the compiler performs semantic checks; for example, the RETURN statement, according to the grammar, is allowed to be used everywhere. However, the semantic phase checks that RETURN statements only are used inside user functions and statements, and if they are not, errors are generated.

```
program                : statementList
                        ;

statementList           : /* empty */
                        | statementList programStatement
                        ;

programStatement       : declaration
                        | statementCall
                        | assignment
                        | antiNologStatement
                        | blockStatement
                        | caseStatement
                        | forLoopStatement
                        | ifStatement
                        | nologStatement
                        | repeatLoopStatement
                        | tryStatement
                        | whileLoopStatement
                        | withStatement
                        | arrayStatement
                        | breakStatement
                        | callStatement
                        | closeStatement
                        | continueStatement
                        | coroutineNewStatement
                        | coroutineLockStatement
                        | coroutineResumeStatement
                        | coroutineKillStatement
                        | coroutineTerminateStatement
                        | debugStatement
                        | dumpVariablesStatement
                        | executeStatement
                        | exitStatement
                        | exportStatement
                        | fileStatement
                        | importStatement
                        | iosetCreateStatement
                        | iosetSwitchStatement
                        | iosetDeleteStatement
                        | iosetStatement
                        | loadStatement
                        | loadpgmStatement
                        | logStatement
                        | onEdiErrorStatement
                        | onExecuteErrorStatement
                        | onError
                        | openStatement
                        | pipeCloseStatement
                        | pipeOpenStatement
                        | pipeReadStatement
                        | pipeWriteStatement
                        | pipeError
                        | printStatement
                        | printerrStatement
                        | readStatement
```

```

| regexpStatement
| releaseStatement
| returnStatement
| scanStatement
| serializeStatement
| setDateStatement
| sleepStatement
| splitOpenStatement
| splitStatement
| splitCloseStatement
| suspendStatement
| switchStatement
| throwStatement
| truncateStatement
| unloadpgmStatement
| unserializeStatement
| useStatement
| ediX12LoopStatement
| ediSegmentStatement
| ediCompositeElementStatement
| ediDataElementStatement
| ediElementRelationStatement
| databaseOpenStatement
| databaseCloseStatement
| databaseExecuteStatement
| databaseError
| sqlAlterTableStatement
| sqlBeginWorkStatement
| sqlCommitWorkStatement
| sqlCreateStatements
| sqlDeleteStatement
| sqlDropStatements
| sqlGrantStatement
| sqlInsertStatement
| sqlRevokeStatement
| sqlRollbackWorkStatement
| sqlSelectStatement
| sqlUpdateStatement
;

declarationList      : /* empty */
| declarationList declaration
;

declaration          : pragmaDirective
| includeDirective
| moduleDeclaration
| variableDeclaration
| recordDeclaration
| userFunctionDeclaration
| extFunctionDeclaration
| userStatementDeclaration
| extStatementDeclaration
| ';'
| END_OF_FILE
;

pragmaDirective      : PRAGMA TRACE separatorWord ';'
| PRAGMA FILE constantExpression ';'
| PRAGMA LINE constantExpression ';'
| PRAGMA EXPORT FUNCTION modulePrefix possibleFunctionName ';'
| PRAGMA EXPORT STATEMENT modulePrefix possibleStatementName ';'
| PRAGMA WARNINGS separatorWord ';'
| PRAGMA VERSION constantExpression ',' constantExpression ';'
| PRAGMA MAIN VERSION constantExpression ',' constantExpression
  pragmaVersionDirectiveProgram ';'
;

pragmaVersionDirectiveProgram : /* empty */
| ',' constantExpression
;

includeDirective      : INCLUDE constantExpression includeOnceOption ';'
;

includeOnceOption     : /* empty */
| separatorWord
;

moduleDeclaration     : moduleDeclarationBegin declarationList '}'
;

moduleDeclarationBegin : moduleDeclarationHead '{'
;

moduleDeclarationHead : DECLARE publicModifier MODULE allIdentifiers
| DECLARE publicModifier MODULE separatorWord allIdentifiers
;

recordDeclaration     : recordDeclarationBegin recordDeclarationList '}'
| DECLARE publicModifier RECORD allIdentifiers separatorWord
  modulePrefix allIdentifiers ';'
;

recordDeclarationBegin : recordDeclarationHead '{'
;

```

```

recordDeclarationHead      : DECLARE publicModifier RECORD allIdentifiers
                           ;

recordDeclarationList      : /* empty */
                           | recordDeclarationList recordDeclarationItem
                           ;

recordDeclarationItem      : pragmaDirective
                           | includeDirective
                           | variableDeclaration
                           | recordDeclarationEmpty
                           ;

recordDeclarationEmpty     : ';'
                           ;

variableDeclaration        : variableNoRecordDeclaration
                           | variableRecordDeclaration
                           ;

variableNoRecordDeclaration : variableDeclarationBegin VARIABLE_NAME arraySpecification
                           | variableInitialization variableConstantModifier variableType
                           | variableSize ';'
                           ;

variableRecordDeclaration  : variableDeclarationBegin VARIABLE_NAME arraySpecification
                           | variableInitialization variableConstantModifier anonymousRecord
                           ;

variableDeclarationBegin   : DECLARE publicModifier
                           | DECLARE publicModifier VARIABLE
                           | DECLARE publicModifier FIELD
                           ;

variableInitialization     : /* empty */
                           | '=' variableInitExpression
                           | '=' aggregateInitialization
                           ;

aggregateInitialization    : aggregateInitializationStart '}'
                           | aggregateInitializationStart aggregateInitializationList '}'
                           ;

aggregateInitializationStart : '{'
                           ;

aggregateInitializationList: aggregateInitializationItem
                           | aggregateInitializationList ',' aggregateInitializationItem
                           ;

aggregateInitializationItem : variableInitExpression
                           | aggregateInitialization
                           ;

variableInitExpression     : constantArrayExpression
                           ;

variableConstantModifier   : /* empty */
                           | CONSTANT
                           | ONCE
                           ;

variableType               : INTEGER
                           | FLOAT
                           | DATE
                           | CHAR
                           | STRING
                           | RECORD modulePrefix allIdentifiers
                           | TYPEOF modulePrefix VARIABLE_NAME
                           ;

anonymousRecord            : anonymousRecordBegin recordDeclarationList '}'
                           ;

anonymousRecordBegin       : RECORD '{'
                           ;

variableSize               : /* empty */
                           | '(' constantExpression ')'
                           ;

outputVariable             : variableReference
                           ;

variableReference           : modulePrefix VARIABLE_NAME variableReferenceIndexList
                           | variableRecordFieldList
                           ;

variableRecordFieldList    : /* empty */
                           | variableRecordFieldList ',' VARIABLE_NAME
                           | variableReferenceIndexList
                           ;

variableReferenceIndexList : /* empty */
                           | variableReferenceIndexList '[' expression ']'

```

```

        | variableReferenceIndexList '[' expression ',' expression ']'
        ;

userFunctionDeclaration      : userFunctionHead statementList endBlock
        ;

userFunctionHead            : DECLARE publicModifier FUNCTION allIdentifiers
        functionFormalParamList arraySpecification
        functionTypeSpecification beginBlock
        ;

extFunctionDeclaration       : DECLARE publicModifier extensionOrExternalFunction FUNCTION
        allIdentifiers functionFormalParamList arraySpecification
        functionTypeSpecification ';'
        ;

extensionOrExternalFunction : EXTENSION
        | EXTERNAL
        ;

functionFormalParamList     : '(' functionDeclarationArgList ')'
        ;

functionDeclarationArgList  : /* empty */
        | functionDeclarationArgList2
        ;

functionDeclarationArgList2 : functionDeclarationArgVar
        | functionDeclarationArgList2 ',' functionDeclarationArgVar
        ;

functionDeclarationArgVar   : VARIABLE_NAME arraySpecification defaultValue
        functionTypeSpecification
        ;

functionTypeSpecification : /* empty */
        | PRIMITIVE
        | RECORD modulePrefix allIdentifiers
        ;

functionCall                : functionCallNamePath '(' expressionList ')'
        ;

functionCallNamePath        : modulePrefix possibleFunctionName
        ;

userStatementDeclaration    : userStatementHead statementList endBlock
        ;

userStatementHead           : userStatementHeadName beginBlock
        | userStatementHeadName statementDeclarationParamList beginBlock
        ;

userStatementHeadName       : DECLARE publicModifier STATEMENT allIdentifiers
        ;

extStatementDeclaration     : extStatementName ';'
        | extStatementName statementDeclarationParamList ';'
        ;

extStatementName            : DECLARE publicModifier extensionOrExternalStatement STATEMENT
        allIdentifiers
        ;

extensionOrExternalStatement : EXTENSION
        | EXTERNAL
        ;

statementDeclarationParamList : statementDeclarationParamVar
        | statementDeclarationParamList statementSeparator
        statementDeclarationParamVar
        ;

statementDeclarationParamVar : statementDeclarationParamType VARIABLE_NAME arraySpecification
        defaultValue statementDeclarationParamDrop
        statementTypeSpecification
        ;

statementDeclarationParamDrop : /* empty */
        | DROP
        ;

statementDeclarationParamType : /* empty */
        | IN
        | OUT
        | IN OUT
        ;

defaultValue                : /* empty */
        | '=' constantArrayExpression
        ;

statementTypeSpecification  : /* empty */
        | PRIMITIVE
        | RECORD IDENTIFIER
        | RECORD statementTypeModulePrefix IDENTIFIER

```

```

;

statementTypeModulePrefix    : statementTypeModulePrefix2
                             | statementTypeModulePrefix statementTypeModulePrefix2
                             ;

statementTypeModulePrefix2   : IDENTIFIER '.'
                             ;

statementCall                 : statementCallNamePath ';'
                             | statementCallNamePath statementCallParamList ';'
                             ;

statementCallNamePath        : modulePrefix possibleStatementName
                             ;

statementCallParamList       : expression
                             | statementCallParamList statementSeparator expression
                             ;

statementSeparator           : ','
                             | identifier
                             ;

arraySpecification           : /* empty */
                             | arraySpecification '[' ']'
                             ;

publicModifier               : /* empty */
                             | PUBLIC
                             ;

modulePrefix                 : /* empty */
                             | modulePrefix possibleModuleName '.'
                             ;

assignment                   : variableReference '=' expression ';'
                             ;

antiNologStatement          : ANTINOLOG beginBlock statementList endBlock
                             ;

blockStatement               : BLOCK statementBlock
                             | beginBlock statementList endBlock
                             ;

caseStatement                : caseStatementHead whenChoiceList
                             ;

caseStatementHead            : CASE expression
                             ;

forLoopStatement             : forLoopHeadBeginBlock statementList endBlock
                             ;

forLoopHeadBeginBlock       : forLoopHead beginBlock
                             ;

forLoopHead                  : FOR VARIABLE_NAME '=' expression separatorWord expression
                             | forLoopStep
                             ;

forLoopStep                  : /* empty */
                             | separatorWord expression
                             ;

ifStatement                  : ifStatementHead statementBlock ifElseIfClauseList ifElseClause
                             ;

ifStatementHead              : IF expression
                             ;

ifElseIfClauseList           : /* empty */
                             | ifElseIfClauseList ifElseIfClause
                             ;

ifElseIfClause               : ifElseIfClauseHead statementBlock
                             ;

ifElseIfClauseHead           : ELSE IF expression
                             ;

ifElseClause                 : /* empty */
                             | ELSE statementBlock
                             ;

nologStatement               : NOLOG beginBlock statementList endBlock
                             ;

repeatLoopStatement          : REPEAT statementBlock separatorWord expression ';'
                             ;

tryStatement                 : tryStatement1 whenChoiceList
                             ;

tryStatement1                : TRY beginBlock statementList endBlock tryStatementVariable

```



```

;

tryStatementVariable      : separatorWord
                          | separatorWord VARIABLE_NAME
                          ;

whenChoiceList            : whenChoice
                          | whenChoiceList whenChoice
                          ;

whenChoice                : whenSelector statementBlock
                          ;

whenSelector              : WHEN constantExpressionList
                          | WHEN
                          | WHEN OTHERS
                          ;

whileLoopStatement        : whileLoopStatementHead statementBlock
                          ;

whileLoopStatementHead    : WHILE expression
                          ;

withStatement             : withStatementHeadBeginBlock statementList endBlock
                          ;

withStatementHeadBeginBlock : withStatementHead beginBlock
                          ;

withStatementHead         : WITH withStatementVariableList
                          ;

withStatementVariableList : withStatementVariable
                          | withStatementVariableList ',' withStatementVariable
                          ;

withStatementVariable      : VARIABLE_NAME separatorWord expression
                          | VARIABLE_NAME separatorWord expression separatorWord
                          ;

arrayStatement            : ARRAY separatorWord outputVariable separatorWord expression
                          | arrayCount ';'
                          ;

arrayCount                 : /* empty */
                          | separatorWord expression
                          ;

breakStatement            : BREAK ';'
                          ;

callStatement             : CALL expression ';'
                          | CALL expression separatorWord variableReference ';'
                          ;

closeStatement            : CLOSE separatorWord closeSyncOption ';'
                          ;

closeSyncOption           : /* empty */
                          | separatorWord
                          ;

continueStatement         : CONTINUE ';'
                          ;

coroutineNewStatement     : coroutineNewPrefix statementCallNamePath ';'
                          | coroutineNewPrefix statementCallNamePath
                          | statementCallParamList ';'
                          ;

coroutineNewPrefix        : COROUTINE NEW outputVariable CALL
                          | COROUTINE NEW outputVariable COUNT expression CALL
                          ;

coroutineLockStatement    : COROUTINE LOCK beginBlock statementList endBlock
                          ;

coroutineResumeStatement  : COROUTINE RESUME expression ';'
                          ;

coroutineKillStatement    : COROUTINE KILL expression ';'
                          ;

coroutineTerminateStatement : COROUTINE TERMINATE expression ';'
                          ;

debugStatement            : DEBUG separatorWord separatorWord ';'
                          ;

dumpVariablesStatement     : DUMP separatorWord ';'
                          | DUMP separatorWord expression ';'
                          ;

executeStatement          : EXECUTE expression separatorWord expression ';'

```

```

;
exitStatement      : EXIT ';'
                  | EXIT expression ';'
                  | EXIT expression separatorWord ';'
;

exportStatement    : EXPORT expression separatorWord expression ';'
;

fileStatement      : FILE separatorWord expression fileStatementSecondParameter ';'
;

fileStatementSecondParameter : /* empty */
                              | separatorWord expression
;

importStatement    : IMPORT expression separatorWord variableReference ';'
;

ioSetCreateStatement : IOSET CREATE outputVariable ';'
;

ioSetSwitchStatement : IOSET SWITCH ';'
                    | IOSET SWITCH expression ';'
;

ioSetDeleteStatement : IOSET DELETE expression ';'
;

loadStatement      : LOAD expression separatorWord expression ';'
;

loadpgmStatement   : LOADPGM expression ';'
                    | LOADPGM expression separatorWord expression ';'
;

logStatement       : LOG expression logType ';'
;

logType            : /* empty */
                    | separatorWord expression
;

onEdiErrorStatement : ON EDI separatorWord expression ';'
;

onExecuteErrorStatement : ON EXECUTE separatorWord separatorWord ';'
;

openStatement      : OPEN separatorWord separatorWord expression openTypeOption ';'
                    | OPEN separatorWord separatorWord expression separatorWord
                      expression openTypeOption ';'
                    | OPEN separatorWord separatorWord expression separatorWord
                      expression separatorWord expression openTypeOption ';'
;

openTypeOption     : /* empty */
                    | separatorWord
;

pipeCloseStatement : PIPE CLOSE ';'
;

pipeOpenStatement  : PIPE OPEN expression pipeOpenTypeOption ';'
;

pipeOpenTypeOption : /* empty */
                    | separatorWord
;

pipeReadStatement  : PIPE READ variableReference readAmountSpecificationList ';'
;

pipeWriteStatement : PIPE WRITE expression ';'
;

printStatement     : PRINT expression ';'
;

printerrStatement  : PRINTERR expression ';'
;

readStatement      : READ variableReference readAmountSpecificationList ';'
;

readAmountSpecificationList : /* empty */
                              | readAmountSpecificationList readAmountSpecification
;

readAmountSpecification : separatorWord expression
;

```

```

regexStatement      : REGEXP expression separatorWord expression
                    ;
                    regexpStart regexpEnd ';'

regexStart          : /* empty */
                    | START outputVariable
                    ;

regexEnd            : /* empty */
                    | END outputVariable
                    ;

releaseStatement    : RELEASE ';'
                    | RELEASE expression ';'
                    ;

returnStatement     : RETURN returnValue ';'
                    ;

returnValue         : /* empty */
                    | expression
                    ;

scanStatement       : SCAN separatorWord expression ';'
                    ;

serializeStatement  : SERIALIZE expression separatorWord variableReference ';'
                    ;

setDateStatement    : SETDATE outputVariable separatorWord expression
                    ;
                    separatorWord expression ';'

sleepStatement      : SLEEP expression ';'
                    ;

splitOpenStatement  : SPLIT OPEN expression ';'
                    ;

splitStatement      : SPLIT INTO expression ';'
                    | SPLIT INTO expression splitOption expressionList splitIndex ';'
                    ;

splitOption         : AFTER
                    | BEFORE
                    | ON SEGMENT
                    ;

splitIndex          : /* empty */
                    | SET outputVariable
                    ;

splitCloseStatement : SPLIT CLOSE ';'
                    ;

suspendStatement    : SUSPEND ';'
                    | SUSPEND expression ';'
                    ;

switchStatement     : SWITCH expression ';'
                    ;

throwStatement      : THROW ';'
                    | THROW expression ';'
                    ;

truncateStatement   : TRUNCATE separatorWord ';'
                    ;

unloadpgmStatement  : UNLOADPGM expression ';'
                    | UNLOADPGM expression separatorWord expression ';'
                    ;

unserializeStatement : UNSERIALIZE expression separatorWord outputVariable ';'
                    ;

useStatement        : USE separatorWord expression useStatementFrom
                    ;
                    useStatementId AS modulePrefix allIdentifiers ';'

useStatementFrom    : /* empty */
                    | FROM expression
                    ;

useStatementId      : /* empty */
                    | ID expression
                    ;

ediX12LoopStatement : ediX12LoopStatementHead statementBlock
                    ;

ediX12LoopStatementHead : ediX12LoopQualifier ediX12LoopStatementName separatorWord
                    ;
                    expressionListNotEmpty ediSegmentRepeat

```

```

ediX12LoopStatementName      : UNBOUNDED
                              | BOUNDED
                              ;

ediX12LoopQualifier          : MANDATORY
                              | OPTIONAL
                              ;

ediSegmentStatement          : ediSegmentStatementHead statementBlock
                              ;

ediSegmentStatementHead      : ediSegmentQualifier ediSegmentStatementName
                              expressionListNotEmpty ediSegmentRepeat
                              ;

ediSegmentStatementName      : SEGMENT_GROUP
                              | SEGMENT GROUP
                              | SEGMENT
                              ;

ediSegmentQualifier          : MANDATORY
                              | ediConditional
                              | FLOATING
                              ;

ediSegmentRepeat             : /* empty */
                              | separatorWord
                              | separatorWord expression
                              ;

ediCompositeElementStatement : ediCompositeElementHead statementBlock
                              ;

ediCompositeElementHead      : ediCompositeQualifier COMPOSITE ediCompositeString
                              ;

ediCompositeQualifier        : MANDATORY
                              | ediConditional
                              ;

ediCompositeString           : /* empty */
                              | STRING
                              ;

ediDataElementStatement      : ediDataElementStatementHead ediStatementBlockOrEmpty
                              ;

ediDataElementStatementHead  : ediDataElementQualifier ediDataElementVariableOrFixed
                              ediDataElementTypeQualifier expression ediDataElementImplied
                              ediDataElementValidate ediDataElementSetVariable

ediDataElementVerify         : ediDataElementQualifier ediDataElementVariableOrFixed
                              ediDataElementTypeQualifier expression ',' expression
                              ediDataElementImplied ediDataElementValidate
                              ediDataElementSetVariable ediDataElementVerify
                              | ediDataElementQualifier ediDataElementVariableOrFixed
                              ;

ediDataElementQualifier      : MANDATORY
                              | ediConditional
                              ;

ediDataElementVariableOrFixed : /* empty */
                              | VARIABLE
                              | FIXED
                              ;

ediDataElementTypeQualifier  : BINARY
                              | DATE
                              | DECIMAL
                              | NUMBER
                              | NUMERIC
                              | STRING
                              | TIME
                              ;

ediDataElementImplied        : /* empty */
                              | IMPLIED expression
                              ;

ediDataElementValidate       : /* empty */
                              | VALIDATE expression
                              ;

ediDataElementSetVariable    : /* empty */
                              | SET ediDataElementSetFileOption outputVariable
                              ;

ediDataElementSetFileOption  : /* empty */
                              | FILE
                              ;

ediDataElementVerify         : /* empty */
                              | VERIFY expression

```

```

;
ediConditional      : CONDITIONAL
                    | OPTIONAL
                    ;

ediElementRelationStatement : ediElementRelationStatementHead ediStatementBlockOrEmpty
;

ediElementRelationStatementHead : ediElementRelationQualifier RELATION expressionList
;

ediElementRelationQualifier : PAIRED
                             | REQUIRED
                             | EXCLUSION
                             | CONDITIONAL
                             | LIST_CONDITIONAL
                             ;

ediStatementBlockOrEmpty : ';'
                         | statementBlock
                         ;

statementBlock : beginBlock statementList endBlock
;

beginBlock : '{'
;

endBlock : '}'
;

constantExpressionList : constantExpressionNode
                       | constantExpressionList ',' constantExpressionNode
                       ;

constantExpressionNode : constantExpression
;

constantExpression : expression
;

constantArrayExpression : expression
;

expressionList : /* empty */
               | expressionListNotEmpty
               ;

expressionListNotEmpty : expression
                       | expressionListNotEmpty ',' expression
                       ;

expression : '(' expression ')'
           | expression '+' expression
           | expression '-' expression
           | expression '*' expression
           | expression '/' expression
           | expression '%' expression
           | '-' expression %prec UNARY_MINUS
           | expression '&' expression
           | expression '=' expression
           | expression '<' expression
           | expression '>' expression
           | expression NOT_EQUAL expression
           | expression LESS_OR_EQUAL expression
           | expression GREATER_OR_EQUAL expression
           | expression AND expression
           | expression OR expression
           | expression XOR expression
           | NOT expression
           | expression BAND expression
           | expression BOR expression
           | expression BXOR expression
           | BNOT expression
           | expression BSHIFT expression
           | FIDENT '(' modulePrefix possibleFunctionName ')'
           | SIDENT '(' modulePrefix possibleStatementName ')'
           | functionCall
           | variableReference
           | INTEGER_CONSTANT
           | FLOAT_CONSTANT
           | STRING_LITERAL
           ;

databaseOpenStatement : DATABASE OPEN expression databaseOpenType
                     databaseOpenIdentifierDest ';'
                     ;

databaseOpenType : /* empty */
                 | TYPE expression
                 ;

databaseOpenIdentifierDest : /* empty */
                           | DBID outputVariable
                           ;

```

```

databaseCloseStatement      : DATABASE CLOSE databaseIdentifier ';'
;

databaseExecuteStatement    : DATABASE EXECUTE expression databaseIdentifier ';'
;

;

sqlAlterTableStatement      : sqlAlterTablePrefix TABLE sqlBaseTableName
                             sqlAlterTableColumnSpec ';'
;

sqlAlterTablePrefix         : EXEC SQL databaseIdentifier ALTER
;

sqlAlterTableColumnSpec     : ADD sqlColumnSpecification
                             | ADD '(' sqlColumnSpecificationList ')'
;

sqlBeginWorkStatement       : sqlBeginWorkPrefix WORK ';'
;

sqlBeginWorkPrefix          : EXEC SQL databaseIdentifier BEGIN
;

sqlCommitWorkStatement      : sqlCommitWorkPrefix WORK ';'
;

sqlCommitWorkPrefix         : EXEC SQL databaseIdentifier COMMIT
;

sqlCreateStatements         : sqlCreatePrefix sqlCreateIndexStatement
                             | sqlCreatePrefix sqlCreateTableStatement
                             | sqlCreatePrefix sqlCreateViewStatement
;

sqlCreatePrefix             : EXEC SQL databaseIdentifier CREATE
;

sqlCreateIndexStatement     : sqlCreateIndexUniqueOrEmpty INDEX sqlIndexName
                             ON sqlBaseTableName sqlCreateIndexColumnList ';'
;

sqlCreateIndexUniqueOrEmpty : /* empty */
                             | UNIQUE
;

sqlCreateIndexColumnList    : '(' sqlCreateIndexColumnList2 ')'
;

sqlCreateIndexColumnList2   : sqlCreateIndexColumn
                             | sqlCreateIndexColumnList2 ',' sqlCreateIndexColumn
;

sqlCreateIndexColumn        : sqlColumnIdentifier sqlAscOrDesc
;

sqlCreateTableStatement     : TABLE sqlBaseTableName sqlCreateTableColumnElementList ';'
;

sqlCreateTableColumnElementList : '(' sqlCreateTableColumnElemList2 ')'
;

sqlCreateTableColumnElemList2 : sqlCreateTableColumnElement
                             | sqlCreateTableColumnElemList2 ',' sqlCreateTableColumnElement
;

sqlCreateTableColumnElement : sqlColumnSpecification sqlCreateTableNotNull
;

sqlCreateTableNotNull       : /* empty */
                             | NOT NULL
;

sqlCreateViewStatement      : VIEW sqlViewedTableName sqlParanColumnIdentifierList
                             AS sqlQuerySpecification sqlCreateViewWithCheckOption ';'
;

sqlCreateViewWithCheckOption : /* empty */
                             | WITH CHECK OPTION
;

sqlDeleteStatement          : sqlDeletePrefix FROM sqlTableName sqlWhereClauseOrEmpty ';'
;

sqlDeletePrefix             : EXEC SQL databaseIdentifier DELETE
;

sqlDropStatements           : sqlDropPrefix sqlDropIndexStatement
                             | sqlDropPrefix sqlDropTableStatement
                             | sqlDropPrefix sqlDropViewStatement
;

sqlDropPrefix               : EXEC SQL databaseIdentifier DROP
;

```

```

sqlDropIndexStatement      : INDEX sqlIndexName ';'
;

sqlDropTableStatement      : TABLE sqlBaseTableName sqlCascadeOrRestrict ';'
;

sqlDropViewStatement       : VIEW sqlViewedTableName sqlCascadeOrRestrict ';'
;

sqlGrantStatement          : sqlGrantPrefix sqlGrantPrivileges ON sqlTableName
                           TO sqlGrantRevokeUsers sqlGrantWithGrantOption ';'
;

sqlGrantPrefix             : EXEC SQL databaseIdentifier GRANT
;

sqlGrantPrivileges         : ALL
                           | ALL PRIVILEGES
                           | sqlGrantPrivilegesList
;

sqlGrantPrivilegesList     : sqlGrantPrivilege
                           | sqlGrantPrivilegesList ',' sqlGrantPrivilege
;

sqlGrantPrivilege          : DELETE
                           | INSERT
                           | SELECT
                           | UPDATE sqlParanColumnIdentifierList
;

sqlGrantWithGrantOption    : /* empty */
                           | WITH GRANT OPTION
;

sqlInsertStatement         : sqlInsertPrefix INTO sqlTableName sqlParanColumnIdentifierList
                           sqlInsertList ';'
;

sqlInsertPrefix            : EXEC SQL databaseIdentifier INSERT
;

sqlInsertList              : VALUES '(' sqlExpressionList ')'
                           | sqlQuerySpecification
;

sqlRevokeStatement         : sqlRevokePrefix sqlRevokePrivileges ON sqlTableName
                           FROM sqlGrantRevokeUsers sqlCascadeOrRestrict ';'
;

sqlRevokePrefix           : EXEC SQL databaseIdentifier REVOKE
;

sqlRevokePrivileges        : ALL
                           | ALL PRIVILEGES
                           | sqlRevokePrivilegesList
;

sqlRevokePrivilegesList    : sqlRevokePrivilege
                           | sqlRevokePrivilegesList ',' sqlRevokePrivilege
;

sqlRevokePrivilege         : DELETE
| INSERT
                           | SELECT
                           | UPDATE
;

sqlGrantRevokeUsers        : PUBLIC
                           | sqlGrantRevokeUsersList
;

sqlGrantRevokeUsersList    : sqlUserName
                           | sqlGrantRevokeUsersList ',' sqlUserName
;

sqlRollbackWorkStatement   : sqlRollbackWorkPrefix WORK ';'
;

sqlRollbackWorkPrefix      : EXEC SQL databaseIdentifier ROLLBACK
;

sqlSelectStatement         : sqlSelectStatementHead statementList endBlock
;

sqlSelectStatementHead     : sqlSelectPrefix sqlSelectOption sqlSelectList INTO
                           sqlSelectIntoList sqlFromClause sqlWhereClauseOrEmpty
                           sqlSelectStatementClauses beginBlock
;

sqlSelectPrefix            : EXEC SQL databaseIdentifier SELECT
;

```

```

sqlSelectIntoList      : sqlSelectIntoList2
                        ;

sqlSelectIntoList2     : sqlSelectIntoPair
                        | sqlSelectIntoList2 ',' sqlSelectIntoPair
                        ;

sqlSelectIntoPair      : sqlSelectVariableReference
                        | sqlSelectVariableReference sqlSelectVariableReference
                        ;

sqlSelectVariableReference : sqlHostVariableIndicator outputVariable
                        ;

sqlSelectStatementClauses : /* empty */
                        | sqlGroupByClause
                        | sqlHavingClause
                        | sqlOrderByClause
                        | sqlGroupByClause sqlOrderByClause
                        | sqlGroupByClause sqlHavingClause sqlOrderByClause
                        ;

sqlUpdateStatement     : sqlUpdatePrefix sqlTableName SET sqlUpdateList
                        | sqlWhereClauseOrEmpty ','
                        ;

sqlUpdatePrefix        : EXEC SQL databaseIdentifier UPDATE
                        ;

sqlUpdateList          : sqlUpdateItem
                        | sqlUpdateList ',' sqlUpdateItem
                        ;

sqlUpdateItem          : sqlColumnIdentifier '=' sqlExpression
                        ;

sqlQuerySpecification  : SELECT sqlSelectOption sqlSelectList sqlFromClause
                        | sqlQuerySpecificationClauses
                        ;

sqlSelectOption        : /* empty */
                        | ALL
                        | DISTINCT
                        ;

sqlSelectList          : '*'
                        | sqlSelectList2
                        ;

sqlSelectList2         : sqlSelectSubList
                        | sqlSelectList2 ',' sqlSelectSubList
                        ;

sqlSelectSubList       : sqlExpression
                        | sqlUserDefinedName '.' '*'
                        | sqlUserDefinedName '.' sqlUserDefinedName '.' '*'
                        | sqlUserDefinedName ':' sqlUserDefinedName '.' '*'
                        | sqlUserDefinedName '.' sqlUserDefinedName '.'
                        | sqlUserDefinedName '.' '*'
                        ;

sqlQuerySpecificationClauses : /* empty */
                        | sqlWhereClause
                        | sqlGroupByClause
                        | sqlHavingClause
                        | sqlWhereClause sqlGroupByClause
                        | sqlWhereClause sqlGroupByClause sqlHavingClause
                        ;

sqlFromClause          : FROM sqlTableReferenceList
                        ;

sqlTableReferenceList  : sqlTableReference
                        | sqlTableReferenceList ',' sqlTableReference
                        ;

sqlTableReference      : sqlTableName
                        | sqlTableName sqlUserDefinedName
                        ;

sqlWhereClauseOrEmpty  : /* empty */
                        | sqlWhereClause
                        ;

sqlWhereClause         : WHERE sqlSearchCondition
                        ;

sqlGroupByClause       : GROUP BY sqlColumnNameList
                        ;

sqlOrderByClause       : ORDER BY sqlOrderByClauseList
                        ;

sqlOrderByClauseList   : sqlOrderByClauseObject
                        | sqlOrderByClauseList ',' sqlOrderByClauseObject
                        ;

```



```

sqlOrderByClauseObject      : sqlColumnIdentifier sqlAscOrDesc
                             ;

sqlHavingClause             : HAVING sqlSearchCondition
                             ;

sqlSearchCondition          : sqlPredicate
                             | '(' sqlSearchCondition ')'
                             | sqlSearchCondition OR sqlSearchCondition
                             | sqlSearchCondition AND sqlSearchCondition
                             ;

sqlPredicate                : sqlNotOrEmpty sqlColumnIdentifier sqlBasicPredicate
                             | sqlNotOrEmpty sqlSetFunctionReference sqlBasicPredicate
                             | sqlNotOrEmpty sqlExistPredicate
                             ;

sqlBasicPredicate           : sqlBetweenPredicate
                             | sqlComparisionPredicate
                             | sqlInPredicate
                             | sqlLikePredicate
                             | sqlNullPredicate
                             | sqlQuantifiedPredicate
                             ;

sqlBetweenPredicate         : sqlNotOrEmpty BETWEEN sqlExpression AND sqlExpression
                             ;

sqlComparisionPredicate     : sqlComparisionOperator sqlExpression
                             | sqlComparisionOperator '(' sqlQuerySpecification ')'
                             ;

sqlInPredicate              : sqlNotOrEmpty IN '(' sqlInPredicateSubQueryOrList ')'
                             ;

sqlInPredicateSubQueryOrList : sqlQuerySpecification
                             | sqlExpressionList
                             ;

sqlLikePredicate            : sqlNotOrEmpty LIKE sqlExpression
                             ;

sqlNullPredicate            : IS sqlNotOrEmpty NULL
                             ;

sqlQuantifiedPredicate      : sqlComparisionOperator sqlQuantifiedPredicateModifier
                             | '(' sqlQuerySpecification ')'
                             ;

sqlQuantifiedPredicateModifier : ALL
                             | ANY
                             ;

sqlExistPredicate           : EXISTS '(' sqlQuerySpecification ')'
                             ;

sqlComparisionOperator      : '='
                             | '<'
                             | '>'
                             | NOT_EQUAL
                             | LESS_OR_EQUAL
                             | GREATER_OR_EQUAL
                             ;

sqlNotOrEmpty               : /* empty */
                             | NOT
                             ;

sqlAscOrDesc                : /* empty */
                             | ASC
                             | DESC
                             ;

sqlColumnSpecificationList  : sqlColumnSpecificationList2
                             ;

sqlColumnSpecificationList2 : sqlColumnSpecification
                             | sqlColumnSpecificationList2 ',' sqlColumnSpecification
                             ;

sqlColumnSpecification      : sqlColumnIdentifier sqlDataType
                             ;

sqlDataType                 : sqlCharacterDataType sqlDataTypeZeroOrOneArgument
                             | CHARACTER VARYING '(' sqlUnsignedInteger ')'
                             | VARCHAR '(' sqlUnsignedInteger ')'
                             | sqlDecimalDataType sqlDataTypeZeroToTwoArguments
                             | INTEGER
                             | INT
                             | SMALLINT
                             | NUMERIC sqlDataTypeZeroToTwoArguments
                             | FLOAT sqlDataTypeZeroOrOneArgument
                             | DOUBLE PRECISION
                             | REAL
                             ;

```

```

sqlCharacterDataType      : CHARACTER
                          | CHAR
                          ;

sqlDecimalDataType       : DECIMAL
                          | DEC
                          ;

sqlDataTypeZeroOrOneArgument : /* empty */
                          | '(' sqlUnsignedInteger ')'
                          ;

sqlDataTypeZeroToTwoArguments : /* empty */
                          | '(' sqlUnsignedInteger ')'
                          | '(' sqlUnsignedInteger ',' sqlUnsignedInteger ')'
                          ;

sqlUnsignedInteger       : INTEGER_CONSTANT
                          ;

sqlColumnNameList       : sqlColumnIdentifierList
                          ;

sqlColumnName           : sqlColumnIdentifier
                          ;

sqlParanColumnIdentifierList : /* empty */
                          | '(' sqlColumnIdentifierList ')'
                          ;

sqlColumnIdentifierList  : sqlColumnIdentifierList2
                          ;

sqlColumnIdentifierList2 : sqlColumnIdentifier
                          | sqlColumnIdentifierList2 ',' sqlColumnIdentifier
                          ;

sqlColumnIdentifier      : sqlUserDefinedName
                          | sqlUserDefinedName '.' sqlUserDefinedName
                          | sqlUserDefinedName ':' sqlUserDefinedName
                          | sqlUserDefinedName '.' sqlUserDefinedName '.' sqlUserDefinedName
                          | sqlUserDefinedName ':' sqlUserDefinedName '.' sqlUserDefinedName
                          | sqlUserDefinedName '.' sqlUserDefinedName '.'
                          | sqlUserDefinedName ':' sqlUserDefinedName
                          ;

sqlBaseTableName        : sqlTableName
                          ;

sqlIndexName            : sqlTableName
                          ;

sqlViewedTableName      : sqlTableName
                          ;

sqlTableName            : sqlUserDefinedName
                          | sqlUserDefinedName '.' sqlUserDefinedName
                          ;

sqlUserName             : sqlUserDefinedName
                          ;

sqlUserDefinedName      : IDENTIFIER
                          ;

sqlExpressionList       : sqlExpressionList2
                          ;

sqlExpressionList2      : sqlExpression
                          | sqlExpressionList2 ',' sqlExpression
                          ;

sqlExpression           : '(' sqlExpression ')'
                          | sqlExpression '+' sqlExpression
                          | sqlExpression '-' sqlExpression
                          | sqlExpression '*' sqlExpression
                          | sqlExpression '/' sqlExpression
                          | '-' sqlExpression %prec UNARY_MINUS
                          | '+' sqlExpression %prec UNARY_PLUS
                          | sqlColumnName
                          | sqlSetFunctionReference
                          | INTEGER_CONSTANT
                          | FLOAT_CONSTANT
                          | USER
                          | STRING_LITERAL
                          | sqlHostVariable
                          | sqlHostVariable sqlHostVariable
                          ;

sqlHostVariable         : sqlHostVariableIndicator variableReference
                          ;

sqlHostVariableIndicator : ':'

```

```

;

sqlSetFunctionReference      : NULL
                             | COUNT '(' sqlSetFunctionRefCountArgument ')'
                             | sqlSetFunctionReferenceName
                             | '(' sqlSetFunctionReferenceArgument ')'
                             ;

sqlSetFunctionRefCountArgument : '*'
                               | sqlSetFunctionReferenceArgument
                               ;

sqlSetFunctionReferenceName : AVG
                             | MAX
                             | MIN
                             | SUM
                             ;

sqlSetFunctionReferenceArgument : sqlExpression
                                | DISTINCT sqlColumnIdentifier
                                ;

sqlCascadeOrRestrict        : /* empty */
                             | CASCADE
                             | RESTRICT
                             ;

databaseIdentifier          : /* empty */
                             | DBID expression
                             ;

separatorWord               : separatorWord2
                             ;

separatorWord2              : IDENTIFIER
                             | validReservedWords
                             | statementNameReservedWords
                             | nonStatementNameReservedWords
                             ;

allIdentifiers              : IDENTIFIER
                             | validReservedWords
                             | statementNameReservedWords
                             | expressionReservedWords
                             | nonStatementNameReservedWords
                             ;

possibleFunctionName        : IDENTIFIER
                             | validReservedWords
                             | statementNameReservedWords
                             ;

possibleStatementName       : IDENTIFIER
                             | validReservedWords
                             | expressionReservedWords
                             | nonStatementNameReservedWords
                             ;

possibleModuleName          : IDENTIFIER
                             | validReservedWords
                             ;

identifier                  : IDENTIFIER
                             ;

validReservedWords          : AFTER
                             | ALTER
                             | AS
                             | BEFORE
                             | BEGIN
                             | BINARY
                             | CHAR
                             | COMMIT
                             | COMPOSITE
                             | CONSTANT
                             | COUNT
                             | CREATE
                             | DATE
                             | DBID
                             | DECIMAL
                             | DELETE
                             | DROP
                             | EDI
                             | END
                             | EXTENSION
                             | EXTERNAL
                             | FIELD
                             | FIXED
                             | FLOAT
                             | FROM
                             | FUNCTION
                             | GRANT
                             | ID
                             | IMPLIED
                             | INSERT
                             | INTEGER

```

```

| INTO
| KILL
| LINE
| LOCK
| MAIN
| MODULE
| NEW
| NUMBER
| NUMERIC
| ONCE
| PRIMITIVE
| PUBLIC
| RECORD
| RELATION
| RESUME
| REVOKE
| ROLLBACK
| SEGMENT
| SEGMENT_GROUP
| SELECT
| SQL
| START
| STATEMENT
| STRING
| TERMINATE
| TIME
| TRACE
| TYPEOF
| UNBOUNDED
| UPDATE
| VALIDATE
| VARIABLE
| VERIFY
| VERSION
| WARNINGS
| WRITE
;

statementNameReservedWords : BLOCK
| BOUNDED
| ANTINOLOG
| ARRAY
| BREAK
| CALL
| CASE
| CLOSE
| CONDITIONAL
| CONTINUE
| COROUTINE
| DATABASE
| DEBUG
| DECLARE
| DUMP
| ELSE
| EXCLUSION
| EXEC
| EXECUTE
| EXIT
| EXPORT
| FILE
| FLOATING
| FOR
| IF
| IMPORT
| INCLUDE
| IOSET
| LIST_CONDITIONAL
| LOAD
| LOADPGM
| LOG
| MANDATORY
| NOLOG
| ON
| OPEN
| OPTIONAL
| PAIRED
| PIPE
| PRAGMA
| PRINT
| READ
| REGEXP
| RELEASE
| REPEAT
| REQUIRED
| RETURN
| SCAN
| SERIALIZE
| SETDATE
| SLEEP
| SPLIT
| SUSPEND
| SWITCH
| THROW
| TRUNCATE

```

```

| TRY
| UNLOADPGM
| UNSERIALIZE
| USE
| WHEN
| WHILE
;

expressionReservedWords : AND
| NOT
| OR
| XOR
| BAND
| BNOT
| BOR
| BXOR
| BSHIFT
;

nonStatementNameReservedWords : GROUP
| OTHERS
| SET
| TYPE
;

```